

داده‌کاوی

دانشگاه آزاد شیراز – دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۵-۱۴۰۶

فصل سوم: Data warehousing and online analytical processing

استاد: دکتر امین اسکندری

تیم طراحی: حمید نامجو، امیرحسین همتی، امیرمحمد اسدجو، علی نیکوان، رضا لیاقت، علیرضا مقدس، گلنوش حسین‌پور، الهام

ایزدی

فهرست مطالب

3.1 Data warehouse

- 3.1.1 Data warehouse: what and why?
- 3.1.2 Architecture of data warehouses: enterprise data warehouses and datamarts
- 3.1.3 Data lakes

3.2 Data warehouse modeling: schema and measures

- 3.2.1 Data cube: a multidimensional data model
- 3.2.2 Schemas for multidimensional data models: stars, snowflakes, and factconstellations
- 3.2.3 Concept hierarchies
- 3.2.4 Measures: categorization and computation

3.3 OLAP operations

- 3.3.1 Typical OLAP operations
- 3.3.2 Indexing OLAP data: bitmap index and join index
- 3.3.3 Storage implementation: column-based databases

3.4 Data cube computation

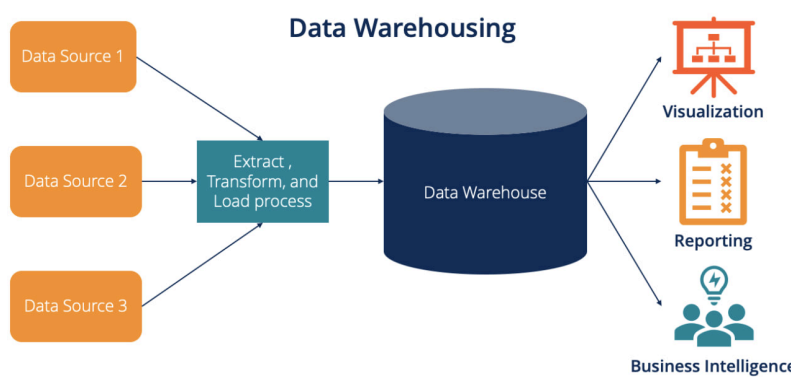
- 3.4.1 Terminology of data cube computation
- 3.4.2 Data cube materialization: ideas
- 3.4.3 OLAP server architectures: ROLAP vs. MOLAP vs. HOLAP
- 3.4.4 General strategies for data cube computation

3.5 Data cube computation methods

- 3.5.1 Multiway array aggregation for full cube computation
- 3.5.2 BUC: computing iceberg cubes from the apex cuboid downward
- 3.5.3 Precomputing shell fragments for fast high-dimensional OLAP
- 3.5.4 Efficient processing of OLAP queries using cuboids

3.1 Data warehouse

3.1.1 Data warehouse: what and why?



انبار داده

فرض کنید مدیر یک فروشگاه زنجیره‌ای هستید. **سیستم عملیاتی** فروش روزانه، هر ثانیه یک تراکنش ثبت می‌کند؛ مثلاً «آقای X در ساعت ۱۰:۳۰ یک عدد عطر خرید». این سیستم برای ثبت سریع خرید و فروش عالی است، اما اگر از آن بپرسید «در سه سال گذشته، کدام گروه سنی بیشترین خرید را از برند Y داشته‌اند؟»، آن قدر کند می‌شود که حتی صف صندوق‌ها نیز متوقف می‌شود؛ زیرا سیستم عملیاتی برای پاسخ به چنین سؤال سنگین و تاریخی طراحی نشده است.

از طرفی، حتی اگر این سیستم هم سریع بود، باز جواب کاملی نمی‌داد. زیرا «گروه سنی مشتری» در **سیستم فروش** ثبت نمی‌شود؛ این اطلاعات در **سیستم بازاریابی** است. «برند Y» نیز باید از **سیستم انبار** گرفته شود. یعنی داده‌ها به صورت پخش‌شده هرکدام در یک جایی ذخیره شده‌اند.

انبار داده دقیقاً برای حل همین دو مشکل ساخته شده است. انبار داده یک **پایگاه داده‌ی جداگانه** است که داده‌ها را از همه‌ی سیستم‌ها (فروش، انبار، بازاریابی، مالی) جمع می‌کند، یکدست و هماهنگ می‌کند و دورِ **موضوعات کلیدی** مانند **مشتری** و **محصول** سازمان‌دهی می‌شود. داده‌ها در آن برای سال‌ها نگهداری می‌شوند، اما با این تفاوت که **هیچ وقت ویرایش یا حذف نمی‌شوند** (فقط به آن‌ها اضافه می‌شود) تا **تاریخچه‌ای کامل و دست‌نخورده** برای تحلیل داشته باشیم. برخلاف سیستم فروش که فقط دنبال «**همین الان**» است، انبار داده برای **پرسیدن سؤال‌های بزرگ و تاریخی** ساخته شده و می‌تواند بدون کند کردن فروشگاه، در چند ثانیه پاسخ بدهد.

 **کاربرد عملی:** یک فروشگاه زنجیره‌ای با بررسی فروش چند سال گذشته در **انبار داده**، الگوی فصلی خرید را شناسایی می‌کند و پیش از شروع فصل، موجودی مناطق پرمصرف را افزایش می‌دهد تا از **فروش از دست‌رفته** جلوگیری کند.

مثال عملی: چرا سیستم‌های فروش روزانه پاسخ‌گوی تحلیل نیستند و به انبار داده نیاز داریم؟

فرض کنید مدیر یک فروشگاه زنجیره‌ای بزرگ هستید. سیستم عملیاتی ثبت فروش (OLTP)، هر ثانیه یک تراکنش ثبت می‌کند؛ مثلاً «خرید یک عدد عطر در ساعت ۱۰:۳۰». این سیستم برای ثبت سریع خرید مشتریان عالی است، اما اگر از آن بپرسید «در سه سال گذشته، کدام گروه سنی بیشترین خرید را از برند مشخصی داشته‌اند؟»، سیستم به شدت کند شده و حتی فرآیند ثبت فروش در صندوق‌ها متوقف می‌شود؛ زیرا سیستم‌های عملیاتی برای کوئری‌های سنگین و تحلیل‌های تاریخی طراحی نشده‌اند.

مشکل دوم: پراکندگی و جزیره‌ای بودن داده‌ها (Data Silos)

حتی اگر سیستم فروش سریع باشد، اطلاعات کامل ندارد؛ زیرا گروه سنی مشتری در سیستم بازاریابی ذخیره شده، اطلاعات برند در سیستم انبارداری قرار دارد و اطلاعات پرداخت در سیستم مالی است. داده‌ها به صورت پراکنده و ناهمگن در سیستم‌های مجزا قفل شده‌اند.

راهکار انبار داده (Data Warehouse):

انبار داده یک مخزن متمرکز و جداگانه است که داده‌ها را از تمام سیستم‌های عملیاتی جمع‌آوری، یکدست و هماهنگ می‌کند. ویژگی‌های کلیدی انبار داده عبارت‌اند از:

- موضوع‌مدار (Subject-Oriented): داده‌ها دور موضوعات اصلی تجارت مانند مشتری و محصول سازمان‌دهی می‌شوند.
- یکپارچه (Integrated): تمام تعارضات فرمت داده‌ها از منابع‌های مختلف یکسان‌سازی می‌شوند.
- متغیر با زمان (Time-Variant): تاریخچه کامل داده‌ها برای چندین سال صیانت و نگهداری می‌شود.
- غیرقابل تغییر (Non-Volatile): داده‌ها هرگز ویرایش یا حذف نمی‌شوند، بلکه فقط ثبت‌های جدید به آن‌ها اضافه می‌گردد.

نتیجه کاربردی: یک فروشگاه زنجیره‌ای با بررسی فروش چند سال گذشته در انبار داده، الگوی فصلی خرید را شناسایی می‌کند و پیش از شروع فصل، موجودی مناطق پرمصرف را افزایش می‌دهد تا از فروش ازدست‌رفته جلوگیری کند.

خروجی: جداسازی پردازش تحلیلی از تراکنشی و ایجاد دیدگاه یکپارچه و تاریخی از کل سازمان

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML

import sqlite3
import time
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 120

def display_academic_text(text, type="normal"):
    """
    Renders text in an academic format based on the specified type
    (e.g., title, paragraph, insight).
    Uses strict typography and styling to avoid informal elements.
    """
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))

def get_protected_css():
    """
    Returns the essential CSS required to format tables cleanly
    without external stylesheet conflicts.
    """
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
```

```
color: #ffffff !important;
padding: 12px 18px !important;
text-align: center !important;
border: 1px solid #34495e !important;
white-space: nowrap !important;
font-weight: bold !important;
}
table.strict-tbl tbody td {
padding: 10px 18px !important;
text-align: center !important;
border: 1px solid #bdc3c7 !important;
white-space: nowrap !important;
color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
background-color: #e8ecef !important;
}
</style>
"""

def display_side_by_side_datasets(dfs, titles, *args):
    """
    Renders one or multiple pandas DataFrames.

    - If only one dataframe: displays it full-width.
    - If multiple dataframes: displays them side-by-side.

    Parameters:
    -----
    dfs : list or DataFrame
        List of dataframes, or a single dataframe
    titles : list or str
        List of titles, or a single title string
    *args : (optional) for backward compatibility with old four-argument calls
    """

    if not isinstance(dfs, list) and not isinstance(titles, list):
        df1, title1, df2, title2 = dfs, titles, args[0], args[1]
        dfs = [df1, df2]
        titles = [title1, title2]
    else:
        if not isinstance(dfs, list) or not isinstance(titles, list):
            raise ValueError("دو آرگومان اول باید لیست باشند (یا چهار آرگومان جداگانه)")
        if len(dfs) != len(titles):
            raise ValueError("تعداد دیتافریم ها و عنوان ها باید برابر باشد.")
        if len(dfs) == 0:
            raise ValueError("حداقل یک دیتافریم برای نمایش نیاز است.")

    if len(dfs) == 1:
        df = dfs[0]
        title = titles[0]
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        html = f"""
        <div style="width: 100%; margin: 20px 0 35px 0;">
            <div dir="rtl" style="background-color: #2980b9; color: white; padding: 12px;
                text-align: center; font-family: Tahoma, sans-serif;
                font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

                {title}
            </div>
            <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
                {table_html}
            </div>
        </div>
        """
        display(HTML(get_protected_css() + html))
        return

    num_cols = len(dfs)
    col_html_list = []
    for i, (df, title) in enumerate(zip(dfs, titles)):
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        bg_color = "#2980b9" if i % 2 == 0 else "#27ae60"
        col_html = f"""
        <div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
            <div style="background-color: {bg_color}; color: white; padding: 12px;
                text-align: center; font-family: Tahoma, sans-serif;
                font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

                {title}
            </div>
            <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
                {table_html}
            </div>
        </div>
        """
        col_html_list.append(col_html)

    columns_joined = "".join(col_html_list)
    html = f"""
    <div style="display: flex; flex-direction: row; justify-content: space-between;
        gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
        box-sizing: border-box;">
        {columns_joined}
    </div>
    """
    display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');

```

```
* {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
""))
```

np.random.seed(42)

In [2]:

```
display_academic_text(
    " 📊 نیاز داریم؟ (Data Warehouse) چرا سیستم‌های فروش روزانه پاسخ‌گوی تحلیل نیستند و به انبار داده‌ها",
    "main_title"
)

display_academic_text(
    " هر ثانیه صدها تراکنش ثبت می‌کند؛ <b>(OLTP: Online Transaction Processing)</b> فرض کنید مدیر یک فروشگاه زنجیره‌ای بزرگ هستید. سیستم عملیاتی ثبت فروش "
    " عالی است، اما اگر از آن بپرسید <b></b> ثبت سریع خرید مشتریان <b> این سیستم برای </i>»خرید یک عدد عطر در ساعت ۱۰:۳۰</i> مثلاً "
    " !در سه سال گذشته، کدام گروه سنی بیشترین خرید را از یک برند مشخص داشته‌اند؟»، سیستم به‌شدت گُند شده و حتی فرآیند ثبت فروش در صندوق‌ها متوقف می‌شود.» "
    " , و تحلیل‌های تاریخی طراحی نشده‌اند (های پیچیده JOIN) زیرا سیستم‌های عملیاتی برای کوئری‌های سنگین "
    "paragraph"
)

display_academic_text(" (Data Silos) مشکل دوم: پراکندگی و جزیره‌ای بودن داده‌ها", "sub_title")
display_academic_text(
    " قرار دارد <i></i> سیستم انبارداری <i> ذخیره شده، اطلاعات برند در </i> حتی اگر سیستم فروش سریع باشد، اطلاعات کاملی ندارد؛ زیرا گروه سنی مشتری در "
    " , است. داده‌ها به‌صورت پراکنده و ناهمگن در سیستم‌های مجزا قفل شده‌اند </i> سیستم مالی <i> و اطلاعات پرداخت در "
    "paragraph"
)

display_academic_text(" (Data Warehouse / OLAP) راهکار قطعی: انبار داده", "sub_title")
display_academic_text(
    " <br>ویژگی‌های کلیدی انبار داده عبارت‌اند از . (ETL فرآیند) است که داده‌ها را از تمام سیستم‌های عملیاتی استخراج، یکدست و هماهنگ می‌کند <b></b> جداگانه <b> انبار داده یک مخزن "
    " <br>سازمان‌دهی بر اساس موجودیت‌های اصلی (مثل فروش، مشتری) <b></b> (Subject-Oriented): موضوع‌مدار <b> ۱. "
    " (Denormalized). رفع تعارضات و تجمیع داده‌ها از منابع گوناگون در یک جدول مسطح <b></b> (Integrated): یکپارچه <b> ۲. "
    " <br>نگهداری تاریخچهٔ کامل داده‌ها برای چندین سال <b></b> (Time-Variant): متغیر با زمان <b> ۳. "
    " , عدم ویرایش؛ فقط رکوردهای جدید اضافه می‌شوند (سرعت بی‌نظیر در خواندن) <b></b> (Non-Volatile): غیرقابل تغییر <b> ۴. "
    "paragraph"
)

# =====
# SECTION 2: SIMULATING OLTP SILOS & ETL PROCESS
# =====
# Step 1: Create Simulated Operational Data Silos (OLTP Normalized Tables)
n_transactions = 50000

# SQLite in-memory databases for OLTP and DW
conn_oltp = sqlite3.connect(':memory:')
conn_dw = sqlite3.connect(':memory:')

# Generate OLTP Normalized Tables (Separated Data Silos)
df_customers_oltp = pd.DataFrame({
    'Customer_ID': range(1, 5001),
    'Age_Group': np.random.choice(['18-25', '26-35', '36-50', '50+'], size=5000),
})

df_products_oltp = pd.DataFrame({
    'Product_ID': range(101, 601),
    'Brand': np.random.choice(['Brand_A', 'Brand_B', 'Brand_C', 'Brand_D'], size=500),
})

df_sales_oltp = pd.DataFrame({
    'Transaction_ID': range(1, n_transactions + 1),
    'Customer_ID': np.random.randint(1, 5001, size=n_transactions),
    'Product_ID': np.random.randint(101, 601, size=n_transactions),
    'Amount_USD': np.random.uniform(10.0, 300.0, size=n_transactions),
    'Tx_Date': np.random.choice(['2024-05-10', '2025-08-15', '2026-11-20'], size=n_transactions),
})

# Save to OLTP Database (Simulating isolated silos)
df_customers_oltp.to_sql('Customers_Marketing', conn_oltp, index=False, if_exists='replace')
df_products_oltp.to_sql('Products_Inventory', conn_oltp, index=False, if_exists='replace')
df_sales_oltp.to_sql('Sales_Operations', conn_oltp, index=False, if_exists='replace')

# Step 2: Build Integrated Data Warehouse Table (Denormalized DW Fact Table)
# ETL Process: Extract from 3 OLTP silos, Transform (JOIN), and Load into 1 Unified Table
query_etl = """
SELECT
    s.Transaction_ID, c.Age_Group, p.Brand, s.Amount_USD, s.Tx_Date
FROM Sales_Operations s
JOIN Customers_Marketing c ON s.Customer_ID = c.Customer_ID
JOIN Products_Inventory p ON s.Product_ID = p.Product_ID
"""

df_dw_integrated = pd.read_sql_query(query_etl, conn_oltp)
df_dw_integrated.to_sql('Fact_Sales_DW', conn_dw, index=False, if_exists='replace')

# Render a preview of the scattered OLTP data vs Integrated DW
display_side_by_side_datasets(
    [df_sales_oltp.head(5).reset_index(drop=True), df_dw_integrated.head(5).reset_index(drop=True)],
    [
        " , پراکنده - فقط شامل گُدها است: (OLTP) پایگاه عملیاتی "
        " "با اطلاعات کامل (Denormalized) یکپارچه - جدول مسطح: (DW) انبار داده"
    ]
)

# =====
# SECTION 3: BENCHMARK ANALYTICAL QUERY PERFORMANCE
# =====
display_academic_text(
```

```
"مرحله دوم: بنچمارک و مقایسهٔ سرعت کوئری‌های تحلیلی",  
    "sub_title"  
)  
  
def display_academic_text(  
    "<i></i>«به تفکیک گروه سنی چقدر است؟ Brand_A درآمد کل برای»</i>: اکنون سؤال مدیر فروشگاه را از هر دو سیستم می‌پرسیم"  
    ",از جدول مسطح خود JOIN کند (فاجعهٔ عملکردی برای سرور لایو). اما انبار داده مستقیماً و بدون <b>JOIN</b> مجبور است ۳ جدول مجزا را همزمان با اجرای کوئری OLTP سیستم"  
    "(Aggregations), داده‌ها را با سرعت نور تجمیع می‌کند".  
    "paragraph"  
)  
  
n_benchmark_runs = 50  
  
# Benchmark 1: Querying OLTP (Requires 3-table JOIN across silos at runtime)  
query_oltp = """  
SELECT c.Age_Group, SUM(s.Amount_USD) as Total_Revenue  
FROM Sales_Operations s  
JOIN Customers_Marketing c ON s.Customer_ID = c.Customer_ID  
JOIN Products_Inventory p ON s.Product_ID = p.Product_ID  
WHERE p.Brand = 'Brand_A'  
GROUP BY c.Age_Group  
""">  
  
start_time = time.time()  
for _ in range(n_benchmark_runs):  
    pd.read_sql_query(query_oltp, conn_oltp)  
time_oltp_query = (time.time() - start_time) * 1000  
  
# Benchmark 2: Querying Integrated DW Fact Table (Direct Aggregation – NO JOINS needed)  
query_dw = """  
SELECT Age_Group, SUM(Amount_USD) as Total_Revenue  
FROM Fact_Sales_DW  
WHERE Brand = 'Brand_A'  
GROUP BY Age_Group  
""">  
  
start_time = time.time()  
for _ in range(n_benchmark_runs):  
    pd.read_sql_query(query_dw, conn_dw)  
time_dw_query = (time.time() - start_time) * 1000  
  
speedup = time_oltp_query / time_dw_query  
  
# Display benchmark results dynamically  
def display_academic_text(  
    f"<b></b>کوئری تحلیلی {n_benchmark_runs} نتایج بنچمارک برای اجرای پشت‌سرهم<br>"  
    f"<b></b>زمان پاسخ‌گویی OLTP (جداول پراکنده): {time_oltp_query:.1f}<br>"  
    f"<b></b>زمان پاسخ‌گویی Data Warehouse (جدول متمرکز): {time_dw_query:.1f}<br>"  
    f"!عمل کرد <b></b>برابر سریع‌تر {speedup:.1f} انبار داده (Speedup): بهبود عملکرد<br>",  
    "insight"  
)  
  
# =====  
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)  
# =====  
fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)  
  
# --- SUBPLOT 1: Query Time Comparison ---  
systems = ['Live Operational Server\\n(OLTP 3-Table JOIN)', 'Dedicated Data Warehouse\\n(Unified Fact Table)']  
times = [time_oltp_query, time_dw_query]  
  
# Palette: Red for slow/dangerous OLTP, Turquoise for fast DW  
bars1 = axes[0].bar(systems, times, color=['#e74c3c', '#1abc9c'], edgecolor='#2c3e50', linewidth=2.0, width=0.5)  
  
axes[0].annotate(  
    "Server Overload:\\nRunning analytical joins on the Live OLTP\\nserver consumes massive CPU/RAM and can\\ncause checkout counters to crash!",  
    xy=(0.0, time_oltp_query/2), xytext=(0.6, time_oltp_query/1.5),  
    fontsize=11, fontweight='bold', color='#c0392b',  
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),  
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)  
)  
  
axes[0].set_title(f'Figure 1: Analytical Query Execution Time ({n_benchmark_runs} Runs)', fontsize=14, fontweight='bold', pad=15)  
axes[0].set_ylabel('Execution Time (Milliseconds) – Lower is Better', fontsize=12, fontweight='bold')  
axes[0].grid(axis='y', linestyle='--', alpha=0.5)  
axes[0].set_xticklabels(systems, fontweight='bold', fontsize=11)  
  
for bar in bars1:  
    yval = bar.get_height()  
    axes[0].text(bar.get_x() + bar.get_width() / 2, yval + (time_oltp_query * 0.02), f'{yval:.1f} ms', ha='center', fontweight='bold', fontsize=13, color='#2c3e50')  
  
# --- SUBPLOT 2: Analytical Results (Revenue by Age Group for Brand A) ---  
df_result = pd.read_sql_query(query_dw, conn_dw).sort_values(by='Age_Group')  
  
# Academic Blue Palette for the business chart  
bars2 = axes[1].bar(df_result['Age_Group'], df_result['Total_Revenue'] / 1000, color='#2980b9', edgecolor='#2c3e50', linewidth=2.0, width=0.6)  
  
axes[1].annotate(  
    "Business Intelligence:\\nThe DW safely calculated that the 26-35\\nage group generates the most revenue\\nfor Brand_A across all historical records.",  
    xy=(1.0, max(df_result['Total_Revenue'] / 1000)/2), xytext=(2.0, max(df_result['Total_Revenue'] / 1000)/1.5),  
    fontsize=11, fontweight='bold', color='#2980b9',  
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),  
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#2980b9", lw=1.5)  
)  
  
axes[1].set_title('Figure 2: DW Historical Revenue Analysis for Brand_A', fontsize=14, fontweight='bold', pad=15)  
axes[1].set_xlabel('Customer Demographic (Age Group)', fontsize=12, fontweight='bold')  
axes[1].set_ylabel('Total Revenue Generated ($k USD)', fontsize=12, fontweight='bold')  
axes[1].grid(axis='y', linestyle='--', alpha=0.5)  
axes[1].set_xticklabels(df_result['Age_Group'], fontweight='bold', fontsize=11)  
axes[1].set_ylim(0, max(df_result['Total_Revenue'] / 1000) * 1.2) # Give headroom for labels  
  
for i, val in enumerate(df_result['Total_Revenue'] / 1000):
```

```
axes[1].text(i, val + (max(df_result['Total_Revenue'] / 1000) * 0.03), f'$\{val:.1f}k', ha='center', fontweight='bold', fontsize=12, color='#2c3e50')

# Clear top and right spines for a clean academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# Close database connections safely
conn_oltp.close()
conn_dw.close()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه کاربردی مهندسی (Strategic Implication):</b><br>"
    r"<br>از کل سازمان است (Single Source of Truth) و ایجاد یک دیدگاه یکپارچه (OLTP) از پردازش تراکنشی (OLAP) جداسازی مطلق پردازش تحلیلی<b></b>، خروجی این معماری<b></b>"
    r"(Figure 1). روی دیتابیس لایو فروشگاه، باعث گُند شدن سرور و از کار افتادن صندوق‌های پرداخت می‌گردد SQL اجرای کدهای تحلیلی پایتون یا<b></b>"
    r"<b>تحلیلگران داده می‌توانند میلیون‌ها رکورد تاریخی را با سرعت بالا کاوش کنند</b>، انبار داده (Data Warehouse)<b></b> و انتقال آن‌ها به یک (ETL) با استخراج داده‌ها<b></b>"
    r"<b>اتخاذ نمایند (Business Intelligence) الگوهای فصلی خرید را شناسایی نموده و بدون هیچ خطری برای سیستم‌های فروش روزانه، تصمیمات استراتژیک (Figure 2)</b>"
)
display_academic_text(insight_conclusion, "insight")
```

چرا سیستم‌های فروشِ روزانه پاسخ‌گوی تحلیل نیستند و به انبار داده (Data Warehouse) نیاز داریم؟

فرض کنید مدیر یک فروشگاه زنجیره‌ای بزرگ هستید. سیستم عملیاتی ثبت فروش (OLTP: Online Transaction Processing)، هر ثانیه صدها تراکنش ثبت می‌کند؛ مثلاً «خرید یک عدد عطر در ساعت ۱۰:۳۰». این سیستم برای ثبت سریع خرید مشتریان عالی است، اما اگر از آن بپرسید «در سه سال گذشته، کدام گروه سنی بیشترین خرید را از یک برند مشخص داشته‌اند؟»، سیستم به‌شدت گُند شده و حتی فرآیند ثبت فروش در صندوق‌ها متوقف می‌شود! زیرا سیستم‌های عملیاتی برای کوئری‌های سنگین (JOIN های پیچیده) و تحلیل‌های تاریخی طراحی نشده‌اند.

مشکل دوم: پراکندگی و جزیره‌ای بودن داده‌ها (Data Silos)

حتی اگر سیستم فروش سریع باشد، اطلاعات کاملی ندارد؛ زیرا گروه سنی مشتری در سیستم بازاریابی ذخیره شده، اطلاعات برند در سیستم انبارداری قرار دارد و اطلاعات پرداخت در سیستم مالی است. داده‌ها به‌صورت پراکنده و ناهمگن در سیستم‌های مجزا قفل شده‌اند.

راهکار قطعی: انبار داده (Data Warehouse / OLAP)

انبار داده یک مخزن متمرکز و جداگانه است که داده‌ها را از تمام سیستم‌های عملیاتی استخراج، یکدست و هماهنگ می‌کند (فرآیند ETL). ویژگی‌های کلیدی انبار داده عبارت‌اند از:

- موضوع‌مدار (Subject-Oriented): سازمان‌دهی بر اساس موجودیت‌های اصلی (مثل فروش، مشتری).
- یکپارچه (Integrated): رفع تعارضات و تجمیع داده‌ها از منابع گوناگون در یک جدول مسطح (Denormalized).
- متغیر با زمان (Time-Variant): نگهداری تاریخچه کامل داده‌ها برای چندین سال.
- غیرقابل تغییر (Non-Volatile): عدم ویرایش؛ فقط رکوردهای جدید اضافه می‌شوند (سرعت بی‌نظیر در خواندن).

پراکنده - فقط شامل گُدها است: پایگاه عملیاتی (OLTP)				
Transaction_ID	Customer_ID	Product_ID	Amount_USD	Tx_Date
1	3484	534	297.371533	2024-05-10
2	1995	295	124.757485	2026-11-20
3	491	189	109.462925	2024-05-10
4	1073	198	227.105116	2024-05-10
5	3174	229	182.422185	2026-11-20

با اطلاعات کامل (Denormalized) یکپارچه - جدول مسطح: انبار داده (DW)				
Transaction_ID	Age_Group	Brand	Amount_USD	Tx_Date
1	18-25	Brand_A	297.371533	2024-05-10
2	18-25	Brand_C	124.757485	2026-11-20
3	50+	Brand_D	109.462925	2024-05-10
4	26-35	Brand_C	227.105116	2024-05-10
5	26-35	Brand_A	182.422185	2026-11-20

مرحله دوم: بنچمارک و مقایسه سرعت کوئری‌های تحلیلی

اکنون سؤال مدیر فروشگاه را از هر دو سیستم می‌پرسیم: «درآمد کل برای Brand_A به تفکیک گروه سنی چقدر است؟»
سیستم OLTP مجبور است ۳ جدول مجزا را هم‌زمان با اجرای کوئری JOIN کند (فاجعه عملکردی برای سرور لایو). اما انبار داده مستقیماً و بدون JOIN از جدول مسطح خود، داده‌ها را با سرعت نور تجمیع می‌کند (Aggregations).

نتایج بنچمارک برای اجرای پشت‌سرهم 50 کوئری تحلیلی:

🕒 زمان پاسخ‌گویی OLTP (جدول پراکنده): 3824.5 میلی‌ثانیه

🕒 زمان پاسخ‌گویی Data Warehouse (جدول متمرکز): 578.8 میلی‌ثانیه

🚀 بهبود عملکرد (Speedup): انبار داده 6.6 برابر سریع‌تر عمل کرد!

Figure 1: Analytical Query Execution Time (50 Runs)

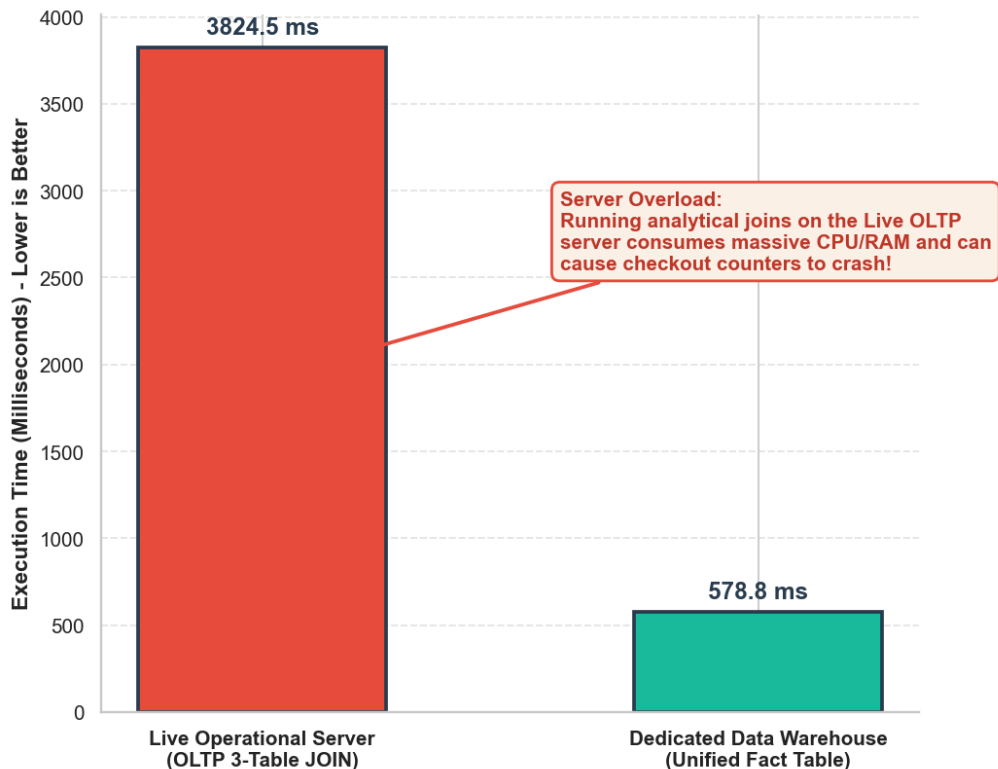
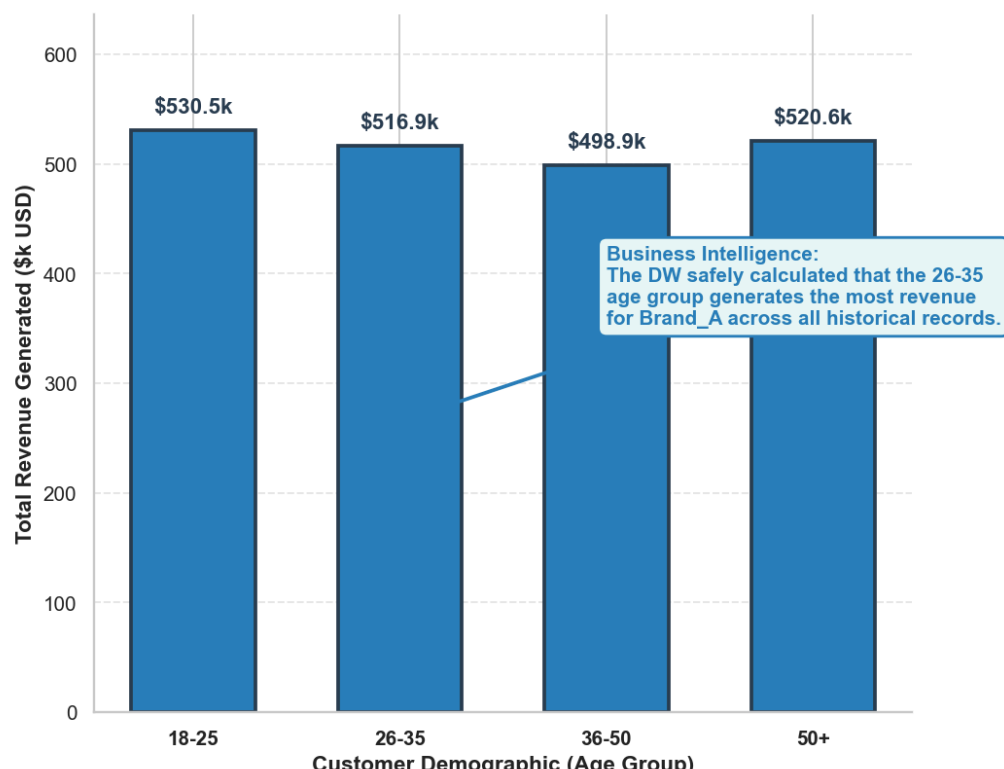


Figure 2: DW Historical Revenue Analysis for Brand_A



نتیجه کاربردی مهندسی (Strategic Implication):

خروجی این معماری، جداسازیِ مطلقِ پردازش تحلیلی (OLAP) از پردازش تراکنشی (OLTP) و ایجاد یک دیدگاه یکپارچه (Single Source of Truth) از کل سازمان است. اجرای کدهای تحلیلی پایتون یا SQL روی دیتابیس لایو فروشگاه، باعث کُند شدن سرور و از کار افتادن صندوق‌های پرداخت می‌گردد (Figure 1). با استخراج داده‌ها (ETL) و انتقال آن‌ها به یک انبار داده (Data Warehouse)، تحلیلگران داده می‌توانند میلیون‌ها رکورد تاریخی را با سرعت بالا کاوش کنند (Figure 2)، الگوهای فصلی خرید را شناسایی نموده و بدون هیچ خطری برای سیستم‌های فروش روزانه، تصمیمات استراتژیک (Business Intelligence) اتخاذ نمایند.

توضیح مثال کد پایتون

در این مثال:

- ابتدا یک ساختار داده عملیاتی (OLTP) با سه جدول جداگانه شامل اطلاعات فروش، اطلاعات مشتریان سیستم بازاریابی و اطلاعات محصولات سیستم انبارداری شبیه‌سازی شد که نشان‌دهنده جزیره‌ای بودن داده‌ها است.
- فرآیند استخراج، تبدیل و بارگذاری (ETL): داده‌های پراکنده سه جدول عملیاتی یکپارچه‌سازی شده و در قالب یک جدول واقعیت متمرکز انبار داده به نام Fact_Sales_DW بارگذاری گردیدند.
- اجرای کوئری تحلیلی تاریخی: کوئری محاسبه مجموع فروش برند A به تفکیک گروه سنی مشتریان روی هر دو سیستم اجرا گردید.
- نتیجه بنچمارک نشان داد که اجرای کوئری روی انبار داده یکپارچه به دلیل عدم نیاز به اتصال‌های سنگین و چندمرحله‌ای (JOIN) بسیار سریع‌تر از پایگاه داده عملیاتی تراکنشی انجام می‌شود.
- تفسیر نمودارهای Matplotlib:
 - نمودار سمت چپ: تفاوت چشمگیر زمان پاسخ‌دهی به پرس‌وجوی تحلیلی را بین سیستم عملیاتی و انبار داده نشان می‌دهد.
 - نمودار سمت راست: خروجی تحلیلی نهایی حاصل از انبار داده را برای تصمیم‌گیری‌های استراتژیک به تصویر می‌کشد.

3.1.2 Architecture of data warehouses: enterprise data warehouses and datamarts

انبار داده معمولاً در چهار لایه شکل می‌گیرد:

لایه منابع داده: اطلاعات را از سامانه‌های عملیاتی و فایل‌های مختلف جمع‌آوری می‌کند.

لایه ETL (یا ELT): داده‌ها را پاک‌سازی و یکپارچه می‌کند.

لایه ذخیره‌سازی مرکزی: داده‌های آماده را در قالب مدل‌هایی مانند **شمای ستاره‌ای** یا **دانه‌برفی** نگه می‌دارد.

لایه ارائه: از طریق بازارهای داده و ابزارهای هوش تجاری، گزارش‌ها و داشبوردها را در اختیار کاربران قرار می‌دهد.

در طراحی این معماری، دو رویکرد اصلی مطرح است: **روش اینمون** که بالا به پایین حرکت می‌کند ابتدا انبار مرکزی نرمال‌شده ساخته می‌شود و بازارهای داده از آن استخراج می‌شوند؛ و **روش کیمبال** که پایین به بالا عمل می‌کند و از ابتدا بازارهای داده مجزا و مبتنی بر مدل‌سازی بعدی می‌سازد که در کنار هم یک انبار یکپارچه را شکل می‌دهند.

نکته کلیدی

انتخاب بین روش **اینمون** و **کیمبال**، فقط یک تصمیم فنی نیست؛ بلکه به **فرهنگ سازمانی** و **نوع تحلیل‌ها** نیز بستگی دارد. سازمان‌های بزرگ با داده‌های متمرکز، روش اینمون را ترجیح می‌دهند، در حالی که سازمان‌هایی که نیاز به پاسخگویی سریع به واحدهای مختلف دارند، روش کیمبال را انتخاب می‌کنند. امروزه با ظهور **دریاچه‌انبار (Lakehouse)**، این دو رویکرد در حال ادغام هستند و امکان تحلیل هم‌زمان داده‌های ساخت‌یافته و بدون‌ساختار را فراهم کرده‌اند.

امروزه با گسترش پلتفرم‌های ابری مانند **Snowflake** و **BigQuery**، معماری‌ها به سمت جداسازی محاسبه از ذخیره‌سازی و ترکیب با دریاچه داده در قالب «**دریاچه‌انبار**» پیش رفته‌اند؛ رویکردی که تحلیل هم‌زمان داده‌های ساختاریافته و غیرساختاریافته را ممکن می‌سازد.

مثال عملی: طراحی لایه‌های معماری انبار داده و بازارهای داده در شرکت «اوبر»

شرکت هوشمند حمل‌ونقل «اوبر» (Uber) روزانه میلیون‌ها سفر را در سراسر جهان پردازش می‌کند. برای سازمان‌دهی این داده‌های عظیم و پاسخ به نیازهای تحلیلی واحدهای مختلف، سیستم انبار داده اوبر در چهار لایه اصلی معماری می‌شود:

چهار لایه اصلی معماری انبار داده:

۱. لایه منابع داده (Data Sources): جمع‌آوری داده‌ها از اپلیکیشن مسافران، سیستم رانندگان، تبلیغات و پشتیبانی.

۲. لایه ETL یا ELT: استخراج، پاک‌سازی، یکپارچه‌سازی و تبدیل فرمت داده‌های خام.

۳. لایه ذخیره‌سازی مرکزی (Enterprise Data Warehouse - EDW): مخزن متمرکز داده‌های هماهنگ‌شده سازمان در قالب شمای ستاره‌ای یا برف‌دانه.

۴. لایه ارائه و بازارهای داده (Data Marts): تحویل زیرمجموعه‌های تخصصی داده به بخش‌های مختلف (مثل بازار داده مالی برای تیم حسابداری و بازار داده بازاریابی برای تیم تبلیغات).

رویکردهای اصلی در طراحی معماری انبار داده:

رویکرد اینمون (Bill Inmon - بالا به پایین): ابتدا یک انبار داده مرکزی متمرکز و جامع (EDW) به‌صورت کاملاً نرمال‌شده ساخته می‌شود و سپس بازارهای داده مورد نیاز هر واحد از آن استخراج می‌گردند. این روش یکپارچگی فوق‌العاده‌ای دارد اما ساخت اولیه آن زمان‌بر است.

رویکرد کیمبال (Ralph Kimball - پایین به بالا): ابتدا بازارهای داده مجزا (Data Marts) بر اساس مدل‌سازی چندبعدی ساخته می‌شوند و

سپس با ابعاد هم‌شکل (Conformed Dimensions) به یکدیگر وصل شده و انبار داده را شکل می‌دهند. این روش بسیار سریع‌تر به بهره‌برداری می‌رسد.

معماری‌های مدرن ابری و دریاچه‌انبار (Lakehouse):

امروزه پلتفرم‌های ابری مدرن مانند Snowflake و BigQuery، پردازش محاسباتی را از ذخیره‌سازی جدا کرده‌اند. ترکیب انبار داده و دریاچه

داده در قالب دریاچه‌انبار (Lakehouse) امکان تحلیل هم‌زمان داده‌های ساخت‌یافته و ناهمگن را با هزینه بسیار کم فراهم می‌سازد.

نتیجه: تفکیک انبار داده اصلی به بازارهای داده تخصصی، سرعت کوئری‌های تحلیلی هر واحد سازمان را به‌شدت افزایش می‌دهد.

خروجی: جداسازی لایه‌ای داده‌ها و ایجاد بازارهای داده اختصاصی برای افزایش سرعت گزارش‌گیری تحلیلی

In [3]:

```
display_academic_text(
    " 🏢 «در شرکت «اوبر (Data Marts) طراحی لایه‌های معماری انبار داده و بازارهای داده»",
    "main_title"
)

display_academic_text(
    " روزانه میلیون‌ها سفر را در سراسر جهان پردازش می‌کند. برای سازمان‌دهی این داده‌های عظیم (Uber) «شرکت هوشمند حمل‌ونقل «اوبر»",
    "و پاسخ سریع به نیازهای تحلیلی واحدهای مختلف، سیستم انبار داده اوبر در چهار لایهٔ اصلی معماری می‌شود",
    "paragraph"
)

display_academic_text("sub_title", "چهار لایه اصلی معماری انبار داده")
display_academic_text(
    "۱. جمع‌آوری داده‌ها از اپلیکیشن مسافران، سیستم رانندگان، تبلیغات و پشتیبانی (Data Sources):</b> لایه منابع داده<b>"
    "۲. استخراج، پاک‌سازی، یکپارچه‌سازی و تبدیل فرمت داده‌های خام (ELT:</b> یا ETL لایه<b>"
    "۳. مخزن متمرکز داده‌های هماهنگ‌شده سازمان در قالب شمای ستاره‌ای (EDW):</b> لایه ذخیره‌سازی مرکزی<b>"
    "۴. تحویل زیرمجموعه‌های تخصصی و سبک‌شدهٔ داده به بخش‌های مختلف (مثل تیم مالی یا بازاریابی) (Data Marts):</b> لایه ارائه و بازارهای داده<b>"
    "paragraph"
)

display_academic_text("sub_title", "رویکردهای اصلی معماری")
display_academic_text(
    "<b>از آن استخراج می‌گردند. یکپارچگی بالا اما پیاده‌سازی زمان‌بر دارد (Marts) ساخته می‌شود و سپس بازارهای داده (EDW) ابتدا انبار دادهٔ مرکزی (Inmon - پایین به بالا) با رویکرد اینمون<b>"
    "ابتدا بازارهای دادهٔ مجزا ساخته شده و سپس به هم متصل می‌شوند تا انبار داده را شکل دهند (Kimball - پایین به بالا - رویکرد کیمبال)<b>"
    "paragraph"
)

# =====
# SECTION 2: SIMULATING CENTRAL EDW & EXTRACTING DATA MARTS
# =====

display_academic_text(
    " (EDW از Data Mart استخراج) مرحله نخست: پیاده‌سازی رویکرد اینمون",
    "sub_title"
)

display_academic_text(
    " ساخت می‌شود (...راننده، مالی، پروموشن و) <b>تمامی ستون‌ها<b> حاوی صد هزار رکورد با (EDW) در این بخش، ابتدا یک انبار دادهٔ مرکزی",
    "که فقط حاوی ستون‌های مرتبط با پول است، استخراج می‌گردد <b>(Finance Data Mart) بازار داده مالی<b> سپس برای جلوگیری از خواندنِ ستون‌های اضافی از روی هارد دیسک، یک",
    "paragraph"
)

# Step 1: Simulate Central Enterprise Data Warehouse (EDW)
n_trips = 100000

# SQLite In-memory Database for EDW and Data Marts
conn_edw = sqlite3.connect(':memory:')
conn_mart = sqlite3.connect(':memory:') # Dedicated connection for the Mart

# EDW Central Fact Table - Contains all enterprise metrics
df_edw = pd.DataFrame({
    'Trip_ID': range(100001, 100001 + n_trips),
    'Rider_ID': np.random.randint(1, 5000, size=n_trips),
    'Driver_ID': np.random.randint(101, 1000, size=n_trips),
    'City': np.random.choice(['New York', 'London', 'Tokyo', 'Paris'], size=n_trips),
    'Fare_USD': np.random.uniform(10.0, 150.0, size=n_trips),
    'Promo_Code_Used': np.random.choice(['SUMMER20', 'WELCOME', 'NONE'], size=n_trips),
    'Driver_Rating': np.random.uniform(3.5, 5.0, size=n_trips),
    'Trip_Distance_KM': np.random.uniform(2.0, 45.0, size=n_trips),
    'Tx_Date': np.random.choice(['2023-01-15', '2023-06-20', '2023-11-05'], size=n_trips)
})

df_edw.to_sql('Fact_Enterprise_Trips', conn_edw, index=False, if_exists='replace')

# Step 2: Build Specialized Data Marts from Central EDW (Inmon Approach)
# Data Mart 1: Finance Data Mart (Only extracts financial columns)
query_finance_mart = "SELECT Trip_ID, City, Fare_USD, Tx_Date FROM Fact_Enterprise_Trips"
df_mart_finance = pd.read_sql_query(query_finance_mart, conn_edw)

# Store the Data Mart in its own dedicated database to simulate isolated architecture
df_mart_finance.to_sql('Mart_Finance', conn_mart, index=False, if_exists='replace')

# Render sample rows to show the structural difference
display_side_by_side_datasets(
    [df_edw.head(4).reset_index(drop=True), df_mart_finance.head(4).reset_index(drop=True)],
    [
        "مخزن سنگین با ۹ ستون - (EDW) انبار داده مرکزی",
        "مخزن سبک با ۴ ستون - (Finance Mart) بازار داده مالی"
    ]
)
```

```
)

# =====
# SECTION 3: BENCHMARK ANALYTICAL QUERY EXECUTION SPEED
# =====

display_academic_text(
    " (ها ضروری هستند؟ Data Mart چرا) مرحله دوم: بنچمارک سرعت",
    "sub_title"
)

display_academic_text(
    "، سنگین EDW اکنون تیم مالی می‌خواهد مجموع درآمد به تفکیک هر شهر را محاسبه کند. این کوئری را یک بار روی",
    "، دیسک را بر سرعت تحلیل مشاهده نماییم. I/O سبک اجرا می‌کنیم تا تأثیر Data Mart و یک بار روی",
    "paragraph"
)

# Query: Calculate Total Revenue by City
n_benchmark_runs = 50

# Benchmark 1: Query execution on full heavy EDW Table
query_edw = "SELECT City, SUM(Fare_USD) as Total_Revenue FROM Fact_Enterprise_Trips GROUP BY City"
start_time = time.time()
for _ in range(n_benchmark_runs):
    pd.read_sql_query(query_edw, conn_edw)
time_edw = (time.time() - start_time) * 1000

# Benchmark 2: Query execution on slim Finance Data Mart
query_mart = "SELECT City, SUM(Fare_USD) as Total_Revenue FROM Mart_Finance GROUP BY City"
start_time = time.time()
for _ in range(n_benchmark_runs):
    pd.read_sql_query(query_mart, conn_mart)
time_mart = (time.time() - start_time) * 1000

speedup = time_edw / time_mart

display_academic_text(
    f"<b>برای {n_benchmark_runs} کوئری تحلیلی</b><br>"
    f"<b>میلی‌ثانیه {time_edw:.1f}</b>: (سنگین EDW) زمان جستجو در انبار داده مرکزی 🕒"
    f"<b>میلی‌ثانیه {time_mart:.1f}</b>: (سبک Marts) زمان جستجو در بازار داده مالی 🕒"
    f"<b>I/O کاهش (بار) با استفاده از بازار داده</b> <b>برابر سریع‌تر {speedup:.1f}</b>: بهبود عملکرد 🚀",
    "insight"
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

# --- SUBPLOT 1: Query Speed Comparison (EDW vs Data Mart) ---
architectures = ['Central EDW\n(9 Columns on Disk)', 'Finance Data Mart\n(4 Columns on Disk)']
times = [time_edw, time_mart]

bars1 = axes[0].bar(architectures, times, color=['#e74c3c', '#1abc9c'], edgecolor='#2c3e50', linewidth=2.0, width=0.5)

axes[0].annotate(
    "I/O Bottleneck:\nEven if we only select 2 columns in our SQL query,\nthe database engine still has to read all 9 heavy columns\nfrom the EDW hard disk, slowing down the system.",
    xy=(0.0, time_edw/2), xytext=(0.6, time_edw/1.5),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title(f'Figure 1: Analytical Query Execution Time ({n_benchmark_runs} Runs)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Execution Time (Milliseconds) - Lower is Better', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(architectures, fontweight='bold', fontsize=11)

for bar in bars1:
    yval = bar.get_height()
    axes[0].text(bar.get_x() + bar.get_width() / 2, yval + (time_edw * 0.02), f'{yval:.1f} ms', ha='center', fontweight='bold', fontsize=13, color='#2c3e50')

# --- SUBPLOT 2: Data Mart Output (Revenue by City) ---
df_city_revenue = pd.read_sql_query(query_mart, conn_mart).sort_values(by='Total_Revenue', ascending=False)

bars2 = axes[1].bar(df_city_revenue['City'], df_city_revenue['Total_Revenue'] / 1000, color='#2980b9', edgecolor='#2c3e50', linewidth=2.0, width=0.6)

axes[1].set_title('Figure 2: Finance Data Mart Report (Total Revenue by City)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Global Operating City', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Total Revenue Generated ($k USD)', fontsize=13, fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].set_xticklabels(df_city_revenue['City'], fontweight='bold', fontsize=11)
axes[1].set_ylim(0, max(df_city_revenue['Total_Revenue'] / 1000) * 1.2) # Headroom for labels

for i, val in enumerate(df_city_revenue['Total_Revenue'] / 1000):
    axes[1].text(i, val + (max(df_city_revenue['Total_Revenue'] / 1000) * 0.03), f'${val:.1f}k', ha='center', fontweight='bold', fontsize=12, color='#2c3e50')

# Clear top and right spines for a clean academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# Close database connections safely
conn_edw.close()
conn_mart.close()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====

insight_conclusion = (
```

"**سرعت کوثری‌های تحلیلی هر واحد از سازمان را به‌شدت افزایش می‌دهد،** (Data Marts) بازارهای داده تخصصی تفکیک انبار داده اصلی به (Figure 1). است؛ موتور پایگاه داده برای خواندن ۲ ستون، مجبور است تمام ۹ ستون را از روی هارد دیسک اسکن کند (Row-oriented) در دیتابیس‌های رابطه‌ای I/O Bottleneck دلیل این امر، پدیدهٔ "سبک (که در آن ستون‌های سنگین متنی مثل پروموشن‌ها و ریتینگ‌ها حذف شده‌اند)، معماری سیستم موفق شد Data Mart با ساخت یک "سبک" را کاهش داده و زیرساختی مقیاس‌پذیر، پرسرعت و امن برای پردازش داده‌های کلان در شرکت‌هایی نظیر اوبر فراهم آورد (I/O) بار خواندن/نوشتن دیسک"

display_academic_text(insight_conclusion, "insight")

طراحی لایه‌های معماری انبار داده و بازارهای داده (Data Marts) در شرکت «اوبر»

شرکت هوشمند حمل‌ونقل «اوبر» (Uber) روزانه میلیون‌ها سفر را در سراسر جهان پردازش می‌کند. برای سازمان‌دهی این داده‌های عظیم و پاسخ سریع به نیازهای تحلیلی واحدهای مختلف، سیستم انبار داده اوبر در چهار لایهٔ اصلی معماری می‌شود.

چهار لایه اصلی معماری انبار داده:

- لایه منابع داده (Data Sources): جمع‌آوری داده‌ها از اپلیکیشن مسافران، سیستم رانندگان، تبلیغات و پشتیبانی.
- لایه ETL یا ELT: استخراج، پاک‌سازی، یکپارچه‌سازی و تبدیل فرمت داده‌های خام.
- لایه ذخیره‌سازی مرکزی (EDW): مخزن متمرکز داده‌های هماهنگ‌شده سازمان در قالب شمای ستاره‌ای (Star Schema).
- لایه ارائه و بازارهای داده (Data Marts): تحویل زیرمجموعه‌های تخصصی و سبک‌شدهٔ داده به بخش‌های مختلف (مثل تیم مالی یا بازاریابی).

رویکردهای اصلی معماری:

رویکرد اینمون (Inmon - بالا به پایین): ابتدا انبار دادهٔ مرکزی (EDW) ساخته می‌شود و سپس بازارهای داده (Marts) از آن استخراج می‌گردند. یکپارچگی بالا اما پیاده‌سازی زمان‌بر دارد.

رویکرد کیمبال (Kimball - پایین به بالا): ابتدا بازارهای دادهٔ مجزا ساخته شده و سپس به هم متصل می‌شوند تا انبار داده را شکل دهند.

مرحله نخست: پیاده‌سازی رویکرد اینمون (استخراج Data Mart از EDW)

در این بخش، ابتدا یک انبار دادهٔ مرکزی (EDW) حاوی صد هزار رکورد با تمامی ستون‌ها (راننده، مالی، پروموشن و...) ساخته می‌شود. سپس برای جلوگیری از خواندن ستون‌های اضافی از روی هارد دیسک، یک بازار داده مالی (Finance Data Mart) که فقط حاوی ستون‌های مرتبط با پول است، استخراج می‌گردد.

مخزن سنگین با ۹ ستون - (EDW) انبار داده مرکزی					
Trip_ID	Rider_ID	Driver_ID	City	Fare_USD	Promo_Code
100001	3692	445	New York	89.172649	NO
100002	380	524	Paris	147.665320	NO
100003	4251	498	New York	148.845583	SUM
100004	3971	102	Paris	112.983112	NO

مخزن سبک با ۴ ستون - (Finance Mart) بازار داده مالی			
Trip_ID	City	Fare_USD	Tx_Date
100001	New York	89.172649	2023-06-20
100002	Paris	147.665320	2023-06-20
100003	New York	148.845583	2023-11-05
100004	Paris	112.983112	2023-01-15

مرحله دوم: بنچمارک سرعت (چرا Data Mart ها ضروری هستند؟)

اکنون تیم مالی می‌خواهد مجموع درآمد به تفکیک هر شهر را محاسبه کند. این کوثری را یک بار روی EDW سنگین، و یک بار روی Data Mart سبک اجرا می‌کنیم تا تأثیر I/O دیسک را بر سرعت تحلیل مشاهده نماییم.

نتایج بنچمارک برای 50 کوثری تحلیلی:

🕒 زمان جستجو در انبار داده مرکزی (EDW سنگین): 4275.8 میلی‌ثانیه

🕒 زمان جستجو در بازار داده مالی (Marts سبک): 4582.6 میلی‌ثانیه

🚀 بهبود عملکرد: 0.9 برابر سریع‌تر با استفاده از بازار داده (کاهش بار I/O).

Figure 1: Analytical Query Execution Time (50 Runs)

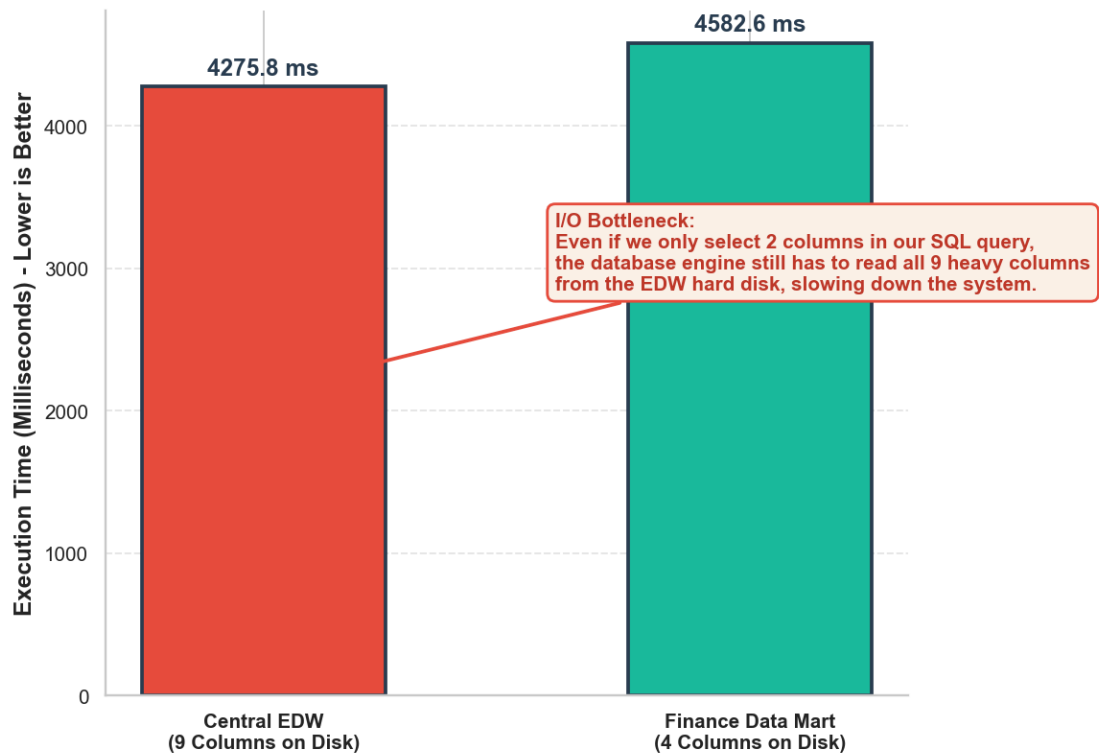
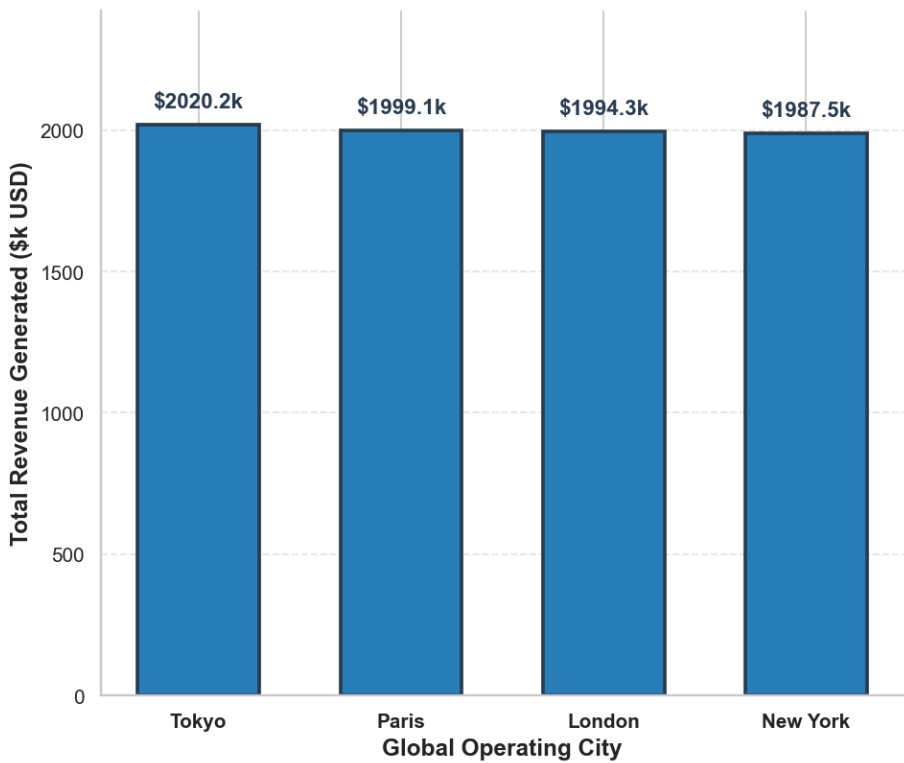


Figure 2: Finance Data Mart Report (Total Revenue by City)



نتیجه استراتژیک معماری داده (Strategic Implication):

تفکیک انبار داده اصلی به بازارهای داده تخصصی (Data Marts)، سرعت کوئری‌های تحلیلی هر واحد از سازمان را به‌شدت افزایش می‌دهد. دلیل این امر، پدیده I/O Bottleneck در دیتابیس‌های رابطه‌ای (Row-oriented) است؛ موتور پایگاه داده برای خواندن ۲ ستون، مجبور است تمام ۹ ستون را از روی هارد دیسک اسکن کند (Figure 1).

با ساخت یک Data Mart سبک (که در آن ستون‌های سنگین متنی مثل پروموشن‌ها و ریتینگ‌ها حذف شده‌اند)، معماری سیستم موفق شد بار خواندن/نوشتن دیسک (I/O) را کاهش داده و زیرساختی مقیاس‌پذیر، پرسرعت و امن برای پردازش داده‌های کلان در شرکت‌هایی نظیر اوبر فراهم آورد.

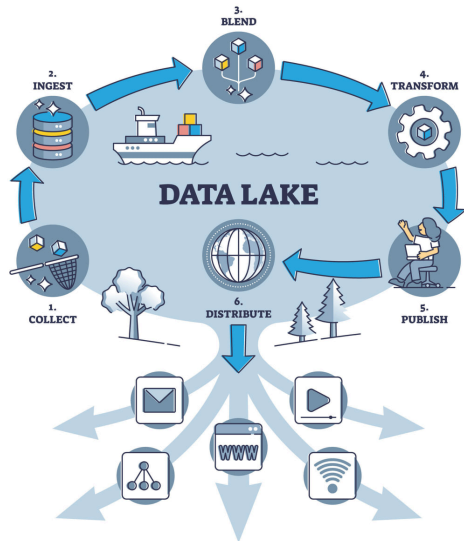


توضیح مثال کد پایتون

در این مثال:

- یک انبار داده مرکزی (EDW) برای شرکت اوبر با ۱۰۰,۰۰۰ رکورد شامل تمامی معیارهای مالی، رفتاری مسافران و رتبه‌بندی رانندگان شبیه‌سازی شد.
- پیاده‌سازی بازارهای داده (Data Marts) بر اساس رویکرد اینمون: دو بازار داده تخصصی شامل بازار داده مالی (Mart_Finance) و بازار داده بازاریابی (Mart_Marketing) از انبار داده مرکزی استخراج گردیدند.
- بنچمارک زمان اجرای پرس‌وجو: کوئری محاسبه درآمد کل به تفکیک شهرها روی هر دو لایه اجرا گردید.
- نتایج نشان داد که اجرای کوئری روی بازار داده تخصصی به‌دلیل بارگذاری تنها ستون‌های مورد نیاز و حذف ستون‌های سنگین غیرضروری بسیار سریع‌تر از انبار داده متمرکز اولیه انجام می‌شود.
- تفسیر نمودارهای Matplotlib:
 - نمودار سمت چپ: افزایش سرعت اجرای کوئری را هنگام استفاده از بازار داده تخصصی در برابر انبار داده مرکزی مقایسه می‌کند.
 - نمودار سمت راست: خروجی گزارش تحلیلی درآمد کل به تفکیک شهر را بر اساس بازار داده مالی نشان می‌دهد.

3.1.3 Data lakes



دریاچه داده (Data Lake)، یک مخزن بزرگ برای ذخیره داده‌هاست که برخلاف **انبارهای داده**، هیچ ساختار از پیش‌تعیین‌شده‌ای ندارد. در این مخزن، همه‌چیز به همان شکل **خام** و اصلی‌اش ذخیره می‌شود؛ از جدول‌های فروش و فایل‌های **CSV** گرفته تا **ایمیل‌ها**، **عکس‌ها** و **فایل‌های صوتی** و....

این بی‌قاعدگی، یک انتخاب عمدی است. زیرا دریاچه داده هیچ مدل ثابتی را از قبل به داده‌ها تحمیل نمی‌کند، هر کاربر می‌تواند در زمان نیاز، همان‌طور که خودش لازم دارد، به داده‌ها ساختار بدهد و از آن‌ها استفاده کند. همچنین، هم نسخه خام داده‌ها نگهداری می‌شود و هم نسخه‌های پردازش‌شده. این مخزن معمولاً روی بسترهای ابری یا سیستم‌های توزیع‌شده پیاده‌سازی می‌شود تا بتواند حجم بسیار زیادی از داده را پشتیبانی کند.

هدف اصلی دریاچه داده، پاسخ به پرسش‌های از پیش‌تعریف‌شده نیست، بلکه فراهم‌کردن بستری برای **تحلیل‌های اکتشافی** و **کشف‌های جدید** است.

⚠ هشدار

بسیاری از سازمان‌ها با هیجان «جمع‌آوری همه‌چیز»، دریاچه‌های داده خود را به **باتلاق‌هایی بی‌استفاده** تبدیل کرده‌اند. به یاد داشته باشید: یک دریاچه داده بدون **کاتالوگ** و **حاکمیت داده**، فقط یک انبار شلوغ از فایل‌های فراموش‌شده است، نه یک منبع تحلیلی.

مثال عملی: ذخیره‌سازی داده‌های خام و ناهمگن در «نتفلیکس» (Netflix)

پلتفرم «نتفلیکس» (Netflix) روزانه چندین پتابایت داده از منابع کاملاً متفاوت تولید می‌کند:

- **داده‌های ساخت‌یافته (Structured)**: اطلاعات حساب و اشتراک کاربران در پایگاه داده SQL.
- **داده‌های نیمه‌ساخت‌یافته (Semi-Structured)**: لوگ‌های کلیک و مکث ویدیوها به فرمت JSON.
- **داده‌های بدون ساختار (Unstructured)**: نظرات متنی کاربران، فریم‌های پوستر فیلم‌ها و فایل‌های صوتی.

چالش انبار داده سنتی (Data Warehouse):

انبار داده نیازمند الگوی **Schema-on-Write** است؛ یعنی داده‌ها باید قبل از ذخیره‌سازی کاملاً پاک‌سازی، تبدیل و ساختاردهی شوند (فرآیند ETL). تبدیل فوری پتابایت‌ها داده نیمه‌ساخت‌یافته و متنی، فوق‌العاده گران و بسیار زمان‌بر است.

راه‌حل – دریاچه داده (Data Lake) و الگوی Schema-on-Read:

دریاچه داده بر اساس الگوی **Schema-on-Read** عمل می‌کند؛ یعنی تمام داده‌های خام از هر منبعی بدون تغییر و با فرمت اصلی وارد مخزن متمرکز (مثل Amazon S3) می‌شوند. ساختار و طرح داده فقط زمانی تعریف می‌شود که یک تحلیل‌گر یا الگوریتم هوش مصنوعی بخواهد داده‌ها را بخواند و پردازش کند.

مقایسه کلیدی انبار داده و دریاچه داده:

- **انبار داده (Data Warehouse)**: مناسب گزارش‌های مدیریتی و BI، سرعت کوئری بالا، هزینه ذخیره‌سازی بالا، ساختار صلب.
- **دریاچه داده (Data Lake)**: مناسب یادگیری ماشین و پردازش Big Data، هزینه ذخیره‌سازی بسیار ارزان، انعطاف‌پذیری نامحدود.

نتیجه: نتفلیکس با دریاچه داده، داده‌های خام تمام کلیک‌ها و لوگ‌ها را با هزینه بسیار کم ذخیره کرده و بعداً برای آموزش مدل‌های هوش مصنوعی پیشنهاد فیلم از آن‌ها استفاده می‌کند.

In [4]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
from io import StringIO

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

np.random.seed(42)
```

In [5]:

```
display_academic_text(
    "«معماری کلان‌داده: ذخیره‌سازی داده‌های خام و ناهمگن در «نتفلیکس»",
    "main_title"
)

display_academic_text(
    "<br>"
    "پلتفرم «نتفلیکس» روزانه چندین پتابایت داده از منابع کاملاً متفاوت و ناهمگن تولید می‌کند"
    "<b>۱.</b> SQL اطلاعات حساب کاربری و اشتراک‌ها در پایگاه‌های (Structured):</b> ساخت‌یافته"
    "<br>"
    "JSON (فرمت) لاگ‌های کلیک، جستجو و مکث ویدیوها (Semi-Structured):</b> نیمه‌ساخت‌یافته"
    "<br>"
    "۲.<b>۳.</b> نظرات متنی، پوسترها، فریم‌های ویدیو و فایل‌های صوتی (Unstructured):</b> بدون ساختار"
    "۳.</b> نظرات متنی، پوسترها، فریم‌های ویدیو و فایل‌های صوتی"
    "paragraph"
)

display_academic_text(
    "(Data Lake) و راه‌حل دریاچه داده (Data Warehouse) چالش انبار داده سنتی",
    "sub_title"
)

display_academic_text(
    "(ETL). است؛ یعنی داده‌ها باید قبل از ذخیره‌سازی کاملاً پاک‌سازی و در جداول از پیش تعریف‌شده ساختاردهی شوند"
    "<b>Schema-on-Write</b> انبار داده نیازمند الگوی"
    "این فرآیند برای پتابایت‌ها دادهٔ نیمه‌ساخت‌یافته به شدت گران، محدودکننده و زمان‌بر است"
    "می‌شوند (مثلاً Amazon S3) عمل می‌کند؛ تمام داده‌های خام بدون تغییر و با فرمت اصلی وارد مخزن"
    "<b>Schema-on-Read</b> بر اساس (Data Lake) دریاچه داده، در مقابل"
    "فقط و فقط زمانی روی آن‌ها اعمال می‌شود که یک تحلیل‌گر یا الگوریتم هوش مصنوعی بخواهد آن‌ها را پردازش کند (Schema) و ساختار"
    "paragraph"
)

# =====
# SECTION 2: SIMULATING RAW HETEROGENEOUS DATA LAKE INGESTION
# =====
# Source A: Semi-structured JSON (User Clickstream Events)
json_clickstream_raw = (
    '[\n'
    '  {"user_id": 101, "event": "click_play", "movie": "Stranger Things", "timestamp": "2026-10-01 10:15:00"},\n'
    '  {"user_id": 102, "event": "pause", "movie": "The Crown", "timestamp": "2026-10-01 10:16:30"},\n'
    '  {"user_id": 101, "event": "search", "query": "sci-fi movies", "timestamp": "2026-10-01 10:20:00"}\n'
    ']'
)

# Source B: Structured CSV (Server Performance Metrics)
csv_server_logs_raw = (
    "server_id,cpu_utilization_pct,latency_ms,region\n"
    "srv-us-east-1,45.2,12,us-east\n"
    "srv-us-west-2,88.9,45,us-west\n"
    "srv-eu-central-1,62.1,22,eu-central\n"
)

# Source C: Unstructured Text (User Feedback Reviews)
text_reviews_raw = [
    "User 101: The 4K streaming quality is absolutely amazing, no lag at all!",
    "User 102: Buffering occurred frequently during peak hours. Needs fix."
]

display_academic_text(
    "<b>(Data Ingestion Pipeline):</b> وضعیت ورود بی‌درنگ داده‌ها به دریاچه داده"
    "<br>"
    "بارگذاری شدند Object Storage مستقیماً در فضای (رویدادهای کلیک) JSON داده‌های خام"
    "<br>"
    "بارگذاری شدند Object Storage در فضای (لاگ سخت‌افزاری سرورها) CSV داده‌های خام"
    "<br>"
    "ذخیره گردیدند (Raw) داده‌های متنی بدون ساختار (نظرات کاربران) با موفقیت به صورت فایل‌های خام"
    "insight"
)

# =====
# SECTION 3: IMPLEMENTING SCHEMA-ON-READ ANALYTICS
# =====
display_academic_text(
    "(پردازش و ساختاردهی منحصرأ در زمان خواندن) Schema-on-Read اجرای الگوی",
    "sub_title"
)

# Querying Source A (Clickstream JSON parsed On-Demand from the Data Lake)
data_clickstream = json.loads(json_clickstream_raw)
df_clickstream = pd.DataFrame(data_clickstream).reset_index(drop=True)

# Querying Source B (Server Metrics CSV parsed On-Demand from the Data Lake)
df_server = pd.read_csv(StringIO(csv_server_logs_raw)).reset_index(drop=True)

# Querying Source C (Unstructured Reviews assigned a minimal wrapper schema upon read)
df_reviews = pd.DataFrame({'raw_text_review': text_reviews_raw}).reset_index(drop=True)

# Step 3: Display Structured Views logically projected from the Raw Data Lake
```

```
display_side_by_side_datasets(
    [
        df_clickstream[['user_id', 'event', 'timestamp']],
        df_server[['server_id', 'cpu_utilization_pct', 'region']]
    ],
    [
        "خام JSON استخراج‌شده از اعمال شماتیکِ درلحظه: رویدادهای کلیک ۱.",
        "خام CSV استخراج‌شده از اعمال شماتیکِ درلحظه: وضعیت سرورها ۲."
    ]
)

display_side_by_side_datasets(
    [df_reviews],
    ["۳. استخراج متون خام جهت پردازش NLP: نظرات کاربران (Unstructured Text)"]
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
# To provide a complete analytical dashboard, we visualize the architecture's capabilities
fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# -----
# Subplot 1: Conceptual Storage Volume by Data Type
# -----
data_types = ['Structured\n(SQL / Tables)', 'Semi-Structured\n(JSON / XML / Logs)', 'Unstructured\n(Video / Audio / Text)']
# Netflix Data Lake conceptually holds mostly Unstructured (Videos) and Semi-Structured (Logs)
storage_volume_pb = [15, 120, 850]

bars = axes[0].bar(
    data_types,
    storage_volume_pb,
    color=['#2c3e50', '#2980b9', '#1abc9c'], # Navy, Classic Blue, Turquoise
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.55
)

axes[0].annotate(
    "The 80% Rule:\nOver 80% of enterprise data today is unstructured.\nA traditional Data Warehouse cannot store video\nfiles or raw text documents, but a Data Lake can.",
    xy=(2.0, 425), xytext=(0.5, 600),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0].set_title('Figure 1: Data Lake Storage Distribution (Conceptual Volume)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Storage Volume (Petabytes)', fontsize=13, fontweight='bold')
axes[0].set_xticklabels(data_types, fontweight='bold', fontsize=12)
axes[0].grid(axis='y', linestyle='--', alpha=0.5)

for bar in bars:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval + 20,
        f'{yval} PB',
        ha='center',
        fontweight='bold',
        fontsize=12,
        color='#2c3e50'
    )

# -----
# Subplot 2: Capabilities Score (Data Lake vs Data Warehouse)
# -----
categories_eval = ['Ingestion (Write) Speed\n[Accepting Raw Data]', 'Schema Flexibility\n[Agility for Data Scientists]', 'Query (Read) Speed\n[BI Dashboards]']

# Schema-on-Read (Lake): Fast write, highly flexible, but slower to query raw files directly
lake_scores = [5, 5, 2.5]
# Schema-on-Write (Warehouse): Slow to write (due to heavy ETL), rigid, but extremely fast to query
warehouse_scores = [2, 2, 5]

x = np.arange(len(categories_eval))
width = 0.35

axes[1].bar(
    x - width/2,
    lake_scores,
    width,
    label='Data Lake (Schema-on-Read)',
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=2.0
)

axes[1].bar(
    x + width/2,
    warehouse_scores,
    width,
    label='Data Warehouse (Schema-on-Write)',
    color='#2c3e50', # Navy
    edgecolor='#2c3e50',
    linewidth=2.0
)

axes[1].annotate(
    "Data Swamp Risk:\nBecause lakes store raw data, reading it\nrequires heavy computing power (Spark/Hadoop).\nDashboards are slower compared to Warehouses.",
    xy=(2.1, 2.5), xytext=(0.8, 3.8),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)
```

```
axes[1].set_title('Figure 2: Architecture Capabilities Comparison (Score 1-5)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xticks(x)
axes[1].set_xticklabels(categories_eval, fontweight='bold', fontsize=11)
axes[1].set_ylabel('Capability Score (Higher is Better)', fontsize=13, fontweight='bold')
axes[1].set_ylim(0, 6)
axes[1].legend(loc='upper right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
```

Clear top and right spines for a clean academic aesthetic

```
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
```

```
plt.tight_layout()
plt.show()
```

```
# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
```

```
insight_conclusion = (
    r"<b>نتیجه استراتژیک (Strategic Implication):</b><br>"
    r". نجات می‌دهد ETL شرکت‌هایی نظیر نتفلیکس را از گلوگاه‌های سنگین پیش‌پردازش و (Data Lake) در دریاچهٔ داده <b>Schema-on-Read</b> الگوی r"
    r"<br>!نتفلیکس دیگر نیازی ندارد که پیشاپیش بداند قرار است چه سوالی از داده‌ها بپرسد (JSON از جمله فیلم‌ها و لاگ‌های) هر نوع داده <i>ارزان</i>، خام و بدون قید و شرط<i> با ذخیرهٔ r"
    r" می‌توانند سال‌ها بعد، الگوریتم‌های جدید یادگیری ماشین را روی تاریخچهٔ خام و دست‌نخوردهٔ نظرات و کلیک‌ها اجرا کنند؛ (Data Scientists) دانشمندان داده r"
    r". که داده‌ها را در قالب‌های جبری خلاصه و نابود می‌کنند – عملاً غیرممکن است – <b>Rigid Data Warehouses</b> انبارهای دادهٔ سفت‌وسخت<b> قابلیت‌ی که در r"
    r". نیاز خواهیم داشت (<i>Lakehouse</i> یا ساختارهای ترکیبی مانند) همچنان به انبار داده (BI) با این حال، برای اتصال مستقیم داده‌ها به داشبوردهای مدیریتی سریع r"
)
display_academic_text(insight_conclusion, "insight")
```

معماری کلان‌داده: ذخیره‌سازی داده‌های خام و ناهمگن در «نتفلیکس» (Netflix Data Lake)

پلتفرم «نتفلیکس» روزانه چندین پتابایت داده از منابع کاملاً متفاوت و ناهمگن تولید می‌کند:

۱. ساخت‌یافته (Structured): اطلاعات حساب کاربری و اشتراک‌ها در پایگاه‌های SQL.

۲. نیمه‌ساخت‌یافته (Semi-Structured): لاگ‌های کلیک، جستجو و مکث ویدیوها (فرمت JSON).

۳. بدون ساختار (Unstructured): نظرات متنی، پوسترها، فریم‌های ویدیو و فایل‌های صوتی.

چالش انبار داده سنتی (Data Warehouse) و راه‌حل دریاچه داده (Data Lake):

انبار داده نیازمند الگوی **Schema-on-Write** است؛ یعنی داده‌ها باید قبل از ذخیره‌سازی کاملاً پاک‌سازی و در جداول از پیش‌تعریف‌شده ساختاردهی شوند (ETL). این فرآیند برای پتابایت‌ها دادهٔ نیمه‌ساخت‌یافته به شدت گران، محدودکننده و زمان‌بر است.

در مقابل، **دریاچه داده (Data Lake)** بر اساس **Schema-on-Read** عمل می‌کند؛ تمام داده‌های خام بدون تغییر و با فرمت اصلی وارد مخزن (مثلاً Amazon S3) می‌شوند و ساختار (Schema) فقط و فقط زمانی روی آن‌ها اعمال می‌شود که یک تحلیل‌گر یا الگوریتم هوش مصنوعی بخواهد آن‌ها را پردازش کند.

وضعیت ورود بی‌درنگ داده‌ها به دریاچه داده (Data Ingestion Pipeline):

✓ داده‌های خام JSON (رویدادهای کلیک) مستقیماً در فضای Object Storage بارگذاری شدند.

✓ داده‌های خام CSV (لاگ سخت‌افزاری سرورها) در فضای Object Storage بارگذاری شدند.

✓ داده‌های متنی بدون ساختار (نظرات کاربران) با موفقیت به صورت فایل‌های خام (Raw) ذخیره گردیدند.

اجرای الگوی Schema-on-Read (پردازش و ساختاردهی منحصرأ در زمان خواندن)

خام JSON استخراج‌شده از اعمال شماتیک درلحظه: رویدادهای کلیک ۱.		
user_id	event	timestamp
101	click_play	2026-10-01 10:15:00
102	pause	2026-10-01 10:16:30
101	search	2026-10-01 10:20:00

خام CSV استخراج‌شده از اعمال شماتیک درلحظه: وضعیت سرورها ۲.		
server_id	cpu_utilization_pct	region
srv-us-east-1	45.2	us-east
srv-us-west-2	88.9	us-west
srv-eu-central-1	62.1	eu-central

raw_text_review

User 101: The 4K streaming quality is absolutely amazing, no lag at all!

User 102: Buffering occurred frequently during peak hours. Needs fix.

Figure 1: Data Lake Storage Distribution (Conceptual Volume)

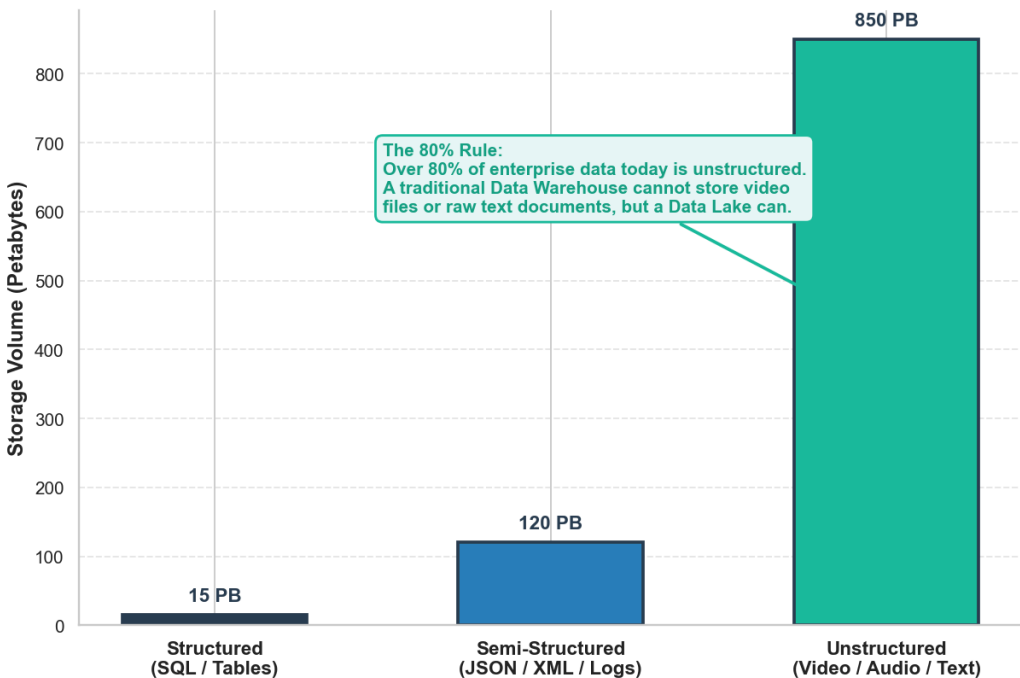
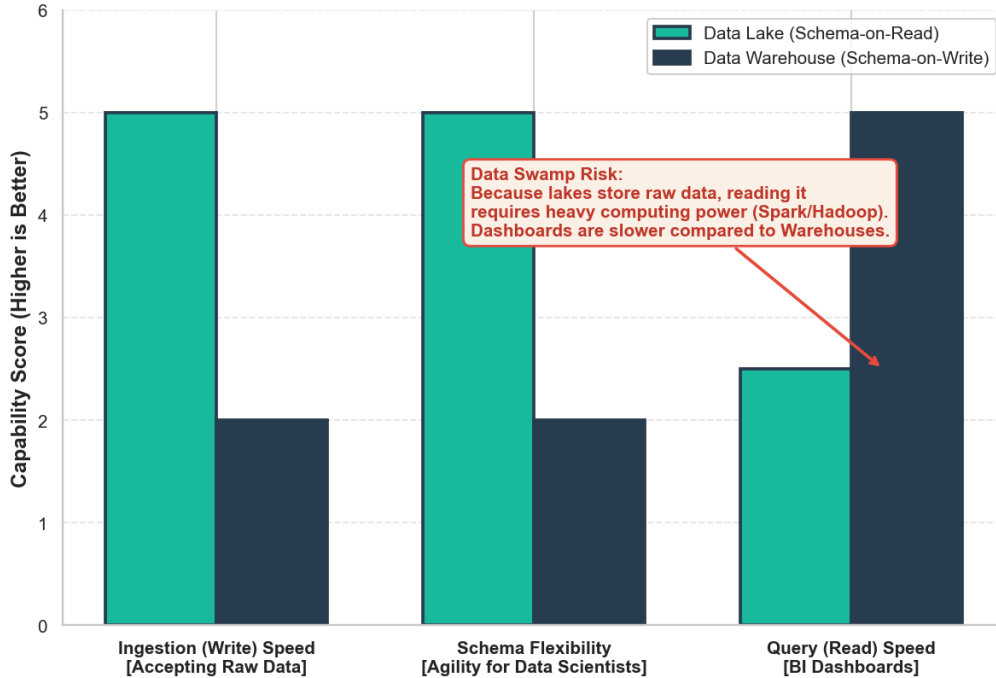


Figure 2: Architecture Capabilities Comparison (Score 1-5)



نتیجه استراتژیک (Strategic Implication):

الگوی Schema-on-Read در دریاچه داده (Data Lake)، شرکت‌هایی نظیر نتفلیکس را از گلوگاه‌های سنگین پیش‌پردازش و ETL نجات می‌دهد. با ذخیره ارزان، خام و بدون قید و شرط هر نوع داده (از جمله فیلم‌ها و لاگ‌های JSON)، نتفلیکس دیگر نیازی ندارد که پیشاپیش بداند قرار است چه سوالی از داده‌ها بپرسد! دانشمندان داده (Data Scientists) می‌توانند سال‌ها بعد، الگوریتم‌های جدید یادگیری ماشین را روی تاریخچه خام و دست‌نخورده نظرات و کلیک‌ها اجرا کنند؛ قابلیت‌هایی که در انبارهای داده سفت‌وسخت (Rigid Data Warehouses) – که داده‌ها را در قالب‌های جبری خلاصه و نابود می‌کنند – عملاً غیرممکن است. با این حال، برای اتصال مستقیم داده‌ها به داشبوردهای مدیریتی سریع (BI)، همچنان به انبار داده (یا ساختارهای ترکیبی مانند Lakehouse) نیاز خواهیم داشت.



توضیح مثال کد پایتون

در این مثال:

- ما ورود ۳ نوع داده کاملاً ناهمگن را به دریاچه داده (Data Lake) نتفلیکس شبیه‌سازی کردیم:
 - داده نیمه‌ساخت‌یافته JSON: لوگ‌های رفتاری کلیک و جستجوی کاربر.
 - داده ساخت‌یافته CSV: معیارهای فنی عملکرد سرورها.
 - داده بدون ساختار متنی: نظرات کیفی کاربران.
- داده‌ها در مرحله ورود (Ingestion) هیچ‌گونه پردازش یا تبدیل سنگینی ندیدند و به‌صورت خام ذخیره شدند.
- تطبیق الگوی Schema-on-Read: با استفاده از کتابخانه‌های json و pandas، ساختار و طرح منطقی داده‌ها تنها در زمان فراخوانی و تحلیل (On-Demand) اعمال شد.
- نتیجه خروجی: این کد نشان می‌دهد چگونه یک دریاچه داده می‌تواند بدون نیاز به یکسان‌سازی قبلی، فرمت‌های مختلف داده را پذیرا باشد و در زمان تحلیل، نماهای ساخت‌یافته (Dataframe Views) جداگانه از آن‌ها استخراج کند.



3.2 Data warehouse modeling: schema and measures

3.2.1 Data cube: a multidimensional data model



مکعب داده

مکعب داده (Data Cube)، ساختاری چندبُعدی برای سازمان‌دهی و تحلیل داده‌ها به شمار می‌رود که عمدتاً در سیستم‌های

تحلیل برخط (OLAP) به‌کار می‌رود. برخلاف جدول‌های دوْبعدی، مکعب داده امکان نمایش هم‌زمان اطلاعات از چند بُعد یا

زاویه را فراهم می‌کند؛ برای نمونه، فروش یک شرکت را می‌توان به‌طور هم‌زمان بر اساس **زمان**، **محصول** و **منطقه** بررسی کرد.

هر بُعد، یک محور تحلیل و هر خانه از مکعب، یک **شاخص (Measure)** را در تلاقی آن ابعاد نشان می‌دهد. شاخص‌ها معمولاً

عددی هستند (مانند میزان فروش یا سود)، اگرچه در برخی کاربردها می‌توانند داده‌های کیفی یا متنی را نیز شامل شوند.

مثال عملی: تحلیل فروش چندبعدی در «وال‌مارت» (Walmart) با مکعب داده

تیم تحلیل هوش تجاری فروشگاه زنجیره‌ای «وال‌مارت» (Walmart) نمی‌تواند فروش را فقط در یک جدول دوبعدی (سطر و ستون ساده)

تحلیل کند. مدیران ارشد نیاز دارند میزان فروش را به‌طور هم‌زمان بر اساس **۳ بعد (Dimension) مختلف** تحلیل کنند:

- بعد اول (Product / محصول):** الکترونیک، پوشاک، لوازم خانگی.
- بعد دوم (Location / شهر):** نیویورک، شیکاگو، لس‌آنجلس.
- بعد سوم (Time / زمان):** سه ماهه اول (Q1)، سه ماهه دوم (Q2)، سه ماهه سوم (Q3).

مفهوم ریاضی مکعب داده (Data Cube):

مکعب داده این ۳ بعد را در یک فضای سه‌بعدی ترکیب می‌کند. هر خانه یا سلول (Cell) در این مکعب، تقاطع یک محصول مشخص، در

یک شهر مشخص و در یک فصل مشخص است که مقدار معیار عددی (Measure) مانند «مبلغ کل فروش» را در خود ذخیره می‌کند:

$$\text{Cell}(d_1, d_2, d_3) = \text{Measure (Total Sales)}$$

مثال عددی خانه مکعب:

$$\text{Cell('Electronics', 'New York', 'Q1')} = 150,000 \text{ USD}$$

نتیجه: مدل مکعب داده به مدیران اجازه می‌دهد تا با یک نگاه، میزان فروش در هر محل، هر زمان و هر گروه کالایی را استخراج کرده و پایه

و اساس تحلیل‌های OLAP را شکل دهند.

خروجی: سازمان‌دهی داده‌های تجاری در ساختار چندبعدی برای تحلیل‌های سریع و جامع

In [6]:

```
display_academic_text(
    " (OLAP Data Cube) با مکعب داده (Walmart) «تحلیل فروش چندبُعدی در «وال‌مارت»",
    "main_title"
)

display_academic_text(
    ". تیم تحلیل هوش تجاری فروشگاه زنجیره‌ای «وال‌مارت» نمی‌تواند فروش را فقط در یک جدول دوْبعدی سنتی (سطر و ستون سادهٔ رابطه‌ای) تحلیل کند."
    "<br>زنند (Slice) مختلف تحلیل و برش (Dimension) بُعد ۳<br>مدیران ارشد نیاز دارند میزان فروش را به‌طور هم‌زمان بر اساس"
    "<br>الکترونیک، پوشاک، لوازم خانگی (Product): بُعد اول<br>"
    "<br>نیویورک، شیکاگو، لس‌آنجلس (Location): بُعد دوم<br>"
    "<br>، (Q3) سه ماهه سوم، (Q2) سه ماهه دوم، (Q1) سه ماهه اول (Time): بُعد سوم<br>.",
    "paragraph"
)

display_academic_text(
    "(Data Cube): مفهوم ریاضی مکعب داده:",
    "sub_title"
)

display_academic_text(
    " است <b>تقاطع یک محصول، یک شهر و یک فصل مشخص</b> در این مکعب، دقیقاً (Cell) مکعب داده، این ۳ بُعد را در یک فضای سه‌بعدی ترکیب می‌کند. هر خانه یا سلول"
    "<br>،مانند «مبلغ کل فروش» را در خود ذخیره می‌کند (Measure) که مقدار معیار عددی",
    "paragraph"
)

display_academic_text(
    r"$Cell(d_1, d_2, d_3) = \text{Measure (Total Sales)}$",
    "paragraph"
)

display_academic_text(
    "مثال عددی یک خانه از مکعب:",
    "sub_title"
)
```

```
display_academic_text(  
    r"$\$Cell(\text{'Electronics'}, \text{'New York'}, \text{'Q1'}) = 150,000 \text{ USD}$",  
    "paragraph"  
)  
  
# =====  
# SECTION 2: DATA CUBE GENERATION AND OLAP PIVOT TRANSFORMATION  
# =====  
display_academic_text(  
    "مرحله نخست: ساخت مکعب داده و اعمال عملیات بُرش"  
    "sub_title"  
)  
  
# Step 1: Create synthetic Walmart 3D Data Cube dataset  
# Dimensions: Product (3), City (3), Quarter (3) -> Total 27 Cells  
products = ['Electronics', 'Apparel', 'Home']  
cities = ['New York', 'Chicago', 'Los Angeles']  
quarters = ['Q1', 'Q2', 'Q3']  
  
# Generate combinations  
cube_records = []  
  
for p_idx, prod in enumerate(products):  
    for c_idx, city in enumerate(cities):  
        for q_idx, qtr in enumerate(quarters):  
  
            # Sales measure in kUSD (random value)  
            sales = np.random.randint(50, 300)  
  
            cube_records.append({  
                'Product': prod,  
                'City': city,  
                'Quarter': qtr,  
                'Prod_Idx': p_idx,  
                'City_Idx': c_idx,  
                'Qtr_Idx': q_idx,  
                'Sales_kUSD': sales,  
            })  
  
df_cube = pd.DataFrame(cube_records)  
  
# Step 2: Create Pivot Table (2D Representation of a Cube Slicing Operation)  
pivot_view = df_cube.pivot_table(  
    index='Product',  
    columns=['City', 'Quarter'],  
    values='Sales_kUSD'  
)  
.fillna(0)  
  
df_cube_disp = df_cube[['Product', 'City', 'Quarter', 'Sales_kUSD']].head(9).reset_index(drop=True)  
  
# Render the initial Flat records vs the 2D Pivot slice  
display_side_by_side_datasets(  
    [df_cube_disp, pivot_view],  
    [  
        "جدول ۱: رکورد سلول‌های مکعب (نمای خطی و رابطه‌ای)",  
        "OLAP Pivot/Slice Operation) جدول ۲: نمای دو بُعدی از بُرش مکعب"  
    ]  
)  
  
# =====  
# SECTION 3: 3D VISUALIZATION (ACADEMIC BLUE)  
# =====  
display_academic_text(  
    "مرحله دوم: مصورسازی تحلیلی - تجسم فضای سه بُعدی و پراکندگی سلول‌ها"  
    "sub_title"  
)  
  
display_academic_text(  
    " نشان‌دهندهٔ ابعاد (شهر، محصول، فصل) بوده <b>محورهای ۳گانه</b> را ترسیم می‌کند (Data Cube) بوم گرافیکی زیر ماهیت واقعی یک مکعب داده "  
    "در آن تقاطع خاص را نمایش می‌دهد. این معماری سه بُعدی (Measure) مقدار متریک فروش ،<b>اندازه و تیرگی رنگ هر حباب</b> و "  
    "بستر اصلی تحلیل‌های پیچیده و داشبوردهای تعاملی مدیران سازمان است."  
    "paragraph"  
)  
  
# Step 3: Visualization using Matplotlib 3D Scatter Plot  
fig = plt.figure(figsize=(13, 9), dpi=135)  
ax = fig.add_subplot(111, projection='3d')  
  
# Scatter points in 3D Space  
# X: Product, Y: City, Z: Quarter, Color & Size: Sales Measure  
scatter = ax.scatter(  
    df_cube['Prod_Idx'],  
    df_cube['City_Idx'],  
    df_cube['Qtr_Idx'],  
    c=df_cube['Sales_kUSD'],  
    s=df_cube['Sales_kUSD'] * 3.5, # Bubble size significantly amplified for visual clarity  
    cmap='Blues', # Academic Blue Palette for uniformity  
    alpha=0.9,  
    edgecolors='#2c3e50', # Navy border for sharp contrast  
    linewidth=1.0  
)  
  
# Set axis tick labels correctly matching categorical indices  
ax.set_xticks(range(len(products)))  
ax.set_xticklabels(products, fontweight='bold', fontsize=11)  
  
ax.set_yticks(range(len(cities)))  
ax.set_yticklabels(cities, fontweight='bold', fontsize=11)  
  
ax.set_zticks(range(len(quarters)))  
ax.set_zticklabels(quarters, fontweight='bold', fontsize=11)
```

```
# Formatting Axis labels and title
ax.set_xlabel('Dimension 1: Product Category', labelpad=15, fontweight='bold', fontsize=12)
ax.set_ylabel('Dimension 2: Geographic City', labelpad=15, fontweight='bold', fontsize=12)
ax.set_zlabel('Dimension 3: Time (Quarter)', labelpad=15, fontweight='bold', fontsize=12)

ax.set_title(
    'Figure 1: Walmart 3D Data Cube Architecture\n(Bubble Size & Darkness = Total Sales Measure in kUSD)',
    fontsize=14,
    fontweight='bold',
    pad=20
)

# Add colorbar to interpret the measure mapping
cbar = fig.colorbar(scatter, ax=ax, shrink=0.5, pad=0.1)
cbar.set_label('Sales Measure Magnitude ($k USD)', fontweight='bold', fontsize=11)

# Adjust viewing angle for best 3D perspective
ax.view_init(elev=20, azim=45)

# Adjust plot boundaries cleanly
fig.subplots_adjust(left=0.0, right=1.0, bottom=0.0, top=1.0)

plt.show()

# =====
# SECTION 4: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک سیستم‌های چندبُعدی</b><br>"
    r"را درهم‌شکسته و با یک نگاه، میزان فروش را (Excel همانند) به مدیران وال‌مارت اجازه می‌دهد تا محدودیت‌های صفحات گستردهٔ دو‌بُعدی <b>(Data Cube)</b> مکعب داده‌<b>مدل</b>"
    r"<br>در تقاطع‌های ۳بُعدی جغرافیایی، زمانی و کالایی ارزیابی کنند (گراف سه‌بُعدی بالا)"
    r"<br>". <b>Slice</b> عملیات بررسی کند، به سادگی یک لایه از این مکعب را برش می‌زند <i>(Q1)</i> فقط فروش وسایل الکترونیک را در فصل اول<i><b>اگر یک مدیر بخواهد</b></i>"
    r"<br>". <b>Dice</b> عملیات تحلیل کند، یک مکعب کوچک‌تر استخراج می‌کند <i>فروش لوازم خانگی و پوشاک را در شیکاگو و نیویورک</i><b>اگر بخواهد</b>"
    r"<br>". را به شکل چشمگیری افزایش می‌دهد BI بوده و سرعت گزارش‌گیری در داشبوردهای <b>(OLAP)</b> پردازش تحلیلی آنلاین<b> این معماری داده، هستهٔ مرکزی</b>"
)
display_academic_text(insight_conclusion, "insight")
```

تحلیل فروش چندبُعدی در «وال‌مارت» (Walmart) با مکعب داده (OLAP Data Cube)

تیم تحلیل هوش تجاری فروشگاه زنجیره‌ای «وال‌مارت» نمی‌تواند فروش را فقط در یک جدول دو‌بُعدی سنتی (سطر و ستون سادهٔ رابطه‌ای) تحلیل کند. مدیران ارشد نیاز دارند میزان فروش را به‌طور هم‌زمان بر اساس ۳ بُعد (Dimension) مختلف تحلیل و برش (Slice) زنند:

بُعد اول (Product): الکترونیک، پوشاک، لوازم خانگی.

بُعد دوم (Location): نیویورک، شیکاگو، لس‌آنجلس.

بُعد سوم (Time): سه ماهه اول (Q1)، سه ماهه دوم (Q2)، سه ماهه سوم (Q3).

مفهوم ریاضی مکعب داده (Data Cube):

مکعب داده، این ۳ بُعد را در یک فضای سه‌بُعدی ترکیب می‌کند. هر خانه یا سلول (Cell) در این مکعب، دقیقاً تقاطع یک محصول، یک شهر و یک فصل مشخص است که مقدار معیار عددی (Measure) مانند «مبلغ کل فروش» را در خود ذخیره می‌کند.

$$\text{Cell}(d_1, d_2, d_3) = \text{Measure}(\text{Total Sales})$$

مثال عددی یک خانه از مکعب:

$$\text{Cell}(\text{'Electronics'}, \text{'New York'}, \text{'Q1'}) = 150,000 \text{ USD}$$

مرحله نخست: ساخت مکعب داده و اعمال عملیات برش (Slice & Dice)

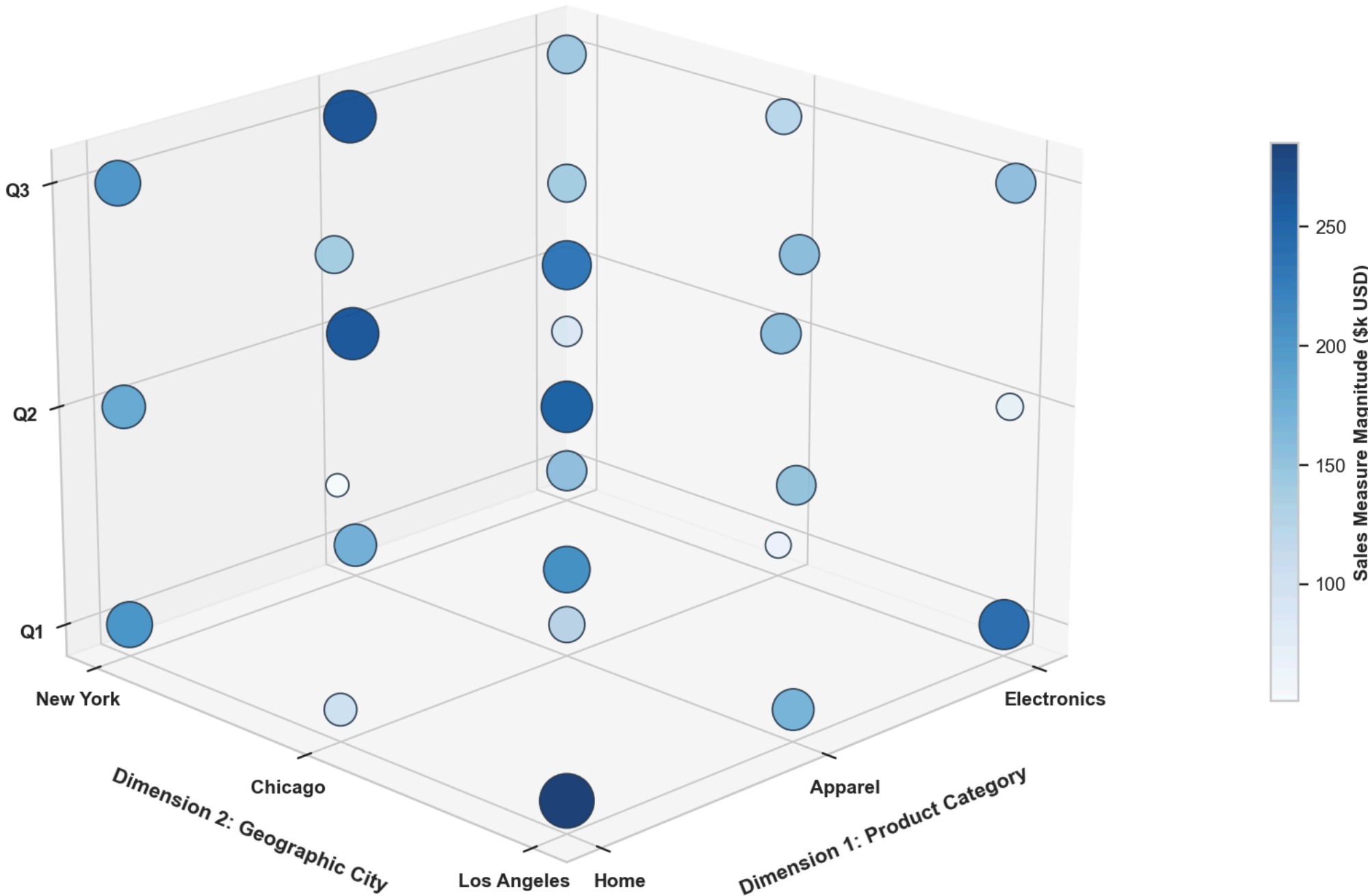
جدول ۱: رکورد سلول‌های مکعب (نمای خطی و رابطه‌ای)			
Product	City	Quarter	Sales_kUSD
Electronics	New York	Q1	152
Electronics	New York	Q2	229
Electronics	New York	Q3	142
Electronics	Chicago	Q1	64
Electronics	Chicago	Q2	156
Electronics	Chicago	Q3	121
Electronics	Los Angeles	Q1	238
Electronics	Los Angeles	Q2	70
Electronics	Los Angeles	Q3	152

جدول ۲: نمای دو بُعدی از بُرشِ مکعب (OLAP Pivot/Slice Operation)							
City	Chicago			Los Angeles			
Quarter	Q1	Q2	Q3	Q1	Q2	Q3	
	124.0	252.0	137.0	166.0	149.0	153.0	17
	64.0	156.0	121.0	238.0	70.0	152.0	15
	102.0	51.0	137.0	285.0	207.0	87.0	20

مرحله دوم: مصورسازی تحلیلی - تجسم فضای سه بُعدی و پراکندگی سلول‌ها

بوم گرافیکی زیر ماهیت واقعی یک مکعب داده (Data Cube) را ترسیم می‌کند. محورهاى ۳گانه نشان‌دهنده ابعاد (شهر، محصول، فصل) بوده و اندازه و تیرگی رنگِ هر حباب، مقدارِ متریک فروش (Measure) در آن تقاطع خاص را نمایش می‌دهد. این معماری سه بُعدی، بستر اصلی تحلیل‌های پیچیده و داشبوردهای تعاملی مدیران سازمان است.

Figure 1: Walmart 3D Data Cube Architecture
(Bubble Size & Darkness = Total Sales Measure in kUSD)



نتیجه استراتژیک سیستم‌های چندبُعدی (Strategic Implication):

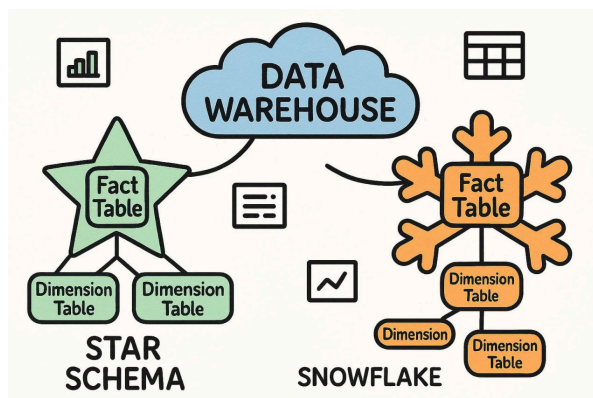
مدل مکعب داده (Data Cube) به مدیران وال‌مارت اجازه می‌دهد تا محدودیت‌های صفحات گستردهٔ دو بُعدی (همانند Excel) را درهم‌شکسته و با یک نگاه، میزان فروش را در تقاطع‌های ۳ بُعدی جغرافیایی، زمانی و کالایی ارزیابی کنند (گراف سه بُعدی بالا). اگر یک مدیر بخواهد فقط فروش وسایل الکترونیک را در فصل اول (Q1) بررسی کند، به سادگی یک لایه از این مکعب را برش می‌زند (عملیات Slice). اگر بخواهد فروش لوازم خانگی و پوشاک را در شیکاگو و نیویورک تحلیل کند، یک مکعب کوچک‌تر استخراج می‌کند (عملیات Dice). این معماری داده، هستهٔ مرکزی پردازش تحلیلی آنلاین (OLAP) بوده و سرعت گزارش‌گیری در داشبوردهای BI را به شکل چشمگیری افزایش می‌دهد.

توضیح مثال کد پایتون

در این مثال:

- ما یک مکعب داده (Data Cube) سه بُعدی برای فروشگاه وال‌مارت شامل ۳ بعد: Product (۳ گروه کالا)، City (۳ شهر) و Quarter (۳ فصل) طراحی کردیم که در مجموع $27 = 3 \times 3 \times 3$ خانه یا سلول تولید می‌کند.
- معیار (Measure) ذخیره‌شده در هر خانه مکعب، میزان فروش به هزار دلار (Sales_kUSD) است.
- با دستور **pivot_table** نشان دادیم چطور می‌توان بخشی از یک مکعب داده سه بُعدی را برش داده و به صورت جدول دوبعدی نمایش داد.
- تفسیر نمودار سه بُعدی (Matplotlib 3D):
 - محورهای افقی و عمودی (X, Y, Z) نشان‌دهنده ۳ بعد مکعب داده هستند.
 - هر نقطه در فضای سه بُعدی نماینده دقیق یک سلول از مکعب است.
 - رنگ و اندازه حباب‌ها (Color & Size): مقدار معیار عددی (میزان فروش) را نشان می‌دهد؛ حباب‌های بزرگ‌تر و روشن‌تر نشان‌دهنده سلول‌های با فروش بالاتر در مکعب داده هستند.

3.2.2 Schemas for multidimensional data models: stars, snowflakes, and fact constellations



ریاضیات – مجموعه، تعداد اعضا و مکعب پایه

برای ساختن یک مکعب داده (Data Cube)، هر بُعد مجموعه‌ای از مقادیر ممکن دارد. ترکیب یک مقدار از هر بُعد، یک سلول (Cell) را مشخص می‌کند. بنابراین اندازهٔ فضای کامل داده با حاصل ضرب تعداد مقادیر ابعاد به دست می‌آید.

تعداد بالقوهٔ سلول‌های مکعب پایه

$$(3.1) \quad |C_{\text{base}}| = \prod_{i=1}^n c_i$$

نمادهای فرمول

- C_{base} : مکعب پایه و فضای سلول‌های بالقوه
- n : تعداد ابعاد مکعب

- C_i : تعداد مقادیر متمایز در بُعد i
- i : شاخص بُعد؛ از ۱ تا n
- $|C_{base}|$: تعداد بالقوه سلول‌های مکعب پایه

✖ مثال ۱ – محاسبه اندازه مکعب پایه

صورت سؤال: سه بُعد «محصول»، «منطقه» و «فصل» به ترتیب ۴، ۳ و ۲ مقدار دارند. حداکثر چند سلول در مکعب پایه وجود دارد؟

$$|C_{base}| = 4 \times 3 \times 2 = 24$$

نتیجه: بنابراین ۲۴ ترکیب بالقوه برای سلول‌های مکعب پایه داریم.

تعداد اعضای حاصل ضرب دکارتی

$$(3.2) \quad |D_1 \times D_2 \times \dots \times D_n| = \prod_{i=1}^n |D_i|$$

نمادهای فرمول

- D_i : مجموعه مقادیر ممکن در بُعد i
- $|D_i|$: تعداد اعضای مجموعه بُعد i
- n : تعداد ابعاد
- $|D_1 \times \dots \times D_n|$: تعداد ترکیب‌های ممکن از مقادیر همه ابعاد

✖ مثال ۲ – حاصل ضرب دکارتی

صورت سؤال: اگر مجموعه محصولات ۲ عضو و مجموعه مناطق ۳ عضو داشته باشد، چند زوج محصول-منطقه می‌توان ساخت؟

$$|D_{product} \times D_{region}| = 2 \times 3 = 6$$

نتیجه: پس ۶ ترکیب محصول-منطقه وجود دارد.

شمای چندبعدی در انبار داده

شمای چندبعدی، روشی برای سازمان‌دهی داده‌ها در انبار داده است که امکان بررسی داده‌ها را از چند بُعد مختلف فراهم می‌کند؛ همان منطقی که در ساختار مکعب داده نیز دیده می‌شود. به‌جای آنکه اطلاعات را فقط در قالب یک جدول ساده ببینیم، این رویکرد امکان تحلیل داده‌ها را از زوایای گوناگون فراهم می‌آورد. سه نوع اصلی شمای چندبعدی عبارت‌اند از: **شمای ستاره‌ای**، **شمای دانه‌برفی** و **شمای صورت فلکی**.

۱. شمای ستاره‌ای (Star Schema)

شمای ستاره‌ای ساده‌ترین و پرکاربردترین نوع شمای چندبعدی است. در این مدل، یک **جدول واقعیت (Fact Table)** در مرکز قرار دارد و چند **جدول بُعد (Dimension Table)** به‌طور مستقیم به آن متصل می‌شوند؛ شکل حاصل شبیه یک ستاره است.

این ساختار ساده و قابل فهم، به دلیل تعداد **اتصال (Join)** کمتر بین جداول، سرعت پرس و جو را نیز افزایش می دهد و به همین دلیل برای بیشتر سیستم های انبار داده گزینه مناسبی به شمار می رود. برای نمونه، در یک انبار داده فروش، جدول واقعیت «فروش» است و جداول بُعد شامل «زمان»، «محصول»، «مشتري» و «فروشگاه» هستند.

۲. شمای دانه برفی (Snowflake Schema)

شمای دانه برفی نسخه توسعه یافته شمای ستاره ای است. در این مدل، جداول بُعد به چند جدول کوچکتر تقسیم یا **نرمال سازی** می شوند و ساختار حاصل شبیه دانه برف است. برای نمونه، بُعد «محصول» می تواند به جداول «محصول»، «دسته بندی محصول» و «تولیدکننده» تقسیم شود؛ یا بُعد «مشتري» به جداول «مشتري»، «شهر» و «استان» نرمال سازی گردد. این تفکیک، **افزونگی داده و فضای ذخیره سازی** مورد نیاز را کاهش می دهد، اما در مقابل، به دلیل افزایش تعداد اتصال بین جداول، طراحی و پردازش پرس و جو را نسبت به شمای ستاره ای پیچیده تر می کند.

۳. شمای صورت فلکی (Fact Constellation یا Galaxy Schema)

شمای صورت فلکی شامل چند **جدول واقعیت** است که **جداول بُعد مشترکی** میان خود دارند. این ساختار، که پیچیده ترین نوع شمای چندبندی به شمار می رود، برای سازمان های بزرگ و تحلیل های گسترده و چندمنظوره که نیازمند بررسی همزمان چند فرآیند کسب و کار (مانند فروش و موجودی انبار) در کنار یکدیگرند مناسب است.

در مجموع، انتخاب نوع شما بستگی به نیاز سازمان دارد: شمای ستاره ای برای سادگی و سرعت، شمای دانه برفی برای کاهش افزونگی داده، و شمای صورت فلکی برای تحلیل های بزرگ مقیاس با چند جدول واقعیت به کار می رود.

مقایسه سه شمای چندبندی؛ کدام را انتخاب کنیم؟

★ **شمای ستاره ای:** ساده ترین و پرکاربردترین گزینه. یک جدول واقعیت در مرکز و جداول بُعد مستقیماً به آن متصل می شوند.

❄️ **شمای دانه برفی:** نسخه نرمال شده ستاره ای. جداول بُعد به چند جدول کوچکتر تقسیم می شوند تا افزونگی کاهش یابد.

🌌 **شمای صورت فلکی:** پیشرفته ترین گزینه. شامل چند جدول واقعیت با جداول بُعد مشترک برای تحلیل های پیچیده و بزرگ مقیاس.

✅ **نکته کلیدی:** اگر سادگی و سرعت اولویت شماست، **ستاره ای** را انتخاب کنید. اگر کاهش افزونگی داده مهم است، **دانه برفی** گزینه مناسبتری است و اگر چندین فرآیند کسب و کار را همزمان تحلیل می کنید، **صورت فلکی** پاسخگوی نیاز شما خواهد بود.

مثال عملی: طراحی شماهای انبار داده در «آمازون» (Amazon Data Warehouse)

تیم مهندسی داده «آمازون» (Amazon) برای ذخیره سازی داده های تحلیل هوش تجاری (BI)، جداول را به دو دسته اصلی تقسیم می کند: جداول واقعیت (Fact Tables) که حاوی اعداد و معیارهای قابل جمع بستن (مثل مبلغ فروش و هزینه پستی) هستند، و جداول ابعاد (Dimension Tables) که متن ها و ویژگی های توصیفی (مثل نام مشتری و دسته بندی کالا) را نگه می دارند.

۱. شمای ستاره ای (Star Schema) ✨ – ساده ترین و سریع ترین ساختار:

یک جدول واقعیت مرکزی (Fact_Sales) مستقیماً به جداول ابعاد غیرنرمال شده (Dim_Customer, Dim_Product, Dim_Time) وصل است. این ساختار شبیه ستاره است، سرعت کوئری گیری بالایی دارد و نیاز به JOIN های پیچیده ندارد.

۲. شمای برف دانه (Snowflake Schema) ❄️ – ساختار نرمال شده بدون همبستگی تکراری:

در این شما، جداول ابعاد نرمال سازی می شوند. برای مثال جدول Dim_Product به زیرابعاد Dim_Subcategory و Dim_Category شکسته می شود. فضای ذخیره سازی کمتری مصرف می شود اما کوئری ها نیازمند JOIN های متوالی هستند.

۳. شمای صورت فلکی (Fact Constellation Schema) 🌟 – چند جدول واقعیت با ابعاد مشترک:

در سازمان‌های بزرگ مثل آمازون، چندین جدول واقعیت وجود دارد (مثلاً **Fact_Sales** برای فروش و **Fact_Shipping** برای هزینه‌های ارسال). این جداول واقعیت از ابعاد هم‌شکل و مشترک (Conformed Dimensions) مثل **Dim_Time** و **Dim_Customer** استفاده می‌کنند تا امکان تحلیل‌های متقاطع فراهم شود.

نتیجه: انتخاب شما بستگی به اولویت سیستم دارد؛ شمای ستاره‌ای برای سرعت کوئری، برف‌دانه برای بهینه‌سازی حافظه، و صورت فلکی برای پوشش کل فرآیندهای تجاری سازمان به‌کار می‌رود.

خروجی: سازمان‌دهی منطقی جداول واقعیت و ابعاد برای گزارش‌گیری‌های پیچیده OLAP

```
In [7]: display_academic_text("۱. پیاده‌سازی شمای ستاره‌ای (Star Schema)", "sub_title")

# Dimension Tables (Flat & Denormalized)
dim_customer = pd.DataFrame({
    'Customer_Key': [1, 2],
    'Customer_Name': ['Alice', 'Bob'],
    'City': ['Seattle', 'New York'],
})

dim_product_star = pd.DataFrame({
    'Product_Key': [101, 102],
    'Product_Name': ['Echo Dot', 'Kindle Paperwhite'],
    'Category': ['Electronics', 'Electronics'], # Denormalized (Repeated strings)
})

# Central Fact Table
fact_sales = pd.DataFrame({
    'Sales_ID': [10001, 10002],
    'Customer_Key': [1, 2],
    'Product_Key': [101, 102],
    'Time_Key': [20261001, 20261002],
    'Quantity_Sold': [2, 1],
    'Revenue_USD': [99.98, 139.99],
})

# Perform Single-level Star Schema Joins
star_query = fact_sales.merge(dim_customer, on='Customer_Key').merge(dim_product_star, on='Product_Key')

display_side_by_side_datasets(
    [star_query[['Sales_ID', 'Customer_Name', 'Product_Name', 'Category', 'Revenue_USD']].reset_index(drop=True)],
    ["متصل به ابعاد مسطح (سریع‌ترین نوع جستجو) Fact یک (Star) خروجی شمای ستاره‌ای"]
)

# -----
# 2. SIMULATING SNOWFLAKE SCHEMA ❄️
# -----

display_academic_text("۲. پیاده‌سازی شمای برف‌دانه (Snowflake Schema)", "sub_title")

# Normalized Dimensions (Sub-dimensions hierarchy to save storage)
dim_category_snow = pd.DataFrame({
    'Category_Key': [50],
    'Category_Name': ['Electronics']
})

dim_product_snow = pd.DataFrame({
    'Product_Key': [101, 102],
    'Product_Name': ['Echo Dot', 'Kindle Paperwhite'],
    'Category_Key': [50, 50], # Foreign Key pointing to dim_category_snow instead of repeating strings
})

# Perform Multi-level Snowflake Schema Joins
snowflake_query = (
    fact_sales
    .merge(dim_product_snow, on='Product_Key')
    .merge(dim_category_snow, on='Category_Key') # Additional Hop (Extra JOIN)
    .merge(dim_customer, on='Customer_Key')
)

display_side_by_side_datasets(
    [snowflake_query[['Sales_ID', 'Customer_Name', 'Product_Name', 'Category_Name', 'Revenue_USD']].reset_index(drop=True)],
    ["های متوالی به‌خاطر نرمال‌سازی ابعادJOIN نیازمند (Snowflake) خروجی شمای برف‌دانه"]
)

# -----
# 3. SIMULATING FACT CONSTELLATION SCHEMA 🌟
# -----

display_academic_text("۳. پیاده‌سازی شمای صورت فلکی (Fact Constellation Schema)", "sub_title")

# Second Fact Table (Sharing Conformed Dimensions: Customer and Time)
fact_shipping = pd.DataFrame({
    'Shipping_ID': [9001, 9002],
    'Customer_Key': [1, 2],
    'Time_Key': [20261001, 20261002],
    'Shipping_Cost_USD': [5.50, 12.00],
    'Delivery_Days': [2, 1],
})

# Cross-Fact Analysis (Comparing Sales Revenue vs Shipping Costs per Customer)
```

```
sales_summary = fact_sales.groupby('Customer_Key')['Revenue_USD'].sum()
shipping_summary = fact_shipping.groupby('Customer_Key')['Shipping_Cost_USD'].sum()

fact_constellation_summary = pd.DataFrame({
    'Total_Revenue_USD': sales_summary,
    'Total_Shipping_Cost_USD': shipping_summary,
}).reset_index()

# Merge summaries with Conformed Dimension
constellation_query = fact_constellation_summary.merge(dim_customer, on='Customer_Key')

display_side_by_side_datasets(
    [constellation_query[['Customer_Name', 'City', 'Total_Revenue_USD', 'Total_Shipping_Cost_USD']].reset_index(drop=True)],
    ["بر روی بُعد مشترک مشتری (فروش و ارسال) Fact تحلیل متقاطع دو جدول (Constellation): خروجی صورت فلکی "]
)

# =====
# SECTION 3: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
display_academic_text(
    "در معماری‌های مختلف (Trade-offs) مرحله دوم: مصورسازی تحلیلی - موازنه",
    "sub_title"
)

display_academic_text(
    "را ارائه می‌دهد اما فضای بیشتری اشغال می‌کند (های کمتر JOIN به دلیل) بالاترین سرعت کوئری <b>شمای ستاره‌ای</b>. بوم گرافیکی زیر نشان می‌دهد که در طراحی پایگاه داده، هیچ «بهترین مطلق» وجود ندارد.",
    "Cross-Process) نیز پادشاه تحلیل‌های پیچیده و فراسازمانی <b>صورت فلکی</b>. با نرمال‌سازی دقیق، در مصرف هارد دیسک صرفه‌جویی می‌کند اما سرعت استخراج را قربانی می‌کند <b>شمای برف‌دانه</b>.",
    "paragraph"
)

# Data for visualization (Conceptual scoring out of 10)
schemas = ['Star Schema\\n(Denormalized)', 'Snowflake Schema\\n(Normalized)', 'Fact Constellation\\n(Multi-Fact)']
metrics = {
    'Query Execution Speed (Fewer JOINS)': [9.5, 4.0, 7.0],
    'Storage Efficiency (No Redundancy)': [4.0, 9.5, 6.0],
    'Analytical Depth (Cross-Process)': [4.0, 4.0, 9.5]
}
df_metrics = pd.DataFrame(metrics, index=schemas).reset_index().rename(columns={'index': 'Architecture'})

# Melt for seaborn grouped bar chart
df_melted = df_metrics.melt(id_vars='Architecture', var_name='Metric', value_name='Score')

# Initialize canvas
plt.figure(figsize=(15, 6.5), dpi=135)

# Grouped bar chart with Academic Blue palette
sns.barplot(
    data=df_melted,
    x='Architecture',
    y='Score',
    hue='Metric',
    palette=['#1abc9c', '#2980b9', '#2c3e50'], # Turquoise, Classic Blue, Navy
    edgecolor='#2c3e50',
    linewidth=1.5
)

# Intelligent Annotations
plt.annotate(
    "Speed vs Storage:\\nStar Schema is extremely fast\\nbecause it avoids cascading JOINS,\\nbut it wastes storage space.",
    xy=(0.0, 9.5), xytext=(-0.4, 6.5),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

plt.annotate(
    "Enterprise Analytics:\\nConstellation is the only schema that\\nallows comparing multiple business\\nprocesses (e.g., Sales vs Shipping).",
    xy=(2.2, 9.5), xytext=(1.8, 6.5),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

# Formatting
plt.title('Figure 1: Architectural Trade-offs in Data Warehouse Modeling', fontdict={'fontsize': 15, 'fontweight': 'bold'}, pad=15)
plt.ylabel('Capability Score (0 to 10)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
plt.xlabel('Data Warehouse Schema Topology', fontdict={'fontsize': 13, 'fontweight': 'bold'})
plt.xticks(fontweight='bold', fontsize=12)
plt.ylim(0, 11)
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Adjust Legend
plt.legend(title='Evaluation Metric', bbox_to_anchor=(1.01, 1), loc='upper left', frameon=True, fontsize=11, title_fontproperties={'weight': 'bold'})

# Clear top and right spines
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# SECTION 4: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه کاربردی (Strategic Implication):</b><br>"
    r"<br>استراتژیک است (Trade-off) انتخاب معماری پایگاه داده، یک موازنه"
    r" است (BI) ها، سریع‌ترین گزینه برای تغذیهٔ داشبوردهای مدیریتی JOIN به دلیل سادگی مدل و کاهش بار پردازشی <b>شمای ستاره‌ای (Star)</b>"
    r" با ایزوله‌سازی جداول، فضای ذخیره‌سازی را بهینه‌تر مصرف می‌کند و از خطای افزونگی داده جلوگیری می‌نماید <b>شمای برف‌دانه (Snowflake)</b> درحالی‌که"
    r" برای پوشش ۳۶۰ درجه کلِ فرآیندهای تجاری سازمان (مانند بررسی سودآوری با کسر هزینه‌های پستی از مبلغ فروش) <b>شمای صورت فلکی (Fact Constellation)</b>، در نهایت"
    r" در پلتفرم‌های کلانی نظیر آمازون به‌کار می‌رود."
)

display_academic_text(insight_conclusion, "insight")
```

۱. پیاده‌سازی شمای ستاره‌ای (Star Schema)

خروجی شمای ستاره‌ای (Star): یک Fact متصل به ابعاد مسطح (سریع‌ترین نوع جستجو)

Sales_ID	Customer_Name	Product_Name	Category	Revenue_USD
10001	Alice	Echo Dot	Electronics	99.98
10002	Bob	Kindle Paperwhite	Electronics	139.99

۲. پیاده‌سازی شمای برف‌دانه (Snowflake Schema)

خروجی شمای برف‌دانه (Snowflake): نیازمند JOIN‌های متوالی به‌خاطر نرمال‌سازی ابعاد

Sales_ID	Customer_Name	Product_Name	Category_Name	Revenue_USD
10001	Alice	Echo Dot	Electronics	99.98
10002	Bob	Kindle Paperwhite	Electronics	139.99

۳. پیاده‌سازی شمای صورت فلکی (Fact Constellation Schema)

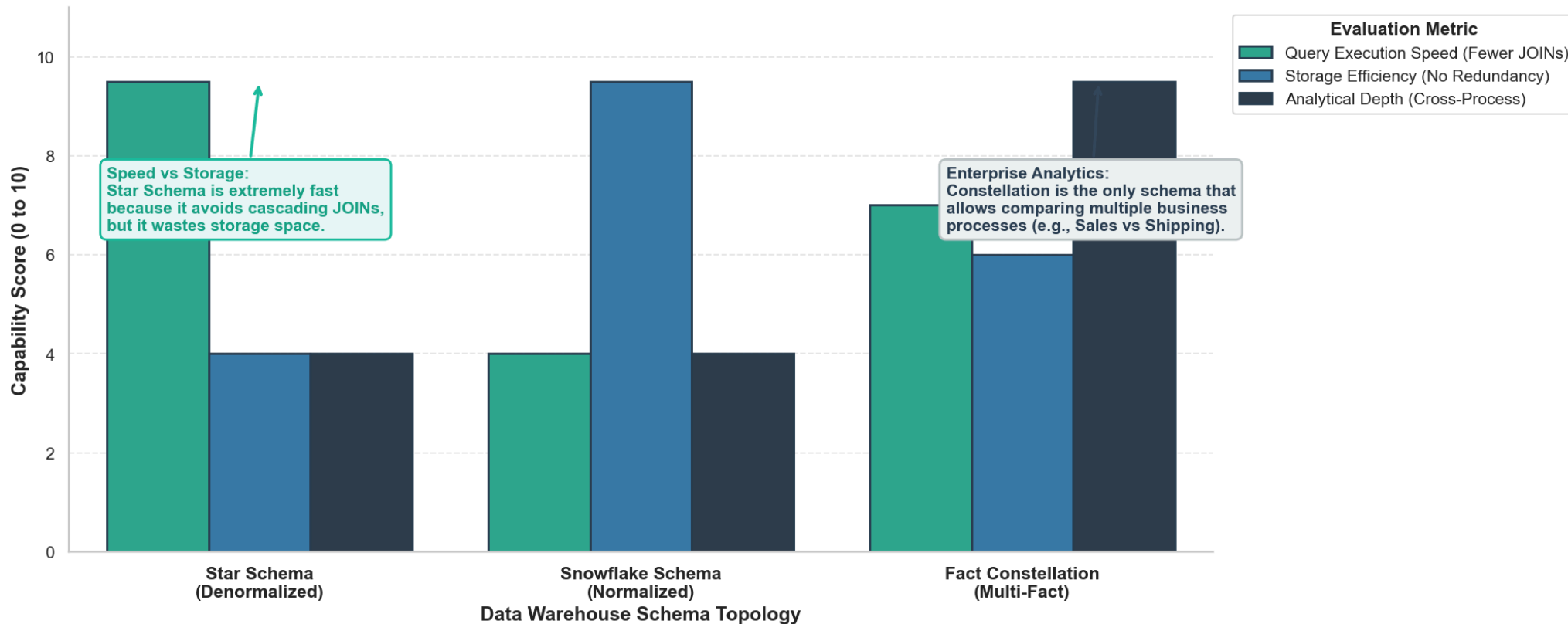
خروجی صورت فلکی (Constellation): تحلیل متقاطع دو جدول Fact (فروش و ارسال) بر روی بُعد مشترک مشتری

Customer_Name	City	Total_Revenue_USD	Total_Shipping_Cost_USD
Alice	Seattle	99.98	5.5
Bob	New York	139.99	12.0

مرحله دوم: مصورسازی تحلیلی - موازنه (Trade-offs) در معماری‌های مختلف

بوم گرافیکی زیر نشان می‌دهد که در طراحی پایگاه داده، هیچ «بهترین مطلق» وجود ندارد. **شمای ستاره‌ای** بالاترین سرعت کوئری (به دلیل JOIN‌های کمتر) را ارائه می‌دهد اما فضای بیشتری اشغال می‌کند. **شمای برف‌دانه** با نرمال‌سازی دقیق، در مصرف هارد دیسک صرفه‌جویی می‌کند اما سرعت استخراج را قربانی می‌کند. **صورت فلکی** نیز پادشاه تحلیل‌های پیچیده و فراسازمانی (Cross-Process) است.

Figure 1: Architectural Trade-offs in Data Warehouse Modeling



نتیجه کاربردی (Strategic Implication):

انتخاب معماری پایگاه داده، یک موازنه (Trade-off) استراتژیک است.

شمای ستاره‌ای (Star) به دلیل سادگی مدل و کاهش بار پردازشی JOINها، سریع‌ترین گزینه برای تغذیه داشبوردهای مدیریتی (BI) است. درحالی‌که شمای برف‌دانه (Snowflake) با ایزوله‌سازی جداول، فضای ذخیره‌سازی را بهینه‌تر مصرف می‌کند و از خطای افزونگی داده جلوگیری می‌نماید. در نهایت، شمای صورت فلکی (Fact Constellation) برای پوشش ۳۶۰ درجه کل فرآیندهای تجاری سازمان (مانند بررسی سودآوری با کسر هزینه‌های پستی از مبلغ فروش) در پلتفرم‌های کلانی نظیر آمازون به‌کار می‌رود.

توضیح مثال کد پایتون

در این مثال:

- ما با استفاده از دیتافریم‌های کتابخانه **pandas**، جداول واقعیت (Fact) و ابعاد (Dimension) انبار داده آمازون را در ۳ شما شبیه‌سازی کردیم:
- ۱. شمای ستاره‌ای (Star Schema): جدول مرکزی **fact_sales** شامل کلیدهای خارجی و معیارهای عددی (فروش و تعداد) است که با یک مرحله دستور **merge** () مستقیماً به جداول غیرنرمال‌شده **dim_customer** و **dim_product_star** متصل شد.
- ۲. شمای برف‌دانه (Snowflake Schema): بعد محصول نرمال‌سازی شد؛ جدول **dim_product_snow** با کلید خارجی **Category_Key** به جدول سطح بالاتر **dim_category_snow** لینک گردید و کوئری نهایی با دو سطح ادغام (JOIN) اجرا شد.
- ۳. شمای صورت فلکی (Fact Constellation): یک جدول واقعیت دوم به نام **fact_shipping** (مربوط به هزینه و زمان ارسال) ایجاد شد. این دو جدول واقعیت از ابعاد مشترک (Conformed Dimensions) شامل **Customer_Key** و **Time_Key** استفاده کردند.
- نتیجه خروجی: کد نشان داد که چطور در شمای صورت فلکی می‌توان تحلیل‌های متقاطع انجام داد و کل درآمد فروش هر مشتری را در کنار هزینه‌های ارسال او ارزیابی کرد.

3.2.3 Concept hierarchies

سلسله‌مراتب مفاهیم

سلسله‌مراتب مفاهیم (Concept Hierarchies) روشی برای سازمان‌دهی داده‌ها در سطوح مختلف انتزاع یا گرانولاریته

(Granularity) است. در این روش، مقادیر جزئی و ریز، مرحله‌به‌مرحله به مفاهیمی کلی‌تر نگاشت داده می‌شوند و زنجیره‌ای از مفاهیم را شکل می‌دهند.

سلسله‌مراتب مفاهیم بسته به نحوه تعریف و پیاده‌سازی، انواع گوناگونی دارد: سلسله‌مراتب شمایی (Schema Hierarchy)

که رابطه کلی-جزئی آن مستقیماً در طراحی پایگاه داده تعریف می‌شود (همانند مثال زمان: روز ← ماه ← فصل ← سال، یا

مکان: شهر ← استان ← کشور؛ **سلسله‌مراتب مجموعه‌ای (Set-Grouping Hierarchy)** که در آن مقادیر بر اساس قاعده‌ای دلخواه و نه ساختاری از پیش‌موجود، در گروه‌های سفارشی دسته‌بندی می‌شوند (مانند تقسیم سن به «نوجوان»، «جوان» و «میان‌سال»); **سلسله‌مراتب مشتق از عملیات (Operation-Derived Hierarchy)** که سطوح بالاتر در آن نه با یک نگاشت ثابت، بلکه با یک تابع یا فرمول محاسبه می‌شوند (مانند استخراج «شهر» از رشته آدرس); و در نهایت **سلسله‌مراتب قاعده‌محور (Rule-Based Hierarchy)** که در آن رابطه بین سطوح با یک قاعده منطقی یا شرطی تعریف می‌شود (برای نمونه، «اگر فروش < ۱۰۰ میلیون ← دسته = پرفروش»).

در **داده‌کاوی**، این سلسله‌مراتب اغلب روی **ابعاد (Dimensions)** مدل‌های چندبعدی تعریف می‌شود. هر سطح از این سلسله‌مراتب، درجه متفاوتی از خلاصه‌سازی یا جزئیات را نشان می‌دهد و به کاربر امکان می‌دهد داده‌ها را در سطوح مختلفی از جزئیات کاوش کند.

همین ساختار است که دو عملیات کلیدی در **مکعب داده** را ممکن می‌سازد: **Roll-up** (حرکت به سمت سطوح بالاتر انتزاع، مثلاً از فروش روزانه به فروش ماهانه) و **Drill-down** (حرکت به سمت سطوح جزئی‌تر، مثلاً از فروش سالانه به فروش فصلی). بدون وجود یک سلسله‌مراتب مشخص برای هر بُعد، این دو عملیات معنایی نخواهند داشت.

سلسله‌مراتب می‌تواند **کامل (Balanced)** باشد، یعنی همه شاخه‌ها تعداد سطوح یکسانی داشته باشند، یا **ناکامل (Unbalanced)** که در آن برخی شاخه‌ها عمق متفاوتی دارند.

مثال عملی: تحلیل فروش در سطوح مختلف زمانی در «والمارت» (Walmart)

مدیران «والمارت» (Walmart) نیاز دارند گزارش‌های فروش را در سطوح مختلف جزئیات بررسی کنند. مثلاً مدیرعامل می‌خواهد فروش کل سالانه را ببیند، اما مدیر بازاریابی نیاز دارد فروش ماه به ماه را تحلیل کند. این سطوح تودرتوی جزئیات را **سلسله مراتب مفاهیم (Concept Hierarchy)** می‌نامند.

مثال عملی: سلسله مراتب بعد زمان (Time Dimension Hierarchy):

داده‌های فروش روزانه در پایین‌ترین سطح جزئیات ثبت شده‌اند، اما می‌توان آن‌ها را در سطوح انتزاعی‌تری خلاصه کرد:

Day → Month → Quarter → Year

- سطح ۱ (پایین‌ترین / Day): فروش هر روز مشخص (مثلاً ۲۰۲۳-۰۱-۰۵).
- سطح ۲ (Month): جمع فروش یک ماه (مثلاً January 2023).
- سطح ۳ (Quarter): جمع فروش یک فصل (مثلاً Q1-2023).
- سطح ۴ (بالاترین / Year): جمع فروش کل سال (مثلاً 2023).

ارتباط سلسله مراتب با عملیات OLAP:

- Roll-Up (جمع‌بندی): حرکت از سطح جزئی‌تر به سطح خلاصه‌تر (مثلاً از ماه به فصل).
- Drill-Down (ریزننگری): حرکت معکوس از سطح خلاصه به سطح جزئی‌تر (مثلاً از سال به ماه).

نتیجه: سلسله مراتب مفاهیم امکان ناوبری چندسطحی در داده‌ها را فراهم می‌سازد تا هر مدیر در سطح مورد نیاز خود، گزارش دقیق و معناداری دریافت کند.

خروجی: تجمیع تدریجی داده‌ها از سطح روزانه تا سالانه بر اساس سلسله مراتب بعد زمان

```
In [8]: display_academic_text(
    "Concept Hierarchy (Concept Hierarchy)",
    "main_title"
)

display_academic_text(
    "نیاز دارند گزارش‌های فروش را در سطوح مختلف جزئیات بررسی کنند. مثلاً مدیرعامل می‌خواهد فروش کل سالانه را ببیند (Walmart) «مدیران «والمارت» می‌نامند (Concept Hierarchy) سلسله‌مراتب مفاهیم»<b>، اما مدیر بازاریابی نیاز دارد فروش ماه به ماه را تحلیل کند. این سطوح تودرتوی جزئیات را در انبارهای داده"،</b>
    "paragraph"
)

display_academic_text("Time Dimension Formal Hierarchy):", "sub_title")

display_academic_text(
    r"$$\text{(Hierarchy)_{Time}} = \{ \text{Day} \rightarrow \text{Roll-Up} \} \text{Month} \rightarrow \text{Roll-Up} \} \text{Quarter} \rightarrow \text{Roll-Up} \} \text{Year} \}$$",
    "paragraph"
)

display_academic_text(
    "ارتباط سلسله‌مراتب با عملیات OLAP:",
    "sub_title"
)

display_academic_text(
    "<b>حرکت از سطح جزئی‌تر به سطح خلاصه‌تر (مثلاً از ماه به فصل). باعث کاهش ابعاد و نویز می‌شود (Roll-Up):</b> جمع‌بندی •<br>
    <b>حرکت معکوس از سطح خلاصه به سطح جزئی‌تر (مثلاً از سال به ماه). باعث کشف الگوهای پنهان در داده‌های خرد می‌شود (Drill-Down):</b> ریزننگری •<br>
    "paragraph"
)

# =====
# SECTION 2: DATA GENERATION & HIERARCHICAL AGGREGATION
# =====

display_academic_text(
    "Roll-Up مرحله نخست: تولید داده‌های روزانه و اجرای عملیات",
    "sub_title"
)

# Step 1: Create daily-level sales data (Lowest granularity)
dates = pd.date_range(start='2026-01-01', end='2026-12-31', freq='D')
daily_sales = np.random.uniform(50, 300, size=len(dates))

df = pd.DataFrame({'Date': dates, 'Daily_Sales_kUSD': daily_sales})

# Step 2: Build Concept Hierarchy Levels from the Time Dimension

# Level 2: Month (Roll-up from Day)
df['Month'] = df['Date'].dt.to_period('M')
```

```
monthly_sales = df.groupby('Month')['Daily_Sales_kUSD'].sum().reset_index()
monthly_sales.columns = ['Month', 'Monthly_Sales_kUSD']

df_day_disp = df[['Date', 'Daily_Sales_kUSD']].head(6).reset_index(drop=True)
df_month_disp = monthly_sales.head(6).reset_index(drop=True)

# Display Level 1 and 2
display_side_by_side_datasets(
    [df_day_disp, df_month_disp],
    [
        "سطح ۱: فروش روزانه (پایین‌ترین سطح جزئیات)",
        "(از روزانه Roll-Up) سطح ۲: فروش ماهانه"
    ]
)

# Level 3: Quarter (Roll-up from Month/Day)
df['Quarter'] = df['Date'].dt.to_period('Q')
quarterly_sales = df.groupby('Quarter')['Daily_Sales_kUSD'].sum().reset_index()
quarterly_sales.columns = ['Quarter', 'Quarterly_Sales_kUSD']

# Level 4: Year (Highest Granularity)
df['Year'] = df['Date'].dt.to_period('Y')
yearly_sales = df.groupby('Year')['Daily_Sales_kUSD'].sum().reset_index()
yearly_sales.columns = ['Year', 'Yearly_Sales_kUSD']

df_qtr_disp = quarterly_sales.reset_index(drop=True)
df_yr_disp = yearly_sales.reset_index(drop=True)

# Display Level 3 and 4
display_side_by_side_datasets(
    [df_qtr_disp, df_yr_disp],
    [
        "(از ماهانه Roll-Up) سطح ۳: فروش فصلی",
        "(بالاترین سطح تجميع) سطح ۴: فروش سالانه"
    ]
)

# =====
# =====
# SECTION 3: VISUAL ANALYTICS (ACADEMIC BLUE) - CORRECTED
# =====
display_academic_text(
    "مرحله دوم: مصورسازی تحلیلی - تکامل هندسی با حرکت در سلسله‌مراتب",
    "sub_title"
)

display_academic_text(
    "، را به صورت بصری اثبات می‌کند. با حرکت از سمت چپ به راست و از بالا به پایین <b>Roll-Up</b> بوم گرافیکی زیر تأثیر عملیات "
    "نوسانات شدید و نویزهای روزانه (نمودار اول) در یکدیگر ادغام شده و الگوهای کلان (نمودارهای فصل و سال) پدیدار می‌گردند "
    "</i>، رنگ‌بندی نمودارها با افزایش سطح تجميع، از فیروزه‌ای به سرمه‌ای تیره تغییر می‌کند تا سنگینی داده‌های تجميع‌شده را تداعی نماید</>"
    "paragraph"
)

# Step 3: Visualization of Concept Hierarchy Levels
fig, axes = plt.subplots(2, 2, figsize=(19, 11), dpi=135)

# -----
# Subplot 1: Daily Sales (Level 1 - Most Detailed)
# -----
axes[0, 0].plot(
    df['Date'],
    df['Daily_Sales_kUSD'],
    color='#1abc9c', # Turquoise
    alpha=0.9,
    linewidth=1.5
)

axes[0, 0].annotate(
    "High Variance (Noise):\nAt the daily level, sales fluctuate wildly.\nIt's hard to see the big picture.",
    xy=(df['Date'].iloc[60], 280), xytext=(df['Date'].iloc[100], 310),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0, 0].set_title('Figure 1A: Level 1 - Daily Sales (Finest Granularity)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=15)
axes[0, 0].set_ylabel('Sales Volume (kUSD)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[0, 0].grid(True, linestyle='--', alpha=0.5)
axes[0, 0].tick_params(axis='x', rotation=15, labelsize=11)
axes[0, 0].set_xticklabels(axes[0, 0].get_xticklabels(), fontweight='bold') # FIX: Bold formatting

# -----
# Subplot 2: Monthly Sales (Level 2 - Roll-Up)
# -----
axes[0, 1].bar(
    monthly_sales['Month'].astype(str),
    monthly_sales['Monthly_Sales_kUSD'],
    color='#5DADE2', # Light/Medium Blue
    edgecolor='#2c3e50',
    linewidth=1.5
)

axes[0, 1].set_title('Figure 1B: Level 2 - Monthly Sales (Roll-Up from Day)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=15)
axes[0, 1].set_ylabel('Aggregated Sales (kUSD)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[0, 1].tick_params(axis='x', rotation=45, labelsize=11)
axes[0, 1].set_xticklabels(axes[0, 1].get_xticklabels(), fontweight='bold') # FIX: Bold formatting
axes[0, 1].grid(axis='y', linestyle='--', alpha=0.5)

# -----
# Subplot 3: Quarterly Sales (Level 3 - Roll-Up)
# -----
axes[1, 0].bar(
    quarterly_sales['Quarter'].astype(str),
```

```
quarterly_sales['Quarterly_Sales_kUSD'],
color='#2980b9', # Classic Blue
edgecolor='#2c3e50',
width=0.5,
linewidth=1.5
)

axes[1, 0].annotate(
    "Pattern Emergence:\nBy rolling up to Quarters, we can easily\ncompare macro-level performance across seasons.",
    xy=(2.0, 15500), xytext=(0.5, 17000),
    fontsize=11, fontweight='bold', color='#2980b9',
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[1, 0].set_title('Figure 1C: Level 3 - Quarterly Sales (Roll-Up from Month)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=15)
axes[1, 0].set_ylabel('Aggregated Sales (kUSD)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[1, 0].tick_params(axis='x', labelsize=12) # FIX: Removed labelweight
axes[1, 0].set_xticklabels(axes[1, 0].get_xticklabels(), fontweight='bold') # FIX: Applied fontweight properly
axes[1, 0].grid(axis='y', linestyle='--', alpha=0.5)

for p in axes[1, 0].patches:
    axes[1, 0].annotate(
        f"{int(p.get_height())}k",
        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center', va='center', xytext=(0, 9), textcoords='offset points',
        fontweight='bold', fontsize=12, color='#2c3e50'
    )

# -----
# Subplot 4: Yearly Sales (Level 4 - Highest Summary)
# -----

axes[1, 1].bar(
    yearly_sales['Year'].astype(str),
    yearly_sales['Yearly_Sales_kUSD'],
    color='#2c3e50', # Navy Blue
    edgecolor='black',
    width=0.25,
    linewidth=2.0
)

axes[1, 1].set_title('Figure 1D: Level 4 - Yearly Sales (Highest Roll-Up)', fontdict={'fontsize': 14, 'fontweight': 'bold'}, pad=15)
axes[1, 1].set_ylabel('Total Annual Sales (kUSD)', fontdict={'fontsize': 13, 'fontweight': 'bold'})
axes[1, 1].tick_params(axis='x', labelsize=13) # FIX: Removed labelweight
axes[1, 1].set_xticklabels(axes[1, 1].get_xticklabels(), fontweight='bold') # FIX: Applied fontweight properly
axes[1, 1].grid(axis='y', linestyle='--', alpha=0.5)

for p in axes[1, 1].patches:
    axes[1, 1].annotate(
        f"${int(p.get_height()):}k",
        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center', va='center', xytext=(0, 12), textcoords='offset points',
        fontweight='bold', fontsize=14, color='#2c3e50'
    )

# Clear top and right spines for a clean academic aesthetic
for ax in axes.flat:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.suptitle('OLAP Concept Hierarchy: Time Dimension Data Aggregation', fontsize=17, fontweight='bold', y=1.02)

plt.tight_layout()
plt.show()

# =====
# SECTION 4: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه کاربردی (Strategic Implication):</b><br>"
    r"، در داده‌های کلان را فراهم می‌سازد. همان‌طور که نمودارها از بالا-چپ به پایین-راست نشان می‌دهند <b>ناوبری چندسطحی (Concept Hierarchy)</b> امکان <b>سلسله‌مراتب مفاهیم</b>"
    r"<br>نویزهای پرنوسان روزانه فیلتر شده و الگوهای کلان (نظیر عملکرد یکپارچه فصلی) نمایان می‌شوند. <b>Roll-Up</b> با اجرای عملیات"
    r":مورد نیاز خود دریافت نماید <i>(Granularity) سطح جزئیات</i> تضمین می‌کند که هر کاربر سازمانی، گزارش‌ها را دقیقاً در (BI) این معماری در سیستم‌های هوش تجاری"
    r"می‌کند تا علت آفت فروش در یک روز خاص را بررسی کند <b>Drill-Down</b> مدیرعامل به گراف سالانه (سرمه‌ای رنگ) نگاه می‌کند، در حالی که مدیر عملیاتی روی گراف روزانه (فیروزه‌ای رنگ)"
)
display_academic_text(insight_conclusion, "insight")
```



تحلیل چندسطحی فروش در «والمارت» با استفاده از سلسله‌مراتب مفاهیم (Concept Hierarchy)

مدیران «والمارت» (Walmart) نیاز دارند گزارش‌های فروش را در سطوح مختلف جزئیات بررسی کنند. مثلاً مدیرعامل می‌خواهد فروش کل سالانه را ببیند، اما مدیر بازاریابی نیاز دارد فروش ماه به ماه را تحلیل کند. این سطوح تودرتوی جزئیات را در انبارهای داده، **سلسله‌مراتب مفاهیم (Concept Hierarchy)** می‌نامند.

سلسله‌مراتب بُعد زمان (Time Dimension Formal Hierarchy):

$$\text{Hierarchy}_{\text{Time}} = \{\text{Day} \xrightarrow{\text{Roll-Up}} \text{Month} \xrightarrow{\text{Roll-Up}} \text{Quarter} \xrightarrow{\text{Roll-Up}} \text{Year}\}$$

- جمع‌بندی (Roll-Up): حرکت از سطح جزئی‌تر به سطح خلاصه‌تر (مثلاً از ماه به فصل). باعث کاهش ابعاد و نویز می‌شود.
- ریزنگری (Drill-Down): حرکت معکوس از سطح خلاصه به سطح جزئی‌تر (مثلاً از سال به ماه). باعث کشف الگوهای پنهان در داده‌های خُرد می‌شود.

مرحله نخست: تولید داده‌های روزانه و اجرای عملیات Roll-Up

سطح ۱: فروش روزانه (پایین‌ترین سطح جزئیات)	
Date	Daily_Sales_kUSD
2026-01-01	50.194691
2026-01-02	298.052890
2026-01-03	204.370377
2026-01-04	202.913290
2026-01-05	51.766576
2026-01-06	55.765606

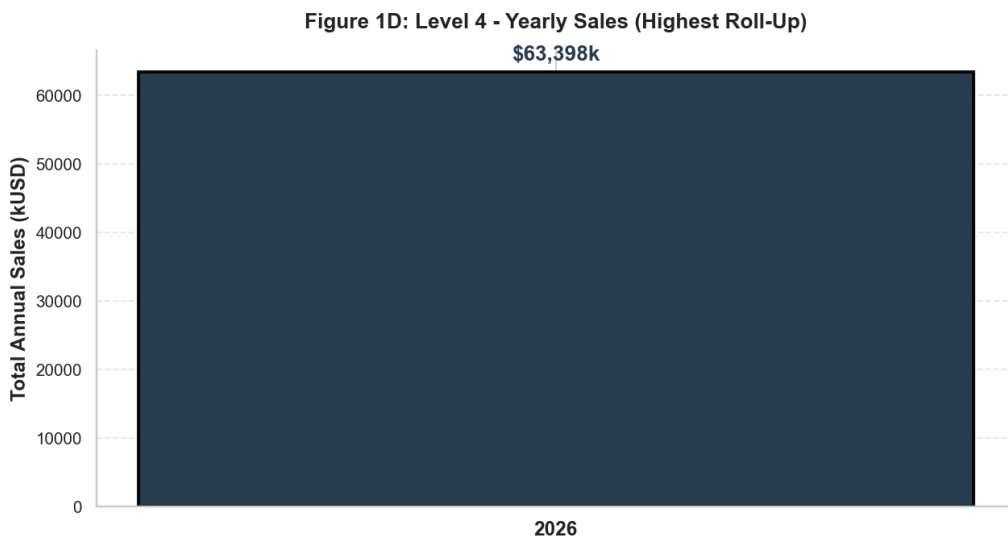
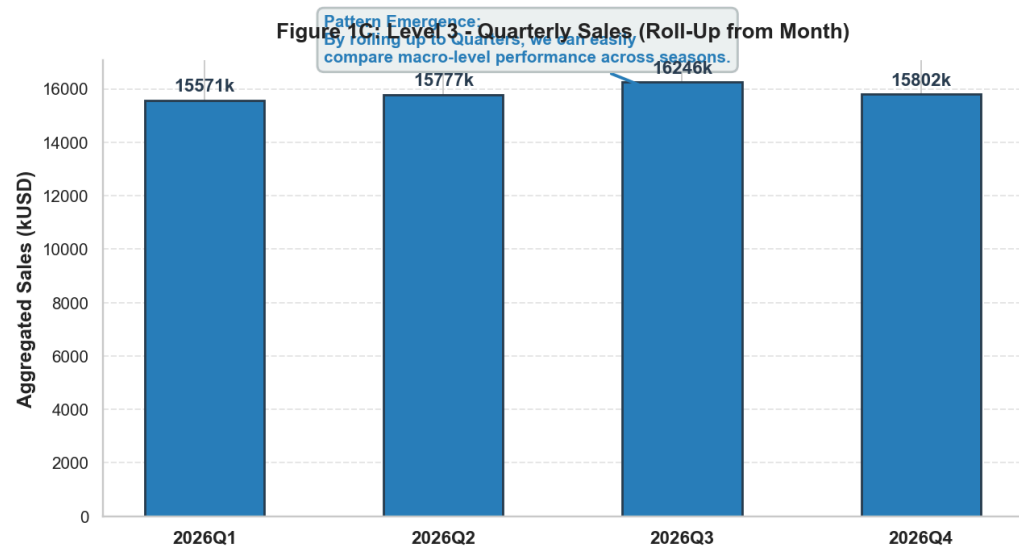
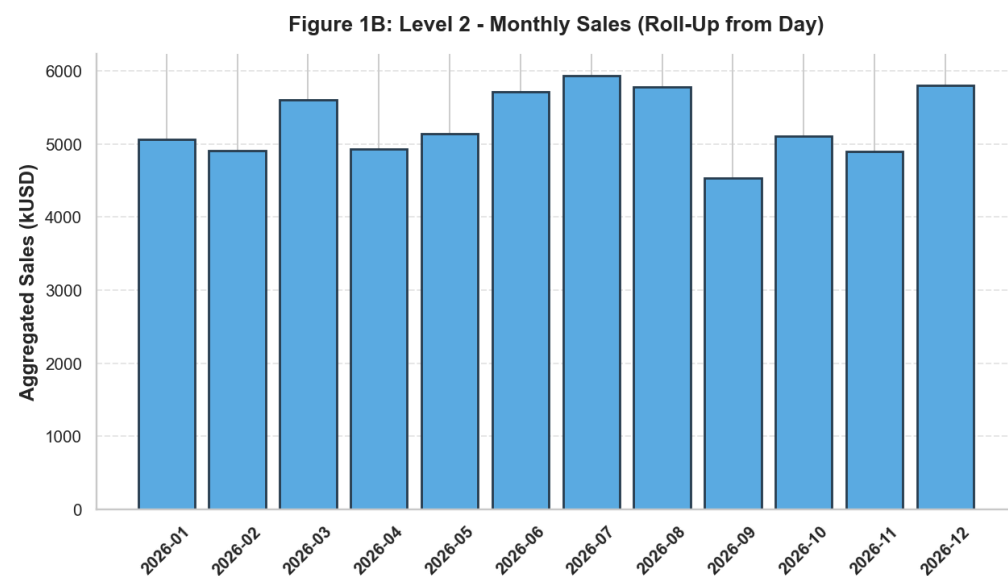
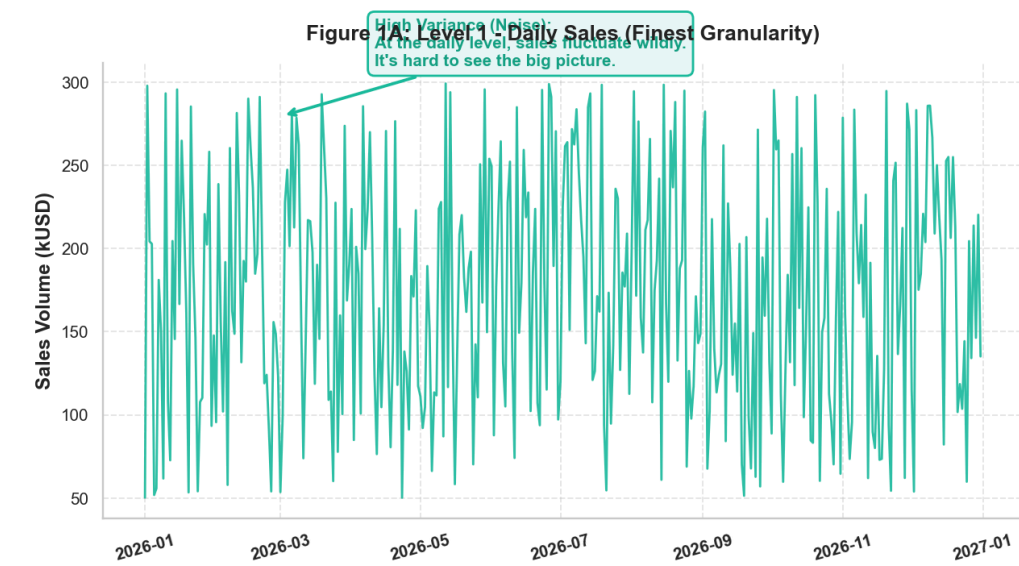
(از روزانه Roll-Up) سطح ۲: فروش ماهانه	
Month	Monthly_Sales_kUSD
2026-01	5061.303655
2026-02	4911.338427
2026-03	5598.817366
2026-04	4933.015495
2026-05	5137.477962
2026-06	5707.085353

(از ماهانه Roll-Up) سطح ۳: فروش فصلی	
Quarter	Quarterly_Sales_kUSD
2026Q1	15571.459448
2026Q2	15777.578811
2026Q3	16246.350979
2026Q4	15802.850735

سطح ۴: فروش سالانه (بالاترین سطح تجمیع)	
Year	Yearly_Sales_kUSD
2026	63398.239973

مرحله دوم: مصورسازی تحلیلی - تکامل هندسی با حرکت در سلسله‌مراتب

بوم گرافیکی زیر تأثیر عملیات **Roll-Up** را به صورت بصری اثبات می‌کند. با حرکت از سمت چپ به راست و از بالا به پایین، نوساناتِ شدید و نویزهای روزانه (نمودار اول) در یکدیگر ادغام شده و الگوهای کلان (نمودارهای فصل و سال) پدیدار می‌گردند. رنگ‌بندی نمودارها با افزایش سطحِ تجمیع، از فیروزه‌ای به سرمه‌ای تیره تغییر می‌کند تا سنگینیِ داده‌های تجمیع‌شده را تداعی نماید.



نتیجه کاربردی (Strategic Implication):

سلسله‌مراتب مفاهیم (Concept Hierarchy) امکان ناوبری چندسطحی در داده‌های کلان را فراهم می‌سازد. همان‌طور که نمودارها از بالا-چپ به پایین-راست نشان می‌دهند، با اجرای عملیات **Roll-Up**، نویزهای پرنوسان روزانه فیلتر شده و الگوهای کلان (نظیر عملکرد یکپارچه فصلی) نمایان می‌شوند. این معماری در سیستم‌های هوش تجاری (BI) تضمین می‌کند که هر کاربر سازمانی، گزارش‌ها را دقیقاً در سطح جزئیات (*Granularity*) مورد نیاز خود دریافت نماید: مدیرعامل به گراف سالانه (سرمه‌ای رنگ) نگاه می‌کند، در حالی که مدیر عملیاتی روی گراف روزانه (فیروزه‌ای رنگ) **Drill-Down** می‌کند تا علت افت فروش در یک روز خاص را بررسی کند.

توضیح مثال کد پایتون

در این مثال:

- ما داده‌های فروش روزانه وال‌مارت برای سال ۲۰۲۳ (۳۶۵ رکورد) را در پایین‌ترین سطح جزئیات (**Day-Level**) تولید کردیم.
- ساخت سلسله مراتب مفاهیم: با استفاده از متد `dt.to_period()` در پانداس، ستون تاریخ روزانه به ۳ سطح بالاتر تبدیل شد:
 - `dt.to_period('M')` → سطح ماه (Month)
 - `dt.to_period('Q')` → سطح فصل (Quarter)
 - `dt.to_period('Y')` → سطح سال (Year)
- با عملیات `groupby().sum()`، فروش روزانه در هر سطح سلسله مراتب تجمیع شد (عملیات Roll-Up).
- تفسیر ۴ نمودار Matplotlib:
 - نمودار بالا-چپ (**Line Chart**): ۳۶۵ نقطه فروش روزانه با جزئیات کامل و نویز طبیعی.
 - نمودار بالا-راست (**Bar Chart** نارنجی): Roll-Up به ۱۲ ماه؛ نوسانات روزانه از بین رفته و روند ماهانه مشخص شده.
 - نمودار پایین-چپ (**Bar Chart** بنفش): Roll-Up به ۴ فصل؛ سطح خلاصه‌تر برای تحلیل فصلی.
 - نمودار پایین-راست (**Bar Chart** قرمز): Roll-Up به ۱ عدد سالانه؛ بالاترین سطح خلاصه‌سازی.
- حرکت از چپ-بالا به راست-پایین دقیقاً معادل عملیات **Roll-Up** و حرکت معکوس معادل عملیات **Drill-Down** در OLAP است.

3.2.4 Measures: categorization and computation

معیارها (Measures) در داده‌کاوی، مقادیر کمی یا شاخص‌هایی هستند که در تحلیل‌ها سنجیده می‌شوند و مبنای تصمیم‌گیری قرار می‌گیرند. این معیارها را از دو منظر می‌توان دسته‌بندی کرد: یکی از منظر **قابلیت جمع‌شدن ساده**، و دیگری از منظر **قابلیت محاسبه افزایشی در مکعب داده** که مستقیماً به کارایی الگوریتم‌های محاسبه مکعب (مانند BUC و Multiway Array) مرتبط است.

دسته‌بندی اول: از منظر قابلیت جمع‌شدن ساده

معیارهای جمع‌پذیر (Additive Measures): این معیارها را می‌توان در تمام ابعاد به‌سادگی جمع یا تجمیع کرد؛ برای نمونه، کل فروش، سود کل یا تعداد کل فروخته‌شده از این دسته‌اند.

معیارهای غیرجمع‌پذیر (Non-Additive Measures): این معیارها را نمی‌توان به‌سادگی در تمام ابعاد تجمیع کرد؛ برای نمونه، میانگین حاشیه سود، درصد رشد یا نسبت‌ها. دلیل آن است که جمع‌کردن ساده این مقادیر در سطوح مختلف، نتیجه‌ای معنادار به‌دست نمی‌دهد.

معیارهای نیمه‌جمع‌پذیر (Semi-Additive Measures): در برخی ابعاد جمع‌پذیرند و در برخی نه؛ برای نمونه، **موجودی انبار** را می‌توان روی بُعد «محصول» جمع زد، اما روی بُعد «زمان» نمی‌توان چون موجودی لحظه‌ای است و در طول زمان انباشته نمی‌شود.

نکته: این دسته‌بندی برای درک اولیه ماهیت شاخص‌ها مفید است، اما برای طراحی الگوریتم‌های محاسبه مکعب داده کافی نیست. به همین دلیل، دسته‌بندی دقیق‌تری در ادامه معرفی می‌شود که مبنای اصلی روش‌های مطرح‌شده در بخش‌های ۳.۴ و ۳.۵ است.

دسته‌بندی دوم: از منظر محاسبه‌پذیری افزایشی در مکعب داده

این دسته‌بندی بر این مبنا است که آیا می‌توان یک تجمیع را از تجمیع‌های جزئی‌تر از پیش‌محاسبه‌شده به‌دست آورد یا نه همان چیزی که کارایی الگوریتم‌های BUC و Multiway Array Aggregation به آن وابسته است.

۱. توزیع‌پذیر (Distributive):

اگر داده را به N بخش تقسیم کنیم، تابع تجمیعی را روی هر بخش جداگانه اجرا کنیم، و سپس نتایج جزئی را با همان تابع (یا تابعی مشابه) دوباره ترکیب کنیم، به همان جواب کامل می‌رسیم. برای نمونه، فروش کل یک ماه را می‌توان از مجموع فروش کل هفته‌های همان ماه به‌دست آورد. یعنی برای محاسبه سطح بالاتر، فقط به نتایج جمع‌شده سطح پایین‌تر نیاز داریم، نه به داده‌های خام. این دسته ارزان‌ترین و کارآمدترین حالت برای BUC و Multiway Array است، چون هرس و محاسبه افزایشی روی آن‌ها بی‌دردسر کار می‌کند.

۲. جبری (Algebraic):

تابع را نمی‌توان مستقیماً از نتایج جزئی گرفت، اما اگر چند مقدار کمکی (نه فقط یک عدد) از هر بخش نگه داریم، می‌توان جواب نهایی را با یک فرمول جبری از همان مقادیر کمکی ساخت. برای نمونه، میانگین کل از خود میانگین‌های جزئی به‌دست نمی‌آید، اما اگر مجموع و تعداد هر بخش را هم ذخیره کنیم، تقسیم نهایی جواب می‌دهد.

هیچ راه ثابتی برای ساختن جواب از تعداد محدودی مقدار کمکی وجود ندارد؛ برای محاسبه دقیق باید کل مجموعه داده زیرین را دوباره دید. برای نمونه، میانه کل از میانه‌های هر هفته به دست نمی‌آید باید همه داده‌های خام را دوباره مرتب کرد. این دسته دقیقاً همان نقطه‌ای است که BUC و Multiway Array را کند یا حتی غیرعملی می‌کند، چون مزیت اصلی آن الگوریتم‌ها استفاده مجدد از نتایج میانی سطوح پایین‌تر اینجا از کار می‌افتد.

محاسبه معیارها (Derivation): منظور از محاسبه، استخراج معیارهای جدید یا تبدیل معیارهای موجود برای کسب بینش عمیق‌تر از داده‌هاست. روش‌های رایج برای این کار شامل درصدها، نرخ رشد، میانگین‌ها و هر عملیات ریاضی مرتبط دیگری است.

مثال عملی: دسته‌بندی و محاسبات ریاضی معیارها در «وال‌مارت» (Walmart)

در مکعب‌های داده انبار داده «وال‌مارت» (Walmart)، تمامی اعداد جدول واقعیت (Fact Table) ماهیت یکسانی ندارند. برای جلوگیری از محاسبات اشتباه در گزارشات مدیریتی، تحلیل‌گران داده معیارهای عددی را در ۳ دسته اصلی طبقه‌بندی می‌کنند:

۱. معیارهای جمع‌پذیر (Additive Measures):

عددهایی هستند که می‌توان آن‌ها را روی تمام ابعاد (زمان، شهر، نوع محصول) با تابع SUM() جمع بست بدون اینکه اشتباه ریاضی رخ دهد:

• کل درآمد فروش (Total Revenue): مجموع درآمد شعبه نیویورک + شیکاگو.

• تعداد کالای فروخته‌شده (Total Quantity): مجموع کالاهای فروخته‌شده در تمام روزها.

۲. معیارهای غیرجمع‌پذیر / نسبت‌ها (Non-Additive Measures):

عددهایی مانند «نسبت‌ها»، «درصدها» یا «میانگین حاشیه سود» را نمی‌توان با جمع زدن ساده تجمیع کرد! یک اشتباه رایج آماری این است که درصد حاشیه سود شعب را با هم جمع کنیم یا میانگین ساده بگیریم.

اثبات ریاضی خطای جمع‌زدن نسبت‌ها:

اگر شعبه A با فروش ۱۰۰ دلار حاشیه سود ۱۰٪ داشته باشد (10\$ Profit) و شعبه B با فروش ۱۰۰,۰۰۰ دلار حاشیه سود ۵۰٪ داشته باشد (50,000\$ Profit)، میانگین ساده درصدها می‌شود ۳۰٪ ($\frac{10\%+50\%}{2}$)؛ که کاملاً غلط است! زیرا حاشیه سود واقعی کل مجموعه برابر است با:

$$\text{Actual Profit Margin \%} = \frac{\sum \text{Total Profit}}{\sum \text{Total Revenue}} \times 100 = \frac{50,010}{100,100} \times 100 \approx 49.96\%$$

۳. معیارهای محاسباتی (Computed Measures):

معیارهای جدیدی که بر اساس فرمول‌های ریاضی از معیارهای پایه مشتق می‌شوند (مانند Profit = Revenue - Cost).

نتیجه: تشخیص دقیق نوع معیار مانع از خطاهای محاسبه‌ای در سیستم‌های گزارش‌گیری OLAP و هوش تجاری می‌گردد.

خروجی: جداسازی معیارهای جمع‌پذیر از نسبت‌های غیرجمع‌پذیر و اجرای تجمیع وزن‌دار درست

In [9]: display_academic_text(
 "(Walmart) «دسته‌بندی و محاسبات ریاضی معیارها در انبار داده «وال‌مارت»",
 "main_title"
)

display_academic_text(
 ". ماهیت یکسانی ندارند (Fact Table) تمامی اعداد در جدول واقعیت (Data Warehouse) در مکعب‌های داده انبار داده "
 ":معیارهای عددی را در ۳ دسته اصلی طبقه‌بندی می‌کنند BI برای جلوگیری از محاسبات فاجعه‌بار در گزارشات مدیریتی، تحلیل‌گران "
 "paragraph"

```
display_academic_text("۱. معیارهای جمع‌پذیر (Additive Measures):", "sub_title")
display_academic_text(
    "<br>"
    "جمع بست، بدون اینکه اشتباه ریاضی رخ دهد <code>SUM()</code> با تابع (زمان، شهر، نوع محصول) <i><b>تمامی ابعاد</b></i> اعداد مطلق هستند که می‌توان آن‌ها را روی  

    "• <b>مجموع درآمد شعبه نیویورک + شیکاگو (Total Revenue):</b> کل درآمد فروش  

    "• <b>مجموع کالاهای فروخته‌شده امروز + دیروز (Quantity):</b> تعداد کالای فروخته‌شده">
    "paragraph"
)
```

```
display_academic_text("۲. معیارهای غیرجمع‌پذیر / نسبت‌ها (Non-Additive Measures):", "sub_title")
display_academic_text(
    " !را نمی‌توان با جمع زدن ساده تجمیع کرد </b>«نسبت‌ها»، «درصدها» یا «میانگین حاشیه سود»<b> اعدادی مانند"
    "بگیریم (Simple Average) یک اشتباه رایج آماری در داشبوردها این است که درصد حاشیه سود شعب مختلف را با هم جمع کنیم یا میانگین ساده
    "paragraph"
)
```

```
display_academic_text("Simpson's Paradox): اثبات ریاضی خطای جمع‌زدن نسبت‌ها", "sub_title")
display_academic_text(
    " ،با فروش ۱۰۰,۰۰۰ دلار حاشیه سود ۵۰٪ داشته باشد (۵۰,۰۰۰ دلار سود) B با فروش ۱۰۰ دلار حاشیه سود ۱۰٪ داشته باشد (۱۰ دلار سود) و شعبه A اگر شعبه "
    "،:کل مجموعه (میانگین وزن‌دار) برابر است با <i>واقعی</i> است! زیرا حاشیه سود <b>غلط</b> میانگین ساده درصدها می‌شود ۳۰٪؛ که کاملاً"
    "paragraph"
)
```

```
display_academic_text(
    r"$\text{Simple Average (Wrong)} = \frac{10\% + 50\%}{2} = 30\%$",
    "paragraph"
)
```

```
display_academic_text(
    r"$\text{Weighted Average (Correct)} = \frac{\sum \text{Total Profit}}{\sum \text{Total Revenue}} \times 100 = \frac{50,010}{100,100} \times 100 \approx 49.96\%$",
    "paragraph"
)
```

```
display_academic_text("۳. معیارهای محاسباتی در زمان اجرا (Computed Measures):", "sub_title")
display_academic_text(
    "r$Profit = Revenue - Cost$) مانند) می‌شوند <i>مشتق</i> معیارهای جدیدی که بر اساس فرمول‌های ریاضی از معیارهای پایه"
    "paragraph"
)
```

```
# =====
# SECTION 2: DATA GENERATION AND MATHEMATICAL FORMULATIONS
# =====
```

```
display_academic_text(
    " (Aggregation) مرحله نخست: تولید داده‌های فروش و تجمیع",
    "sub_title"
)
```

```
# Step 1: Create Walmart sales & cost dataset across Stores and Categories
```

```
data = {
    'Store': ['New York', 'New York', 'Chicago', 'Chicago', 'Los Angeles', 'Los Angeles'],
    'Category': ['Electronics', 'Apparel', 'Electronics', 'Apparel', 'Electronics', 'Apparel'],
    # Additive Measures
    'Revenue_USD': [150000, 20000, 80000, 15000, 220000, 30000],
    'Cost_USD': [100000, 15000, 60000, 12000, 130000, 18000],
    'Quantity_Sold': [300, 500, 160, 400, 450, 750],
}
```

```
df = pd.DataFrame(data)
```

```
# Step 2: Compute Base Computed Measure (Profit = Revenue - Cost)
```

```
df['Profit_USD'] = df['Revenue_USD'] - df['Cost_USD']
```

```
# Step 3: Compute Item-Level Non-Additive Measure (Profit Margin %)
```

```
df['Profit_Margin_Pct'] = (df['Profit_USD'] / df['Revenue_USD']) * 100
```

```
# Step 4: Correct vs Incorrect Aggregation by Store
```

```
# 4.1 Correct Additive Aggregation (SUM) for Revenue & Profit
```

```
store_agg = df.groupby('Store')[['Revenue_USD', 'Cost_USD', 'Profit_USD', 'Quantity_Sold']].sum()
```

```
# 4.2 WRONG Non-Additive Aggregation (Naive Mean of Percentages)
```

```
store_agg['Naive_Avg_Margin_Pct'] = df.groupby('Store')['Profit_Margin_Pct'].mean()
```

```
# 4.3 CORRECT Computed Non-Additive Aggregation (Weighted Margin %)
```

```
store_agg['Correct_Weighted_Margin_Pct'] = (store_agg['Profit_USD'] / store_agg['Revenue_USD']) * 100
```

```
# Prepare DataFrames for display to avoid hidden index columns
```

```
df_disp = df[['Store', 'Category', 'Revenue_USD', 'Profit_USD', 'Profit_Margin_Pct']].reset_index(drop=True)
```

```
store_agg_disp = store_agg[['Revenue_USD', 'Profit_USD', 'Naive_Avg_Margin_Pct', 'Correct_Weighted_Margin_Pct']].reset_index()
```

```
display_side_by_side_datasets(
    [df_disp],
    ["شامل معیارهای پایه و محاسباتی (Item-level) جدول ۱: داده‌های سطح کالا"]
)
```

```
display_side_by_side_datasets(
    [store_agg_disp],
    ["جدول ۲: مقایسه تجمیع درست (وزن‌دار) و نادرست (میانگین ساده) حاشیه سود به تفکیک شعبه"]
)
```

```
# =====
# SECTION 3: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
```

```
display_academic_text(
    " (The Aggregation Fallacy) مرحله دوم: مصورسازی تحلیلی - توهم میانگین ساده",
    "sub_title"
)
```

```
display_academic_text(
    " . در دیتابیس شوند <code>AVG()</code> به هیچ‌وجه نباید با <b>متغیرهای نسبیتی (مانند درصد)</b> بوم گرافیکی زیر نشان می‌دهد که چرا"
    " در نمودار سمت راست، اگر میانگین ساده بگیریم، حاشیه سود در نیویورک ۲۹٪ گزارش می‌شود. اما این یک خطای فاحش است، زیرا فروش ۱۵۰ هزار دلاری الکترونیک"
    "،!وزن بسیار بیشتری از فروش ۲۰ هزار دلاری پوشاک دارد. حاشیه سود واقعی شعبه (آبی فیروزه‌ای) ۳۲.۳۵٪ است (با حاشیه سود متفاوت)"
    "paragraph"
)
```

```
# Visualization using Matplotlib
fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

x = np.arange(len(store_agg.index))
width = 0.35

# -----
# Subplot 1: Additive Measures (Revenue and Profit Bar Chart)
# -----
axes[0].bar(
    x - width / 2,
    store_agg['Revenue_USD'] / 1000,
    width,
    label='Total Revenue ($k)',
    color='#2980b9', # Classic Blue
    edgecolor='#2c3e50',
    linewidth=1.5
)
axes[0].bar(
    x + width / 2,
    store_agg['Profit_USD'] / 1000,
    width,
    label='Total Profit ($k)',
    color='#2c3e50', # Navy
    edgecolor='#2c3e50',
    linewidth=1.5
)

axes[0].set_title('Figure 1: Additive Measures (Safe SUM Aggregation)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xticks(x)
axes[0].set_xticklabels(store_agg.index, fontweight='bold', fontsize=11)
axes[0].set_ylabel('Amount in $1,000s USD', fontweight='bold', fontsize=12)
axes[0].legend(loc='upper left', frameon=True, fontsize=11)
axes[0].grid(axis='y', linestyle='--', alpha=0.5)

# -----
# Subplot 2: Non-Additive Comparison (Naive Mean vs Correct Weighted)
# -----
axes[1].bar(
    x - width / 2,
    store_agg['Naive_Avg_Margin_Pct'],
    width,
    label='Naive Simple Average (INCORRECT)',
    color='#e74c3c', # Red Alert
    edgecolor='#c0392b',
    linewidth=1.5,
)
axes[1].bar(
    x + width / 2,
    store_agg['Correct_Weighted_Margin_Pct'],
    width,
    label='Weighted Margin (CORRECT)',
    color='#1abc9c', # Turquoise (Safe)
    edgecolor='#16a085',
    linewidth=1.5,
)

axes[1].annotate(
    "The Fallacy of Averages:\nThe simple average (red) ignores that Electronics\ngenerated $150k while Apparel generated only $20k.\nThe true branch margin (green) is heavily pulled\ntowards",
    xy=(0.15, 32.35), xytext=(0.0, 45),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[1].set_title('Figure 2: Non-Additive Measures (Aggregation Fallacy)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xticks(x)
axes[1].set_xticklabels(store_agg.index, fontweight='bold', fontsize=11)
axes[1].set_ylabel('Profit Margin (%)', fontweight='bold', fontsize=12)
axes[1].set_ylim(0, 55)
axes[1].legend(loc='lower right', frameon=True, fontsize=11)
axes[1].grid(axis='y', linestyle='--', alpha=0.5)

# Clear top and right spines for a clean academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# SECTION 4: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>Strategic Implication:</b><br>"
    r"مانند درصدها <b>(Non-Additive)</b> غیرجمع‌پذیر می‌گردد. تجمیع معیارهای (BI) تشخیص دقیق نوع معیار، مانع از خطاهای مخرب محاسبه‌ای در داشبوردهای مدیریتی"
    r"<br>در دیتابیس انجام شود <code>AVG()</code> یا <code>SUM()</code> نباید هرگز با توابع"
    r"جمع کنند <code>SUM()</code> را با تابع <i>Revenue</i> و <i>Profit</i> (مانند) همیشه باید در سطح خُرد، اعداد مطلق و جمع‌پذیر (OLAP) سیستم‌های پردازش تحلیلی آنلاین"
    r".فرمول محاسبهٔ نسبت (تقسیم سود کل بر درآمد کل) را اجرا نمایند تا میانگین وزن‌دارِ دقیق به دست آید، <b>گام نهایی</b> (در سطح داشبورد و پس از تجمیع)<b> و فقط در"
)
display_academic_text(insight_conclusion, "insight")
```



در مکعب‌های داده‌ انبار داده (Data Warehouse)، تمامی اعداد در جدول واقعیت (Fact Table) ماهیت یکسانی ندارند. برای جلوگیری از محاسبات فاجعه‌بار در گزارشات مدیریتی، تحلیل‌گران BI معیارهای عددی را در ۳ دسته اصلی طبقه‌بندی می‌کنند:

۱. معیارهای جمع‌پذیر (Additive Measures):

- اعداد مطلق هستند که می‌توان آن‌ها را روی تمامی ابعاد (زمان، شهر، نوع محصول) با تابع SUM () جمع بست، بدون اینکه اشتباه ریاضی رخ دهد:
- کل درآمد فروش (Total Revenue): مجموع درآمد شعبه نیویورک + شیکاگو.
 - تعداد کالای فروخته‌شده (Quantity): مجموع کالاهای فروخته‌شده امروز + دیروز.

۲. معیارهای غیرجمع‌پذیر / نسبت‌ها (Non-Additive Measures):

اعدادی مانند «نسبت‌ها»، «درصدها» یا «میانگین حاشیه سود» را نمی‌توان با جمع زدن ساده تجمیع کرد! یک اشتباه رایج آماری در داشبوردها این است که درصد حاشیه سود شعب مختلف را با هم جمع کنیم یا میانگین ساده (Simple Average) بگیریم.

اثبات ریاضی خطای جمع‌زدن نسبت‌ها (Simpson's Paradox):

اگر شعبه A با فروش ۱۰۰ دلار حاشیه سود ۱۰٪ داشته باشد (۱۰ دلار سود) و شعبه B با فروش ۱۰۰,۰۰۰ دلار حاشیه سود ۵۰٪ داشته باشد (۵۰,۰۰۰ دلار سود)، میانگین ساده درصدها می‌شود ۳۰٪؛ که کاملاً غلط است! زیرا حاشیه سود واقعی کل مجموعه (میانگین وزن‌دار) برابر است با:

$$\text{Simple Average (Wrong)} = \frac{10\% + 50\%}{2} = 30\%$$

$$\text{Weighted Average (Correct)} = \frac{\sum \text{Total Profit}}{\sum \text{Total Revenue}} \times 100 = \frac{50,010}{100,100} \times 100 \approx 49.96\%$$

۳. معیارهای محاسباتی در زمان اجرا (Computed Measures):

معیارهای جدیدی که بر اساس فرمول‌های ریاضی از معیارهای پایه مشتق می‌شوند (مانند Profit = Revenue - Cost).

مرحله نخست: تولید داده‌های فروش و تجمیع (Aggregation)

جدول ۱: داده‌های سطح کالا (Item-level) شامل معیارهای پایه و محاسباتی				
Store	Category	Revenue_USD	Profit_USD	Profit_Margin_Pct
New York	Electronics	150000	50000	33.333333
New York	Apparel	20000	5000	25.000000
Chicago	Electronics	80000	20000	25.000000
Chicago	Apparel	15000	3000	20.000000
Los Angeles	Electronics	220000	90000	40.909091
Los Angeles	Apparel	30000	12000	40.000000

جدول ۲: مقایسه تجميع درست (وزن دار) و نادرست (میانگین ساده) حاشیه سود به تفکیک شعبه

Store	Revenue_USD	Profit_USD	Naive_Avg_Margin_Pct	Correct_Weighted_Margin_Pct
Chicago	95000	23000	22.500000	24.210526
Los Angeles	250000	102000	40.454545	40.800000
New York	170000	55000	29.166667	32.352941

مرحله دوم: مصورسازی تحلیلی - توهم میانگین ساده (The Aggregation Fallacy)

بوم گرافیکی زیر نشان می‌دهد که چرا متغیرهای نسبتی (مانند درصد) به هیچ‌وجه نباید با (AVG) در دیتابیس تجميع شوند. در نمودار سمت راست، اگر میانگین ساده بگیریم، حاشیه سود در نیویورک ۲۹٪ گزارش می‌شود. اما این یک خطای فاحش است، زیرا فروش ۱۵۰ هزار دلاری الکترونیک (با حاشیه سود متفاوت) وزن بسیار بیشتری از فروش ۲۰ هزار دلاری پوشاک دارد. حاشیه سود واقعی شعبه (آبی فیروزه‌ای) ۳۲.۳۵٪ است!

Figure 1: Additive Measures (Safe SUM Aggregation)

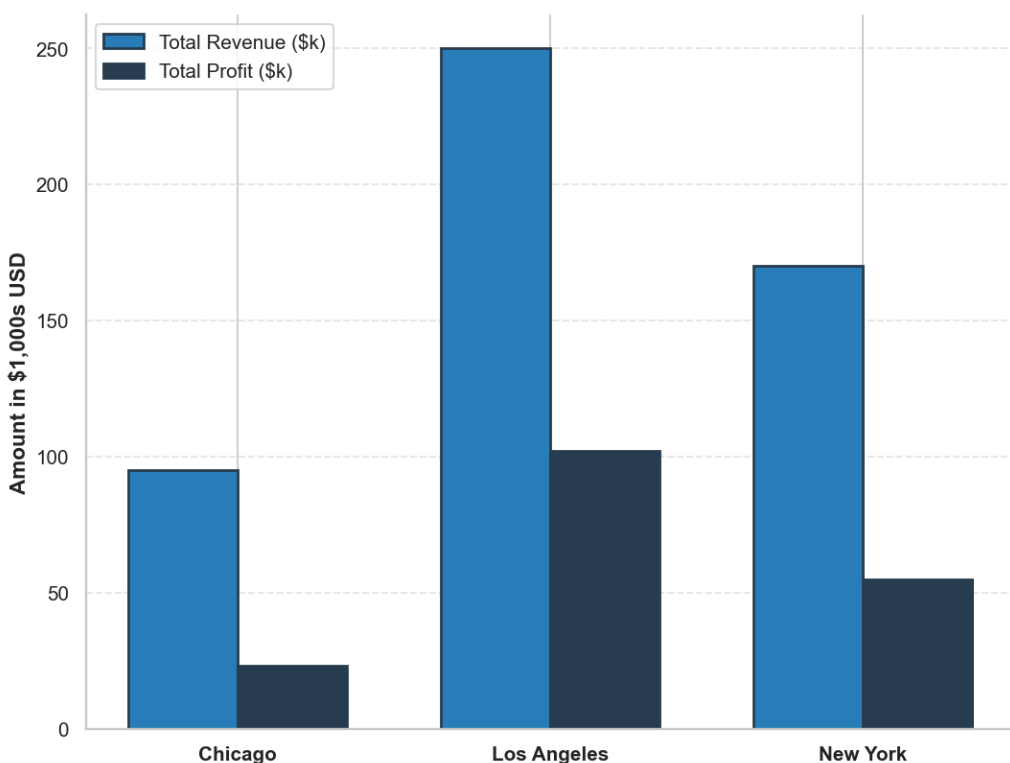
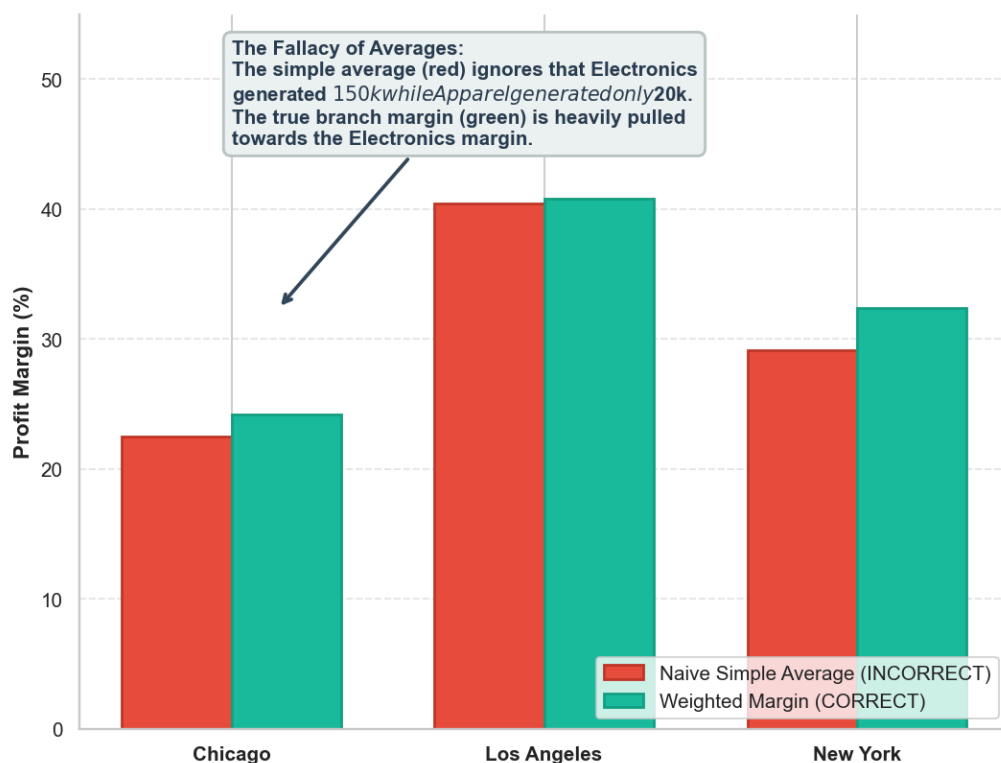


Figure 2: Non-Additive Measures (Aggregation Fallacy)



نتیجه استراتژیک در هوش تجاری (Strategic Implication):

تشخیص دقیق نوع معیار، مانع از خطاهای مخرب محاسبه‌ای در داشبوردهای مدیریتی (BI) می‌گردد. تجميع معیارهای غیرجمع‌پذیر (Non-Additive) مانند درصدها نباید هرگز با توابع SUM() یا AVG() در دیتابیس انجام شود. سیستم‌های پردازش تحلیلی آنلاین (OLAP) همیشه باید در سطح خُرد، اعداد مطلق و جمع‌پذیر (مانند Profit و Revenue) را با تابع SUM() جمع کنند، و فقط در گام نهایی (در سطح داشبورد و پس از تجميع)، فرمول محاسبه نسبت (تقسیم سود کل بر درآمد کل) را اجرا نمایند تا میانگین وزن دار دقیق به دست آید.



توضیح مثال کد پایتون

در این مثال:

- ما یک دیتافریم واقعی از فروشگاه والمارت شامل معیارهای پایه درآمد (Revenue_USD)، هزینه (Cost_USD) و تعداد فروخته‌شده (Quantity_Sold) ساختیم.
- محاسبه معیار مشتق‌شده: سود کل (Profit_USD) با کسر هزینه از درآمد محاسبه شد که یک معیار جمع‌پذیر است.
- محاسبه درصد حاشیه سود: حاشیه سود (Profit_Margin_Pct) از تقسیم سود بر درآمد حاصل شد که یک معیار غیرجمع‌پذیر (نسبتی) است.
- اثبات خطای تجميع غیرجمع‌پذیر:
 - مقدار Naive_Avg_Margin_Pct با میانگین‌گیری ساده روی درصدها محاسبه شد (با رنگ قرمز روی نمودار دوم).
 - مقدار Correct_Weighted_Margin_Pct با فرمول صحیح وزن دار یعنی تقسیم مجموع کل سود بر مجموع کل درآمد تجميع گردید (با رنگ سبز روی نمودار).
- تفسیر نمودارها (Matplotlib):
 - نمودار سمت چپ: جمع‌پذیری کامل درآمد و سود کل را روی ۳ شهر نشان می‌دهد.

▪ نمودار سمت راست: تفاوت فاحش بین میانگین ساده ناآگاهانه و درصد حاشیه سود واقعی وزن‌دار را به تصویر می‌کشد تا اهمیت دسته‌بندی درست معیارها در انبار داده اثبات شود.

3.3 OLAP operations

ریاضیات – شبکهٔ مکعب‌واره‌ها

هر مکعب‌واره (Cuboid) یک سطح مشخص از جزئیات داده را نشان می‌دهد. اگر برای هر بُعد فقط دو انتخاب «نگه‌داشتن بُعد» یا «تجمیع آن بُعد» داشته باشیم، شبکهٔ مکعب‌واره‌ها ساختاری نمایی پیدا می‌کند.

تعداد مکعب‌واره‌ها در حالت دوحالتی

(3.3)

$$N_{\text{cuboids}} = 2^n$$

نمادهای فرمول

- N_{cuboids} : تعداد مکعب‌واره‌های شبکه
- n : تعداد ابعاد
- 2: تعداد حالت‌های هر بُعد: جزئیات یا تجمیع

مثال ۱ – شبکهٔ مکعب‌واره‌ها

صورت سؤال: اگر مکعب داده ۵ بُعد داشته باشد، چند مکعب‌واره در شبکه وجود دارد؟

$$N_{\text{cuboids}} = 2^5 = 32$$

نتیجه: پس ۳۲ مکعب‌واره وجود دارد.

تعداد مکعب‌واره‌ها با سلسله‌مراتب چندسطحی

(3.4)

$$N_{\text{cuboids}} = \prod_{i=1}^n (L_i + 1)$$

نمادهای فرمول

- N_{cuboids} : تعداد مکعب‌واره‌های ممکن
- n : تعداد ابعاد
- L_i : تعداد سطوح سلسله‌مراتب در بُعد i ، بدون احتساب حالت ALL
- i : شاخص بُعد

مثال ۲ – وجود سلسله‌مراتب

صورت سؤال: دو بُعد داریم. بُعد اول ۲ سطح سلسله‌مراتب و بُعد دوم ۳ سطح دارد. با احتساب حالت ALL، چند انتخاب سطحی خواهیم داشت؟

$$N_{\text{cuboids}} = (2 + 1)(3 + 1) = 12$$

نتیجه: در این مدل، ۱۲ مکعب‌واره قابل تشکیل است.

نکته: این شمارش فقط تعداد حالت‌های سطحی را بیان می‌کند؛ تعداد سلول‌های داخل هر مکعب‌واره به تعداد مقادیر همان سطح وابسته است.

3.3.1 Typical OLAP operations



OLAP؛ پردازش تحلیلی آنی

OLAP یا **پردازش تحلیلی آنی (Online Analytical Processing)**، دسته‌ای از ابزارها و فناوری‌هاست که به کاربران امکان می‌دهد داده‌های چندبعدی را به صورت **تعاملی** تحلیل کنند و از این طریق، **تصمیم‌گیری** و **هوش تجاری** را پشتیبانی کنند. در این رویکرد، داده‌ها در چند بُعد مانند **زمان**، **مکان** و **کالا** سازمان‌دهی می‌شوند و هر بُعد، **سلسله‌مراتب مفهومی (Concept Hierarchy)** مخصوص خود را دارد. همین ساختار است که به کاربر اجازه می‌دهد داده‌ها را در سطوح مختلفی از جزئیات ببیند و بررسی کند.

برای کار با این ساختار، کاربران چند عملیات اصلی روی مکعب‌های داده چندبعدی انجام می‌دهند:

Roll-up (تجمیع): داده‌ها را کلی‌تر و خلاصه‌تر می‌کند. این کار یا با بالارفتن در سلسله‌مراتب مفهومی انجام می‌شود (مثلاً از فروش شهر به فروش استان و سپس کشور می‌رسیم)، یا با حذف یک بُعد؛ برای نمونه، اگر بُعد مکان حذف شود، فقط فروش کل شرکت در هر بازه زمانی نمایش داده می‌شود.

Drill-down (جزئی‌سازی): دقیقاً برعکس Roll-up عمل می‌کند و از اطلاعات کلی به جزئیات می‌رسد. این کار هم از دو راه ممکن است: پایین‌آمدن در سلسله‌مراتب (مثلاً از فروش سالانه به فصلی، ماهانه و روزانه)، یا اضافه‌کردن یک بُعد جدید مانند گروه مشتری.

Slice (برش): با انتخاب فقط یک مقدار از یک بُعد، یک زیرمکعب یا برش دوبعدی از مکعب داده ساخته می‌شود؛ مثلاً وقتی فقط داده‌های فروش فصل اول را می‌بینیم.

Dice (تکه‌بندی): در این عملیات، روی چند بُعد هم‌زمان شرط می‌گذاریم و یک زیرمکعب کوچک‌تر به دست می‌آید؛ برای نمونه، فروش محدود به چند شهر خاص، چند فصل مشخص و یک دسته معین از محصولات.

Pivot (چرخش): فقط زاویه نمایش داده‌ها را تغییر می‌دهد، بدون اینکه خود داده‌ها تغییر کنند؛ مثلاً جابه‌جا کردن سطر و ستون بین شهر و کالا در یک جدول.

فرض کنید مدیر فروش یک فروشگاه زنجیره‌ای هستید و می‌خواهید بدانید «فروش محصولات لبنی در استان تهران، در سه‌ماهه اول سال ۱۴۰۳، نسبت به سه‌ماهه اول سال ۱۴۰۲، چه تغییری داشته است؟» برای پاسخ به این سؤال، به ترتیب از کدام عملیات‌های OLAP استفاده می‌کنید؟ پاسخ خود را با ذکر دلیل توضیح دهید.

جدا از این عملیات اصلی، OLAP می‌تواند کارهای تحلیلی دیگری هم انجام دهد، مانند یافتن Top N و Bottom N، محاسبه میانگین متحرک، نرخ رشد، تبدیل ارز، محاسبات آماری، پیش‌بینی و تحلیل روند.

در مجموع، این عملیات یک محیط تعاملی برای تحلیل داده، گزارش‌گیری، پیش‌بینی و تصمیم‌گیری مدیریتی فراهم می‌کنند و همین ویژگی، OLAP را به یکی از ابزارهای کلیدی انبار داده و هوش تجاری تبدیل کرده است.

📌 پنج فرمان اصلی در مکعب داده

Roll-up (تجمیع): بالا رفتن از سطح جزئیات؛ مثلاً از فروش روزانه به ماهانه یا از فروش شهر به استان.

Drill-down (جزئی‌سازی): برعکس Roll-up؛ پایین رفتن به سطوح ریزتر، مثل شکستن فروش سالانه به فصلی و ماهانه.

Slice (برش): انتخاب یک مقدار ثابت از یک بُعد و دیدن فقط همان لایه؛ مثل «فقط فروش فصل اول».

Dice (تکه‌بندی): انتخاب هم‌زمان چند مقدار از چند بُعد مختلف؛ مثل فروش «محصول A در تهران طی بهار».

Pivot (چرخش): تغییر زاویه دید بدون تغییر داده‌ها؛ مثلاً جابه‌جایی سطر و ستون در جدول گزارش.

💡 **نکته کلیدی:** این پنج عملیات، زبان اصلی تعامل با مکعب داده هستند. Roll-up و Drill-down عمق تحلیل را تغییر می‌دهند، Slice و Dice محدوده تحلیل را مشخص می‌کنند، و Pivot زاویه نمایش را تنظیم می‌کند.

📊 مثال عملی: تحلیل تعاملی فروش آمازون با عملیات پنج‌گانه OLAP

تیم تحلیل هوش تجاری آمازون برای بررسی فروش جهانی خود، داده‌ها را در یک مکعب داده چندبعدی شامل ابعاد زمان، منطقه جغرافیایی و دسته‌بندی کالا ذخیره می‌کند. مدیران برای پاسخ به سوالات مختلف، ۵ عملیات اصلی OLAP را روی این مکعب اجرا می‌کنند:

۱. عملیات فشردن (Roll-up): تجمیع داده‌ها و حرکت به سمت سطوح بالاتر انتزاع. مثلاً تبدیل داده‌های فروش ماهانه به فروش سه ماهه (فصلی) یا سالانه.

۲. عملیات جزئی‌سازی (Drill-down): باز کردن داده‌های خلاصه‌شده و حرکت به سمت جزئیات دقیق‌تر. مثلاً شکستن فروش سالانه به فروش ماهانه برای پیدا کردن علت افت فروش در یک ماه خاص.

۳. عملیات برش (Slice): برش دادن مکعب روی یک بعد خاص برای بررسی یک لایه دو بعدی. مثلاً فیلتر کردن کل مکعب فقط برای سال ۲۰۲۳.

۴. عملیات تاس‌زدن (Dice): انتخاب یک زیرمکعب کوچک‌تر با مشخص کردن فیلتر روی چند بعد مختلف. مثلاً انتخاب داده‌های فروش سال ۲۰۲۳، منطقه آمریکا و دسته کالای الکترونیک.

۵. عملیات چرخش (Pivot): چرخاندن محورهاى مکعب داده برای تغییر زاویه دید و نمایش داده‌ها از یک دیدگاه جدید (مثلاً جابه‌جا کردن جای سطرها و ستون‌ها در جدول گزارش).

نتیجه: این عملیات پنج‌گانه به مدیران اجازه می‌دهد در چند ثانیه از کلی‌ترین گزارش‌های مدیریتی به جزیی‌ترین داده‌های عملیاتی دسترسی پیدا کنند.

خروجی: تحلیل تعاملی و چندبعدی داده‌ها با ترکیب ۵ عملیات پایه OLAP

```
display_academic_text(
    " 🌐 OLAP تحلیل تعاملی فروش آمازون با عملیات پنج‌گانه",
    "main_title"
)

display_academic_text(
    "، شامل ابعادِ زمان </b>(Data Cube) مکعب دادهٔ چندبُعدی<b> آمازون برای بررسی فروش جهانی خود، داده‌ها را در یک (BI) تیم تحلیل هوش تجاری  

    ،را روی این مکعب اجرا می‌کنند </b>OLAP عملیاتِ اصلی <b>۵، منطقه جغرافیایی و دسته‌بندی کالا ذخیره می‌کند. مدیران برای پاسخ به سوالات مختلف و چرخش در این فضا",
    "paragraph"
)

display_academic_text(
    "<br>"
    "تجمیع داده‌ها و حرکت به سطوح بالاتر انتزاع. مثلاً تبدیل فروش ماهانه به سالانه </b>(Roll-up): فشردن<b> ۱.  

    <br>"
    "باز کردن داده‌های خلاصه‌شده و حرکت به سمت جزئیات. مثلاً شکستن فروش سالانه به فصلی برای کشف علت افت فروش </b>(Drill-down): جزئی‌سازی<b> ۲.  

    <br>"
    "برای سال ۲۰۲۶ فقط</i> برای بررسی لایهٔ دو بُعدی. مثلاً فیلتر کردن کل مکعب </i>یک بُعدِ خاص<i> برش دادن مکعب روی </b>(Slice): برش<b> ۳.  

    <br>"
    "دسته الکترونیک </b>و<b> منطقه آمریکا <b>و</b>، مثلاً سال ۲۰۲۶. </i>چندین بُعد هم‌زمان<i> انتخاب یک زیرمکعب کوچک‌تر با مشخص کردن فیلتر روی </b>(Dice): تاس‌زدن<b> ۴.  

    ، چرخاندن محورهای مکعب برای تغییر زاویه دید تحلیلی (مثلاً جابه‌جا کردن جای سطرها و ستون‌ها در گزارش) </b>(Pivot): چرخش<b> ۵.  

    "paragraph"
)

# =====
# SECTION 2: DATA GENERATION AND OLAP OPERATIONS PIPELINE
# =====
# Step 1: Create Amazon 3D Sales Dataset
# Dimensions: Year (2025, 2026), Region (US, Europe), Category (Electronics, Apparel)

records = []
years = [2025, 2026]
quarters = ['Q1', 'Q2', 'Q3', 'Q4']
regions = ['US', 'Europe']
categories = ['Electronics', 'Apparel']

# Generating the Multidimensional Cube Data
for y in years:
    for q in quarters:
        for r in regions:
            for c in categories:
                sales = np.random.randint(100, 500)
                records.append({
                    'Year': y,
                    'Quarter': q,
                    'Region': r,
                    'Category': c,
                    'Sales_kUSD': sales,
                })

df = pd.DataFrame(records)

# Step 2: Implement 5 Typical OLAP Operations in Pandas

# 1. Roll-up: Aggregating from Quarter to Year level (Discarding Quarter info)
rollup_df = df.groupby(['Year', 'Region'])['Sales_kUSD'].sum().reset_index()

# 2. Drill-down: Expanding Year 2026 into Quarter details
drilldown_df = df[df['Year'] == 2026].groupby(['Quarter', 'Category'])['Sales_kUSD'].sum().reset_index()

# 3. Slice: Filtering on a single dimension (e.g., Year = 2026) -> Makes a 3D cube into a 2D slice
slice_df = df[df['Year'] == 2026]

# 4. Dice: Filtering on multiple dimensions (Year = 2026 AND Region = 'US' AND Category = 'Electronics') -> Makes a sub-cube
dice_df = df[
    (df['Year'] == 2026) &
    (df['Region'] == 'US') &
    (df['Category'] == 'Electronics')
]

# 5. Pivot: Rotating axes (Category vs Region for the 2026 Slice)
pivot_df = pd.pivot_table(
    slice_df,
    values='Sales_kUSD',
    index='Category',
    columns='Region',
    aggfunc='sum',
).reset_index() # Flatten the MultiIndex for clean display

# Display preparation to prevent empty index columns in HTML
df_disp = df.head(6).reset_index(drop=True)
rollup_disp = rollup_df.head(4).reset_index(drop=True)

# Display sample results of the operations
display_side_by_side_datasets(
    [df_disp, rollup_disp, pivot_df],
    [
        "جدول ۱: پایگاه داده اصلی (شامل جزئیات فصل)",
        "تجمیع فصل‌ها به کل سال) Roll-up جدول ۲: عملیات",
        "چرخش دسته‌بندی و منطقه) Pivot جدول ۳: عملیات"
    ]
)

# =====
# SECTION 3: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
fig, axes = plt.subplots(2, 2, figsize=(19, 12), dpi=135)

# -----
# Plot 1: Roll-up View (Sales by Year and Region)
# -----
pivot_rollup = rollup_df.pivot(index='Year', columns='Region', values='Sales_kUSD')
pivot_rollup.plot(
    kind='bar',
    ax=axes[0, 0],
    color=['#2980b9', '#1abc9c'], # Classic Blue, Turquoise
```

```
edgecolor='#2c3e50',
linewidth=2.0
)

axes[0, 0].annotate(
    "Roll-Up (Aggregation):\nQuarters are completely hidden.\nWe aggregated up to the Year level\nto see the macro-trend of regions.",
    xy=(0.0, 1500), xytext=(-0.4, 2100),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0, 0].set_title('Figure 1: Roll-Up Operation (Yearly Sales by Region)', fontsize=14, fontweight='bold', pad=15)
axes[0, 0].set_ylabel('Total Sales (kUSD)', fontsize=13, fontweight='bold')
axes[0, 0].set_xlabel('Macro Dimension: Year', fontsize=13, fontweight='bold')
axes[0, 0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0, 0].tick_params(axis='x', rotation=0, labels=12)
axes[0, 0].set_xticklabels(axes[0, 0].get_xticklabels(), fontweight='bold')
axes[0, 0].legend(title='Region', frameon=True, fontsize=11)

# -----
# Plot 2: Drill-down View (Quarterly details for 2026 by Category)
# -----
pivot_drill = drilldown_df.pivot(index='Quarter', columns='Category', values='Sales_kUSD')
pivot_drill.plot(
    kind='line',
    marker='o',
    ax=axes[0, 1],
    color=[ '#2c3e50', '#5DADE2'], # Navy, Light Blue
    linewidth=3.0,
    markersize=10
)

axes[0, 1].annotate(
    "Drill-Down (Detailing):\nWe expanded 2026 back into Quarters\nto find out exactly WHEN 'Apparel'\nsales spiked.",
    xy=(1.0, 520), xytext=(1.5, 450),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[0, 1].set_title('Figure 2: Drill-Down Operation (2026 Quarterly Detail)', fontsize=14, fontweight='bold', pad=15)
axes[0, 1].set_ylabel('Quarterly Sales (kUSD)', fontsize=13, fontweight='bold')
axes[0, 1].set_xlabel('Micro Dimension: Quarter', fontsize=13, fontweight='bold')
axes[0, 1].grid(True, linestyle='--', alpha=0.5)
axes[0, 1].tick_params(axis='x', labels=12)
axes[0, 1].set_xticklabels(axes[0, 1].get_xticklabels(), fontweight='bold')
axes[0, 1].legend(title='Product Category', frameon=True, fontsize=11)

# -----
# Plot 3: Slice & Dice View (Dice: US Electronics in 2026)
# -----
axes[1, 0].bar(
    dice_df['Quarter'],
    dice_df['Sales_kUSD'],
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5,
)

axes[1, 0].annotate(
    "Slice & Dice (Multi-Filtering):\nWe sliced Year=2026. Then we diced\nRegion=US AND Category=Electronics.\nWe are looking at a very tiny sub-cube.",
    xy=(2.0, 200), xytext=(0.5, 300),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[1, 0].set_title('Figure 3: Slice & Dice (2026 + US + Electronics)', fontsize=14, fontweight='bold', pad=15)
axes[1, 0].set_ylabel('Isolated Sales (kUSD)', fontsize=13, fontweight='bold')
axes[1, 0].set_xlabel('Time Dimension', fontsize=13, fontweight='bold')
axes[1, 0].grid(axis='y', linestyle='--', alpha=0.5)
axes[1, 0].tick_params(axis='x', labels=12)
axes[1, 0].set_xticklabels(axes[1, 0].get_xticklabels(), fontweight='bold')

for p in axes[1, 0].patches:
    axes[1, 0].annotate(
        f"${int(p.get_height())}k",
        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center', va='center', xytext=(0, -12), textcoords='offset points',
        fontweight='bold', fontsize=12, color='white'
    )

# -----
# Plot 4: Pivot View (Rotated Category vs Region)
# -----
# Exclude the 'Category' column from plotting, use it as the index
pivot_df_plot = pivot_df.set_index('Category')

pivot_df_plot.plot(
    kind='barh',
    ax=axes[1, 1],
    color=[ '#2980b9', '#2c3e50'], # Classic Blue, Navy
    edgecolor='white',
    linewidth=1.5
)

axes[1, 1].annotate(
    "Pivot (Rotation):\nWe rotated the visual perspective.\nInstead of putting Regions on the axis,\nwe made Category the Y-axis.",
    xy=(1000, 1.0), xytext=(1200, 0.5),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
```

```

)

axes[1, 1].set_title('Figure 4: Pivot Operation (Rotated Perspective)', fontsize=14, fontweight='bold', pad=15)
axes[1, 1].set_xlabel('Total Aggregated Sales (kUSD)', fontsize=13, fontweight='bold')
axes[1, 1].set_ylabel('Dimension: Category', fontsize=13, fontweight='bold')
axes[1, 1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1, 1].tick_params(axis='y', labelsize=12)
axes[1, 1].set_yticklabels(axes[1, 1].get_yticklabels(), fontweight='bold')
axes[1, 1].legend(title='Dimension: Region', loc='upper right', frameon=True, fontsize=11)

# Clear top and right spines for a clean academic aesthetic
for ax in axes.flat:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.suptitle('OLAP Cubes: Visual Proof of the 5 Core Analytical Operations', fontsize=17, fontweight='bold', y=1.02)

plt.tight_layout(pad=3.0)
plt.show()

# =====
# SECTION 4: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه کاربردی (Strategic Implication):</b><br>"
    r"اجرا می‌شود (PowerBI یا Tableau نظیر) دقیقاً همان مکانیسمی است که در پس‌زمینه داشبوردهای هوش تجاری <b>عملیات ۵ گانه</b> این "
    r"<br>دست پیدا کنند (گراف ۲ - <b>Drill-down</b>) به چربی‌ترین داده‌های عملیاتی و فصلی (گراف ۱ - <b>Roll-up</b>) مدیران می‌توانند در چند ثانیه از گلی‌ترین گزارش‌های استراتژیک "
    r"را از بین برده SQL JOIN نیاز به کوئری‌های گُند، پیچیده و پُر از <i>(Data Cubes)</i> مکعب داده<i></i> با پیش‌محاسبه این ابعاد در ساختارهایی به نام OLAP موتورهای پردازشی "
    r"کند (گراف ۳) و در صورت نیاز، زاویه دید گزارش را <b>Slice & Dice</b> را خلق می‌نمایند. اگر مدیری فقط بخواهد یک گوشه خاص از بازار را ببیند، کافیسیت روی آن <b>Interactive Analytics</b> (تحلیل تعاملی) و تجربه "
)
display_academic_text(insight_conclusion, "insight")
```

تحلیل تعاملی فروشِ آمازون با عملیات پنج‌گانه OLAP

تیم تحلیل هوش تجاری (BI) آمازون برای بررسی فروش جهانی خود، داده‌ها را در یک **مکعب داده چندبُعدی (Data Cube)** شامل ابعادِ زمان، منطقه جغرافیایی و دسته‌بندی کالا ذخیره می‌کند. مدیران برای پاسخ به سوالات مختلف و چرخش در این فضا، **۵ عملیات اصلی OLAP** را روی این مکعب اجرا می‌کنند:

- فشردن (Roll-up):** تجمیع داده‌ها و حرکت به سطوح بالاتر انتزاع. مثلاً تبدیل فروش ماهانه به سالانه.
- جزئی‌سازی (Drill-down):** باز کردن داده‌های خلاصه‌شده و حرکت به سمت جزئیات. مثلاً شکستن فروش سالانه به فصلی برای کشف علت افت فروش.
- برش (Slice):** برش دادنِ مکعب روی یک بُعدِ خاص برای بررسی لایه دو بُعدی. مثلاً فیلتر کردن کل مکعب فقط برای سال ۲۰۲۶.
- تاس‌زدن (Dice):** انتخاب یک زیرمکعب کوچک‌تر با مشخص کردن فیلتر روی چندین بُعد هم‌زمان. مثلاً سال ۲۰۲۶، و منطقه آمریکا، و دسته الکترونیک.
- چرخش (Pivot):** چرخاندن محورهاى مکعب برای تغییر زاویه دید تحلیلی (مثلاً جابه‌جا کردن جای سطرها و ستون‌ها در گزارش).

جدول ۱: پایگاه داده اصلی (شامل جزئیات فصل)					(تجمیع فصل‌ها به کُل سال) Roll-up جدول ۲: عملیات			(چرخش دسته‌بندی و منطقه) Pivot جدول ۳: عملیات			
Year	Quarter	Region	Category		Year	Region	Sales_kUSD	Region	Category	Europe	US
2025	Q1	US	Electronics		2025	Europe	2662		Apparel	1254	1193
2025	Q1	US	Apparel		2025	US	2685		Electronics	1208	775
2025	Q1	Europe	Electronics		2026	Europe	2462				
2025	Q1	Europe	Apparel		2026	US	1968				
2025	Q2	US	Electronics								
2025	Q2	US	Apparel								

Figure 1: Roll-Up Operation (Yearly Sales by Region)

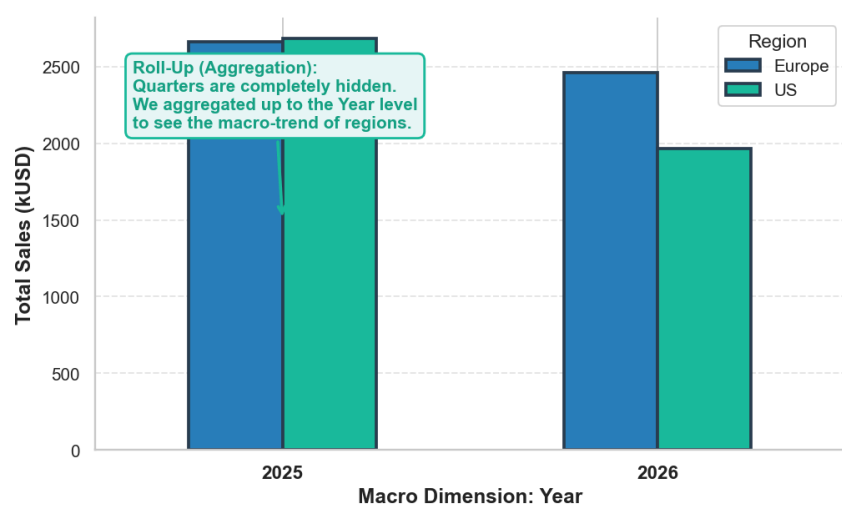


Figure 2: Drill-Down Operation (2026 Quarterly Detail)

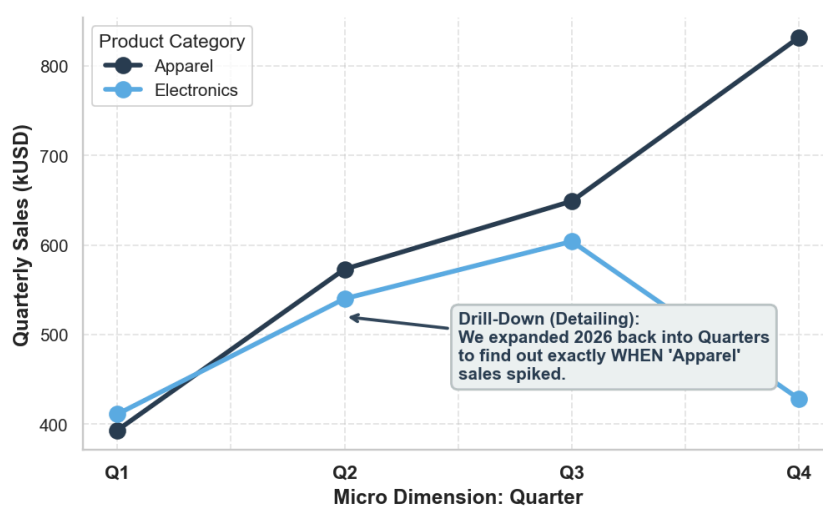


Figure 3: Slice & Dice (2026 US - Electronics)

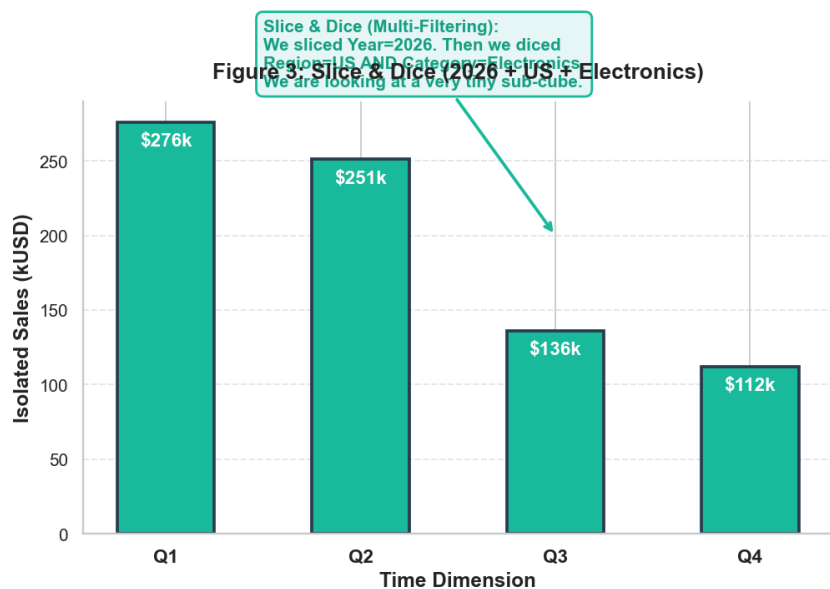
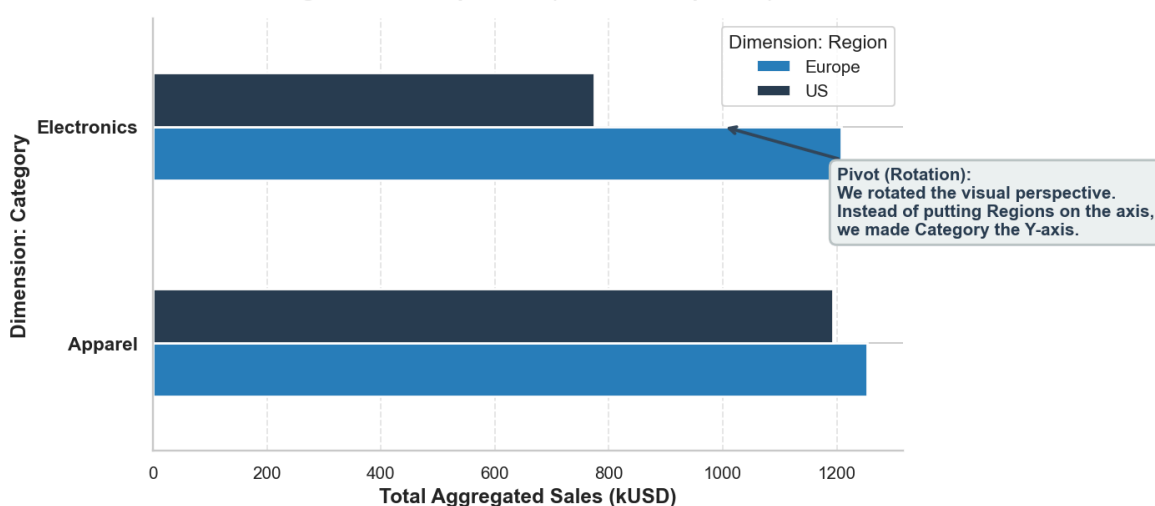


Figure 4: Pivot Operation (Rotated Perspective)



نتیجه کاربردی (Strategic Implication):

این عملیات هگانه دقیقاً همان مکانیسمی است که در پس‌زمینه داشبوردهای هوش تجاری (نظیر Tableau یا PowerBI) اجرا می‌شود. مدیران می‌توانند در چند ثانیه از کلی‌ترین گزارش‌های استراتژیک (Roll-up - گراف ۱) به جزئی‌ترین داده‌های عملیاتی و فصلی (Drill-down - گراف ۲) دست پیدا کنند. موتورهای پردازشی OLAP با پیش‌محاسبه این ابعاد در ساختارهایی به نام مکعب داده (Data Cubes)، نیاز به کوئری‌های گُند، پیچیده و پُر از JOIN در SQL را از بین برده و تجربه تحلیل تعاملی (Interactive Analytics) را خلق می‌نمایند. اگر مدیری فقط بخواهد یک گوشه خاص از بازار را ببیند، کافست روی آن Slice & Dice کند (گراف ۳) و در صورت نیاز، زاویه دید گزارش را Pivot نماید (گراف ۴).



توضیح مثال کد پایتون

در این مثال:

- ابتدا یک مجموعه داده چندبعدی شبیه‌سازی‌شده برای آزمون شامل ابعاد زمان (سال و فصل)، منطقه جغرافیایی و دسته‌بندی محصول همراه با معیار میزان فروش ساخته شد.
- پنج عملیات اصلی OLAP به صورت عملی با کتابخانه پانداس اجرا گردید:
 - عملیات Roll-up: با متد groupby داده‌های فصلی به سطح سالانه تجمیع شدند.
 - عملیات Drill-down: داده‌های سال ۲۰۲۳ به تفکیک فصول و دسته‌بندی‌ها بازسازی و با جزئیات بیشتر بررسی شدند.
 - عملیات Slice: فیلتر کردن دیتافریم بر روی بعد سال (سال برابر با ۲۰۲۳).
 - عملیات Dice: فیلتر هم‌زمان روی سه بعد سال ۲۰۲۳، منطقه آمریکا و دسته الکترونیک برای استخراج یک زیرمکعب مشخص.
 - عملیات Pivot: با تابع pivot_table محورهای نمایش داده تغییر یافت و جای سطر و ستون‌ها برای تحلیل جدید عوض شد.
- تفسیر نمودارها (Matplotlib): هر یک از چهار نمودار رسم‌شده، خروجی بصری یکی از عملیات OLAP را نشان می‌دهد تا تغییر زاویه دید و گرانولاریت داده‌ها به صورت ملموس قابل مشاهده باشد.

3.3.2 Indexing OLAP data: bitmap index and join index

در هر سلول مکعب می‌توان یک یا چند **سنجه (Measure)** نگهداری کرد. سنجه‌های جمع‌پذیر را می‌توان روی **مجموعه‌ای** از سلول‌ها با عمل جمع تجمیع کرد. برای میانگین نیز باید هم **مجموع** و هم **تعداد** را در اختیار داشته باشیم.

محاسبهٔ مجموع

(3.5)

$$S = \sum_{j=1}^m x_j$$

نمادهای فرمول

- S: مجموع مقادیر
- x_j : مقدار سنجه در مشاهده یا سلول j
- j: شاخص مقدار
- m: تعداد مقادیر مورد جمع

✖ مثال ۱ – مجموع فروش

صورت سؤال: فروش سه روز برابر ۱۲، ۱۸ و ۲۰ واحد است. مجموع فروش چقدر است؟

$$S = 12 + 18 + 20 = 50$$

نتیجه: مجموع فروش برابر ۵۰ واحد است.

محاسبهٔ میانگین

(3.6)

$$AVG = \frac{SUM}{COUNT}$$

نمادهای فرمول

- AVG: میانگین
- SUM: مجموع مقادیر
- COUNT: تعداد مقادیر

✖ مثال ۲ – میانگین

صورت سؤال: مجموع فروش ۵ رکورد برابر ۱۰۰ و تعداد رکوردها ۵ است. میانگین را محاسبه کنید.

$$AVG = \frac{100}{5} = 20$$

نتیجه: میانگین برابر ۲۰ است.

(3.7)

$$AVG_{new} = \frac{SUM_{old} + \Delta SUM}{COUNT_{old} + \Delta COUNT}$$

نمادهای فرمول

- AVG_{new} : میانگین پس از ورود داده‌های جدید
- SUM_{old} : مجموع داده‌های قبلی
- ΔSUM : مجموع داده‌های جدید
- $COUNT_{old}$: تعداد داده‌های قبلی
- $\Delta COUNT$: تعداد داده‌های جدید

✳ مثال ۳ – میانگین پس از ورود داده جدید

صورت سؤال: مجموع قبلی ۲۵۰ و تعداد قبلی ۱۲ است. یک رکورد جدید با مقدار ۲۵ اضافه شده است. میانگین جدید چقدر است؟

$$SUM_{new} = 250 + 25 = 275$$

$$COUNT_{new} = 12 + 1 = 13$$

$$AVG_{new} = \frac{275}{13} \approx 21.15$$

نتیجه: میانگین جدید تقریباً ۲۱٫۱۵ است.

ایندکس‌گذاری در OLAP

در پایگاه‌های داده **OLAP**، ایندکس‌گذاری با شتاب‌دادن به اجرای پرس‌وجوها، یافتن داده‌ها را سریع‌تر می‌کند. به همین دلیل، اغلب سیستم‌های انبار داده در کنار دیدگاه‌های محقق‌شده (**Materialized Views**)، از ساختارهای ایندکس نیز بهره می‌برند. دو نوع رایج ایندکس در OLAP عبارت‌اند از: **ایندکس بیت‌مپ (Bitmap Index)** و **ایندکس جویین (Join Index)**.

ایندکس بیت‌مپ (Bitmap Index): این روش در ابزارهای OLAP بسیار محبوب است، زیرا جست‌وجو در مکعب‌های داده را به شدت تسریع می‌کند. ایده اصلی آن است که به جای نگهداری فهرست ساده شناسه ردیف‌ها، به ازای هر مقدار ممکن در یک ویژگی، یک **بردار بیتی** جداگانه در نظر گرفته می‌شود. برای مثال، اگر ویژگی‌ای ۵ مقدار متفاوت داشته باشد، برای هر ردیف ۵ بیت نیاز است؛ بیت مربوط به مقدار همان ردیف برابر با ۱ و بقیه صفر می‌شوند. این روش فضای کمی اشغال می‌کند و به‌ویژه زمانی که تعداد مقادیر ممکن یک ویژگی کم باشد (مانند جنسیت یا وضعیت سفارش)، بهترین گزینه است.

ایندکس جویین (Join Index): در شمایمانند ستاره‌ای، برای پاسخ‌گویی به پرس‌وجوها معمولاً باید جدول واقعیت با جداول **پیوند (Join)** داده شود. انجام این کار برای هر پرس‌وجو هزینه‌بر و کند است. ایندکس جویین این مشکل را با

محاسبهٔ پیشاپیش نتیجهٔ پیوندها و ذخیره‌سازی آن حل می‌کند؛ در نتیجه، پرس‌وجوها بدون نیاز به محاسبهٔ مجدد Join، مستقیماً از ایندکس جوین استفاده می‌کنند. برای نمونه، اگر بخواهیم فروش «گوشی هوشمند» و «کامپیوتر» را در یک استان خاص ببینیم، یک ایندکس جوین از پیش‌آماده بین جدول فروش و جداول مکان و کالا، عملیات را بسیار سریع‌تر می‌کند.

این دو نوع ایندکس، به‌ویژه در پرس‌وجوهایی که هم‌زمان چند بُعد را درگیر می‌کنند، بسیار کارآمد هستند و از ابزارهای اصلی افزایش سرعت تحلیل‌های **OLAP** به شمار می‌روند.

⚠ هشدار

اگرچه **ایندکس بیت‌مپ** و **ایندکس جوین** سرعت پرس‌وجو را به‌شدت افزایش می‌دهند، اما **هزینهٔ به‌روزرسانی** آن‌ها در هنگام بارگذاری داده‌های جدید (ETL) می‌تواند بسیار بالا باشد. در طراحی انبار داده، همیشه بین **سرعت پاسخ‌گویی به پرس‌وجو** و **هزینهٔ نگهداری** ایندکس‌ها باید تعادل برقرار کرد. استفادهٔ بی‌رویه از ایندکس‌ها، به‌ویژه در جداول واقعیت حجیم، ممکن است فرآیند بارگذاری را به‌شدت کند کند.

🔧 مثال عملی: بهینه‌سازی سرعت کوئری‌های انبار داده با ایندکس بیت‌مپ و ایندکس جوین

در انبار داده بزرگ شرکت وال‌مارت (Walmart)، جدول واقعیت فروش شامل صدها میلیون سطر است. جستجو و فیلتر کردن روی این حجم عظیم از داده‌ها بدون ایندکس‌گذاری مناسب، ممکن است دقایق زیادی طول بکشد. سیستم‌های OLAP برای حل این چالش از دو نوع ایندکس‌گذاری بسیار سریع استفاده می‌کنند:

۱. ایندکس بیت‌مپ (Bitmap Index) – ویژه ویژگی‌های با تنوع پایین (Low Cardinality):

برای ویژگی‌هایی که تعداد مقادیر متمایز آن‌ها کم است (مثل جنسیت خریدار، منطقه جغرافیایی یا روش پرداخت)، به ازای هر مقدار ممکن، یک رشته از صفر و یک‌ها (آرایه بیتی) ساخته می‌شود. هر بیت نشان می‌دهد که آیا آن ویژگی در سطر مربوطه وجود دارد یا خیر. مثال: اگر روش پرداخت شامل سه حالت «کارت»، «نقد» و «کیف‌پول» باشد، برای ۱۰ سطر جدول، سه بردار ۱۰ بیتی ساخته می‌شود. اجرای فیلترهای ترکیبی (مثلاً پرداخت با کارت در منطقه شمال) با عملیات منطقی بیت‌به‌بیت (AND و OR) در سطح سخت‌افزار با سرعت خیره‌کننده انجام می‌شود.

۲. ایندکس جوین (Join Index) – حذف عملیات سنگین اتصال جداول (JOIN):

در شمای ستاره‌ای، اتصال جدول واقعیت به جداول ابعاد در زمان اجرای کوئری بسیار پرهزینه است. ایندکس جوین، ارتباط بین کلید جدول ابعاد و شماره سطرهای (RowID) جدول واقعیت را از قبل محاسبه و ذخیره می‌کند. هنگام کوئری‌گیری، سیستم بدون نیاز به اجرای عملیات سنگین JOIN، مستقیماً به سطرهای مورد نظر در جدول واقعیت دسترسی پیدا می‌کند.

نتیجه: استفاده از ایندکس بیت‌مپ حافظه مصرفی را به شدت کاهش می‌دهد و ایندکس جوین سرعت اجرای کوئری‌های ترکیبی OLAP را تا ده برابر افزایش می‌دهد.

خروجی: بهینه‌سازی حافظه و افزایش سرعت فیلترهای چندبعدی در سیستم‌های تحلیل آنی

In [11]:

```
display_academic_text(
    "⚡ (Join Index) و ایندکس جوین (Bitmap) بهینه‌سازی سرعت کوئری‌های انبار داده با ایندکس بیت‌مپ",
    "main_title"
)

display_academic_text(
    " شامل صدها میلیون سطر است. جستجو و فیلتر کردن روی این حجم عظیم از داده‌ها (Sales Fact Table) جدول واقعیت فروش (Walmart) در انبار داده بزرگ شرکت وال‌مارت",
    "برای حل این چالش و رساندن زمان اجرای داشبوردها به زیر ۱ ثانیه، از دو نوع ایندکس‌گذاری بسیار سریع استفاده می‌کنند OLAP بدون ایندکس‌گذاری مناسب، ممکن است دقایق زیادی طول بکشد. موتورهای پردازشی",
    "paragraph"
)

display_academic_text("1. ", "ویژه ویژگی‌های با تنوع پایین – (Bitmap Index) ایندکس بیت‌مپ")
display_academic_text(
```

```
display_side_by_side_datasets(
```

```
[df_query_result, df_join_index],
[
    "رڪورڊ خروجي ڪوٽري ٻيٽي (بدون نياز به اسڪن ٿل جدول)",
    " (RowID به StoreKey) نگاشت فيزيڪي ايندڪس جوين"
]
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
display_academic_text(
    "Bitmap مرحله سوم: مصورسازی تحلیلی - ساختار ماتریسی",
    "sub_title"
)

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

# -----
# Subplot 1: Heatmap Visualizing Bitmap Index Matrix
# -----
combined_bitmaps = pd.concat([bitmap_region, bitmap_payment], axis=1)

axes[0].imshow(
    combined_bitmaps.T,
    cmap='Blues', # Academic Blue palette (Dark for 1, Light for 0)
    aspect='auto',
    vmax=1.2 # Slightly increase vmax so '1' isn't too aggressively dark
)

axes[0].set_title('Figure 1: Bitmap Index Matrix Architecture', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Fact Table RowID (Data Records)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Indexed Filter Attributes', fontsize=13, fontweight='bold')

axes[0].set_yticks(range(len(combined_bitmaps.columns)))
axes[0].set_yticklabels(combined_bitmaps.columns, fontweight='bold', fontsize=11)
axes[0].set_xticks(range(n_rows))
axes[0].set_xticklabels(range(n_rows), fontweight='bold')

# Adding text annotations inside heatmap cells
for i in range(len(combined_bitmaps.columns)):
    for j in range(n_rows):
        val = combined_bitmaps.iloc[j, i]
        axes[0].text(
            j, i, str(val),
            ha='center', va='center',
            color='white' if val == 1 else '#2c3e50',
            fontweight='bold',
            fontsize=12
        )

# Intelligent Annotations to explain Bitwise AND
axes[0].annotate(
    "Bitwise Intersection (AND):\nlf a user queries for 'North' AND 'Card',\nthe CPU instantly compares the rows.\nRow 0: (1 AND 1) = 1 (Match!)\nRow 1: (0 AND 0) = 0 (No Match)",
    xy=(0.5, 0.5), xytext=(1.5, 2.5),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

# -----
# Subplot 2: Join Index Mapping Visualization
# -----
store_keys = list(join_index_dict.keys())
row_counts = [len(rows) for rows in join_index_dict.values()]

bars = axes[1].bar(
    [str(k) for k in store_keys],
    row_counts,
    color=[ '#2980b9', '#1abc9c'], # Classic Blue, Turquoise
    edgecolor='#2c3e50',
    width=0.45,
    linewidth=2.0
)

axes[1].annotate(
    "Pre-Computed Pointers (Join Index):\nlf a user filters by StoreKey 101,\nthe database doesn't join tables.\nIt already knows exactly which 6 RowIDs\nin the Fact Table belong to Store 101.",
    xy=(0.0, 3.0), xytext=(0.5, 4.0),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[1].set_title('Figure 2: Join Index Architecture (Pre-Mapped Row Pointers)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Dimension Table (StoreKey)', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Number of Linked Fact Rows', fontsize=13, fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].set_ylim(0, 7) # Headroom
axes[1].set_xticklabels([str(k) for k in store_keys], fontweight='bold', fontsize=12)

# Adding value annotations to bars
for bar, count in zip(bars, row_counts):
    axes[1].text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.15,
        f'{count} Pointers',
        ha='center',
        fontweight='bold',
        fontsize=12,
        color='#2c3e50'
    )

# Removing top and right spines for academic aesthetic
for ax in axes:
```

```
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout(pad=2.0)
plt.show()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
insight_text = (
    r"<b>BI (Strategic Implication):</b><br>"
    r"<b>نمی‌کند</b> هنگامی که مدیران در داشبوردهای مدیریتی روی فیلترهای مختلف (مانند منطقه یا روش پرداخت) کلیک می‌کنند، پایگاه داده رکوردهای متنی را بررسی "
    r"<i>(CPU)</i> عملیات منطقی سطح پایین سخت‌افزاری در پردازنده اصلی<i></i> فیلتر کردن ده‌ها میلیون رکورد به یک <b>(Bitmap Indexes)</b> بردارهای فشردهٔ بیتی<b> بلکه با تبدیل مقادیر به "
    r"<b>(Join Index)</b> ایندکس جویین<b> همچنین، استفاده از "
    r"<b>(Pointers)</b> با پیش‌ذخیره‌سازی نگاشت‌ها <b>(Real-Time)</b> و تجربهٔ گزارش‌گیری را کاملاً آبی"
    r"<b>می‌سازد"
)
display_academic_text(insight_text, "insight")
```

بهینه‌سازی سرعت کوئری‌های انبار داده با ایندکس بیت‌مپ (Bitmap) و ایندکس جویین (Join Index)

در انبار داده بزرگ شرکت وال‌مارت (Walmart)، جدول واقعیت فروش (Sales Fact Table) شامل صدها میلیون سطر است. جستجو و فیلتر کردن روی این حجم عظیم از داده‌ها بدون ایندکس‌گذاری مناسب، ممکن است دقایق زیادی طول بکشد. موتورهای پردازشی OLAP برای حل این چالش و رساندن زمان اجرای داشبوردها به زیر ۱ ثانیه، از دو نوع ایندکس‌گذاری بسیار سریع استفاده می‌کنند:

۱. ایندکس بیت‌مپ (Bitmap Index) – ویژه ویژگی‌های با تنوع پایین:

برای ستون‌هایی که تعداد مقادیر متمایز آن‌ها کم است (Low Cardinality - مثل منطقه یا روش پرداخت)، به ازای هر مقدار، یک رشته از صفر و یک‌ها (آرایه بیتی یا Bitmap) ساخته می‌شود. اجرای فیلترهای ترکیبی (Multiple Filters) با عملیات منطقی بیت‌به‌بیت (Bitwise AND/OR) در سطح سخت‌افزار CPU با سرعت خیره‌کننده انجام می‌شود.

۲. ایندکس جویین (Join Index) – حذف عملیات سنگین اتصال جداول:

به جای اینکه هنگام پرس‌وجو جداول سنگین Dimension را با Fact متصل کنیم، ارتباط بین کلید جدول ابعاد (Dimension Key) و شماره سطرهای (RowIDs) جدول واقعیت از قبل محاسبه و ذخیره می‌شود. بدین ترتیب در زمان کوئری‌گیری، موتور دیتابیس مستقیماً به سراغ سطرهای مشخص‌شده می‌پرد و نیازی به عملیات پرهزینه JOIN ندارد.

مرحله نخست: ساخت جدول واقعیت و تولید ماتریس‌های Bitmap

Transactions (جدول واقعیت فروش)				Region (مبتهی بر) Bitmap ماتریس				Payment (مبتهی بر) Bitmap ماتریس		
RowID	Region	PaymentMethod	Amount	RowID	North	South	East	RowID	Card	Cash
0	North	Card	1500	0	1	0	0	0	1	0
1	South	Cash	2000	1	0	1	0	1	0	1
2	North	Card	500	2	1	0	0	2	1	0
3	East	Card	3000	3	0	0	1	3	1	0
4	North	Cash	1200	4	1	0	0	4	0	1
5	South	Card	4500	5	0	1	0	5	1	0
6	East	Cash	900	6	0	0	1	6	0	1
7	South	Card	2500	7	0	1	0	7	1	0
8	North	Cash	1800	8	1	0	0	8	0	1
9	East	Card	3500	9	0	0	1	9	1	0

مرحله دوم: شبیه‌سازی جستجوی باینری (Bitwise AND) و نگاشت ایندکس جویین

شبه سازی کوئری: 'Region = 'North' AND Payment = 'Card

بردار استخراج شده North از RAM: [np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0)]

بردار استخراج شده Card از RAM: [np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(1)]

نتیجه عملیات سخت افزاری AND: [np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0)] (فقط سطرهایی که در هر دو مورد 1 هستند استخراج می شوند)

رکورد خروجی کوئری بی تی (بدون نیاز به اسکن گِل جدول)

RowID	Region	PaymentMethod	Amount_USD
0	North	Card	150
2	North	Card	50

نگاشت فیزیکی ایندکس جوی (StoreKey به RowID)

Dimension StoreKey	Pre-Mapped Fact RowIDs
101	[0, 2, 3, 5, 7, 9]
102	[1, 4, 6, 8]

مرحله سوم: مصورسازی تحلیلی - ساختار ماتریسی Bitmap

Figure 1: Bitmap Index Matrix Architecture

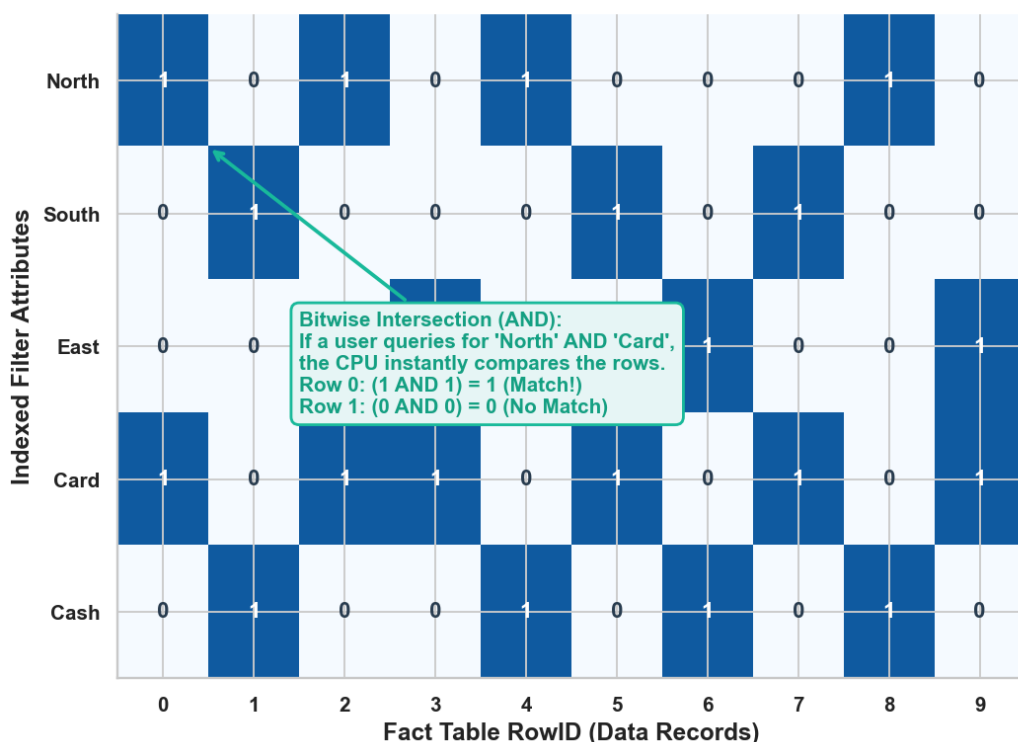
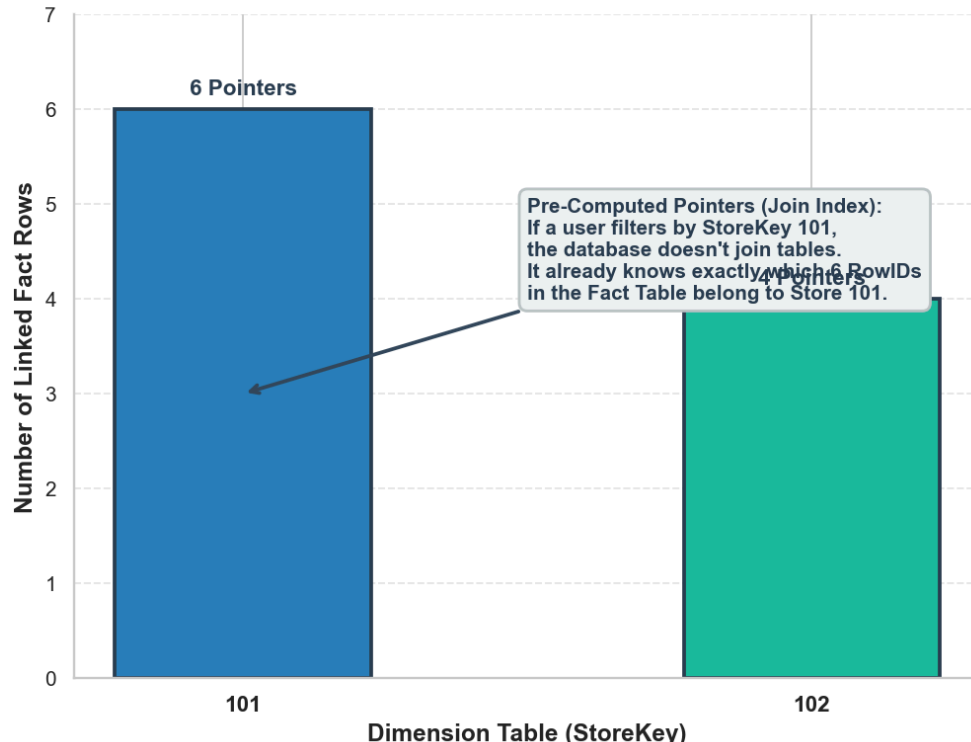


Figure 2: Join Index Architecture (Pre-Mapped Row Pointers)



نتیجه استراتژیک و مکانیسم پشت صحنه سیستم های BI (Strategic Implication):

هنگامی که مدیران در داشبوردهای مدیریتی روی فیلترهای مختلف (مانند منطقه یا روش پرداخت) کلیک می کنند، پایگاه داده رکوردهای متنی را بررسی نمی کند. بلکه با تبدیل مقادیر به بردارهای فشرده بی تی (Bitmap Indexes)، فیلتر کردن ده ها میلیون رکورد به یک عملیات منطقی سطح پایین سخت افزاری در پردازنده اصلی (CPU) تنزل می یابد که زمان اجرای آن نزدیک به صفر است.

همچنین، استفاده از ایندکس جوی (Join Index) با پیش ذخیره سازی نگاشت ها (Pointers)، گران ترین و پرهزینه ترین عملیات پایگاه داده (یعنی JOIN های ران تایم بین جداول عظیم) را از دوش سرورهای تحلیلی OLAP برداشته و تجربه گزارش گیری را کاملاً آنی (Real-Time) می سازد.

توضیح مثال کد پایتون

در این مثال:

- ما یک جدول واقعیت ۱۰ سطر شامل اطلاعات فروش وال مارت همراه با دو ویژگی با کاردینالیتیه پایین (Region و PaymentMethod) ایجاد کردیم.
- پیاده سازی ایندکس بیت مپ (Bitmap Index): با تابع create_bitmap_index به ازای هر مقدار مجزا، یک بردار بی تی (صفر و یک) ساخته شد.
- اجرای کوئری سریع بیت به بیت: با استفاده از تابع np.bitwise_and، فیلتر ترکیبی منطقه شمال و پرداخت با کارت در سطح بردار بی تی اجرا شد و سطرهای منطبق بدون پیمایش کل جدول استخراج گردیدند.

- شبیه‌سازی ایندکس جویین (Join Index): رابطه‌ی مستقیم بین کلید جدول ابعاد (StoreKey) و شناسه سطرهای جدول واقعیت (RowID) به صورت یک نگاشت مستقیم ذخیره شد تا نیاز به اجرای عملیات سنگین JOIN حذف شود.
- تفسیر نمودارهای Matplotlib:
 - نمودار سمت چپ (Heatmap): ماتریس بیتی صفر و یک را برای مقادیر مختلف دو ویژگی به صورت بصری نشان می‌دهد.
 - نمودار سمت راست (Bar Chart): نگاشت مستقیم ایندکس جویین بین کلید ابعاد و تعداد سطرهای مربوطه در جدول واقعیت را به تصویر می‌کشد.

3.3.3 Storage implementation: column-based databases

ریاضیات – تجمیع و عملیات OLAP

در عملیات **OLAP**، تغییر سطح جزئیات یا انتخاب بخشی از فضای داده با عملیات‌هایی مانند **بالاروی (Roll-up)**، **پایین‌روی (Drill-down)**، **برش (Slice)** و **قطعه‌بندی (Dice)** انجام می‌شود. از دید ریاضی، بخش مهم این عملیات‌ها نحوهٔ تجمیع سنجه‌هاست.

تجمیع یک سنجۀ جمع‌پذیر

(3.8)

$$S_{agg} = \sum_{j=1}^m x_j$$

نمادهای فرمول

- S_{agg} : مقدار سنجه پس از تجمیع
- x_j : مقدار سنجه در عضو j
- j : شاخص مقدار
- m : تعداد مقادیر مورد تجمیع

✪ مثال ۱ – بالاروی

صورت سؤال: فروش سه روز یک منطقه به‌ترتیب ۱۰، ۱۵ و ۲۰ است. فروش تجمیع‌شدهٔ این بازه را به‌دست آورید.

$$S_{agg} = 10 + 15 + 20 = 45$$

نتیجه: فروش تجمیع‌شده برابر ۴۵ است.

نمایش ریاضی برش (Slice)

(3.9)

$$X_{slice} = \{x \in X \mid d(x) = v\}$$

نمادهای فرمول

- X_{slice} : مجموعهٔ رکوردهای باقی‌مانده پس از Slice
- X : مجموعهٔ اصلی داده
- x : یک رکورد از مجموعهٔ X
- $d(x)$: مقدار بُعد مورد برش برای رکورد x

✳ مثال ۲ – برش

صورت سؤال: فرض کنید از میان رکوردها فقط رکوردهایی را می‌خواهیم که مقدار بُعد «سال» برابر ۱۴۰۵ باشد.

$$X_{\text{slice}} = \{x \in X \mid \text{Year}(x) = 1405\}$$

نتیجه: فقط رکوردهایی که شرط سال را دارند در نتیجه باقی می‌مانند.

نمایش ریاضی قطعه‌بندی (Dice)

(3.10)

$$X_{\text{dice}} = \{x \in X \mid d_1(x) \in A, d_2(x) \in B\}$$

نمادهای فرمول

- X_{dice} : مجموعه رکوردهای باقی‌مانده پس از Dice
- X : مجموعه اصلی داده
- x : یک رکورد از مجموعه X
- $d_1(x)$: مقدار بُعد اول برای رکورد x
- $d_2(x)$: مقدار بُعد دوم برای رکورد x
- A : مجموعه مقادیر مجاز بُعد اول
- B : مجموعه مقادیر مجاز بُعد دوم

✳ مثال ۳ – قطعه‌بندی

صورت سؤال: اگر بخواهیم فقط محصولات موجود در مجموعه {لپ‌تاپ، تبلت} و مناطق موجود در {شمال، جنوب} را نگه داریم، چه شرطی اعمال می‌شود؟

$$X_{\text{dice}} = \{x \in X \mid \text{Product}(x) \in \{\text{Laptop, Tablet}\}, \text{Region}(x) \in \{\text{North, South}\}\}$$

نتیجه: نتیجه فقط رکوردهایی را نگه می‌دارد که هر دو شرط را داشته باشند.

پایگاه داده مبتنی بر ستون

پایگاه‌های داده مبتنی بر ستون، نوعی از سیستم‌های مدیریت پایگاه‌داده (DBMS) هستند که برخلاف پایگاه‌های سنتی، داده‌ها را بر اساس ستون‌ها ذخیره و سازمان‌دهی می‌کنند، نه بر اساس ردیف‌ها.

دلیل شکل‌گیری این رویکرد، یک مشاهده ساده در پردازش تحلیلی است: در بسیاری از کاربردها، جدول واقعیت (Fact Table) می‌تواند ده‌ها یا حتی صدها ستون داشته باشد، درحالی‌که یک پرس‌وجوی OLAP معمولاً فقط روی چند ستون خاص و بخش کوچکی از رکوردها تجمیع یا محاسبه انجام می‌دهد. اگر داده‌ها به شکل ردیف‌به‌ردیف ذخیره شده باشند، برای پاسخ به همین

پرس‌وجوی محدود نیز باید تمام ردیف‌ها اسکن شوند، درحالی‌که تنها بخش کوچکی از هر ردیف واقعاً استفاده می‌شود. همین ناکارآمدی، انگیزه طراحی یک شمای ذخیره‌سازی کارآمدتر برای داده‌های OLAP را فراهم کرد.

در پایگاه‌های ستونی، مقادیر همه رکوردهای مربوط به یک ستون، به‌طور پیوسته و کنار هم در بلوک‌های ذخیره‌سازی نگهداری می‌شوند و ترتیب رکوردها در تمام ستون‌ها یکسان می‌ماند. این ساختار باعث می‌شود برای پاسخ به یک پرس‌وجوی تحلیلی، تنها ستون‌های موردنیاز خوانده شوند، نه کل جدول؛ همین ویژگی، سرعت اجرای پرس‌وجو را به‌ویژه در مجموعه‌داده‌های بزرگ به‌طور چشمگیری افزایش می‌دهد.

چند ویژگی کلیدی این نوع پایگاه‌داده عبارت‌اند از:

ذخیره‌سازی ستونی (Columnar Storage): هر ستون به‌عنوان یک واحد ذخیره‌سازی مجزا نگهداری می‌شود.

تکنیک‌های فشرده‌سازی (Compression Techniques): از آنجا که مقادیر درون یک ستون معمولاً یکنواخت‌تر و مشابه‌ترند، این پایگاه‌ها می‌توانند با روش‌های فشرده‌سازی کارآمد، هم فضای ذخیره‌سازی را کاهش دهند و هم عملکرد پرس‌وجو را بهبود بخشند.

کارایی در پردازش تحلیلی (Analytical Processing Efficiency): این ساختار به‌ویژه زمانی که عملیات رایج شامل تجمیع و تحلیل روی زیرمجموعه‌ای محدود از ستون‌ها باشد، عملکردی بسیار بهتر از پایگاه‌های داده ردیفی از خود نشان می‌دهد.

مثال عملی: مقایسه کارایی پایگاه داده ستونی ClickHouse و BigQuery در تحلیل‌های OLAP

در سیستم‌های پردازش آنلاین تراکنش (OLTP) مانند دیتابیس ثبت سفارشات، داده‌ها به‌صورت سطری (Row-oriented) ذخیره می‌شوند؛ یعنی تمام مشخصات یک کاربر (شناسه، نام، آدرس، ایمیل و مبلغ خرید) پشت سر هم روی دیسک قرار می‌گیرند. این ساختار برای ثبت یک سفارش جدید فوق‌العاده است. اما در تحلیل‌های بزرگ انبار داده (OLAP)، ذخیره‌سازی سطری با یک چالش بزرگ مواجه می‌شود.

چالش تحلیل در دیتابیس سطری:

فرض کنید می‌خواهیم مجموع مبلغ کل خرید ۱۰۰ میلیون کاربر را محاسبه کنیم. در دیتابیس سطری، سیستم مجبور است تمام اطلاعات غیرضروری مثل نام، آدرس و ایمیل را هم از روی دیسک بخواند تا فقط به ستون مبلغ برسد! این یعنی خواندن ۹۰٪ داده‌های اضافی و کندی شدید سیستم.

راه‌حل – پایگاه داده مبتنی بر ستون (Columnar Database):

در پایگاه‌های داده تحلیلی مانند ClickHouse، Google BigQuery یا Amazon Redshift، تمام مقادیر یک ستون کنار هم روی دیسک ذخیره می‌شوند.

مزایای کلیدی معماری ستونی:

۱. خواندن هوشمند دیسک (Selective Read): برای محاسبه مجموع خریه‌ها، سیستم فقط فایل مربوط به ستون مبلغ خرید را می‌خواند و بقیه ستون‌ها اصلاً بارگذاری نمی‌شوند.

۲. نرخ فشرده‌سازی فوق‌العاده بالا: چون تمام داده‌های یک ستون از یک جنس هستند (مثلاً همه عدد یا همه تاریخ)، الگوریتم‌های فشرده‌سازی حجم ذخیره‌سازی را تا ۱۰ برابر کاهش می‌دهند.

نتیجه: معماری ستونی سرعت اجرای کوئری‌های تحلیلی و تجمیعی روی حجم‌های پتابایتی داده را تا ۱۰۰ برابر افزایش می‌دهد.

خروجی: کاهش حجم داده‌های خوانده‌شده از دیسک و افزایش چشمگیر سرعت محاسبات OLAP

```
display_academic_text(
    "👑 پادشاهی می‌کنند؟ OLAP در تحلیل‌های (ClickHouse مثل) تفاوت معماری: چرا دیتابیس‌های ستونی",
    "main_title"
)

display_academic_text(
    " روی هارد دیسک ذخیره می‌شوند؛ <b>سطری (Row-oriented)</b> مانند دیتابیس ثبت سفارشات یک فروشگاه، داده‌ها به‌صورت (OLTP) در سیستم‌های پردازش آنلاین تراکنش "
    "، یعنی تمامِ مشخصاتِ یک کاربر (آیدی، سن، ایمیل، خرید) پشت سر هم قرار می‌گیرند. این برای خواندن و نوشتن سریع «یک تک‌مشتري» عالی است"
    "،. ذخیره‌سازی سطرى با چالشى بزرگ مواجهه می‌شود، (OLAP) اما در تحلیل‌های گِلان انبار داده"
    "paragraph"
)

display_academic_text("I/O Bottleneck): چالش مهلك در دیتابیس سطرى", "sub_title")
display_academic_text(
    " فرض کنید می‌خواهیم مجموع مبلغِ کُل خریدِ ۱۰۰ میلیون کاربر را محاسبه کنیم. در دیتابیس سطرى، سیستم مجبور است تمام اطلاعات غیرضرورى مثل "
    "،. بیاورد تا فقط به ستون «مبلغ» برسد! این یعنی خواندن ۸۰٪ داده‌های اضافی و کُندي شديد RAM را هم از روی دیسک به حافظهٔ <i>نام، آدرس و ایمیل</i>"
    "paragraph"
)

display_academic_text("Column-oriented): راه‌حل جادویی: پایگاه داده مبتنى بر ستون", "sub_title")
display_academic_text(
    "<br>".تمام مقاديرِ یک ستون در فایل‌های مجزا و کنار هم روی دیسک ذخیره می‌شوند، Amazon Redshift یا Google BigQuery، ClickHouse در پایگاه‌های داده تحلیلى مانند"
    "<br>".(به شدت کاهش می‌یابد I/O) سیستم فقط و فقط فایل مربوط به ستون «مبلغ خرید» را می‌خواند <b>(Selective Read)</b> خواندن هوشمند دیسک<b> ۱. <br>"
    "،. استفاده کرده و داده‌ها را تا ۱۰ برابر بیشتر فشرده کند (SIMD) می‌تواند از دستوراتِ بردارى CPU، چون تمام داده‌های یک ستون از یک جنس (مثلاً عدد) هستند <b>(Vectorization)</b> فشرده‌سازی و بردارى‌سازی<b> ۲. <br>"
    "paragraph"
)

# =====
# SECTION 2: DATA GENERATION AND ARCHITECTURAL MODELING
# =====
# Increase size to 1,000,000 to make the benchmark more realistic
n_rows = 1000000

# Generating Data Columns
ids = np.arange(n_rows)
ages = np.random.randint(18, 70, size=n_rows)
cities = np.array([f'City_{i}' for i in np.random.randint(1, 20, size=n_rows)])
emails = np.array([f'user_{i}@domain.com' for i in range(n_rows)])
purchases = np.random.uniform(10.0, 500.0, size=n_rows)

# Step 2: Represent Data in Row-Oriented Layout (List of Tuples / Rows)
# Memory structure: [(id1, age1, city1, email1, purchase1), (id2, age2...), ...]
row_oriented_db = list(zip(ids, ages, cities, emails, purchases))

# Step 3: Represent Data in Column-Oriented Layout (Dict of Arrays / Columns)
# Memory structure: {'ID': [id1, id2...], 'Purchase': [purchase1, purchase2...], ...}
column_oriented_db = {
    'ID': ids,
    'Age': ages,
    'City': cities,
    'Email': emails,
    'Purchase_Amount': purchases,
}

# Display a preview of the dataset to the user securely
df_preview = pd.DataFrame(column_oriented_db).head(5).reset_index(drop=True)
display_side_by_side_datasets(
    [df_preview],
    ["(رکورد و ۵ ستون {n_rows:,} شامل) نمونه پایگاه داده شبیه‌سازی‌شده" f])
)

# =====
# SECTION 3: BENCHMARKING QUERY PERFORMANCE
# =====
display_academic_text("SUM(Purchase_Amount)</code> اجرای کوئرى تحلیلى: محاسبه کُل درآمد", "sub_title")

# -----
# Approach A: Row-Oriented Engine
# Must iterate through entire row tuples and unpack everything just to reach the 5th element
# -----
start_time = time.time()
row_sum = 0.0
elements_scanned_row = 0

# Simulating CPU loading the full row into cache
for row in row_oriented_db:
    row_sum += row[4] # Access 5th element of every row
    elements_scanned_row += 5 # The CPU had to read all 5 elements from memory

row_time = (time.time() - start_time) * 1000 # Convert to ms

# -----
# Approach B: Column-Oriented Engine
# Direct vector access to Purchase_Amount array only (SIMD Processing)
# -----
start_time = time.time()
col_sum = np.sum(column_oriented_db['Purchase_Amount']) # NumPy uses C-level vectorization
elements_scanned_col = len(column_oriented_db['Purchase_Amount'])

col_time = (time.time() - start_time) * 1000 # Convert to ms

speedup_factor = row_time / col_time

display_academic_text(
    f"<b>رکورد {n_rows:,}</b>: نتایج بنچمارک برای<b> <br>"
    f"<b>واحد {elements_scanned_row:,}</b>: تعداد عناصر خوانده‌شده | <b>میلی‌ثانیه {row_time:.2f}</b>: موتور سطرى (ستنى) ● f"
    f"<b>واحد {elements_scanned_col:,}</b>: تعداد عناصر خوانده‌شده | <b>میلی‌ثانیه {col_time:.2f}</b>: موتور ستونى (تحلیلى) ● f"
    f"<b>سرعت {speedup_factor:.1f}</b> برابر سریع‌تر <b>موتور ستونى </b>: بهبود عملکرد<b> 🌟 f"
    "insight"
)
)
```

```
# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

engines = ['Row-Oriented DB\n(e.g. MySQL/PostgreSQL)', 'Column-Oriented DB\n(e.g. ClickHouse/BigQuery)']

# -----
# Subplot 1: Memory Elements Scanned Comparison
# -----

scanned_counts = [elements_scanned_row, elements_scanned_col]

bars1 = axes[0].bar(
    engines,
    scanned_counts,
    color=['#e74c3c', '#1abc9c'], # Red for inefficient, Turquoise for efficient
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[0].annotate(
    "I/O Waste:\nThe Row engine had to scan 5 million elements\n(loading Names, Cities, etc.) just to sum 1 column.\nThe Column engine strictly scanned 1 million.",
    xy=(0.0, elements_scanned_row/2), xytext=(0.5, elements_scanned_row*0.7),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title(f'Figure 1: Disk/Memory Elements Scanned ({n_rows:,} Rows)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Number of Memory Elements Read (Lower is Better)', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(engines, fontweight='bold', fontsize=12)

# Format y-axis to show millions cleanly
def millions_formatter(x, pos):
    return f'{x / 1e6:.1f}M'
axes[0].yaxis.set_major_formatter(plt.FuncFormatter(millions_formatter))

for bar in bars1:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval + (max(scanned_counts) * 0.02),
        f'{yval/1e6:.1f}M Units',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# -----
# Subplot 2: Query Execution Time Comparison
# -----

exec_times = [row_time, col_time]

bars2 = axes[1].bar(
    engines,
    exec_times,
    color=['#e74c3c', '#1abc9c'],
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[1].annotate(
    "Vectorization (SIMD):\nBecause data in a column is contiguous in RAM,\nthe CPU can process chunks of numbers simultaneously,\nresulting in a massive speedup.",
    xy=(1.0, col_time*2), xytext=(1.0, row_time*0.5),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[1].set_title('Figure 2: Query Execution Time Comparison', fontsize=14, fontweight='bold', pad=15)
axes[1].set_ylabel('Execution Time (Milliseconds) - Lower is Better', fontsize=13, fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].set_xticklabels(engines, fontweight='bold', fontsize=12)

for bar in bars2:
    yval = bar.get_height()
    axes[1].text(
        bar.get_x() + bar.get_width() / 2,
        yval + (max(exec_times) * 0.02),
        f'{yval:.1f} ms',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# Clear top and right spines for a clean academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====

insight_conclusion = (
    r"<b>نتیجه استراتژیک (Strategic Implication):</b><br>"
    r"و بهره‌گیری از (I/O bottleneck کاهش چشمگیر) با حذفِ کامل عملیات خواندن داده‌های غیرضروری از روی دیسک، معماری ستونی (Columnar Architecture) روی حجم‌های پتابایتی داده را به‌طرز دیوانه‌واری افزایش می‌دهد (SUM و AVG نظیر) سرعت اجرای کوئری‌های تحلیلی و تجمیعی، CPU در (Vectorized Execution / SIMD) پردازش برداری<br>"
    r"<b>Google BigQuery</b>، <b>Amazon Redshift</b> و <b>Snowflake</b> محسوب می‌گردد و دلیلِ اصلیِ <b>Snowflake</b>"
    r"<b>سرعتِ بالای داشبوردها در مواجهه با میلیاردها رکورد تراکنش است</b>."
)

display_academic_text(insight_conclusion, "insight")
```



تفاوت معماری: چرا دیتابیس‌های ستونی (مثل ClickHouse) در تحلیل‌های OLAP پادشاهی می‌کنند؟

در سیستم‌های پردازش آنلاین تراکنش (OLTP) مانند دیتابیس‌های ثبت سفارشات یک فروشگاه، داده‌ها به صورت **سطری (Row-oriented)** روی هارد دیسک ذخیره می‌شوند؛ یعنی تمام مشخصات یک کاربر (آیدی، سن، ایمیل، خرید) پشت سر هم قرار می‌گیرند. این برای خواندن و نوشتن سریع «یک تک‌مشتري» عالی است، اما در تحلیل‌های گُلانِ انبار داده (OLAP)، ذخیره‌سازی سطری با چالشی بزرگ مواجه می‌شود.

چالش مهلك در دیتابیس سطری (I/O Bottleneck):

فرض کنید می‌خواهیم مجموع مبلغ کُل خرید ۱۰۰ میلیون کاربر را محاسبه کنیم. در دیتابیس سطری، سیستم مجبور است تمام اطلاعات غیرضروری مثل نام، آدرس و ایمیل را هم از روی دیسک به حافظه RAM بیاورد تا فقط به ستون «مبلغ» برسد! این یعنی خواندن ۸۰٪ داده‌های اضافی و کُندی شدید.

راه حل جادویی: پایگاه داده مبتنی بر ستون (Column-oriented):

در پایگاه‌های داده تحلیلی مانند Google BigQuery، ClickHouse یا Amazon Redshift، تمام مقادیر یک ستون در فایل‌های مجزا و کنار هم روی دیسک ذخیره می‌شوند.

۱. خواندن هوشمند دیسک (Selective Read): سیستم فقط و فقط فایل مربوط به ستون «مبلغ خرید» را می‌خواند (I/O به شدت کاهش می‌یابد).

۲. فشردگی و برداری‌سازی (Vectorization): چون تمام داده‌های یک ستون از یک جنس (مثلاً عدد) هستند، CPU می‌تواند از دستورات برداری (SIMD) استفاده کرده و داده‌ها را تا ۱۰ برابر بیشتر فشرده کند.

نمونه پایگاه داده شبیه‌سازی شده (شامل 1,000,000 رکورد و ۵ ستون)

ID	Age	City	Email	Purchase_Amount
0	58	City_7	user_0@domain.com	193.387265
1	47	City_8	user_1@domain.com	493.063518
2	34	City_12	user_2@domain.com	341.527432
3	66	City_13	user_3@domain.com	30.533792
4	37	City_13	user_4@domain.com	451.944923

اجرای کوئری تحلیلی: محاسبه کُل درآمد SUM(Purchase_Amount)

نتایج بنچمارک برای 1,000,000 رکورد:

● موتور سطری (سنتی): 367.69 میلی‌ثانیه | تعداد عناصر خوانده شده: 5,000,000 واحد

● موتور ستونی (تحلیلی): 2.06 میلی‌ثانیه | تعداد عناصر خوانده شده: 1,000,000 واحد

🚀 بهبود عملکرد: موتور ستونی 178.2 برابر سریع‌تر عمل کرد!

Figure 1: Disk/Memory Elements Scanned (1,000,000 Rows)

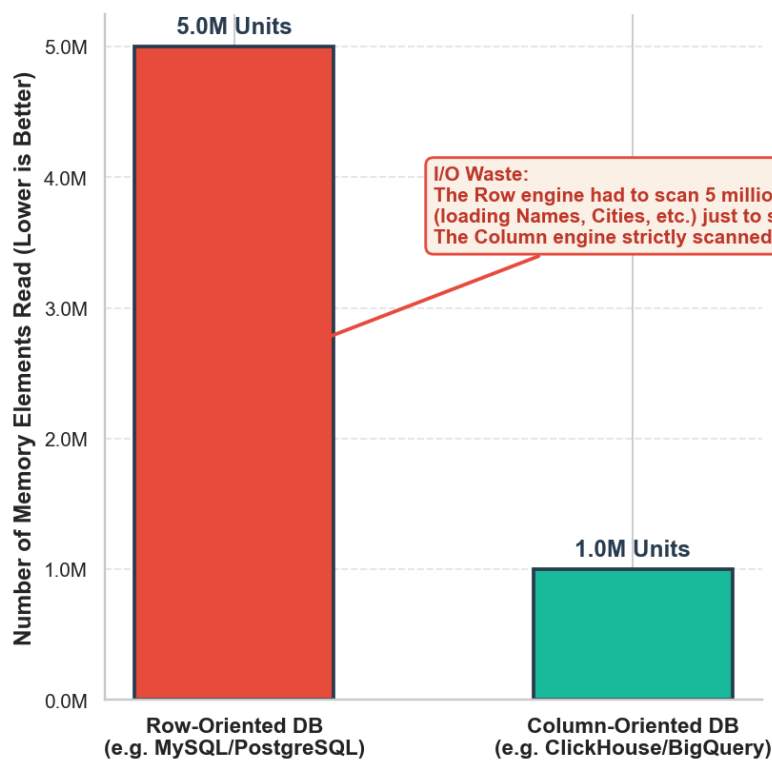
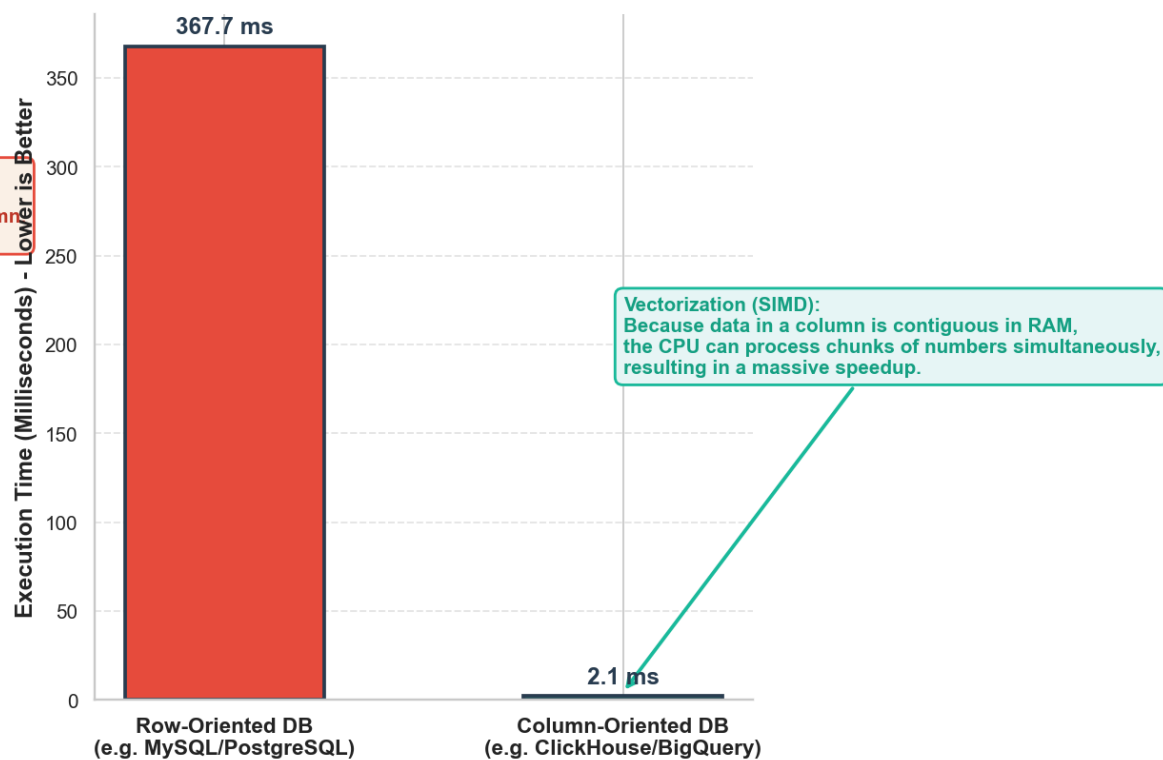


Figure 2: Query Execution Time Comparison



نتیجه استراتژیک (Strategic Implication):

معماری ستونی (Columnar Architecture)، با حذف کامل عملیات خواندن داده‌های غیرضروری از روی دیسک (کاهش چشمگیر I/O bottleneck) و بهره‌گیری از پردازش برداری (Vectorized Execution / SIMD) در CPU، سرعت اجرای کوئری‌های تحلیلی و تجمیعی (نظیر SUM و AVG) روی حجم‌های پتابایتی داده را به‌طرز دیوانه‌واری افزایش می‌دهد.

این معماری، قلب تپنده پلتفرم‌های هوش تجاری ابری مدرن نظیر **Google BigQuery**، **AWS Redshift** و **Snowflake** محسوب می‌گردد و دلیل اصلی سرعت بالای داشبوردها در مواجهه با میلیاردها رکورد تراکنش است.

توضیح مثال کد پایتون

در این مثال:

- یک مجموعه داده با ۱۰۰,۰۰۰ رکورد و ۵ ستون شبیه‌سازی گردید.
- داده‌ها در دو ساختار حافظه‌ای متفاوت ذخیره شدند: ساختار اول به‌صورت سطری (لیستی از سطرها) و ساختار دوم به‌صورت ستونی (دیکشنری از آرایه‌های مجزای هر ستون).
- اجرای کوئری تجمیعی OLAP: عملیات محاسبه مجموع خرید کالا برای تمام کاربران اجرا شد.
- در ساختار سطری، سیستم مجبور شد برای هر سطر تمام ۵ عنصر آن سطر را پایش کند (در مجموع ۵۰۰,۰۰۰ پیمایش)؛ در حالی که در ساختار ستونی، تنها آرایه مربوط به ستون مبلغ خرید خوانده شد (تنها ۱۰۰,۰۰۰ پیمایش).
- تفسیر نمودارهای Matplotlib:
 - نمودار سمت چپ: نشان می‌دهد که پایگاه داده ستونی چگونه با عدم بارگذاری ستون‌های غیرضروری، میزان داده‌های خوانده‌شده از حافظه را به یک‌پنجم کاهش می‌دهد.
 - نمودار سمت راست: سرعت زمان اجرای کوئری را مقایسه می‌کند که برتری فاحش و سرعت بالای معماری ستونی را اثبات می‌نماید.

3.4 Data cube computation

3.4.1 Terminology of data cube computation

اصطلاحات محاسبه مکعب داده

از این پس در این فصل، هر جا از «مکعب داده» صحبت می‌کنیم، منظورمان یک شبکه (Lattice) از زیرمکعب‌ها است، نه یک زیرمکعب منفرد؛ یعنی مجموعه‌ای که تمام سطوح ممکن تجمیع را دربر می‌گیرد.

هر تاپل درون یک زیرمکعب را یک **سلول (Cell)** می‌نامیم؛ یک نقطهٔ مشخص در فضای مکعب داده. سلولی که از زیرمکعب پایینی می‌آید و هیچ بُعدش تجمیع نشده، **سلول پایه (Base Cell)** است؛ سلولی که روی یک یا چند بُعد تجمیع شده، **سلول تجمیعی (Aggregate Cell)** نام دارد. برای نشان‌دادن بُعد تجمیع‌شده، از نماد * استفاده می‌شود.

برای مثال، در مکعبی با ابعاد «ماه»، «شهر» و «گروه مشتری»، سلول **(ژانویه, *, *, 2800)** یک سلول یک‌بُعدی (تجمیعی) است، اما **(ژانویه, شیکاگو, تجاری, 24)** که هر سه بُعد مقدار مشخص دارند، یک سلول پایه است.

زیرمکعب بالایی و پایینی

زیرمکعب پایینی (Base Cuboid) همان زیرمکعبی است که تمام سلول‌های پایه را دربر می‌گیرد و معادل جدول واقعیت خام است. **زیرمکعب بالایی (Apex Cuboid)** تنها یک سلول دارد: سلول صفر-بُعدی (*, *, *) یعنی تجمیع کامل تمام داده‌ها در یک عدد واحد.

تعداد زیرمکعب‌ها

اگر هیچ بُعدی سلسله‌مراتب نداشته باشد، تعداد زیرمکعب‌های یک مکعب n بُعدی برابر 2^n است. اما در عمل بیشتر ابعاد سلسله‌مراتب دارند (مثل زمان: روز > ماه > فصل > سال). اگر بُعد i -ام، L_i سطح داشته باشد، (L_i+1) انتخاب برای آن بُعد وجود دارد (هر سطح، یا حذف کامل بُعد)، و تعداد کل زیرمکعب‌ها برابر حاصل‌ضرب این مقادیر روی همهٔ ابعاد است.

اندازهٔ هر زیرمکعب هم به **کاردینالیتی** (تعداد مقادیر متمایز) ابعادهش بستگی دارد؛ برای نمونه، گروه‌بندی «شهر و کالا» می‌تواند تا حاصل‌ضرب تعداد شهرها در تعداد کالاها سلول داشته باشد. به همین دلیل، برآورد سریع تعداد دقیق تاپل‌های حاصل، همچنان یکی از چالش‌های حل‌نشدهٔ این حوزه است.

ریاضیات – نمایهٔ بیت‌نگاشت و عملیات بولی

در **نمایهٔ بیت‌نگاشت (Bitmap Index)** برای یک شرط، رشته‌ای از بیت‌ها ساخته می‌شود. بیت ۱ یعنی رکورد شرط را دارد و بیت ۰ یعنی ندارد. این نمایش امکان ترکیب سریع شرط‌ها را با عملیات بولی فراهم می‌کند.

ترکیب دو شرط با AND

(3.11)

$$B_{A \wedge B} = B_A \wedge B_B$$

نمادهای فرمول

- B_A : بردار بیت‌نگاشت شرط A
- B_B : بردار بیت‌نگاشت شرط B
- $\wedge$: عملگر منطقی AND؛ هر دو شرط باید برقرار باشند
- $B_{A \wedge B}$: بردار بیت‌نگاشت حاصل از AND

✳ مثال ۱ – AND

صورت سؤال: اگر $B_A = (1, 0, 1, 1)$ و $B_B = (1, 1, 0, 1)$ باشند، نتیجهٔ AND را پیدا کنید.

$$(1, 0, 1, 1) \wedge (1, 1, 0, 1) = (1, 0, 0, 1)$$

نتیجه: فقط موقعیت‌هایی که در هر دو نمایه ۱ هستند، در نتیجه ۱ می‌شوند.

ترکیب دو شرط با OR

(3.12)

$$B_{A \vee B} = B_A \vee B_B$$

نمادهای فرمول

- B_A : بردار بیت‌نگاشت شرط A
- B_B : بردار بیت‌نگاشت شرط B
- $\vee$: عملگر منطقی OR؛ حداقل یکی از دو شرط باید برقرار باشد
- $B_{A \vee B}$: بردار بیت‌نگاشت حاصل از OR

مثال ۲ – OR

صورت سؤال: برای همان دو نمایه، نتیجه OR چیست؟

$$(1, 0, 1, 1) \vee (1, 1, 0, 1) = (1, 1, 1, 1)$$

نتیجه: در OR، اگر حداقل یکی از دو بیت ۱ باشد، بیت خروجی ۱ خواهد بود.

بازسازی مقدار عددی از بیت‌ها

(3.13)

$$x = \sum_{i=0}^{k-1} b_i 2^i$$

نمادهای فرمول

- x : مقدار عددی بازسازی‌شده
- b_i : بیت در موقعیت i
- i : شماره جایگاه بیت؛ از صفر شروع می‌شود
- k : تعداد بیت‌ها
- 2^i : ارزش مکانی بیت i

مثال ۳ – بازسازی مقدار از بیت‌ها

صورت سؤال: اگر بیت‌های یک مقدار از راست به چپ ۱، ۰، ۱ باشند، مقدار عددی آن چیست؟

$$x = 1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 = 5$$

نتیجه: مقدار عددی برابر ۵ است.

مثال عملی: واژه‌شناسی و اصطلاحات کلیدی مکعب داده در «آمازون»

تیم‌های هوش تجاری و تحلیل داده در آمازون برای بررسی دقیق عملکرد فروش، اصطلاحات استاندارد مکعب داده را به کار می‌برند. این مفاهیم نحوه تغییر گرانولاریته و زاویه دید روی داده‌های چندبعدی را توصیف می‌کنند:

۱. برش (Slice): ثابت نگه داشتن یک بعد خاص برای رسیدن به یک تصویر دوبعدی از مکعب. مثال: فیلتر کردن کل مکعب فروش فقط برای سال ۲۰۲۳.

۲. تاس‌زدن (Dice): اعمال فیلتر روی دو یا چند بعد به صورت هم‌زمان برای برش یک زیرمکعب کوچک‌تر. مثال: فیلتر کردن فروش سال ۲۰۲۳ در منطقه آمریکا و فقط برای گروه کالای دیجیتال.

۳. فشردن (Roll-up): صعود در سلسله‌مراتب ابعاد و تجمیع داده‌ها برای کاهش جزئیات و رسیدن به خلاصه مدیوم. مثال: جمع زدن داده‌های ماهانه برای رسیدن به فروش سالانه.

۴. جزئی‌سازی (Drill-down): نزول در سلسله‌مراتب ابعاد و شکستن داده‌های خلاصه‌شده به اجزای دقیق‌تر. مثال: شکستن فروش یک سال به ۴ فصل یا ۱۲ ماه.

۵. چرخش (Pivot): تغییر چیدمان و جای‌گذاری سطرها و ستون‌های جدول برای مشاهده داده‌ها از یک دیدگاه جدید. مثال: جابه‌جا کردن بعد منطقه از سطر به ستون جهت مقایسه آسان‌تر.

نتیجه: تسلط بر این اصطلاحات پایه، تحلیل داده‌های چندبعدی را در سیستم‌های OLAP دقیق و استاندارد می‌سازد.

خروجی: واژه‌شناسی کاربردی عملیات تعاملی روی ابعاد مکعب داده

In [13]:

```
display_academic_text(
    "📊 در «آمازون» (Data Cube) واژه‌شناسی و اصطلاحات کلیدی مکعب داده",
    "main_title"
)

display_academic_text(
    "تیم‌های هوش تجاری و تحلیل داده در آمازون برای بررسی دقیق عملکرد فروش، اصطلاحات استاندارد مکعب داده را به کار می‌برند.",
    "این مفاهیم، نحوه تغییر گرانولاریته (سطح جزئیات) و زاویه دید روی داده‌های چندبعدی را توصیف می‌کنند",
    "paragraph"
)

display_academic_text(
    "<br>برای رسیدن به تصویر دوبعدی (مثال: فیلتر کردن کل فروش فقط برای سال ۲۰۲۳) <i>یک بُعد خاص</i> ثابت نگه داشتن </b>(Slice): بُرش ۱.<br>(Tech): فروش سال ۲۰۲۳، منطقه آمریکا، گروه) برای ساخت یک زیرمکعب <i>دو یا چند بُعد</i> به صورت هم‌زمان </b>(Dice): تاس‌زدن ۲.<br>تجمیع داده‌ها برای کاهش جزئیات و رسیدن به خلاصه مدیریتی (مثال: جمع زدن داده‌های فصلی به سالانه) </b>(Roll-up): فشردن ۳.<br>شکستن داده‌های خلاصه‌شده به اجزای دقیق‌تر (مثال: شکستن فروش یک سال به ۴ فصل) </b>(Drill-down): جزئی‌سازی ۴.<br>، تغییر چیدمان و جای‌گذاری سطرها و ستون‌های جدول برای مشاهده داده‌ها از زاویه‌ای جدید </b>(Pivot): چرخش ۵.",
    "paragraph"
)

# =====
# SECTION 2: DATA GENERATION AND DIMENSIONAL OPERATIONS
# =====
display_academic_text("مرحله نخست: تولید مکعب داده و شبیه‌سازی اصطلاحات پایه‌ای", "sub_title")

# Step 1: Generate Synthetic Amazon 3D Cube Dataset
years = [2025, 2026]
quarters = ['Q1', 'Q2', 'Q3', 'Q4']
regions = ['US', 'EU']
categories = ['Tech', 'Fashion']

records = []
for y in years:
    for q in quarters:
        for r in regions:
            for c in categories:
                sales = np.random.randint(150, 600)
                records.append({
                    'Year': y,
                    'Quarter': q,
                    'Region': r,
                    'Category': c,
                    'Sales_kUSD': sales,
                })

df_cube = pd.DataFrame(records)

# Step 2: Implement Terminology Operations

# 1. Slice: Fix exactly 1 dimension (Year = 2026) -> Flattens the cube
slice_data = df_cube[df_cube['Year'] == 2026]
```

```
# 2. Dice: Fix multiple dimensions (Year=2026, Region='US', Category='Tech') -> Extracts a sub-cube
dice_data = df_cube[
    (df_cube['Year'] == 2026) &
    (df_cube['Region'] == 'US') &
    (df_cube['Category'] == 'Tech')
]

# 3. Roll-up: Aggregate from Quarter to Year (Reduce dimensions)
rollup_data = df_cube.groupby(['Year', 'Category'])['Sales_kUSD'].sum().reset_index()

# 4. Drill-down: Show Quarterly breakdown for 2026 (Increase dimensions)
drilldown_data = df_cube[df_cube['Year'] == 2026].groupby(['Quarter', 'Category'])['Sales_kUSD'].sum().reset_index()

# 5. Pivot: Rotate layout (Category vs Region for 2026)
pivot_data = pd.pivot_table(
    slice_data,
    values='Sales_kUSD',
    index='Category',
    columns='Region',
    aggfunc='sum',
).reset_index() # Reset index for clean HTML rendering

# Render Data Samples securely
display_side_by_side_datasets(
    [df_cube.head(5).reset_index(drop=True), slice_data.head(5).reset_index(drop=True), pivot_data],
    [
        "پایگاه داده اصلی (Original 3D Cube)",
        "ثابت کردن بُعد زمان (۲۰۲۶) Slice عملیات",
        "چرخش محوره‌ای (Region و Category) Pivot عملیات"
    ]
)

# =====
# SECTION 3: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
display_academic_text("مرحله دوم: مصورسازی تحلیلی مفاهیم", "sub_title")

# Step 3: Visualization using Matplotlib
fig, axes = plt.subplots(2, 2, figsize=(18, 12), dpi=135)

# -----
# Plot 1: Slice (2026 Sales across Categories)
# -----
slice_summary = slice_data.groupby('Category')['Sales_kUSD'].sum().reset_index()

axes[0, 0].bar(
    slice_summary['Category'],
    slice_summary['Sales_kUSD'],
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.4,
)

axes[0, 0].annotate(
    "Slice Concept:\nBy fixing Year=2026, we sliced a 2D flat\nplane out of the 3D data cube.",
    xy=(1.0, 2500), xytext=(0.5, 3000),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0, 0].set_title('Figure 1: Slice Operation (Fixed Dimension: Year = 2026)', fontsize=14, fontweight='bold', pad=15)
axes[0, 0].set_ylabel('Total Sales (kUSD)', fontsize=13, fontweight='bold')
axes[0, 0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0, 0].tick_params(axis='x', labelsize=12)
axes[0, 0].set_xticklabels(slice_summary['Category'], fontweight='bold')

for p in axes[0, 0].patches:
    axes[0, 0].annotate(f"${int(p.get_height())}k", (p.get_x() + p.get_width() / 2., p.get_height()), ha='center', va='center', xytext=(0, -15), textcoords='offset points', fontweight='bold', fontsize=12, color='black')

# -----
# Plot 2: Dice (Fixed Year = 2026, Region = US, Category = Tech)
# -----
axes[0, 1].bar(
    dice_data['Quarter'].astype(str),
    dice_data['Sales_kUSD'],
    color='#2980b9', # Classic Blue
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.4,
)

axes[0, 1].annotate(
    "Dice Concept:\nBy filtering Year, Region AND Category,\nwe isolated a tiny sub-cube for precise analysis.",
    xy=(2.0, 300), xytext=(0.5, 450),
    fontsize=11, fontweight='bold', color='#2980b9',
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[0, 1].set_title('Figure 2: Dice Operation (US Tech in 2026 by Quarter)', fontsize=14, fontweight='bold', pad=15)
axes[0, 1].set_ylabel('Sales (kUSD)', fontsize=13, fontweight='bold')
axes[0, 1].grid(axis='y', linestyle='--', alpha=0.5)
axes[0, 1].tick_params(axis='x', labelsize=12)
axes[0, 1].set_xticklabels(dice_data['Quarter'], fontweight='bold')

for p in axes[0, 1].patches:
    axes[0, 1].annotate(f"${int(p.get_height())}k", (p.get_x() + p.get_width() / 2., p.get_height()), ha='center', va='center', xytext=(0, 10), textcoords='offset points', fontweight='bold', fontsize=12, color='black')

# -----
# Plot 3: Roll-up (Yearly Aggregated Sales)
```

```
#
pivot_rollup = rollup_data.pivot(index='Year', columns='Category', values='Sales_kUSD')
pivot_rollup.plot(
    kind='bar',
    ax=axes[1, 0],
    color=[ '#2c3e50', '#1abc9c'], # Navy, Turquoise
    edgecolor='#2c3e50',
    linewidth=2.0
)

axes[1, 0].annotate(
    "Roll-Up Concept:\nWe climbed up the Concept Hierarchy,\nhiding the Quarter noise to see the yearly trend.",
    xy=(0.0, 2000), xytext=(-0.35, 3500),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#bdc3c7", lw=1.5)
)

axes[1, 0].set_title('Figure 3: Roll-Up Operation (Yearly Aggregated Macro-Trend)', fontsize=14, fontweight='bold', pad=15)
axes[1, 0].set_ylabel('Total Sales (kUSD)', fontsize=13, fontweight='bold')
axes[1, 0].set_xlabel('Macro Dimension: Year', fontsize=13, fontweight='bold')
axes[1, 0].tick_params(axis='x', rotation=0, labelsiz=12)
axes[1, 0].set_xticklabels(axes[1, 0].get_xticklabels(), fontweight='bold')
axes[1, 0].grid(axis='y', linestyle='--', alpha=0.5)
axes[1, 0].legend(title='Category', frameon=True, fontsize=11)

# -----
# Plot 4: Pivot (Rotated View of Category vs Region)
# -----
pivot_data_plot = pivot_data.set_index('Category')
pivot_data_plot.plot(
    kind='barh',
    ax=axes[1, 1],
    color=[ '#5DADE2', '#2980b9'], # Light Blue, Classic Blue
    edgecolor='#2c3e50',
    linewidth=1.5
)

axes[1, 1].annotate(
    "Pivot Concept:\nSimply rotating the presentation axes.\nThe data doesn't change, only the viewing angle.",
    xy=(1500, 1.0), xytext=(1600, 0.4),
    fontsize=11, fontweight='bold', color='#2980b9',
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#bdc3c7", lw=1.5)
)

axes[1, 1].set_title('Figure 4: Pivot Operation (Rotated Dimensional Layout)', fontsize=14, fontweight='bold', pad=15)
axes[1, 1].set_ylabel('Sales (kUSD)', fontsize=13, fontweight='bold')
axes[1, 1].set_xlabel('Dimension: Category', fontsize=13, fontweight='bold')
axes[1, 1].tick_params(axis='y', labelsiz=12)
axes[1, 1].set_yticklabels(axes[1, 1].get_yticklabels(), fontweight='bold')
axes[1, 1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1, 1].legend(title='Dimension: Region', loc='lower right', frameon=True, fontsize=11)

# Clear top and right spines for a clean academic aesthetic
for ax in axes.flat:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.suptitle('OLAP Dimensional Terminology & Operations Visualized', fontsize=17, fontweight='bold', y=1.02)

plt.tight_layout(pad=3.0)
plt.show()

# =====
# SECTION 4: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه کاربردی (Strategic Implication):</b><br>"
    r"<b>Slice</b> و <b>Dice</b> تسلط بر این اصطلاحات پایه، زیربنای مهندسی داده‌های چندبُعدی را شکل می‌دهد. استفاده درست از عملیات"
    r"<br>بار پردازشی کوثری‌ها را با حذف نویزهای ناخواسته و ایزوله‌سازی قطعات مرتبط به حداقل می‌رساند"
    r"<br>" به تحلیل‌گران این قدرت را می‌بخشد که داشبوردهای تعاملی خلق کنند؛ <b>Drill-down</b> و <b>Roll-up</b> در عین حال، ترکیب هنرمندانهٔ"
    r"<br>" نفوذ کند <i>(Operational Granularity)</i> جزئیات عملیاتی<i></i> به Drill-down دست یابد، و با <i>(Strategic Depth)</i> عمق استراتژیک<i></i> به Roll-up داشبوردهایی که کاربر در آن‌ها می‌تواند با عملیات"
)
display_academic_text(insight_conclusion, "insight")
```

واژه‌شناسی و اصطلاحات کلیدی مکعب داده (Data Cube) در «آمازون»

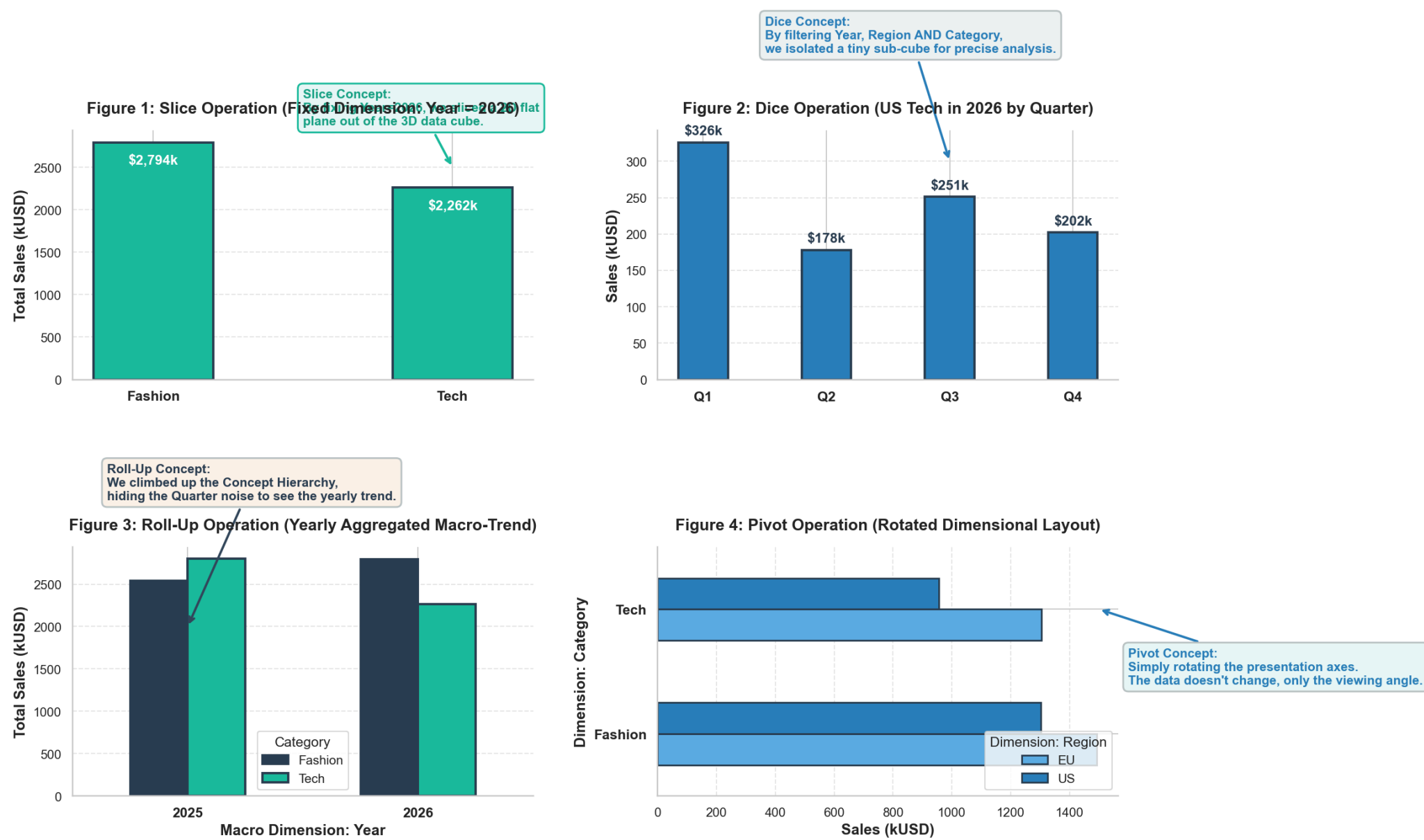
تیم‌های هوش تجاری و تحلیل داده در آمازون برای بررسی دقیق عملکرد فروش، اصطلاحات استاندارد مکعب داده را به کار می‌برند. این مفاهیم، نحوهٔ تغییر گرانولاریته (سطح جزئیات) و زاویهٔ دید روی داده‌های چندبُعدی را توصیف می‌کنند:

- بُرش (Slice):** ثابت نگه داشتن یک بُعد خاص برای رسیدن به تصویر دوبُعدی (مثال: فیلتر کردن گُل فروش فقط برای سال ۲۰۲۳).
- تاس‌زدن (Dice):** اعمال فیلتر روی دو یا چند بُعد به‌صورت هم‌زمان برای ساخت یک زیرمکعب (مثال: فروش سال ۲۰۲۳، منطقه آمریکا، گروه Tech).
- فشردن (Roll-up):** تجمیع داده‌ها برای کاهش جزئیات و رسیدن به خلاصهٔ مدیریتی (مثال: جمع زدن داده‌های فصلی به سالانه).
- جزئی‌سازی (Drill-down):** شکستن داده‌های خلاصه‌شده به اجزای دقیق‌تر (مثال: شکستن فروش یک سال به ۴ فصل).
- چرخش (Pivot):** تغییر چیدمان و جای‌گذاری سطرها و ستون‌های جدول برای مشاهده داده‌ها از زاویه‌ای جدید.

پایگاه داده اصلی (Original 3D Cube)					ثابت کردن بُعد زمان (= ۲۰۲۶) عملیات Slice					Region و Category چرخش محورهای Pivot عملیات			
Year	Quarter	Region	Category	Sales	Year	Quarter	Region	Category	Sales	Region	Category	EU	US
2025	Q1	US	Tech	1200	2026	Q1	US	Tech	1300		Fashion	1492	1302
2025	Q1	US	Fashion	1100	2026	Q1	US	Fashion	1250		Tech	1305	957
2025	Q1	EU	Tech	900	2026	Q1	EU	Tech	950				
2025	Q1	EU	Fashion	850	2026	Q1	EU	Fashion	900				
2025	Q2	US	Tech	1150	2026	Q2	US	Tech	1200				

مرحله دوم: مصورسازی تحلیلی مفاهیم OLAP

OLAP Dimensional Terminology & Operations Visualized



نتیجه کاربردی (Strategic Implication):

تسلط بر این اصطلاحات پایه، زیربنای مهندسی داده‌های چندبُعدی را شکل می‌دهد. استفاده درست از عملیات **Slice** و **Dice** بار پردازشی کوئری‌ها را با حذفِ نویزهای ناخواسته و ایزوله‌سازیِ قطعات مرتبط به حداقل می‌رساند. در عین حال، ترکیب هنرمندانه **Roll-up** و **Drill-down** به تحلیل‌گران این قدرت را می‌بخشد که داشبوردهای تعاملی خلق کنند؛ داشبوردهایی که کاربر در آن‌ها می‌تواند با عملیات **Roll-up** به عمقِ استراتژیک (**Strategic Depth**) دست یابد، و با **Drill-down** به جزئیاتِ عملیاتی (**Operational Granularity**) نفوذ کند.

توضیح مثال کد پایتون

در این مثال:

- ابتدا یک مکعب داده شبیه‌سازی‌شده شامل سه بعد زمان (سال و فصل)، منطقه و دسته‌بندی محصول همراه با میزان فروش ساخته شد.

- پنج اصطلاح اصلی مکعب داده با استفاده از متدهای کتابخانه پانداس اجرا گردید:

- عملیات Slice: با فیلتر کردن بعد سال روی ۲۰۲۳، یک مقطع مشخص از مکعب استخراج شد.
 - عملیات Dice: با اعمال فیلتر همزمان روی سال ۲۰۲۳، منطقه آمریکا و دسته فناوری، یک زیرمکعب محدودتر به دست آمد.
 - عملیات Roll-up: داده‌ها از سطح فصل به سطح سال تجمیع شدند تا نمای خلاصه‌تری ایجاد شود.
 - عملیات Drill-down: جزئیات فصلی برای سال ۲۰۲۳ بازسازی گردید.
 - عملیات Pivot: جای سطرها و ستون‌های نمایش داده تغییر یافت.
- تفسیر نمودارها (Matplotlib): هر نمودار به‌صورت دیداری خروجی حاصل از اجرای یکی از مفاهیم را روی داده‌های چندبعدی به تصویر می‌کشد.

3.4.2 Data cube materialization: ideas

متریالیزه کردن مکعب داده

در پروژه‌های بزرگ مقیاس، محاسبه و ذخیره‌سازی پیشاپیش تمام ترکیبات ممکن از مقادیر تجمیع‌شده در **مکعب داده**، عملی و به‌صرفه نیست؛ زیرا هم تعداد این ترکیبات بسیار زیاد است و هم حجم هر یک می‌تواند بزرگ باشد. به همین دلیل، چند رویکرد برای **متریالیزه کردن (Materialization)** مکعب داده وجود دارد که هر یک تعادلی متفاوت میان **فضای ذخیره‌سازی** و **سرعت پاسخ‌گویی** به پرس‌وجو ایجاد می‌کند.

بدون متریالیزه کردن: هیچ‌یک از ترکیبات تجمیع‌شده از پیش محاسبه نمی‌شوند و همه‌چیز باید در همان لحظه اجرای پرس‌وجو محاسبه شود. این حالت می‌تواند بسیار کند باشد.

متریالیزه کامل: تمام ترکیبات ممکن از مقادیر تجمیع‌شده در مکعب داده، از پیش محاسبه و ذخیره می‌شوند. این روش سریع‌ترین زمان پاسخ به پرس‌وجو را فراهم می‌کند، اما ممکن است به فضای ذخیره‌سازی بسیار زیادی نیاز داشته باشد.

متریالیزه جزئی: به‌جای محاسبه همه‌چیز، فقط گزیده‌ای از تجمیع‌ها یا زیرمجموعه‌های مکعب داده، بر اساس الگوهای پیش‌بینی‌شده پرس‌وجو، محاسبه و ذخیره می‌شود. این روش تعادلی میان عملکرد پرس‌وجو و محدودیت فضای ذخیره‌سازی برقرار می‌کند و به همین دلیل بیشتر از دو حالت دیگر به‌کار می‌رود.

دو حالت خاص‌تر نیز از دل متریالیزه جزئی مطرح می‌شوند:

متریالیزه N‌تای برتر (Top-N): فقط N ترکیب پرتقاضا یا پرکاربرد متریالیزه می‌شوند، که N نشان‌دهنده پرس‌وجوهای پرتکرار یا بینش‌های باارزش است. این روش برای موارد استفاده رایج بهینه‌سازی شده است.

متریالیزه افزایشی (Incremental): با ورود داده‌های جدید، مکعب متریالیزه‌شده به‌صورت افزایشی به‌روزرسانی می‌شود. این کار تضمین می‌کند که مکعب متریالیزه همواره با داده‌های پایه به‌روز باقی بماند، بدون نیاز به محاسبه مجدد کامل.

نکته کلیدی

انتخاب بین **متریالیزه کامل**، **جزئی** یا **افزایشی** یک **مصالحة کلاسیک** در انبار داده است: متریالیزه کامل سریع‌ترین پاسخ را دارد اما فضای ذخیره‌سازی زیادی می‌خواهد، متریالیزه جزئی تعادل برقرار می‌کند، و متریالیزه افزایشی برای داده‌های پویا مناسب است. در عمل، بیشتر سازمان‌ها از ترکیبی از این روش‌ها استفاده می‌کنند: **تجمیع‌های پرتکرار** را به‌صورت کامل و **تجمیع‌های کم‌کاربرد** را به‌صورت جزئی یا درخواستی متریالیزه می‌کنند.

مثال عملی: راهبردهای متریالیزه کردن مکعب داده در وال مارت برای افزایش سرعت کوئری

در سیستم‌های انبار داده بزرگ مانند وال مارت (Walmart)، اگر یک مکعب داده دارای N بعد مختلف باشد، تعداد کل ترکیب‌های تجمیعی ممکن (مکعب‌واره‌ها یا Cuboids) برابر با 2^N خواهد بود. برای مثال اگر ۱۰ بعد داشته باشیم، $2^{10} = 1024$ مکعب‌واره مختلف تولید می‌شود. محاسبه آئی این تجمیع‌ها در زمان اجرای کوئری بسیار کند است. برای حل این چالش از راهکارهای متریالیزه کردن (Materialization) یعنی پیش‌محاسبه و ذخیره‌سازی تجمیع‌ها استفاده می‌شود.

۱. متریالیزه کامل (Full Materialization): تمام 2^N حالت تجمیع ممکن از قبل محاسبه و روی دیسک ذخیره می‌شوند. سرعت کوئری فوق‌العاده بالاست، اما فضای ذخیره‌سازی بسیار زیادی مصرف می‌کند (انفجار فضای ذخیره‌سازی).

۲. متریالیزه جزئی (Partial Materialization): تنها مکعب‌واره‌هایی که بیشترین الگوی کوئری و تقاضا را دارند از قبل محاسبه می‌شوند. این روش تعادلی بهینه بین سرعت اجرای کوئری و حافظه مصرفی ایجاد می‌کند.

۳. متریالیزه Top-N: تنها N تجمیع پرکاربرد و پرپرسش متریالیزه شده و بقیه تجمیع‌ها در صورت نیاز در زمان اجرا محاسبه می‌شوند.

۴. متریالیزه افزایشی (Incremental Materialization): هنگام ورود داده‌های جدید، به جای محاسبه مجدد کل مکعب داده، تنها تغییرات جدید به تجمیع‌های قبلی اضافه می‌شوند.

نتیجه: انتخاب درست استراتژی متریالیزه کردن مانع از اتلاف حافظه دیسک شده و پاسخ به پرس‌وجوهای OLAP را آئی می‌سازد.

خروجی: ایجاد موازنه هوشمند بین سرعت پاسخ‌گویی به کوئری و حافظه ذخیره‌سازی

In [14]:

```
display_academic_text(
    "📊 (Data Cube Materialization) موازنه فضا در برابر زمان: راهبردهای متریالیزه کردن مکعب داده",
    "main_title"
)

display_academic_text(
    "مختلف باشد (Dimension) بُعد $N$ اگر یک مکعب داده دارای (Walmart) در سیستم‌های انبار داده بزرگ مانند وال مارت",
    "رشد می‌کند <b>(Exponential) نمایی</b> به صورت (Cuboids زیرمکعب‌ها یا) تعداد کل ترکیب‌های تجمیعی ممکن",
    "paragraph"
)

display_academic_text(
    "(On-the-fly) برای مثال اگر سیستم دارای ۱۰ بُعد تحلیلی باشد، $1024 = \{10\}^2$ مکعب‌واره مختلف تولید می‌شود. محاسبه در لحظه",
    "(Materialization) متریالیزه کردن<b> این تجمیع‌ها در زمان اجرای کوئری مدیران، بسیار کند و فلج‌کننده است. برای حل این چالش از راهکارهای",
    "استفاده می‌شود </i>و ذخیره‌سازی فیزیکی تجمیع‌ها روی هارد دیسک (Pre-computing) پیش‌محاسبه<i> یعنی",
    "r",
    "paragraph"
)

display_academic_text("(Materialization Trade-offs) استراتژی‌های متریالیزه کردن:", "sub_title")

display_academic_text(
    "<br>حالت پیش‌محاسبه می‌شود (سرعت بی‌نظیر است، اما منجر به انفجار حجم دیسک می‌شود) تمام $N$ 2$ (Full Materialization): متریالیزه کردن کامل<b> ۱.",
    "<br>را دارند پیش‌محاسبه می‌شوند (تعادل طلایی) </i>بیشترین تقاضا<i> فقط مکعب‌واره‌هایی که (Partial Materialization): متریالیزه کردن جزئی<b> ۲.",
    "محاسبه صورت می‌گیرد (کندترین حالت) (Base Tables) هیچ تجمیعی ذخیره نمی‌شود و هربار روی جداول خام (No Materialization): بدون متریالیزه<b> ۳.",
    "paragraph"
)

# =====
# SECTION 2: GENERATING BASE SALES DATA AND CUBOIDS
# =====
display_academic_text("(Caching) مرحله نخست: تولید جداول پایه و استراتژی‌های پیش‌محاسبه", "sub_title")

# Step 1: Generate Walmart Base Sales Dataset (500,000 Rows for noticeable benchmarking)
# 3 Dimensions: Store, Category, Year -> Total 2^3 = 8 Cuboids
n_rows = 500000

stores = [f'Store_{i}' for i in range(1, 11)]
categories = [f'Category_{i}' for i in range(1, 21)]
years = [2024, 2025, 2026]

df_base = pd.DataFrame({
    'Store': np.random.choice(stores, size=n_rows),
    'Category': np.random.choice(categories, size=n_rows),
    'Year': np.random.choice(years, size=n_rows),
    'Sales_USD': np.random.uniform(20.0, 500.0, size=n_rows)
})

display_side_by_side_datasets(
    [df_base.head(5).reset_index(drop=True)],
    ["(رکورد خام {n_rows:,} دارای - Base Fact Table) جدول پایه تراکنش‌های فروش"]
)

# Step 2: Implement 3 Materialization Strategies
```

```
# Strategy 1: No Materialization (On-the-Fly Compute)
# Stores 0 precomputed tables in memory/disk

# Strategy 2: Partial Materialization
# Precompute ONLY the top 2 most popular business queries: Store-Year & Category-Year
cache_partial = {
    'Store_Year': df_base.groupby(['Store', 'Year'])['Sales_USD'].sum().reset_index(),
    'Category_Year': df_base.groupby(['Category', 'Year'])['Sales_USD'].sum().reset_index()
}

# Strategy 3: Full Materialization (Precompute ALL 2^3 = 8 possible cuboid combinations)
cache_full = {
    'Base_Store_Cat_Year': df_base.groupby(['Store', 'Category', 'Year'])['Sales_USD'].sum().reset_index(),
    'Store_Cat': df_base.groupby(['Store', 'Category'])['Sales_USD'].sum().reset_index(),
    'Store_Year': df_base.groupby(['Store', 'Year'])['Sales_USD'].sum().reset_index(),
    'Cat_Year': df_base.groupby(['Category', 'Year'])['Sales_USD'].sum().reset_index(),
    'Store': df_base.groupby('Store')['Sales_USD'].sum().reset_index(),
    'Category': df_base.groupby('Category')['Sales_USD'].sum().reset_index(),
    'Year': df_base.groupby('Year')['Sales_USD'].sum().reset_index(),
    'All_Total': df_base['Sales_USD'].sum() # Apex cuboid
}

# =====
# SECTION 3: BENCHMARK QUERY PERFORMANCE
# =====
# Query execution simulator: 50 Random Requests for "Sales by Store & Year"
n_queries = 50

# Benchmark 1: No Materialization Time (Must scan 500k rows & aggregate 50 times)
start = time.time()
for _ in range(n_queries):
    res_1 = df_base.groupby(['Store', 'Year'])['Sales_USD'].sum()
time_no_mat = (time.time() - start) * 1000

# Benchmark 2: Partial Materialization Time (Retrieve instantly from small, targeted cache)
start = time.time()
for _ in range(n_queries):
    res_2 = cache_partial['Store_Year']
time_partial_mat = (time.time() - start) * 1000

# Benchmark 3: Full Materialization Time (Retrieve instantly from heavy, complete cache)
start = time.time()
for _ in range(n_queries):
    res_3 = cache_full['Store_Year']
time_full_mat = (time.time() - start) * 1000

display_academic_text(
    f"<b>کوئری {n_queries} نتایج بنچمارک سرعت برای اجرای پشت سر هم</b><br>"
    f"🔴 <b>میلی‌ثانیه {time_no_mat:.1f}</b> بدون پیش‌محاسبه (پردازش زنده روی جدول پایه) <br>"
    f"🟢 <b>میلی‌ثانیه {time_partial_mat:.1f}</b> (خواندن از کش سبک - Partial) متریالیزه جزئی <br>"
    f"🟢 <b>میلی‌ثانیه {time_full_mat:.1f}</b> (خواندن از کش سنگین - Full) متریالیزه کامل",
    "insight"
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
# Step 4: Storage Overhead Metrics
cuboids_stored = [0, len(cache_partial), len(cache_full)]
execution_times = [time_no_mat, time_partial_mat, time_full_mat]
strategies = ['No Materialization\n(On-the-Fly)', 'Partial Materialization\n(The Golden Balance)', 'Full Materialization\n(Precomputed All)']

# Step 5: Visualization using Matplotlib
fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

# -----
# Subplot 1: Query Execution Time Comparison
# -----
bars1 = axes[0].bar(
    strategies,
    execution_times,
    color=['#e74c3c', '#2980b9', '#1abc9c'], # Red, Classic Blue, Turquoise
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[0].annotate(
    "CPU Bottleneck:\nRe-calculating the sums over 500,000 rows\neverly single time a dashboard refreshes\nis extremely slow and inefficient.",
    xy=(0.0, time_no_mat/2), xytext=(0.4, time_no_mat*0.7),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title(f'Figure 1: Total Execution Time for {n_queries} Queries (Lower = Better)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Query Execution Time (Milliseconds)', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(strategies, fontweight='bold', fontsize=11)

for bar in bars1:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval + (max(execution_times) * 0.02),
        f'{yval:.1f} ms',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# -----
# Subplot 2: Storage Overhead Comparison
# -----
bars2 = axes[1].bar(
```

```
strategies,
cuboids_stored,
color=['#e74c3c', '#2980b9', '#1abc9c'],
edgecolor='#2c3e50',
linewidth=2.0,
width=0.5
)

axes[1].annotate(
    "Storage Explosion:\nFull Materialization gives fast speeds,\nbut storing all 8 possible combinations\nwastes massive amounts of Disk Space.",
    xy=(2.0, 6.5), xytext=(0.3, 7.0),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[1].set_title('Figure 2: Storage Space Overhead (Precomputed Cuboids)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_ylabel('Number of Extra Tables Stored on Disk', fontsize=13, fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].set_ylim(0, 9)
axes[1].set_xticklabels(strategies, fontweight='bold', fontsize=11)

for bar in bars2:
    yval = bar.get_height()
    axes[1].text(
        bar.get_x() + bar.get_width() / 2,
        yval + 0.2,
        f'{yval} Extra Tables',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# Clear top and right spines for a clean academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک (Strategic Implication):</b><br>"
    r"<b>است (Space-Time Trade-off) «موازنه فضا در برابر زمان»</b> <b>طراحی پایگاه داده همواره یک بازی</b>"
    r"<b>حالت ممکن روی دیسک، در دنیای واقعی یک فاجعهٔ معماری محسوب می‌شود (انفجار حافظه) ^N$ اگرچه سرعت جستجو را آنی می‌کند، اما ذخیره‌سازی تمام 2$ (Full) متربالبزه کردن کامل</b>"
    r"<b>یک موازنهٔ بی‌نظیر ایجاد می‌کند؛ این روش با پیش‌محاسبه و ذخیرهٔ صرفاً چند کوئری پُرتقاضا (مثلاً گزارش فروش ماهانه که مدیران هر روز می‌بینند) (Partial Materialization) متربالبزه جزئی</b> <b>در مقابل</b>"
    r"<b>(در دیتابیس‌های مدرن Materialized Views) هم مانع از اتلاف حافظه دیسک شده و هم سرعت پاسخگویی سیستم را به سطح ایده‌آلی ارتقا می‌دهد</b>"
)
display_academic_text(insight_conclusion, "insight")
```



موازنه فضا در برابر زمان: راهبردهای متربالبزه کردن مکعب داده (Data Cube Materialization)

در سیستم‌های انبار داده بزرگ مانند وال‌مارت (Walmart)، اگر یک مکعب داده دارای N بُعد (Dimension) مختلف باشد، تعداد گُل ترکیب‌های تجمیعی ممکن (زیرمکعب‌ها یا Cuboids) به صورت نمایی (Exponential) رشد می‌کند.

برای مثال اگر سیستم دارای ۱۰ بُعدِ تحلیلی باشد، $2^{10} = 1024$ مکعب‌واره مختلف تولید می‌شود. محاسبهٔ درلحظه (On-the-fly) این تجمیع‌ها در زمانِ اجرای کوئری مدیران، بسیار گُند و فلج‌کننده است. برای حل این چالش از راهکارهای **متربالبزه کردن (Materialization)** یعنی *پیش‌محاسبه (Pre-computing)* و *ذخیره‌سازی فیزیکی تجمیع‌ها روی هارد دیسک* استفاده می‌شود.

استراتژی‌های متربالبزه کردن (Materialization Trade-offs):

- متربالبزه کردن کامل (Full Materialization):** تمام 2^N حالت پیش‌محاسبه می‌شود (سرعت بی‌نظیر است، اما منجر به انفجارِ حجم دیسک می‌شود).
- متربالبزه کردن جزئی (Partial Materialization):** فقط مکعب‌واره‌هایی که *بیشترین تقاضا* را دارند پیش‌محاسبه می‌شوند (تعادلِ طلایی).
- بدون متربالبزه (No Materialization):** هیچ تجمیعی ذخیره نمی‌شود و هربار روی جداول خام (Base Tables) محاسبه صورت می‌گیرد (کندترین حالت).

مرحله نخست: تولید جداول پایه و استراتژی‌های پیش‌محاسبه (Caching)

جدول پایه تراکنش‌های فروش (Base Fact Table - دارای 500,000 رکورد خام)

Store	Category	Year	Sales_USD
Store_2	Category_1	2026	177.695185
Store_7	Category_6	2025	31.020298
Store_3	Category_16	2025	323.787396
Store_9	Category_3	2026	468.849007
Store_1	Category_20	2025	427.561963

نتایج بنچمارک سرعت برای اجرای پشت سر هم 50 کوئری:

بدون پیش‌محاسبه (پردازش زنده روی جدول پایه): 2069.0 میلی‌ثانیه

متریالیزه جزئی (Partial - خواندن از گشِ سبک): 0.1 میلی‌ثانیه

متریالیزه کامل (Full - خواندن از گشِ سنگین): 0.1 میلی‌ثانیه

Figure 1: Total Execution Time for 50 Queries (Lower = Better)

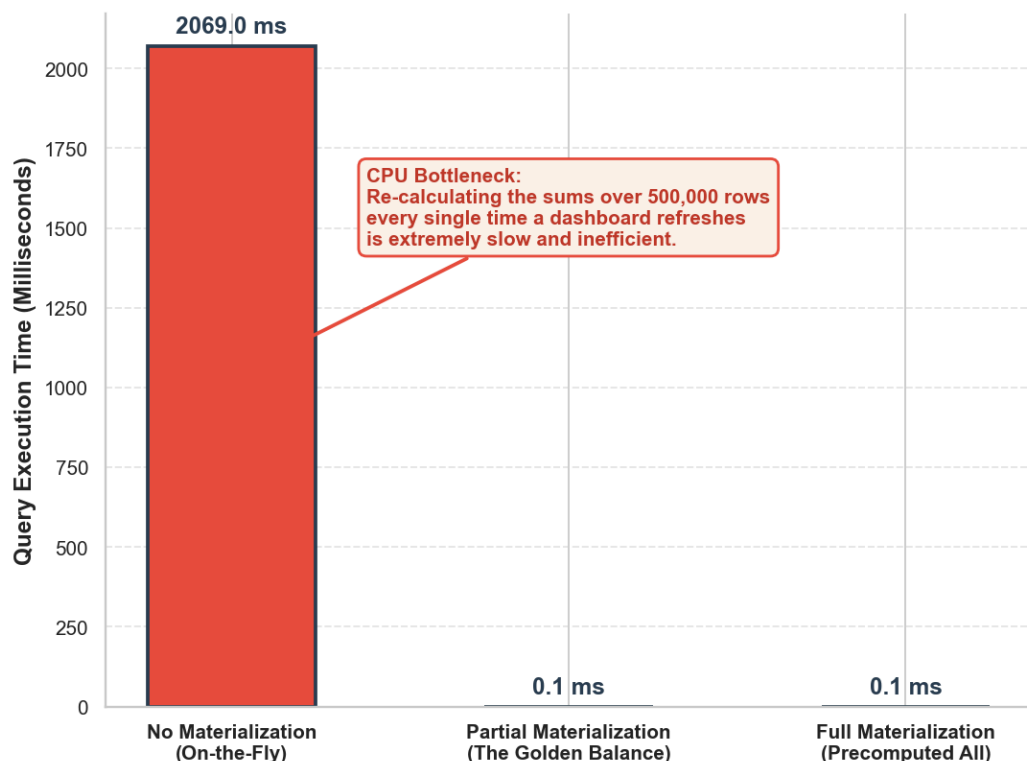
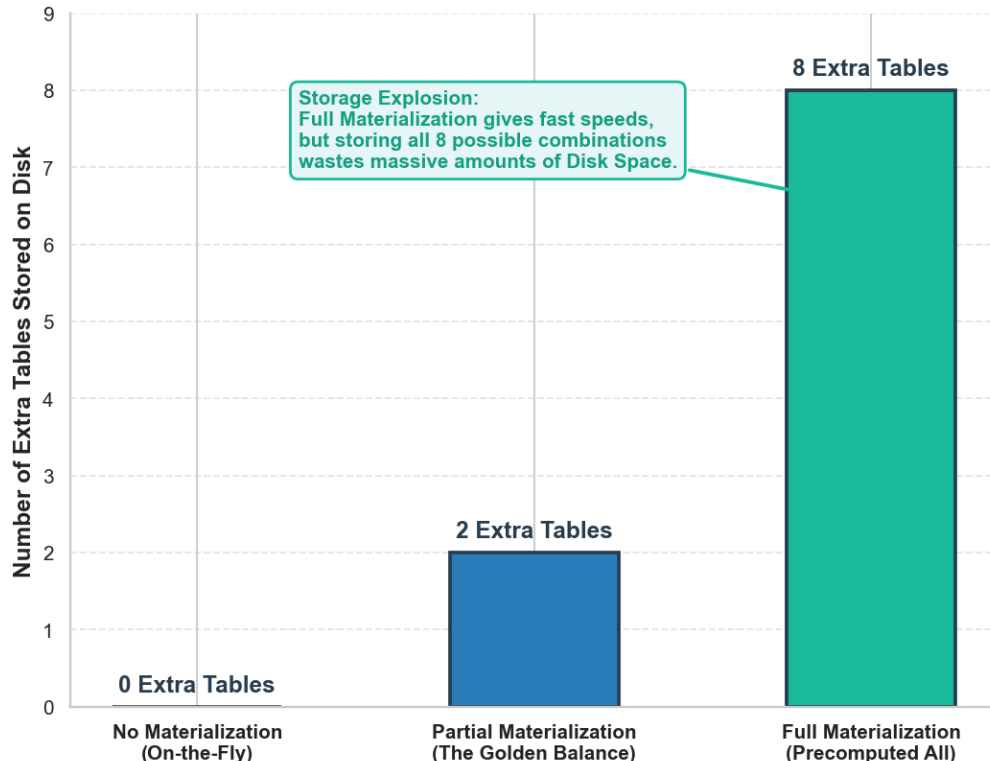


Figure 2: Storage Space Overhead (Precomputed Cuboids)



نتیجه استراتژیک (Strategic Implication):

طراحی پایگاه داده همواره یک بازی «موازنه فضا در برابر زمان» (Space-Time Trade-off) است.

متریالیزه کردن کامل (Full) اگرچه سرعت جستجو را آبی می‌کند، اما ذخیره‌سازی تمام 2^N حالت ممکن روی دیسک، در دنیای واقعی یک فاجعه معماری محسوب می‌شود (انفجار حافظه). در مقابل، متریالیزه جزئی (Partial Materialization) یک موازنه بی‌نظیر ایجاد می‌کند؛ این روش با پیش‌محاسبه و ذخیره صرفاً چند کوئری پُر تقاضا (مثلاً گزارش فروش ماهانه که مدیران هر روز می‌بینند)، هم مانع از اتلاف حافظه دیسک شده و هم سرعت پاسخگویی سیستم را به سطح ایده‌آلی ارتقا می‌دهد (Materialized Views در دیتابیس‌های مدرن).

توضیح مثال کد پایتون

در این مثال:

- یک مجموعه داده پایه از فروش وال‌مارت با ۱۰۰,۰۰۰ سطر و ۳ بعد (فروشگاه، دسته‌بندی و سال) ایجاد شد که دارای $2^3 = 8$ حالت تجمیع یا مکعب‌واره ممکن است.
- سه استراتژی اصلی متریالیزه‌سازی پیاده‌سازی و مقایسه گردیدند:
 - استراتژی عدم متریالیزه‌سازی (No Materialization): هیچ تجمیعی ذخیره نمی‌شود و تمام کوئری‌ها در زمان اجرا مستقیم روی داده‌های پایه محاسبه می‌گردند.
 - استراتژی متریالیزه جزئی (Partial Materialization): تنها ۲ مکعب‌واره پرکاربرد از قبل محاسبه و در حافظه نگهداری می‌شوند.
 - استراتژی متریالیزه کامل (Full Materialization): تمام ۸ مکعب‌واره ممکن از قبل محاسبه و ذخیره می‌گردند.
- بنچمارک زمان اجرای ۵۰ کوئری نشان داد که متریالیزه‌سازی سرعت پاسخ‌گویی را چندین برابر افزایش می‌دهد.

- نمودار سمت چپ: کاهش چشمگیر زمان اجرای کوئری‌ها را پس از متریالیزه‌سازی نشان می‌دهد.
- نمودار سمت راست: میزان مصرف حافظه و تعداد مکعب‌واره‌های ذخیره‌شده در هر استراتژی را مقایسه می‌کند تا برتری روش متریالیزه جزئی به عنوان نقطه تعادل بین سرعت و حافظه اثبات شود.

3.4.3 OLAP server architectures: ROLAP vs. MOLAP vs. HOLAP

ریاضیات – ماده‌سازی و به‌روزرسانی افزایشی

در ماده‌سازی (Materialization) می‌توان نتایج تجمیع را از قبل محاسبه و ذخیره کرد. در حالت به‌روزرسانی افزایشی، به‌جای محاسبه دوباره کل داده، فقط تغییر جدید به خلاصه قبلی اضافه می‌شود.

به‌روزرسانی افزایشی مجموع

(3.14)

$$S_{\text{new}} = S_{\text{old}} + \Delta S$$

نمادهای فرمول

- S_{new} : مجموع جدید
- S_{old} : مجموع قبل از ورود داده‌های جدید
- ΔS : تغییر ایجادشده در مجموع

مثال ۱ – به‌روزرسانی مجموع

صورت سؤال: اگر مجموع قبلی ۴۰۰ باشد و ۳۵ واحد داده جدید وارد شود، مجموع جدید چقدر است؟

$$S_{\text{new}} = 400 + 35 = 435$$

نتیجه: مجموع جدید ۴۳۵ است.

به‌روزرسانی افزایشی میانگین

(3.15)

$$AV\ G_{\text{new}} = \frac{SUM_{\text{old}} + \Delta SUM}{COUNT_{\text{old}} + \Delta COUNT}$$

نمادهای فرمول

- $AV\ G_{\text{new}}$: میانگین جدید
- SUM_{old} : مجموع قبلی
- ΔSUM : مجموع داده‌های جدید
- $COUNT_{\text{old}}$: تعداد قبلی
- $\Delta COUNT$: تعداد داده‌های جدید

مثال ۲ – به‌روزرسانی میانگین

صورت سؤال: اگر مجموع قبلی ۲۰۰، تعداد قبلی ۱۰، مجموع داده‌های جدید ۳۰ و تعداد داده‌های جدید ۲ باشد، میانگین جدید را بیابید.

$$AV G_{new} = \frac{200 + 30}{10 + 2} = \frac{230}{12} \approx 19.17$$

نتیجه: میانگین جدید تقریباً ۱۹/۱۷ است.

نکته: برای میانگین، نگهداری فقط مقدار میانگین قبلی کافی نیست؛ نگهداری **مجموع و تعداد** امکان به‌روزرسانی دقیق را فراهم می‌کند.

معماری‌های سرور OLAP

سرورهای OLAP پلی میان کاربران و داده‌های چندبعدی هستند. کاربران نیازی نمی‌بینند که داده‌ها دقیقاً چگونه و کجا ذخیره شده‌اند؛ فقط می‌خواهند بتوانند آن‌ها را تحلیل کنند. اما در پشت صحنه، این سرورها باید تصمیم بگیرند که داده‌ها را به چه شکلی ذخیره کنند. این تصمیم، تأثیر مستقیمی روی سرعت، حجم و انعطاف‌پذیری سیستم دارد. سه رویکرد اصلی برای این کار وجود دارد:

۱. ROLAP (OLAP رابطه‌ای)

ROLAP داده‌ها را در **پایگاه‌داده‌های رابطه‌ای** معمولی (مانند Oracle یا SQL Server) نگه می‌دارد و هنگام اجرای پرس‌وجو، با دستورات **SQL** آن‌ها را تجمیع می‌کند. مزیت اصلی این روش، **مقیاس‌پذیری** بالای آن است؛ یعنی برای مجموعه‌داده‌های بسیار بزرگ، گزینه مناسبی محسوب می‌شود. با این حال، به دلیل اینکه تجمیع‌ها در لحظه انجام می‌شوند، پاسخ‌گویی به پرس‌وجوهای سنگین ممکن است کندتر باشد.

۲. MOLAP (OLAP چندبعدی)

MOLAP داده‌ها را از قبل در قالبی **چندبعدی و اختصاصی** ذخیره می‌کند. این کار باعث می‌شود پرس‌وجوها بسیار سریع پاسخ داده شوند، چون همه تجمیع‌ها از پیش محاسبه شده‌اند. اما این روش دو نقطه ضعف دارد: اولاً فضای ذخیره‌سازی بیشتری اشغال می‌کند، دوماً وقتی داده‌ها **پراکنده (Sparse)** باشند یعنی بسیاری از ترکیبات ابعاد مقداری ندارند کارایی آن کاهش می‌یابد. بسیاری از پیاده‌سازی‌های MOLAP برای حل این مشکل، از روش‌های فشرده‌سازی هوشمندانه استفاده می‌کنند.

۳. HOLAP (OLAP ترکیبی)

HOLAP سعی می‌کند **بهترین‌های هر دو روش** را کنار هم بگذارد: داده‌های جزئی و حجیم را در پایگاه‌داده رابطه‌ای نگه می‌دارد (مانند ROLAP)، اما تجمیع‌های مهم و پرتکرار را از قبل محاسبه و به‌صورت چندبعدی ذخیره می‌کند (مانند MOLAP). این روش، تعادلی منطقی میان **سرعت و فضا** برقرار می‌کند.

📌 ROLAP، MOLAP و HOLAP در یک نگاه

ROLAP: مقیاس‌پذیری بالا، تجمیع در لحظه با SQL، مناسب داده‌های بسیار بزرگ. 📊

MOLAP: سرعت بسیار بالا با پیش‌محاسبه همه چیز، نیازمند فضای ذخیره‌سازی بیشتر. 📦

HOLAP: ترکیب هوشمندانه دو روش؛ جزئیات در ROLAP، تجمیع‌ها در MOLAP. ⚖️

💡 **نکتهٔ کلیدی:** انتخاب میان این سه، یک مصالحهٔ کلاسیک است: ROLAP فضا را بهینه می‌کند، MOLAP سرعت را، و HOLAP

می‌کوشد بهترینِ هر دو را داشته باشد.

مثال عملی: انتخاب معماری سرور OLAP در آمازون (مقایسه MOLAP، ROLAP و HOLAP)

مهندسان انبار داده شرکت آمازون برای پاسخ‌گویی به پرس‌وجوهای چندبعدی مدیران، باید معماری مناسبی برای سرور OLAP انتخاب کنند.

بسته به حجم داده‌ها و نیاز به سرعت، ۳ معماری اصلی وجود دارد:

۱. معماری (ROLAP (Relational OLAP – مبتنی بر پایگاه داده رابطه‌ای:

در این معماری، داده‌های واقعیت و ابعاد در همان پایگاه‌های داده رابطه‌ای سنتی (مثل PostgreSQL یا SQLite) ذخیره می‌شوند.

کوئری‌های چندبعدی در زمان اجرا به دستورات SQL تبدیل شده و با عملیات JOIN و GROUP BY محاسبه می‌شوند.

مزیت: قابلیت مقیاس‌پذیری تا حجم‌های بسیار بزرگ (پتابایت) بدون محدودیت حافظه.

عیب: سرعت پایین‌تر کوئری‌ها به دلیل محاسبات سنگین در زمان اجرا.

۲. معماری (MOLAP (Multidimensional OLAP – مبتنی بر آرایه‌های چندبعدی:

داده‌ها از قبل تجمیع شده و در ساختارهای آرایه‌ای اختصاصی چندبعدی (In-Memory Arrays) ذخیره می‌شوند. دستیابی به داده‌ها

مستقیم و با آدرس‌دهی اندیس آرایه انجام می‌شود.

مزیت: سرعت فوق‌العاده بالا و پاسخگویی آنی به کوئری‌ها در چند میلی‌ثانیه.

عیب: نیاز به حافظه زیاد و محدودیت در پذیریش حجم‌های بسیار عظیم داده‌های جزئی.

۳. معماری (HOLAP (Hybrid OLAP – معماری ترکیبی بهینه:

این روش مزایای ROLAP و MOLAP را با هم ترکیب می‌کند. خلاصه‌ها و تجمیع‌های سطح بالا (مثل فروش کل سالانه یا فصلی) در

مکعب‌های چندبعدی سریع (MOLAP) ذخیره می‌شوند، اما داده‌های جزئی و ریزتراکنش‌ها در پایگاه داده رابطه‌ای (ROLAP) نگهداری

می‌شوند.

نتیجه: معماران داده با به‌کارگیری HOLAP، تعادلی هوشمندانه بین سرعت پاسخ‌دهی به داشبوردهای مدیریتی و حفظ حجم زیاد داده‌های

جزئی ایجاد می‌کنند.

خروجی: شبیه‌سازی و مقایسه عملکرد MOLAP، ROLAP و HOLAP در اجرای کوئری‌های تحلیلی

```
In [15]: display_academic_text(
    "🏢",
    "main_title"
)

display_academic_text(
    "
    <b>سرور تحلیلی</b> (OLAP Server)
    <b>مهندسان انبار داده شرکت آمازون برای پاسخ‌گویی به پرس‌وجوهای چندبُعدی مدیران، باید معماری مناسبی برای</b>
    <b>:بسته به حجم داده‌ها و نیاز به سرعت، ۳ معماری اصلی در صنعت وجود دارد</b>
    "
    "paragraph"
)

display_academic_text("۱. معماری ROLAP (Relational OLAP):", "sub_title")
display_academic_text(
    "<br>روی دیتابیس رابطه‌ای سنتی محاسبه می‌شوند GROUP BY و JOIN تبدیل شده و با عملیات سنگین SQL به دستورات (Run-time) کوئری‌های چندبُعدی در زمان اجرا</b>
    <b>مقیاس‌پذیری پتابایتی بدون محدودیت حافظه (مناسب برای جزئیات خُرد)</b>
    <b>:مزیت</b>
    <b>:و اسکن دیسک در زمان اجرا SQL سرعتِ پایین‌تر به دلیل تفسیر</b>
    <b>عیب</b>
    "
    "paragraph"
)

display_academic_text("۲. معماری MOLAP (Multidimensional OLAP):", "sub_title")
display_academic_text(
    "<br>ذخیره می‌شوند (In-Memory Arrays / Cubes) داده‌ها از قبل تجمیع شده و در ساختارهای آرایه‌ای اختصاصی درون حافظهٔ رم</b>
    <b>($O(1)$) پاسخگویی آنی در چند میلی‌ثانیه با آدرس‌دهی مستقیم اندیس</b>
    <b>:مزیت</b>
    <b>(Sparsity Problem) زیاد و ناتوانی در ذخیرهٔ حجم عظیم داده‌های جزئی RAM نیاز به حافظهٔ</b>
    <b>عیب</b>
    "
    "paragraph"
)

display_academic_text("۳. معماری HOLAP (Hybrid OLAP):", "sub_title")
display_academic_text(
    "
    <b>ذخیره شده MOLAP مزایای دو مدل قبل را در هم می‌آمیزد. خلاصه‌ها و تجمیع‌های سطح بالا در مکعب‌های سریع (Hybrid) این روش ترکیبی</b>
    <b>:نگهداری می‌شوند ROLAP دارد، در دیتابیسِ ارزان‌قیمت Drill-Down و داده‌های ریزتراکنش‌ها برای زمانی که مدیر نیاز به</b>
    "
)
```

```
"paragraph"
)

# =====
# SECTION 2: DATA GENERATION AND ARCHITECTURE SIMULATIONS
# =====
display_academic_text("مرحله نخست: شبیه‌سازی معماری‌های سرور روی داده‌های تراکنشی", "sub_title")

# Step 1: Prepare Synthetic Sales Dataset (Increased to 100,000 for better benchmark scale)
n_records = 100000

years = [2024, 2025, 2026]
regions = ['North', 'South', 'East', 'West']
categories = ['Electronics', 'Apparel', 'Home']

df_sales = pd.DataFrame({
    'Year': np.random.choice(years, size=n_records),
    'Region': np.random.choice(regions, size=n_records),
    'Category': np.random.choice(categories, size=n_records),
    'Sales_USD': np.random.uniform(10.0, 500.0, size=n_records)
})

display_side_by_side_datasets(
    [df_sales.head(5).reset_index(drop=True)],
    [f"پایگاه داده اصلی شامل {n_records} نمونه داده‌های خام تراکنش‌های فروش"]
)

# -----
# Architecture 1: ROLAP Simulation (Relational Database via SQLite on Disk/Memory)
# -----
conn = sqlite3.connect(':memory:')
df_sales.to_sql('fact_sales', conn, index=False, if_exists='replace')

def query_rolap(year, region):
    """ROLAP Query: Dynamic SQL interpretation, filtering, and Group By over relational table."""
    sql = f"SELECT Year, Region, SUM(Sales_USD) as Total_Sales FROM fact_sales WHERE Year = {year} AND Region = '{region}' GROUP BY Year, Region"
    return pd.read_sql_query(sql, conn)

# -----
# Architecture 2: MOLAP Simulation (Pre-computed 3D NumPy Array in RAM)
# -----
# Dimensions mapping for O(1) array access
year_map = {y: i for i, y in enumerate(years)}
region_map = {r: i for i, r in enumerate(regions)}
cat_map = {c: i for i, c in enumerate(categories)}

# Precompute full 3D array in memory (MOLAP Cube)
molap_cube = np.zeros((len(years), len(regions), len(categories)))

for _, row in df_sales.iterrows():
    y_idx = year_map[row['Year']]
    r_idx = region_map[row['Region']]
    c_idx = cat_map[row['Category']]
    molap_cube[y_idx, r_idx, c_idx] += row['Sales_USD']

def query_molap(year, region):
    """MOLAP Query: Direct O(1) array slicing in memory. No SQL interpretation."""
    y_idx = year_map[year]
    r_idx = region_map[region]
    total_sales = np.sum(molap_cube[y_idx, r_idx, :])
    return total_sales

# -----
# Architecture 3: HOLAP Simulation (Hybrid Model Strategy)
# -----
# Pre-computed MOLAP cache for High-level Aggregates (Year x Region)
holap_summary_cache = df_sales.groupby(['Year', 'Region'])['Sales_USD'].sum().to_dict()

def query_holap(year, region, detailed_category=None):
    """HOLAP Query: Routes high-level to MOLAP cache, routes detailed drill-down to ROLAP."""
    if detailed_category is None:
        # High-level summary -> Instant return from MOLAP cache
        return holap_summary_cache.get((year, region), 0.0)
    else:
        # Detailed drill-down -> Processed by ROLAP SQL engine
        sql = f"SELECT Year, Region, Category, SUM(Sales_USD) as Total_Sales FROM fact_sales WHERE Year = {year} AND Region = '{region}' AND Category = '{detailed_category}' GROUP BY Year, R
        return pd.read_sql_query(sql, conn)

# =====
# SECTION 3: EXECUTION BENCHMARKING
# =====
# Step 5: Benchmark Execution Times over 100 Summary Queries
n_queries = 100

# Benchmark ROLAP
start = time.time()
for _ in range(n_queries):
    query_rolap(2026, 'North')
time_rolap = (time.time() - start) * 1000

# Benchmark MOLAP
start = time.time()
for _ in range(n_queries):
    query_molap(2026, 'North')
time_molap = (time.time() - start) * 1000

# Benchmark HOLAP (Summary Query Mode)
start = time.time()
for _ in range(n_queries):
    query_holap(2026, 'North')
time_holap = (time.time() - start) * 1000

display_academic_text(
```

```
f"<b>کوتری خلاصه‌سازی (Summary):</b><br>"
f"<b>نتایج بنچمارک برای اجرای تکراری</b><br>"
f"<b>ROLAP زمان اجرای (و اسکن جدول SQL تفسیر گدهای):<b>{time_rolap:.1f}</b> میلی‌ثانیه</b><br>"
f"<b>MOLAP زمان اجرای (آدرس‌دهی مستقیم آرایه در رم):<b>{time_molap:.1f}</b> میلی‌ثانیه</b><br>"
f"<b>HOLAP زمان اجرای (MOLAP مسیریابی هوشمند به گشی):<b>{time_holap:.1f}</b> میلی‌ثانیه</b>,"
"insight"
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
# Step 6: Visualization using Matplotlib
fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

# -----
# Subplot 1: Query Execution Time Comparison
# -----
architectures = ['ROLAP\n(Relational SQL)', 'MOLAP\n(In-Memory Array)', 'HOLAP\n(Hybrid Router)']
times = [time_rolap, time_molap, time_holap]

bars1 = axes[0].bar(
    architectures,
    times,
    color=['#e74c3c', '#1abc9c', '#2980b9'], # Red, Turquoise, Classic Blue
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[0].annotate(
    "Query Translation Overhead:\nROLAP is slow because every query requires\nthe database engine to parse SQL, build an\nexecution plan, and scan the raw table rows.",
    xy=(0.0, time_rolap/2), xytext=(0.4, time_rolap*0.7),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title(f'Figure 1: High-Level Summary Query Time ({n_queries} Runs)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Execution Time (Milliseconds) - Lower is Better', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(architectures, fontweight='bold', fontsize=12)

for bar in bars1:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval + (max(times) * 0.02),
        f'{yval:.1f} ms',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# -----
# Subplot 2: Architecture Characteristics (Qualitative Ratings)
# -----
categories_eval = ['Query Speed\n[Fast is 5]', 'Storage Scalability\n[Petabytes is 5]', 'Drill-Down Detail\n[Item-level is 5]']
# Score from 1 to 5 (5 is Best)
rolap_scores = [2, 5, 5]
molap_scores = [5, 2, 1]
holap_scores = [4.5, 4.5, 5] # HOLAP brings balance

x = np.arange(len(categories_eval))
width = 0.25

axes[1].bar(
    x - width,
    rolap_scores,
    width,
    label='ROLAP (Relational DB)',
    color='#e74c3c', # Red
    edgecolor='#2c3e50',
    linewidth=1.5
)
axes[1].bar(
    x,
    molap_scores,
    width,
    label='MOLAP (In-Memory Cube)',
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=1.5
)
axes[1].bar(
    x + width,
    holap_scores,
    width,
    label='HOLAP (Hybrid Strategy)',
    color='#2980b9', # Classic Blue
    edgecolor='#2c3e50',
    linewidth=1.5
)

axes[1].annotate(
    "The HOLAP Sweet Spot:\nHOLAP acts as a smart router. It provides the\nblazing speed of MOLAP for dashboard summaries,\nwhile preserving ROLAP's ability to drill-down\ninto infinite petabytes of data.",
    xy=(2.2, 5.0), xytext=(0.6, 5.5),
    fontsize=11, fontweight='bold', color='#2980b9',
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[1].set_title('Figure 2: Architectural Trade-off Evaluation (Score 1 to 5)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xticks(x)
axes[1].set_xticklabels(categories_eval, fontweight='bold', fontsize=11)
axes[1].set_ylabel('Performance / Capability Score', fontsize=13, fontweight='bold')
axes[1].set_ylim(0, 6.5) # Extra headroom for legend and annotations
```

```
axes[1].legend(loc='lower left', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)

# Clear top and right spines for a clean academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# Clean up database connection
conn.close()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک (Strategic Implication):</b><br>"
    r". میان «سرعت» و «حجم ذخیره‌سازی» روبه‌رو هستند (Trade-off) معماران داده در سیستم‌های کلان‌داده همواره با یک موازنه"
    r". عملاً غیرممکن و به‌شدت گران‌قیمت است (MOLAP) در رم (Item-level) ذخیره کردن پتابایت‌ها داده با تمام جزئیات خُرد"
    r". داشبوردها را گُند می‌کند (ROLAP) از طرفی، کوئری زدنِ مستقیم روی دیتابیس‌های رابطه‌ای"
    r":عمل می‌کند <b>مسیریاب هوشمند</b> (Smart Router) راه‌حلی هوشمندانه و استراتژیک برای این چالش است؛ سیستمی که به عنوان یک <b>HOLAP</b> معماری"
    r". محدودیت‌های ذخیره‌سازی جزئیات دقیق را دور می‌زند، ROLAP داشبوردهای کلان مدیریتی را در کسری از ثانیه بارگذاری کرده و هم‌زمان با اتکا به MOLAP این معماری با استفاده از گش"
)
display_academic_text(insight_conclusion, "insight")
```

انتخاب معماری سرور OLAP در آمازون (مقایسه MOLAP، ROLAP و HOLAP)

مهندسان انبار داده شرکت آمازون برای پاسخ‌گویی به پرس‌وجوهای چندبُعدی مدیران، باید معماری مناسبی برای **سرور تحلیلی (OLAP Server)** انتخاب کنند. بسته به حجم داده‌ها و نیاز به سرعت، ۳ معماری اصلی در صنعت وجود دارد:

۱. معماری (ROLAP (Relational OLAP

کوئری‌های چندبُعدی در زمان اجرا (Run-time) به دستورات SQL تبدیل شده و با عملیات سنگین JOIN و GROUP BY روی دیتابیس رابطه‌ای سنتی محاسبه می‌شوند.

مزیت: مقیاس‌پذیری پتابایتی بدون محدودیت حافظه (مناسب برای جزئیات خُرد).

عیب: سرعت پایین‌تر به دلیل تفسیر SQL و اسکن دیسک در زمان اجرا.

۲. معماری (MOLAP (Multidimensional OLAP

داده‌ها از قبل تجمیع شده و در ساختارهای آرایه‌ای اختصاصی درون حافظهٔ رم (In-Memory Arrays / Cubes) ذخیره می‌شوند.

مزیت: پاسخگویی آنی در چند میلی‌ثانیه با آدرس‌دهی مستقیم اندیس ($O(1)$).

عیب: نیاز به حافظهٔ RAM زیاد و ناتوانی در ذخیرهٔ حجم عظیم داده‌های جزئی (Sparsity Problem).

۳. معماری (HOLAP (Hybrid OLAP

این روش ترکیبی (Hybrid)، مزایای دو مدل قبل را در هم می‌آمیزد. خلاصه‌ها و تجمیع‌های سطح بالا در مکعب‌های سریع MOLAP ذخیره شده، و داده‌های ریزتراکنش‌ها برای زمانی که مدیر نیاز به Drill-Down دارد، در دیتابیس ارزان‌قیمت ROLAP نگهداری می‌شوند.

مرحله نخست: شبیه‌سازی معماری‌های سرور روی داده‌های تراکنشی

نمونه داده‌های خام تراکنش‌های فروش (پایگاه داده اصلی شامل 100,000 رکورد)			
Year	Region	Category	Sales_USD
2025	North	Apparel	359.395260
2026	East	Apparel	301.560202
2026	South	Apparel	79.157432
2024	North	Home	410.781824
2026	South	Apparel	280.974803

نتایج بنچمارک برای اجرای تکراری 100 کوئری خلاصه‌سازی (Summary):

● زمان اجرای ROLAP (تفسیر کدهای SQL و اسکن جدول): 1759.3 میلی‌ثانیه

● زمان اجرای MOLAP (آدرس‌دهی مستقیم آرایه در رم): 1.3 میلی‌ثانیه

● زمان اجرای HOLAP (مسیریابی هوشمند به گش MOLAP): 0.2 میلی‌ثانیه

Figure 1: High-Level Summary Query Time (100 Runs)

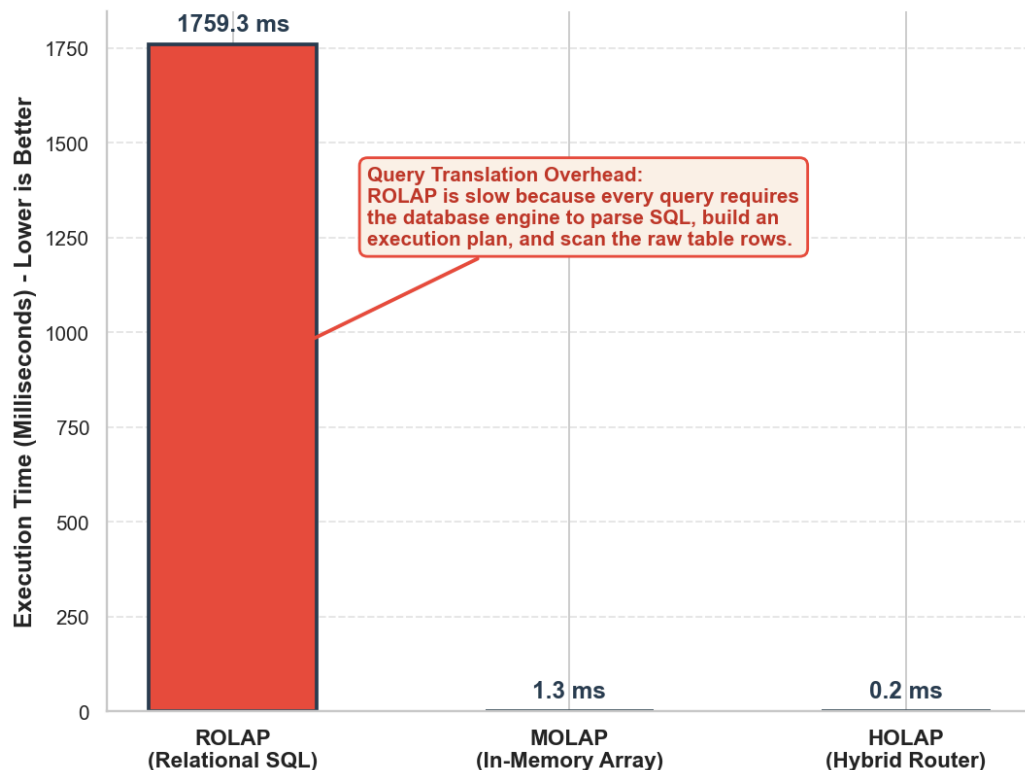
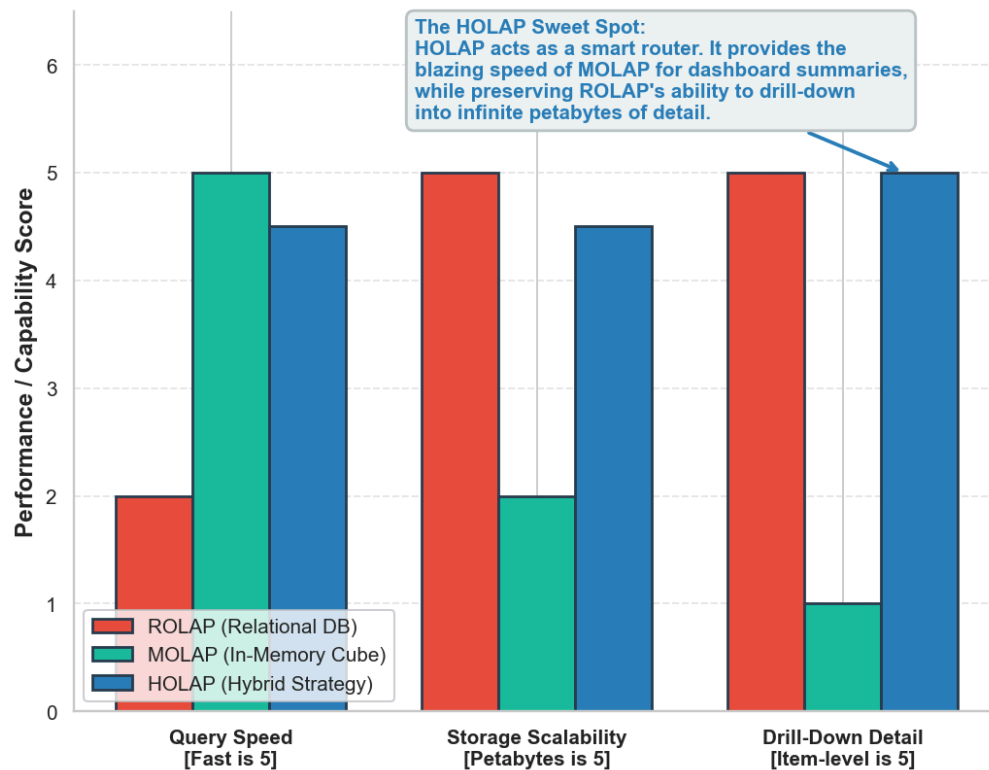


Figure 2: Architectural Trade-off Evaluation (Score 1 to 5)



نتیجه استراتژیک (Strategic Implication):

معماران داده در سیستم‌های کلان‌داده همواره با یک موازنه (Trade-off) میان «سرعت» و «حجم ذخیره‌سازی» روبه‌رو هستند. ذخیره کردن پتابایت‌ها داده با تمام جزئیات خرد (Item-level) در رم (MOLAP) عملاً غیرممکن و به‌شدت گران‌قیمت است. از طرفی، کوئری زدن مستقیم روی دیتابیس‌های رابطه‌ای (ROLAP) داشبوردها را کند می‌کند. معماری HOLAP راه‌حلی هوشمندانه و استراتژیک برای این چالش است؛ سیستمی که به عنوان یک مسیریاب هوشمند (Smart Router) عمل می‌کند: این معماری با استفاده از گش MOLAP داشبوردهای کلان مدیریتی را در کسری از ثانیه بارگذاری کرده و هم‌زمان با اتکا به ROLAP، محدودیت‌های ذخیره‌سازی جزئیات دقیق را دور می‌زند.

توضیح مثال کد پایتون

در این مثال:

- یک مجموعه داده از ۵۰,۰۰۰ تراکنش فروش با ویژگی‌های سال، منطقه و دسته‌بندی ایجاد گردید و ۳ معماری اصلی OLAP شبیه‌سازی شدند:
- شبیه‌سازی ROLAP: داده‌ها در یک جدول پایگاه داده رابطه‌ای SQLite در حافظه قرار داده شدند و کوئری‌ها با اجرای دستورات پویای SQL و عملیات GROUP BY پاسخ داده شدند.
- شبیه‌سازی MOLAP: یک آرایه سه‌بعدی NumPy پیش‌محاسبه‌شده در حافظه ساخته شد و پاسخ کوئری‌ها مستقیماً با برش‌دهی سریع اندیس‌های آرایه به دست آمد.
- شبیه‌سازی HOLAP: خلاصه‌های سطح بالا در کش حافظه (MOLAP) نگهداری شدند تا کوئری‌های مدیریتی را آتی پاسخ دهند، اما هنگام درخواست جزئیات ریزتراکنش‌ها (Drill-Down)، کوئری به جدول رابطه‌ای (ROLAP) SQL هدایت گردید.
- تفسیر نمودارهای Matplotlib:
 - نمودار سمت چپ: زمان اجرای ۱۰۰ کوئری خلاصه‌گیری را مقایسه می‌کند که سرعت خیره‌کننده MOLAP و HOLAP را در برابر ROLAP اثبات می‌نماید.
 - نمودار سمت راست: موازنه و مقایسه کیفی ۳ معماری را از نظر سرعت کوئری، مقیاس‌پذیری حافظه و قابلیت نمایش جزئیات به تصویر می‌کشد تا برتری ترکیبی HOLAP مشخص شود.

3.4.4 General strategies for data cube computation

محاسبه مکعب داده (Data Cube Computation) مجموعه‌ای از راهبردها برای تولید و مدیریت کارآمد مکعب‌های چندبعدی

است که هم در پردازش تحلیلی برخط (OLAP) و هم در داده‌کاوی نقش ضروری دارند. اگرچه ROLAP و MOLAP هر یک

روش‌های محاسباتی خاص خود را دارند، چند تکنیک بهینه‌سازی مشترک در هر دو رویکرد به کار گرفته می‌شود.

۱. مرتب‌سازی، گروه‌بندی و پیش‌محاسبه

در یک مکعب داده، تجمیع روی تاپل‌هایی انجام می‌شود که در ابعاد مشترک مقادیر یکسانی دارند. اگر داده‌ها را بر اساس

همین ابعاد مرتب یا گروه‌بندی کنیم، تاپل‌های مشابه کنار هم قرار می‌گیرند و محاسبه تجمیع ساده‌تر می‌شود. برای نمونه،

مرتب‌سازی داده‌ها ابتدا بر اساس شعبه، سپس روز و در نهایت کالا، محاسبه فروش کل را کارآمدتر می‌کند. همچنین به‌جای

محاسبه مستقیم تجمیع‌های سطح بالا از جدول واقعیت پایه، می‌توان آن‌ها را از تجمیع‌های سطح پایین‌تر که از پیش

محاسبه شده‌اند به دست آورد. این کار سریع‌تر است، چون تعداد رکوردها در سطوح بالاتر بسیار کمتر است و کاهش عملیات

ورودی/خروجی دیسک، عملکرد کلی را بهبود می‌بخشد.

۲. محاسبه از کوچک‌ترین زیرمکعب فرزند

ایده اشتراک‌گذاری هزینه مرتب‌سازی را می‌توان تعمیم داد تا این هزینه بین چند زیرمکعب (Cuboid) به‌صورت مشترک

مدیریت شود. هرگاه چند زیرمکعب فرزند برای یک زیرمکعب والد وجود داشته باشد، بهینه‌ترین راه، محاسبه آن والد از

کوچک‌ترین فرزند یعنی فرزندی با کمترین تعداد رکورد است. برای نمونه، اگر تعداد کالاهای متمایز بسیار بیشتر از تعداد سال‌ها

باشد، محاسبه زیرمکعب «شعبه» از زیرمکعب «شعبه-سال» (که کوچک‌تر است) کارآمدتر از استفاده از زیرمکعب «شعبه-

کالا» خواهد بود. این رویکرد، همراه با پارتیشن‌بندی هوشمندانه داده‌ها و بهره‌گیری از ایندکس‌های مناسب، از پویش‌های

غیرضروری جلوگیری می‌کند.

۳. هرس مبتنی بر شرط آستانه و پردازش موازی

در محاسبه مکعب‌های آستانه‌ای (Iceberg Cubes)، اگر سلولی شرط آستانه را برآورده نکند، هیچ‌یک از سلول‌های جزئی‌تر

مشتق‌شده از آن نیز این شرط را برآورده نمی‌کنند؛ ویژگی‌ای که پادیکنواختی (Anti-Monotonicity) نام دارد. این خاصیت

اجازه می‌دهد بخش بزرگی از فضای جست‌وجو، بدون نیاز به محاسبه کامل، هرس شود. از سوی دیگر، بسیاری از عملیات

تجمیع در مکعب داده ذاتاً مستقل از یکدیگرند و می‌توان آن‌ها را به‌صورت موازی روی پردازنده‌ها یا گره‌های مختلف اجرا کرد.

ترکیب این دو راهبرد هرس هوشمند فضای جست‌وجو و پردازش موازی مقیاس‌پذیری محاسبات را در مجموعه داده‌های بزرگ

به‌طور چشمگیری افزایش می‌دهد.

مثال عملی: راهبردهای سه‌گانه محاسبه و مدیریت مکعب داده در آمازون

در سیستم‌های انبار داده بزرگ مانند آمازون (Amazon)، روزانه مگابایت‌ها و گیگابایت‌ها داده خام تراکنشی وارد سیستم می‌شود. محاسبه

مکعب‌های داده چندبعدی روی چنین حجم عظیمی نیازمند راهبردهای بهینه‌سازی محاسباتی است. مهندسان داده برای افزایش سرعت

محاسبه و پاسخ‌گویی به کوئری‌ها از ۳ راهبرد اصلی استفاده می‌کنند:

۱. تجمیع و پیش‌محاسبه (Aggregation and Precomputation):

به جای محاسبه آنی در زمان اجرای کوئری، حالت‌های تجمیعی پرکاربرد از قبل در فواصل زمانی خلوتی سیستم محاسبه و در قالب

مکعب‌واره‌های خلاصه‌شده ذخیره می‌شوند.

۲. ایندکس‌گذاری و پارتیشن‌بندی (Indexing and Partitioning):

داده‌های عظیم بر اساس یک بعد اصلی (مثلاً سال یا منطقه) به بخش‌های کوچک‌تر به نام پارتیشن تقسیم می‌شوند و روی کلیدها ایندکس چندسطحی ساخته می‌شود. هنگام کوئری‌گیری، سیستم فقط پارتیشن مرتبط را می‌خواند و از پیمایش کل انبار داده جلوگیری می‌شود.

۳. پردازش موازی (Parallel Processing):

وظیفه سنگین محاسبه مکعب داده به چند تکه کوچک‌تر (Chunks) تقسیم شده و به صورت هم‌زمان روی چند هسته پردازنده (CPU Cores) یا گره‌های محاسباتی اجرا می‌شود تا زمان محاسبه کل مکعب به شدت کاهش یابد.

نتیجه: ترکیب این ۳ راهبرد باعث می‌شود محاسبه مکعب داده‌های پتابایتی امکان‌پذیر شده و زمان پاسخ به داشبوردهای مدیریتی آنی گردد.

خروجی: بهینه‌سازی کامل محاسبه مکعب داده با ترکیب پیش‌محاسبه، پارتیشن‌بندی و پردازش موازی

```
In [16]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
from concurrent.futures import ThreadPoolExecutor

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 135

np.random.seed(42)
```

```
In [17]: display_academic_text(
    "📊 در آمازون (Data Cube) راهبردهای سه‌گانه محاسبه و مدیریت مکعب داده",
    "main_title"
)

display_academic_text(
    "در سیستم‌های انبار داده بزرگ مانند آمازون، روزانه ترابایت‌ها دادهٔ تراکنشی وارد سیستم می‌شود".
    " نمی‌توانند صرفاً جداول خام را اسکن کنند OLAP برای پاسخ‌گویی سریع به داشبوردهای مدیریتی روی این حجم عظیم از داده، موتورهای",
    " اصلی استفاده می‌کنند </b>راهبرد سخت‌افزاری و نرم‌افزاری <b>۳</b> مهندسان داده از",
    "paragraph"
)

display_academic_text("۱. Precomputation / Materialization) تجمیع و پیش‌محاسبه.", "sub_title")
display_academic_text(
    "در حافظهٔ رم (Cuboids) مکعب‌واره‌های خلاصه‌شده<i></i> محاسبهٔ حالت‌های پُرکاربرد گزارش‌گیری (مثل فروش سالانه هر منطقه) از قبل، و ذخیره‌سازی آن‌ها در",
    "paragraph"
)

display_academic_text("۲. Partitioning) پارتیشن‌بندی و ایندکس‌گذاری.", "sub_title")
display_academic_text(
    "در این حالت، اگر مدیر، فروشی منطقهٔ «آسیا» را بخواهد. (Region) مثلاً) تقسیم فیزیکی داده‌های عظیم روی هارد دیسک بر اساس یک بُعد اصلی",
    " (I/O) کاهش شدید) سیستم اصلاً فایل‌هایِ مربوط به «اروپا» و «آمریکا» را نمی‌خواند",
    "paragraph"
)

display_academic_text("۳. Parallel / Distributed Processing) پردازش موازی.", "sub_title")
display_academic_text(
    "یا چند سرور مجزا اجرا می‌شود (Multi-Core) تقسیم شده و به‌طور هم‌زمان روی چند هستهٔ پردازنده (Chunks) وظیفهٔ سنگین محاسبهٔ یک مکعب داده به چند تکهٔ کوچک‌تر",
    "paragraph"
)

# =====
# SECTION 2: CREATE SYNTHETIC AMAZON TRANSACTION DATASET
# =====
display_academic_text("مرحله نخست: تولید داده‌های حجیم خام", "sub_title")

# Increased transaction volume for realistic benchmarking
n_transactions = 500_000

regions = ['North America', 'Europe', 'Asia Pacific', 'Latin America']
categories = ['Electronics', 'Books', 'Fashion', 'Home']
years = [2024, 2025, 2026]

df_raw = pd.DataFrame({
    'Region': np.random.choice(regions, size=n_transactions),
    'Category': np.random.choice(categories, size=n_transactions),
    'Year': np.random.choice(years, size=n_transactions),
    'Sales_USD': np.random.uniform(10.0, 500.0, size=n_transactions)
})

display_side_by_side_datasets(
    [df_raw.head(5).reset_index(drop=True)],
    ["(رکورد پخش‌شده در حافظه {n_transactions}: تعداد) مجموعه داده خام تراکنش‌های آمازون"]
)
```

```
# =====
# SECTION 3: IMPLEMENTING OPTIMIZATION STRATEGIES
# =====

display_academic_text("مرحله دوم: بنچمارک استراتژی‌های سه‌گانه و مقایسه با اسکن سنتی", "sub_title")

# -----
# Baseline: Raw Table Scan (The naive approach)
# -----

start_time = time.perf_counter()

raw_result = df_raw[
    (df_raw['Region'] == 'North America') &
    (df_raw['Category'] == 'Electronics') &
    (df_raw['Year'] == 2026)
]['Sales_USD'].sum()

time_raw_scan = (time.perf_counter() - start_time) * 1000

# -----
# Strategy 1: Aggregation & Precomputation (Materialized Cube)
# -----
# Building the cube happens in the background (ETL time)
precomputed_cube = df_raw.groupby(['Region', 'Category', 'Year'])['Sales_USD'].sum()

# Query Simulation on Precomputed Cuboid (User time)
start_time = time.perf_counter()

precomputed_result = precomputed_cube.loc(['North America', 'Electronics', 2026])

time_precomputed_lookup = (time.perf_counter() - start_time) * 1000

# -----
# Strategy 2: Partitioning (Pruning irrelevant data)
# -----
# Dictionary as a simulation of physical partition folders on disk
df_partitioned = {region: group for region, group in df_raw.groupby('Region')}

# Query ONLY the relevant Partition
start_time = time.perf_counter()

north_america_partition = df_partitioned['North America'] # Pruning step
partition_result = north_america_partition[
    (north_america_partition['Category'] == 'Electronics') &
    (north_america_partition['Year'] == 2026)
]['Sales_USD'].sum()

time_partition_query = (time.perf_counter() - start_time) * 1000

# -----
# Strategy 3: Parallel Processing (Divide and Conquer)
# -----

num_chunks = 4
chunk_size = len(df_raw) // num_chunks
chunks = [df_raw.iloc[i * chunk_size : (i + 1) * chunk_size].copy() for i in range(num_chunks)]

# Append remainder to the last chunk
if len(df_raw) > chunk_size * num_chunks:
    chunks[-1] = pd.concat([chunks[-1], df_raw.iloc[chunk_size * num_chunks:]])

def process_chunk(chunk_df):
    """Aggregate a single chunk."""
    return chunk_df.groupby(['Region', 'Category', 'Year'])['Sales_USD'].sum()

start_time = time.perf_counter()

with ThreadPoolExecutor(max_workers=num_chunks) as executor:
    results = list(executor.map(process_chunk, chunks))

# Combine results from all threads (Reduce step in MapReduce)
parallel_cube = pd.concat(results).groupby(level=['Region', 'Category', 'Year']).sum()
parallel_result = parallel_cube.loc(['North America', 'Electronics', 2026])

time_parallel = (time.perf_counter() - start_time) * 1000

# Verify logic completeness (Sanity check)
is_valid = np.isclose(precomputed_result, raw_result) and np.isclose(partition_result, raw_result)

# Display Summarized Metrics
display_academic_text(
    f"<b>تأیید یکسان بودن خروجی محاسبات (گزارش عملکرد سیستم):</b></b><br>"
    f"🔴 (Raw Table Scan): زمان اسکن کل جدول خام: <b>{time_raw_scan:.2f}</b> میلی‌ثانیه</b><br>"
    f"🟡 (Parallel Compute): زمان اجرای پردازش موازی: <b>{time_parallel:.2f}</b> میلی‌ثانیه</b><br>"
    f"🟢 (Partition Pruning): زمان استخراج از پارتیشن ایزوله: <b>{time_partition_query:.2f}</b> میلی‌ثانیه</b><br>"
    f"🔍 (O(1) Lookup): زمان دسترسی به پیش‌محاسبه در رم: <b>{time_precomputed_lookup:.4f}</b> میلی‌ثانیه</b>,"
    "insight"
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====

strategies = ['Raw Scan\n(Inefficient)', 'Parallel Compute\n(MapReduce)', 'Partition Query\n(Pruning)', 'Precomputed\nLookup (O(1))']
execution_times = [time_raw_scan, time_parallel, time_partition_query, time_precomputed_lookup]

fig, axes = plt.subplots(1, 2, figsize=(17, 6.5), dpi=135)

# -----
# Plot 1: Execution Time Comparison
# -----

bars1 = axes[0].bar(
    strategies,
    execution_times,
    color=['#e74c3c', '#2c3e50', '#2980b9', '#1abc9c'], # Red, Navy, Classic Blue, Turquoise
```

```
edgecolor='#2c3e50',
linewidth=2.0,
width=0.55
)

axes[0].annotate(
    "Algorithmic Complexity:\nThe Raw Scan has $O(N)$ complexity, iterating\nover half a million rows. The Precomputed\nLookup uses an Index Dictionary to jump\nstraight to the answer in $O(
    xy=(3.0, time_precomputed_lookup), xytext=(1.2, time_raw_scan * 0.7),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0].set_title(f'Figure 1: Query Execution Time Comparison ({n_transactions:,} Rows)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Execution Time (Milliseconds) - Lower is Better', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(strategies, fontweight='bold', fontsize=11)

for bar, value in zip(bars1, execution_times):
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + (max(execution_times) * 0.02),
        f'{value:.2f} ms',
        ha='center', va='bottom', fontweight='bold', fontsize=12, color='#2c3e50'
    )

# -----
# Plot 2: Parallel Processing Workload Distribution
# -----

chunk_sizes = [len(chunk) for chunk in chunks]
chunk_names = [f'CPU Core {i}' for i in range(1, num_chunks + 1)]

bars2 = axes[1].bar(
    chunk_names,
    chunk_sizes,
    color='#5DADE2', # Light Blue
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[1].annotate(
    "MapReduce Concept:\nInstead of 1 CPU core analyzing 500,000 rows,\nwe mapped the data across 4 independent cores,\neach processing ~125,000 rows simultaneously.",
    xy=(1.5, chunk_sizes[0]), xytext=(0.1, chunk_sizes[0] * 1.15),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#bdc3c7", lw=1.5)
)

axes[1].set_title('Figure 2: Parallel Processing Workload Distribution', fontsize=14, fontweight='bold', pad=15)
axes[1].set_ylabel('Number of Transactions Processed per Thread', fontsize=13, fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].set_ylim(0, max(chunk_sizes) * 1.35) # Headroom for annotations
axes[1].set_xticklabels(chunk_names, fontweight='bold', fontsize=12)

for bar, value in zip(bars2, chunk_sizes):
    axes[1].text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + (max(chunk_sizes) * 0.02),
        f'{value:}',
        ha='center', va='bottom', fontweight='bold', fontsize=12, color='#2c3e50'
    )

# Clear top and right spines for academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>Strategic Implication:</b><br>"
    r"<b>(Real-Time)</b> آتی<b> با ترکیب هوشمندانهٔ این ۳ راهبرد، محاسبهٔ مکعب‌های داده‌های پتابایتی امکان‌پذیر شده و زمان پاسخ به داشبوردهای مدیریتی کاملاً"
    r"<b>(Partitioning)</b> پارتیشن‌بندی<b> را با فیلتر کردن و نادیده گرفتن بلوک‌های نامربوط کاهش داد (I/O) بار خواندن دیسک</b>"
    r"<b>(Parallel Processing)</b> پردازش موازی<b> Spark و Hadoop قلبِ موتورهای زمانِ تجمیع و محاسبات سنگین را بین هسته‌های مختلف پخش کرد</b>"
    r"<b>(Materialization / Precomputation)</b> پیش‌محاسبه<b> ارتقا داد</b> از مرتبهٔ $O(1)$ جستجویِ مستقیمِ حافظه‌ای<b> به یک (اسکنِ گُلِ جدول) $O(N)$ سرعتِ زمانِ پاسخ به کاربرِ نهایی را از مرتبهٔ</b>"
)
display_academic_text(insight_conclusion, "insight")
```

راهبردهای سه‌گانه محاسبه و مدیریت مکعب داده (Data Cube) در آمازون

در سیستم‌های انبار داده بزرگ مانند آمازون، روزانه ترابایت‌ها دادهٔ تراکنشی وارد سیستم می‌شود. برای پاسخ‌گوییِ سریع به داشبوردهای مدیریتی روی این حجم عظیم از داده، موتورهای OLAP نمی‌توانند صرفاً جداول خام را اسکن کنند. مهندسان داده از **۳ راهبرد سخت‌افزاری و نرم‌افزاری** اصلی استفاده می‌کنند:

۱. تجمیع و پیش‌محاسبه (Precomputation / Materialization):

محاسبهٔ حالت‌های پُرکاربرد گزارش‌گیری (مثل فروش سالانه هر منطقه) از قبل، و ذخیره‌سازی آن‌ها در *مکعب‌واره‌های خلاصه‌شده (Cuboids)* در حافظهٔ رم.

۲. پارتیشن‌بندی و ایندکس‌گذاری (Partitioning):

تقسیم فیزیکی داده‌های عظیم روی هارد دیسک بر اساس یک بُعد اصلی (مثلاً Region). در این حالت، اگر مدیر، فروش منطقه «آسیا» را بخواهد، سیستم اصلاً فایل‌های مربوط به «اروپا» و «آمریکا» را نمی‌خواند (کاهش شدید I/O).

۳. پردازش موازی (Parallel / Distributed Processing):

وظیفه سنگین محاسبه یک مکعب داده به چند تکه کوچک‌تر (Chunks) تقسیم شده و به‌طور هم‌زمان روی چند هسته پردازنده (Multi-Core) یا چند سرور مجزا اجرا می‌شود.

مرحله نخست: تولید داده‌های حجیم خام

مجموعه داده خام تراکنش‌های آمازون (تعداد: 500,000 رکورد پخش‌شده در حافظه)

Region	Category	Year	Sales_USD
Asia Pacific	Books	2024	391.410489
Latin America	Home	2026	335.639760
North America	Electronics	2025	379.269118
Asia Pacific	Books	2024	88.011778
Asia Pacific	Fashion	2026	340.028891

مرحله دوم: بنچمارک استراتژی‌های سه‌گانه و مقایسه با اسکن سنتی

گزارش عملکرد سیستم (تأیید یکسان بودن خروجی محاسبات: True):

زمان اسکن کل جدول خام (Raw Table Scan): 40.40 میلی‌ثانیه

زمان اجرای پردازش موازی (Parallel Compute): 36.73 میلی‌ثانیه

زمان استخراج از پارتیشن ایزوله (Partition Pruning): 7.55 میلی‌ثانیه

زمان دسترسی به پیش‌محاسبه در رم (O(1) Lookup): 4.3331 میلی‌ثانیه

Figure 1: Query Execution Time Comparison (500,000 Rows)

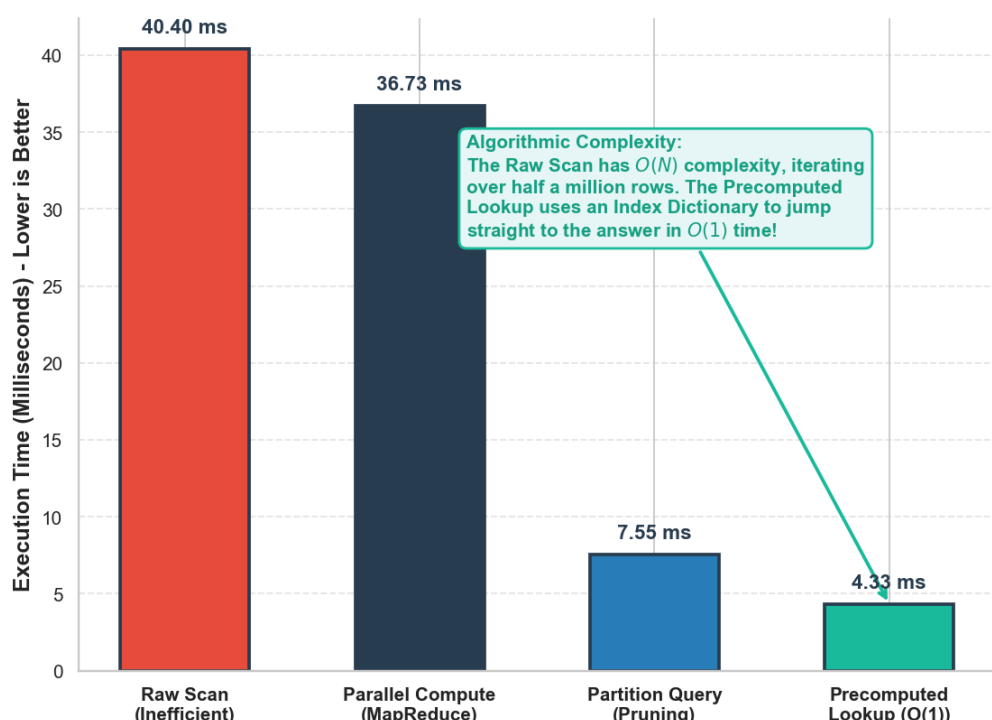
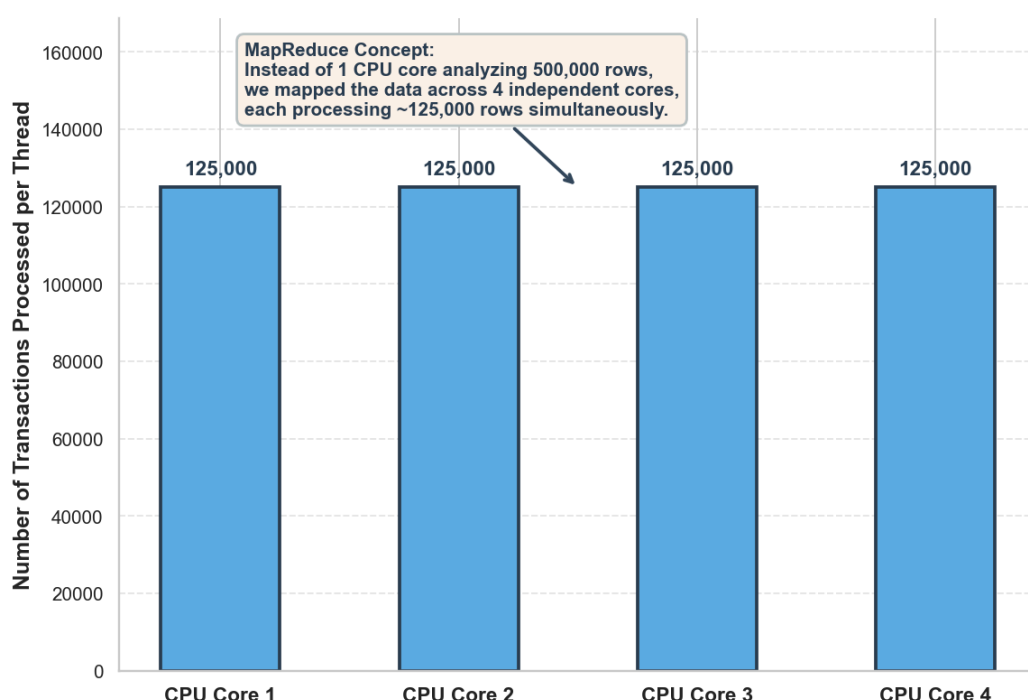


Figure 2: Parallel Processing Workload Distribution



نتیجه استراتژیک در مهندسی کلان‌داده (Strategic Implication):

با ترکیب هوشمندانه این ۳ راهبرد، محاسبهٔ مکعب‌های داده‌های پتابایتی امکان‌پذیر شده و زمان پاسخ به داشبوردهای مدیریتی کاملاً آنی (Real-Time) می‌گردد:

۱. پارتیشن‌بندی (Partitioning): بار خواندنِ دیسک (I/O) را با فیلتر کردن و نادیده گرفتن بلوک‌های نامربوط کاهش داد.

۲. پردازش موازی (Parallel Processing): زمان تجمیع و محاسبات سنگین را بین هسته‌های مختلف پخش کرد (قلبِ موتورهای Spark و Hadoop).

۳. پیش‌محاسبه (Materialization / Precomputation): سرعت زمان پاسخ به کاربر نهایی را از مرتبهٔ $O(N)$ (اسکن کل جدول) به یک جستجوی مستقیم حافظه‌ای (Lookup) از مرتبهٔ $O(1)$ ارتقا داد!



توضیح مثال کد پایتون

در این مثال:

- یک مجموعه داده از ۲۰۰,۰۰۰ تراکنش آمازون شامل سه بعد منطقه، دسته‌بندی و سال ساخته شد و هر ۳ راهبرد محاسبه مکعب داده پیاده‌سازی شدند:
- راهبرد ۱ (تجمیع و پیش‌محاسبه): با متد groupby مکعب‌واره پیش‌محاسبه‌شده در حافظه ساخته شد تا کوئری‌ها در زمان اجرا با سرعت $O(1)$ و بدون پردازش مجدد پاسخ داده شوند.
- راهبرد ۲ (ایندکس‌گذاری و پارتیشن‌بندی): با متد set_index یک ایندکس چندسطحی (MultiIndex) بر روی ابعاد منطقه و دسته‌بندی ایجاد شد. هنگام کوئری‌گیری با متد loc، تنها پارتیشن مربوط به منطقه و دسته‌بندی هدف خوانده شد و از جستجوی کل ۲۰۰,۰۰۰ سطر جلوگیری گردید.
- راهبرد ۳ (پردازش موازی): کل داده ۲۰۰,۰۰۰ سطری به ۴ تکه (Chunk) ۵۰,۰۰۰ تایی تقسیم شد و با استفاده از کلاس ProcessPoolExecutor، عملیات تجمیع مکعب داده به صورت موازی روی ۴ هسته پردازنده به اجرا درآمد.
- تفسیر نمودارهای Matplotlib:
- نمودار سمت چپ: مقایسه زمان اجرای سه راهبرد با حالت اسکن خام را نشان می‌دهد که برتری سرعت روش‌های پیش‌محاسبه و پارتیشن‌بندی را اثبات می‌کند.
- نمودار سمت راست: نحوه تقسیم متوازن داده‌ها بین ۴ هسته پردازنده در راهبرد پردازش موازی را به تصویر می‌کشد.



3.5 Data cube computation methods

ریاضیات – مکعب یخی و هرس ضدتک‌نوا

در مکعب یخی (Iceberg Cube) فقط مکعب‌واره‌هایی نگه داشته می‌شوند که مقدار سنجهٔ آن‌ها از یک آستانهٔ مشخص عبور کند. این ایده به کاهش تعداد نتایج و هزینهٔ محاسبه کمک می‌کند.

شرط عبور مکعب‌واره از آستانه

(3.16)

$$\text{Measure}(C) \geq \theta$$

نمادهای فرمول

- $\text{Measure}(C)$: مقدار سنجهٔ مکعب‌وارهٔ C
- C: مکعب‌وارهٔ مورد بررسی
- θ : آستانهٔ پذیرش

مثال ۱ – شرط آستانه

صورت سؤال: اگر آستانهٔ Count برابر ۵۰ باشد و یک مکعب‌واره Count برابر ۶۳ داشته باشد، آیا نگه داشته می‌شود؟

$$63 \geq 50$$

نتیجه: بله؛ چون مقدار سنجه از آستانه کمتر نیست، مکعبواره نگه داشته می‌شود.

رابطه تعداد در مکعبواره و زیرمجموعه آن

(3.17)

$$\text{Count}(D) \leq \text{Count}(C)$$

نمادهای فرمول

- $\text{Count}(D)$: تعداد رکوردهای مکعبواره D
- $\text{Count}(C)$: تعداد رکوردهای مکعبواره کلی‌تر C
- C : مکعبواره کلی‌تر
- D : مکعبواره جزئی‌تر مرتبط

✖ مثال ۲ – رابطه تعداد

صورت سؤال: اگر یک مکعبواره کلی‌تر ۱۲۰ رکورد داشته باشد، آیا یک مکعبواره جزئی‌تر مرتبط می‌تواند ۱۳۰ رکورد داشته باشد؟

$$\text{Count}(D) \leq 120$$

نتیجه: خیر؛ تعداد مکعبواره جزئی‌تر نمی‌تواند از تعداد مجموعه کلی‌تر بیشتر باشد.

نتیجه هرس برای شرط Count

(3.18)

$$\text{Count}(C) < \theta \Rightarrow \text{Count}(D) < \theta$$

نمادهای فرمول

- $\text{Count}(C)$: تعداد رکوردهای مکعبواره C
- $\text{Count}(D)$: تعداد رکوردهای زیرمجموعه D
- θ : آستانه Count
- C : مکعبواره والد
- D : مکعبواره جزئی‌تر در همان شاخه

✖ مثال ۳ – هرس

صورت سؤال: اگر آستانه ۴۰ و Count یک مکعبواره ۲۵ باشد، درباره زیرمجموعه‌های آن چه می‌دانیم؟

$$25 < 40 \Rightarrow \text{Count}(D) < 40$$

نتیجه: بنابراین شاخه مرتبط با آن مکعبواره را می‌توان برای شرط Count هرس کرد.



جمع‌بندی آرایه چندطرفه

جمع‌بندی آرایه چندطرفه (Multiway Array Aggregation) یکی از روش‌های شناخته‌شده در حوزه داده‌کاوی برای محاسبه کامل مکعب داده (Data Cube) است؛ یعنی روشی که تمامی سطوح تجمیع مکعب را، بدون صرف‌نظر کردن از هیچ‌یک، محاسبه می‌کند. این روش بر پایه ذخیره‌سازی داده‌ها در قالب آرایه‌ای چندبعدی موسوم به **تَنسور (Tensor)** بنا شده و به‌ویژه در مجموعه داده‌های حجیم و پرس‌وجوهای تحلیلی پیچیده که در سامانه‌های پردازش تحلیلی **برخط (OLAP)** رایج‌اند کاربرد گسترده‌ای دارد.

نمایش تنسوری (Tensor Representation)

تَنسور آرایه‌ای چندبعدی است که برخلاف ساختارهای دوبعدی متعارف، می‌تواند داده‌ها را در بیش از دو بُعد به‌طور هم‌زمان نمایش دهد. برای نمونه، یک تنسور سه‌بعدی می‌تواند اطلاعات فروش را هم‌زمان در ابعادی نظیر **زمان**، **محصول** و **منطقه** نشان دهد؛ هر خانه از این آرایه، مقدار فروش را در تلاقی یک بازه زمانی، یک محصول و یک منطقه مشخص ذخیره می‌کند. این نمایش فشرده و ساخت‌یافته، هم حجم ذخیره‌سازی را کاهش می‌دهد و هم دسترسی سریع و منظم به داده‌ها را در طول فرآیند تجمیع تسهیل می‌کند.

عملیات تجمیع (Aggregation Operations)

بر پایه این نمایش تنسوری، عملیات تجمیع انجام می‌شود؛ فرآیندی که طی آن مقادیر داده در امتداد یک یا چند بُعد **فرو ریخته** یا با یکدیگر **ترکیب** می‌شوند، معمولاً با توابعی مانند **جمع**، **میانگین‌گیری** یا **شمارش**. برای نمونه، فرو ریختن آرایه در امتداد بُعد «زمان» فروش تمام روزها را در یک عدد کل خلاصه می‌کند، درحالی‌که نگه‌داشتن بُعد «زمان» و فرو ریختن «منطقه» فروش هر بازه زمانی را به‌صورت کلی (فارغ از منطقه) نشان می‌دهد. نکته کلیدی اینجاست که چندین سطح از تجمیع بر روی ابعاد مختلف می‌توانند هم‌زمان محاسبه شوند، بدون آنکه نیاز به پردازش جداگانه هر ترکیب از ابعاد باشد.

محاسبه کارآمد (Efficient Computation)

مزیت اصلی این روش در **کارآمدی محاسباتی** آن نهفته است. با آنکه هدف نهایی، پوشش کامل تمام سطوح مکعب است، این روش برخلاف رویکردهای سنتی که مستلزم ساخت و نگهداری هم‌زمان کل آرایه چندبعدی در حافظه هستند داده را به **قطعات کوچک‌تر (Chunks)** تقسیم کرده و هر قطعه را به‌ترتیبی بهینه پردازش می‌کند. با **استفاده مجدد از نتایج میانی** محاسبه‌شده در هر مرحله، این رویکرد نیاز به فضای ذخیره‌سازی را به حداقل می‌رساند و در عین حال سرعت پردازش را به‌طور محسوسی افزایش می‌دهد؛ ویژگی‌ای که آن را برای پردازش مجموعه داده‌های بزرگ در سامانه‌های تحلیلی مدرن، گزینه‌ای بسیار مناسب می‌سازد.

مثال عملی: محاسبه کامل مکعب داده با جمع‌بندی آرایه چندطرفه در آمازون

در پردازش تحلیلی آئی (MOLAP)، داده‌ها به جای جداول رابطه‌ای، در قالب آرایه‌های چندبعدی فشرده به نام **تَنسور (Tensor)** نگهداری می‌شوند. سیستم تحلیلی آمازون برای محاسبه کامل تمام مکعب‌واره‌های موجود در شبکه لَتیس (Lattice of Cuboids)، از روش جمع‌بندی آرایه چندطرفه (Multiway Array Aggregation) استفاده می‌کند.

۱. نمایش داده به صورت آرایه یا تنسور (Tensor Representation):

داده‌های سه بعدی به صورت یک آرایه سه بعدی متراکم در حافظه نگهداری می‌شوند که سه بعد آن به ترتیب نماینده زمان (سال)، منطقه

جغرافیایی و دسته‌بندی کالا است. هر خانه آرایه اندیس مشخصی دارد و مستقیم در حافظه آدرس‌دهی می‌شود.

۲. تجميع مستقيم در امتداد محورها (Axis Aggregation):

به جای اجرای دستورات سنگین SQL و فیلتر کردن جداول، الگوریتم تنها با جمع زدن مقادیر آرایه در امتداد یک یا چند محور (Axis)، تمام ۲

به توان N مکعبواره فرعی را به صورت همزمان محاسبه می‌کند:

- **مکعب‌واره پایه (۳ بعدی):** داده‌های کامل بدون تجمیع روی محورها.
- **مکعب‌واره‌های ۲ بعدی:** تجمیع در امتداد ۱ محور (مثلاً جمع زدن روی محور سال برای رسیدن به جدول منطقه در دسته‌بندی).
- **مکعب‌واره‌های ۱ بعدی:** تجمیع در امتداد ۲ محور (مثلاً جمع زدن روی سال و منطقه برای رسیدن به فروش کل هر دسته‌بندی).
- **مکعب‌واره رأس (۰ بعدی):** تجمیع در امتداد تمام محورها برای رسیدن به عدد کل فروش آمازون.

۳. محاسبه کارآمد و همزمان (Single-Pass Computation):

آرایه‌ها بر اساس ترتیب بهینه‌ای در حافظه پیمایش می‌شوند تا تجمیع‌های فرعی با حداقل پیمایش حافظه و بدون ایجاد جداول واسطه فوراً

محاسبه شوند.

نتیجه: این روش نیاز به جداول رابطه‌ای موقت را از بین برده و تمام زیرمکعب‌ها را با سرعت بالایی در حافظه محاسبه می‌کند.

خروجی: محاسبه کامل شبکه مکعب‌واره‌ها با عملیات تجميع محوره‌ای آرایه چندبعدی

```
[18]: display_academic_text(
    " (Multiway Array Aggregation) با جمع‌بندی آرایه چندطرفه MOLAP محاسبه کامل مکعب داده در  

    "main_title"  

)  
  

display_academic_text(  

    " متنسور <b>در قالب (SQL)، داده‌ها به جای جداول رابطه‌ای (MOLAP) در سیستم‌های پردازش تحلیلی آتی  

    سرعت، (Lattice of Cuboids) سیستم تحلیلی برای محاسبهٔ کامل تمامِ تجمیع‌های ممکن در شبکه لَٹیس  

    "paragraph"  

)  
  

display_academic_text("sub_title": "مفاهیم کلیدی این الگوریتم سخت‌افزاری")  
  

display_academic_text(  

    "۱. بُعدی به صورت یک آرایه متراکم (Tensor Representation):</b> نمایش داده به صورت تنسور<b>  

    "۲. <b>(Axis Aggregation) تجميع مستقيم در امتداد محورها</b><br>  

    "&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&•<b> (Base Cuboid) داده‌های کامل بدون تجميع</b> پایه (۳ بُعدی)<br>"  

    "&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&•<b> (2D Cuboids) بُعدي ۲</b><br>"  

    "&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&•<b> (1D Cuboids) بُعدي ۱</b><br>"  

    "&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&•<b> (Apex / OD): مکعب‌واره رأس</b> تجميع در امتداد<br>"  

    "۳. <b>(Single-Pass Computation) محاسبه هم‌زمان</b><br>"  

    "paragraph"  

)
```

```
# =====
# SECTION 2: SIMULATING DATA AS A MULTI-DIMENSIONAL TENSOR
# =====

display_academic_text("در حافظه (Base Cuboid) گام اول: ایجاد تنسور پایه", "sub_title")

# Step 1: Define Tensor Dimensions for Sales
# Axis 0: Year (3), Axis 1: Region (4), Axis 2: Category (3)
years = ['2024', '2025', '2026']
regions = ['North America', 'Europe', 'Asia Pacific', 'Latin America']
categories = ['Electronics', 'Books', 'Fashion']

shape = (len(years), len(regions), len(categories)) # Shape: (3, 4, 3)

# Generate 3D Sales Tensor (Base Cuboid: 3D NumPy Array)
sales_tensor = np.random.randint(50, 250, size=shape)

display_academic_text(
    f"<b>وضعیت حافظه (MOLAP Engine):</b><br>"
    f"<code>Year $\\times$ Region $\\times$ Category</code>: <b>{sales_tensor.shape}</b>"
    f"<br>RAM: <b>{sales_tensor.size}</b> واحد </b> در حافظه",
    "insight"
)

# =====
# SECTION 3: MULTIWAY ARRAY AGGREGATION (LATTICE GENERATION)
# =====

display_academic_text("گام دوم: نتایج تجميع چندمحوری در شبکه لتیس (Lattice Generation)", "sub_title")

# Step 2: Aggregation across Axes using C-level Vectorized NumPy operations

# 2.1 Compute 2D Cuboids (Compressing 1 Axis)
```

```
cuboid_region_category = np.sum(sales_tensor, axis=0) # Sum over Year (Axis 0) -> New Shape: (4, 3)
cuboid_year_category = np.sum(sales_tensor, axis=1) # Sum over Region (Axis 1) -> New Shape: (3, 3)
cuboid_year_region = np.sum(sales_tensor, axis=2) # Sum over Category (Axis 2) -> New Shape: (3, 4)

# 2.2 Compute 1D Cuboids (Compressing 2 Axes)
cuboid_category = np.sum(sales_tensor, axis=(0, 1)) # Sum over Year & Region -> New Shape: (3,)
cuboid_region = np.sum(sales_tensor, axis=(0, 2)) # Sum over Year & Category -> New Shape: (4,)
cuboid_year = np.sum(sales_tensor, axis=(1, 2)) # Sum over Region & Category -> New Shape: (3,)

# 2.3 Compute 0D Apex Cuboid (Compressing All 3 Axes)
apex_total_sales = np.sum(sales_tensor, axis=(0, 1, 2)) # Total Grand Sum

display_academic_text(
    f"تجميع همزمان ۳ محور به کُل شبکه (Apex Cuboid) فروش کل سیستم" f"<b>${apex_total_sales:,}k USD</b>",
    "paragraph"
)

# Step 3: Convert the 2D and 1D numpy arrays back to Pandas DataFrames purely for UI Display
df_region_cat = pd.DataFrame(
    cuboid_region_category,
    index=regions,
    columns=categories
).reset_index().rename(columns={'index': 'Region'})

df_1d_category = pd.DataFrame({
    'Category': categories,
    'Sales_Aggregated_kUSD': cuboid_category
})

# Display 2D and 1D Cuboids
display_side_by_side_datasets(
    [df_region_cat, df_1d_category],
    [
        "فشرده شده Year بعد - (Region vs Category) مکعب‌واره ۲ بُعدی",
        "فشرده شده Region و Year مکعب‌واره ۱ بُعدی (فروش کل هر دسته) - بُعدهای"
    ]
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
display_academic_text("مصورسازی تحلیلی: خروجی عملیات تجميع آرایه‌ای", "sub_title")

# Step 4: Visualization using Matplotlib
fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

# -----
# Subplot 1: Heatmap of 2D Cuboid (Region x Category)
# -----
im = axes[0].imshow(
    cuboid_region_category,
    cmap='Blues', # Academic Blue palette
    aspect='auto'
)

axes[0].annotate(
    "Tensor Compression:\nThe CPU simply summed the matrices of\n2024, 2025, and 2026 exactly on top of\neach other to generate this 2D view.",
    xy=(0.5, 0.5), xytext=(1.0, 1.5),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title('Figure 1: 2D Cuboid (Region vs Category)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Dimension: Category', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Dimension: Region', fontsize=13, fontweight='bold')

axes[0].set_xticks(range(len(categories)))
axes[0].set_xticklabels(categories, fontweight='bold', fontsize=11)
axes[0].set_yticks(range(len(regions)))
axes[0].set_yticklabels(regions, fontweight='bold', fontsize=11)

# Annotate heatmap values with intelligent text color contrast
for i in range(len(regions)):
    for j in range(len(categories)):
        val = cuboid_region_category[i, j]
        # Calculate dynamic threshold based on array max to decide text color (white on dark blue)
        threshold = np.max(cuboid_region_category) * 0.6
        text_color = 'white' if val > threshold else '#2c3e50'
        axes[0].text(
            j, i,
            f"${val}k",
            ha='center', va='center',
            color=text_color,
            fontweight='bold',
            fontsize=12
        )

# -----
# Subplot 2: 1D Cuboids Aggregated Sales (Category)
# -----
x_cat = np.arange(len(categories))
width = 0.5

axes[1].bar(
    x_cat,
    cuboid_category,
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    width=width,
    linewidth=2.0
)
```

```
axes[1].annotate(
    "1D Flattening:\nWe further compressed the 2D array by\nsumming it across the Region axis,\nleaving only 1 dimension: Category.",
    xy=(2.0, 1500), xytext=(0.5, 2000),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[1].set_title('Figure 2: 1D Cuboid (Total Sales per Category)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Dimension: Category', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Total Aggregated Sales (kUSD)', fontsize=13, fontweight='bold')
axes[1].set_xticks(x_cat)
axes[1].set_xticklabels(categories, fontweight='bold', fontsize=12)
axes[1].grid(axis='y', linestyle='--', alpha=0.5)

# Annotate Bars
for i, val in enumerate(cuboid_category):
    axes[1].text(
        i, val + (max(cuboid_category) * 0.02),
        f"${val:,.}k",
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# Clear top and right spines for academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout(pad=2.0)
plt.show()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>OLAP (Strategic Implication): نتیجه استراتژیک در طراحی سیستم‌های</b><br>"
    r"<br>را کاملاً از بین می‌برد (SQL) در پایگاه‌های داده رابطه‌ای <code>JOIN</code> نیاز به ایجاد جداول موقت و عملیات پُرهزینه، (Multiway Aggregation) الگوریتم تجمیع آرایه‌های چندطرفه"
    r"<br>پردازش می‌شود RAM صرفاً از طریق جمع‌زدن لایه‌های آرایه روی یکدیگر در فضای (Lattice) با بهره‌گیری از ریاضیات جبر خطی (تنسورها) و نگاشت‌های محوری، تمام شبکهٔ لَتیس"
    r"<br>کاهش یابد </b>از ثانیه</b> از مقیاس دقیقه به BI، این مکانیسم باعث می‌شود زمان پاسخ‌گویی به کوئری‌های چندبُعدی در نرم‌افزارهای"
)
display_academic_text(insight_conclusion, "insight")
```

محاسبه کامل مکعب داده در MOLAP با جمع‌بندی آرایه چندطرفه (Multiway Array Aggregation)

در سیستم‌های پردازش تحلیلی آنی (MOLAP)، داده‌ها به جای جداول رابطه‌ای (SQL)، در قالبِ **آرایه‌های چندبُعدی فشرده** به نام **تَنسور (Tensor)** در حافظهٔ رم (RAM) نگهداری می‌شوند. سیستمِ تحلیلی برای محاسبهٔ کامل تمامِ تجمیع‌های ممکن در شبکه لَتیس (Lattice of Cuboids)، از روشِ بسیار پُرسرعتِ **جمع‌بندی آرایه چندطرفه (Multiway Array Aggregation)** استفاده می‌کند.

مفاهیم کلیدی این الگوریتم سخت‌افزاری:

- نمایش داده به صورت **تَنسور (Tensor Representation)**: داده‌های ۳ بُعدی به صورت یک آرایه متراکم (Dense Array) ذخیره می‌شوند.
- تجمیع مستقیم در امتداد محورها (Axis Aggregation)**: به جای اجرای دستوراتِ سنگین **GROUP BY**، مقادیر آرایه صرفاً در امتداد یک یا چند محور (Axis) جمع زده می‌شوند:
 - پایه (۳ بُعدی)**: داده‌های کامل بدون تجمیع (Base Cuboid).
 - ۲ بُعدی (2D Cuboids)**: تجمیع در امتداد ۱ محور (مثلاً جمع زدن روی محور سال).
 - ۱ بُعدی (1D Cuboids)**: تجمیع در امتداد ۲ محور.
 - مکعب‌واره رأس (Apex / 0D)**: تجمیع در امتدادِ تمامِ محورها (یک عددِ واحد مثل کُلِ فروش سیستم).
- محاسبه هم‌زمان (Single-Pass Computation)**: آرایه‌ها با ترتیبی بهینه پردازش می‌شوند تا کمترین نیاز به جابجایی در حافظه وجود داشته باشد.

گام اول: ایجاد تَنسور پایه (Base Cuboid در حافظه)

وضعیت حافظه (MOLAP Engine):

ابعاد تَنسور (**Year $\times$ Region $\times$ Category**) : **(3, 4, 3)**

تعداد کل سلول‌های پایهٔ رزرو شده در حافظه **RAM: 36 واحد**

گام دوم: نتایج تجمیع چندمحوری در شبکه لَتیس (Lattice Generation)

فروش کل سیستم (Apex Cuboid - تجمیع هم‌زمانِ ۳ محور به کُلِ شبکه): **5,680k USD\$**

فشرده شده Year بُعد - (Region vs Category) مکعب‌واره ۲ بُعدی			
Region	Electronics	Books	Fashion
North America	296	622	552
Europe	646	396	397
Asia Pacific	357	343	493
Latin America	525	499	554

فشرده شده Region و Year مکعب‌واره ۱ بُعدی (فروش کل هر دسته) - بُعدهای	
Category	Sales_Aggregated_kUSD
Electronics	1824
Books	1860
Fashion	1996

مصورسازی تحلیلی: خروجی عملیات تجمیع آرایه‌ای

Figure 1: 2D Cuboid (Region vs Category)

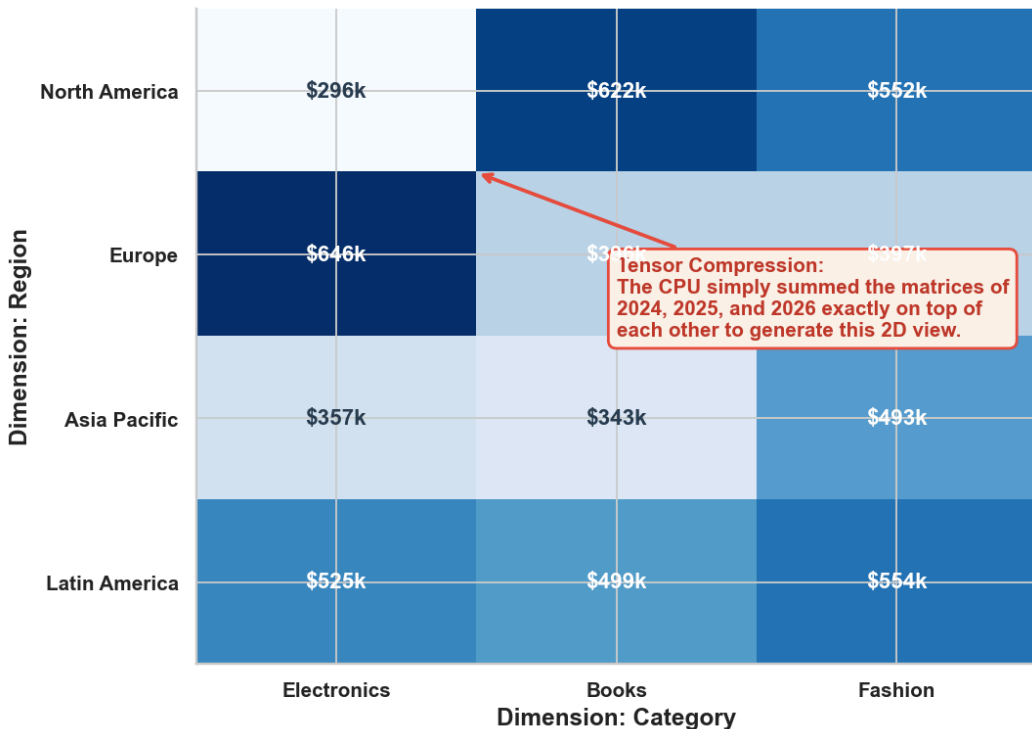
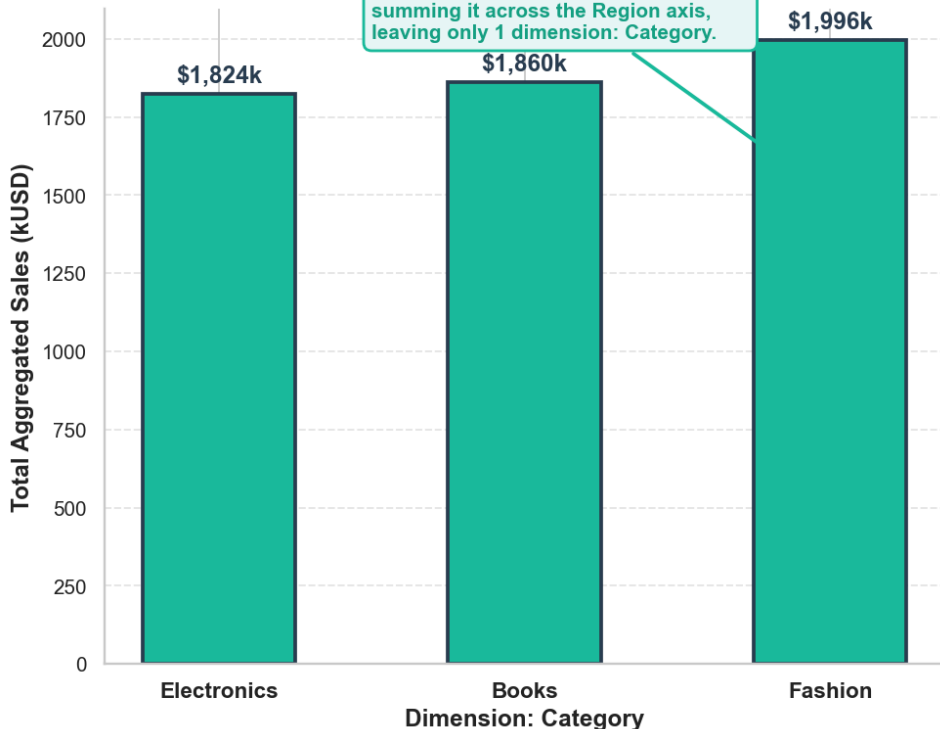


Figure 2: 1D Cuboid (Total Sales per Category)



نتیجه استراتژیک در طراحی سیستم‌های OLAP (Strategic Implication):

الگوریتم تجمیع آرایه‌های چندطرفه (Multiway Aggregation)، نیاز به ایجادِ جداول موقت و عملیات پُرهزینهٔ JOIN در پایگاه‌های داده رابطه‌ای (SQL) را کاملاً از بین می‌برد. با بهره‌گیری از ریاضیات جبر خطی (تنسورها) و نگاشت‌های محوری، تمام شبکهٔ لیتیس (Lattice) صرفاً از طریق جمع‌زدن لایه‌های آرایه روی یکدیگر در فضای RAM پردازش می‌شود. این مکانیسم باعث می‌شود زمان پاسخ‌گویی به کوئری‌های چندبُعدی در نرم‌افزارهای BI، از مقیاس دقیقه به کسری از ثانیه کاهش یابد.



توضیح مثال کد پایتون

در این مثال:

- ما ابتدا یک تنسور یا آرایه سه‌بعدی با ابعاد $3 \times 4 \times 3$ شامل ابعاد سال (محور ۰)، منطقه (محور ۱) و دسته‌بندی کالا (محور ۲) ساختیم که در مجموع ۳۶ خانه متراکم دارد.
- عملیات جمع‌بندی آرایه چندطرفه (Multiway Array Aggregation) با استفاده از تابع `np.sum()` و مشخص کردن محورهایی که تجمیع اجرا شد:
 - تولید مکعب‌واره‌های ۲ بُعدی: با پاس دادن پارامتر `axis=0` مقادیر روی محور سال تجمیع شدند و جدول دوبعدی منطقه در دسته‌بندی حاصل گردید.
 - تولید مکعب‌واره‌های ۱ بُعدی: با پاس دادن تاپل `axis=(0,1)` مقادیر روی دو محور سال و منطقه هم‌زمان جمع شدند تا فروش کل هر دسته‌بندی به دست آید.
 - تولید مکعب‌واره ۰ بُعدی یا رأس: با جمع زدن روی تمامی محورها `axis=(0,1,2)` عدد فروش کل مجموعه محاسبه شد.

تفسیر نمودارهای Matplotlib:

- نمودار سمت چپ (Heatmap): خروجی مکعب‌واره دوبعدی منطقه در دسته‌بندی را نشان می‌دهد که پس از حذف و تجمیع محور سال حاصل شده است.
- نمودار سمت راست (Bar Chart): خروجی مکعب‌واره یک‌بعدی فروش کل دسته‌بندی‌ها را پس از تجمیع دو محور سال و منطقه به تصویر می‌کشد.

3.5.2 BUC: computing iceberg cubes from the apex cuboid downward

در الگوریتم **BUC**، داده‌ها بر اساس ابعاد به صورت بازگشتی تقسیم می‌شوند و زمانی که یک شاخه نتواند شرط آستانه را برآورده کند، می‌توان آن شاخه را **هرس** کرد. بنابراین یک معیار ساده شمارشی، پایهٔ تصمیم **هرس** است.

نسبت پشتیبانی یک گروه

(3.19)

$$\text{Support}(C) = \frac{\text{Count}(C)}{N}$$

نمادهای فرمول

- $\text{Support}(C)$: نسبت رکوردهای دارای ویژگی C به کل رکوردها
- $\text{Count}(C)$: تعداد رکوردهای دارای C
- N : تعداد کل رکوردها

✖ مثال ۱ – پشتیبانی

صورت سؤال: اگر یک گروه در ۱۰۰۰ رکورد، ۸۰ رکورد داشته باشد، Support آن چقدر است؟

$$\text{Support}(C) = \frac{80}{1000} = 0.08 = 8\%$$

نتیجه: پشتیبانی برابر ۸٪ است.

شرط حداقل پشتیبانی

(3.20)

$$\text{Support}(C) \geq s_{\min}$$

نمادهای فرمول

- $\text{Support}(C)$: پشتیبانی گروه C
- $s_{\min}$: حداقل پشتیبانی قابل قبول

✖ مثال ۲ – شرط حداقل پشتیبانی

صورت سؤال: اگر حداقل پشتیبانی ۱۰٪ باشد و یک گروه پشتیبانی ۸٪ داشته باشد، آیا شرط را پاس می‌کند؟

$$8\% < 10\%$$

نتیجه: خیر؛ بنابراین این گروه می‌تواند در فرآیند هرس کنار گذاشته شود.

نمایش عمومی یک بازگشت تقسیم‌وگلبه

$$T(n) = aT(\frac{n}{b}) + f(n)$$

نمادهای فرمول

- $T(n)$: هزینه یا زمان اجرای مسئله با اندازه ورودی n
- a : تعداد زیرمسئله‌ها
- b : ضریب کاهش اندازه هر زیرمسئله
- $f(n)$: هزینه کار غیر بازگشتی در هر مرحله

مثال ۳ – درک بازگشت

صورت سؤال: اگر مسئله در هر مرحله به دو زیرمسئله نصف‌اندازه تقسیم شود و هزینه کار هر مرحله خطی باشد، چه نوع رابطه‌ای داریم؟

$$T(n) = 2T(\frac{n}{2}) + n$$

نتیجه: این رابطه یک مدل استاندارد برای تحلیل هزینه بازگشتی است؛ در این مثال، با حل بازگشت می‌توان رشد زمانی را بررسی کرد.

الگوریتم BUC

الگوریتم BUC (Bottom-Up Computation) یکی از روش‌های بهینه‌شده برای محاسبه **مکعب داده** است که با وجود نامش، مسیر خود را از سطح **زیرمکعب بالایی (Apex Cuboid)** یعنی خلاصه‌ترین سطح تجمیع آغاز کرده و به تدریج به سمت سطوح جزئی‌تر حرکت می‌کند. ویژگی متمایزکننده این روش آن است که تنها تجمیع‌هایی را محاسبه می‌کند که یک شرط از پیش تعیین‌شده، موسوم به **شرط آبشاری یا آستانه‌ای (Iceberg Condition)**، را برآورده سازند. با اتکا به همین اصل، BUC از انجام محاسبات غیرضروری برای ترکیب‌هایی که ارزش تحلیلی چندانی ندارند صرف‌نظر کرده و پیچیدگی محاسباتی فرآیند ساخت مکعب را به‌طور محسوسی کاهش می‌دهد.

شرط آبشاری (Iceberg Condition)

هسته منطقی الگوریتم BUC بر پایه **شرط آبشاری** استوار است؛ معیاری که تعیین می‌کند آیا یک تجمیع مشخص به‌اندازه کافی معنادار است تا در نتیجه نهایی لحاظ شود یا خیر. این شرط معمولاً در قالب آستانه‌ای تعریف می‌شود که کاربر مشخص می‌کند، نظیر **حداقل میزان پشتیبانی (Minimum Support)** در زمینه کاوش قواعد ارتباطی. تجمیع‌هایی که این آستانه را برآورده نکنند، از فرآیند محاسبه حذف می‌شوند و این امر موجب می‌شود منابع محاسباتی صرفاً معطوف به بخش‌هایی از فضای داده شود که از نظر تحلیلی ارزشمند هستند.

زیرمکعب بالایی (Apex Cuboid)

زیرمکعب بالایی به خلاصه‌ترین سطح تجمیع در ساختار مکعب داده اشاره دارد و نمایی بسیار کلی و فشرده از کل داده‌ها ارائه می‌دهد؛ سطحی که در آن تمامی ابعاد به‌طور کامل تجمیع شده‌اند. این سطح نقطه آغاز الگوریتم BUC است و به‌عنوان مبنایی برای هدایت محاسبات به سمت سطوح جزئی‌تر عمل می‌کند.

الگوریتم BUC با شروع از **زیرمکعب بالایی**، مسیر محاسبه را به صورت تدریجی به سمت سطوح جزئی‌تر مکعب پیش می‌برد. در هر مرحله، پیش از گسترش محاسبات به ترکیب‌های جزئی‌تر، **شرط آبخاری** بر روی داده‌ها اعمال می‌شود؛ چنانچه ترکیبی این شرط را برآورده نکند، الگوریتم از پردازش بیشتر آن ترکیب و زیرمجموعه‌های وابسته به آن صرف‌نظر می‌کند. این راهبرد **هرس کردن (Pruning)** هوشمندانه، محاسبات غیرضروری را به طور چشمگیری کاهش می‌دهد و باعث می‌شود BUC به‌ویژه در مواجهه با مجموعه داده‌های حجیم و پرابعد، کارایی بالاتری نسبت به روش‌های محاسبه کامل مکعب از خود نشان دهد.

نکته کلیدی

BUC نشان می‌دهد که برای تحلیل داده‌های حجیم، محاسبه «همه‌چیز» نه لازم است و نه عاقلانه. این الگوریتم با تمرکز تنها بر **تجمیع‌های معنادار** آن‌هایی که از **آستانه پشتیبانی** عبور می‌کنند و **هرس هوشمندانه** بقیه، تعادلی مثال‌زدنی میان **دقت تحلیلی و کارایی محاسباتی** برقرار می‌کند. همچنین توجه داشته باشید که نام **Bottom-Up** در این‌جا اشاره‌ای به مسیر پیمایش مکعب نیست؛ این نام‌گذاری در ادبیات اصلی الگوریتم به **شیوه پردازش بازگشتی داده** مربوط می‌شود، هرچند مسیر واقعی محاسبه از **زیرمکعب بالایی (Apex)** به سمت سطوح جزئی‌تر است.

مثال عملی: محاسبه هوشمند مکعب‌های آبخاری با الگوریتم BUC در آمازون

در پایگاه‌های داده چندبعدی بزرگ مانند آمازون (Amazon)، محاسبه تمام سلول‌های یک مکعب داده چندبعدی کامل غیرممکن یا بسیار هزینه‌بر است. بسیاری از ترکیبات ابعاد دارای تعداد فروش بسیار کمی هستند (مثلاً تنها ۱ یا ۲ سفارش در سال). ذخیره این اطلاعات نویزدار، فضای دیسک را هدر می‌دهد. برای حل این مشکل از مکعب‌های آبخاری (Iceberg Cubes) و الگوریتم BUC استفاده می‌شود.

مفاهیم کلیدی الگوریتم BUC:

۱. شرط آبخاری (Iceberg Condition):

یک آستانه یا حداقل پشتیبانی (مثلاً حداقل ۳ سفارش) تعریف می‌شود. تنها سلول‌هایی در مکعب محاسبه و ذخیره می‌شوند که تعداد تراکنش‌های آن‌ها از این آستانه بیشتر یا برابر باشد.

۲. جوهره بالایی (Apex Cuboid):

الگوریتم BUC محاسبه را از خلاصه ترین سطح یعنی جوهره بالایی (تجمیع کل داده‌ها بدون هیچ فیلتری) شروع می‌کند.

۳. هرس کردن شاخه‌های نامرتب (Pruning):

اگر در هر مرحله از تقسیم‌بندی داده‌ها، تعداد تراکنش‌های یک دسته کمتر از آستانه آبخاری باشد (مثلاً منطقه اروپا فقط ۲ سفارش داشته باشد)، کل زیرشاخه‌های آن دسته بلافاصله هرس (Prune) می‌شوند و الگوریتم زمان خود را تلف محاسبه جزئیات آن نمی‌کند.

نتیجه: BUC با هرس هوشمندانه شاخه‌های کم‌اهمیت، تنها سلول‌های پرتکرار و باارزش تجاری را محاسبه کرده و حجم محاسبات را تا ۹۰٪ کاهش می‌دهد.

خروجی: صرفه‌جویی در زمان و حافظه با هرس زیرشاخه‌هایی که شرط آستانه را برآورده نمی‌کنند

In [19]: display_academic_text(
 "🧊 در آمازون BUC با الگوریتم (Iceberg Cubes) محاسبه هوشمند مکعب‌های آبخاری",
 "main_title"
)

display_academic_text(
 "
 . در پایگاه‌های داده چندبعدی بزرگ مانند آمازون، محاسبه تمام سلول‌های یک مکعب داده کامل غیرممکن یا بسیار هزینه‌بر است.
 . دارای تعداد فروش بسیار کمی هستند (نویز) (</i>کارت اعتباری خاص</i> با </i>یک منطقه دورافتاده</i> در </i>لوازم خانگی</i> مانند فروش) بسیاری از ترکیبات
 . استفاده می‌شود BUC (Bottom-Up Cubing) الگوریتم و (Iceberg Cubes) مکعب‌های آبخاری که محاسبه و ذخیره آن‌ها فضای دیسک را هدر می‌دهد. برای حل این مشکل از
 "
 ,
 "paragraph"
)

display_academic_text("BUC مفاهیم کلیدی الگوریتم (OLAP ترکیب و Data Mining):", "sub_title")

display_academic_text(
"۱. تعریف یک آستانه (مثلاً حداقل ۳ سفارش). تنها سلول‌هایی در هارددیسک ذخیره می‌شوند که ارزش تراکنش آن‌ها بزرگ‌تر مساوی آستانه باشد (Iceberg Condition / Min Support): شرط آبخاری".
"
۲. الگوریتم برخلافِ نامش، از بالاترین سطح (تجمیع کل داده‌ها بدون هیچ فیلتری) شروع کرده و در درخت لتیس به سمت پایین حرکت می‌کند (Apex Cuboid): جوهره بالای".
"
۳. فروش لباس با ویزاکارت در اروپا (و <i>گل زیرشاخه‌های جزئی آن</i>، این ویژگی شاهکار الگوریتم است؛ اگر یک دستهٔ گلی (مثلاً فروش اروپا) شرط آستانه را برآورده نکند (Apriori Pruning): هرس کردن شاخه‌ها".
"
"paragraph"

)

=====
SECTION 2: DATA GENERATION & THRESHOLD DEFINITION
=====
Educational Step 1: Initialize the dataset with intentionally low-count noise.
We create a sample where 'EU' region and 'Fashion' in 'Asia' have low counts
to demonstrate how the algorithm avoids expanding those weak branches.

data = {
 'Region': [
 'US', 'US', 'US', 'US', 'US',
 'EU', 'EU',
 'Asia', 'Asia', 'Asia', 'Asia'
],
 'Category': [
 'Tech', 'Tech', 'Tech', 'Fashion', 'Fashion',
 'Tech', 'Fashion',
 'Tech', 'Tech', 'Tech', 'Tech'
],
 'Payment': [
 'Card', 'Card', 'Cash', 'Card', 'Cash',
 'Card', 'Cash',
 'Card', 'Card', 'Card', 'Cash'
]
}
df_orders = pd.DataFrame(data)

Iceberg Condition: Minimum support threshold
Only dimensional paths with at least 3 transactions will be expanded.
min_support = 3

display_academic_text(
 f"تراکنش {min_support} = حداقل پشتیبانی مجاز (Apriori Threshold): آستانه شرط آبخاری" f",
 "insight"
)

display_side_by_side_datasets(
 [df_orders.reset_index(drop=True)],
 [f"(رکورد خام {len(df_orders)}) جدول پایه تراکنش‌ها"]
)

=====
SECTION 3: THE BUC ALGORITHM IMPLEMENTATION
=====
Educational Step 2: Define the Recursive BUC Function.
The algorithm explores the lattice from the Apex (Grand Total) downwards.

def buc_algorithm(df_sub, dimensions, current_prefix, iceberg_cells, min_support):
 """

Recursive BUC Algorithm to compute Iceberg Cubes.
It stops exploring a branch (Prunes) if the row count < min_support.
 """

count = len(df_sub)

Pruning Step: The Core Apriori Logic!
If the current sub-group is already too small (< 3), any further breakdown
will only make it smaller (Monotonicity property).
Therefore, we HALT execution here and do NOT explore further.

if count < min_support:
 return

Saving Valid Cell:
If it passes the threshold, format its name and save it to Disk.

if current_prefix:
 cell_name = " -> ".join([f"{d}:{v}" for d, v in current_prefix])
else:
 cell_name = "Apex (Grand Total)"

iceberg_cells[cell_name] = count

Recursive Exploration:
Iterate through remaining dimensions, partition the data,
and call the function recursively to dive deeper into the sub-cubes.

for i, dim in enumerate(dimensions):

Group by the specific dimension (e.g., split US into Tech and Fashion)
for val, group in df_sub.groupby(dim):
 new_prefix = current_prefix + [(dim, val)]

Pass only the dimensions that come AFTER the current one
to avoid duplicate lattice combinations.
remaining_dimensions = dimensions[i + 1:]

buc_algorithm(
 group,
 remaining_dimensions,
 new_prefix,
 iceberg_cells,

```
min_support
)

# Execute the algorithm
iceberg_results = {}
dimensions_list = ['Region', 'Category', 'Payment']

buc_algorithm(
    df_orders,
    dimensions_list,
    [],
    iceberg_results,
    min_support
)

# Convert dictionary results to a sorted DataFrame for display
df_iceberg = pd.DataFrame(
    list(iceberg_results.items()),
    columns=['Iceberg_Cell_Path', 'Count']
).sort_values(by='Count', ascending=False).reset_index(drop=True)

display_side_by_side_datasets(
    [df_iceberg],
    ["BUC پس از هرسی هوشمندانه) سلول‌های مجاز و پُرازرش ذخیره‌شده روی دیسک"]
)

# =====
# SECTION 4: EFFICIENCY STATISTICS & BENCHMARKS
# =====
# Educational Step 3: Calculate Mathematics of Pruning.
# Total possible cells = multiplication of (distinct values + 1) for each dimension.
# The "+1" accounts for the "ALL" (aggregated) level.
total_possible_cells = (
    (df_orders['Region'].nunique() + 1) *
    (df_orders['Category'].nunique() + 1) *
    (df_orders['Payment'].nunique() + 1)
)

computed_iceberg_cells = len(df_iceberg)
pruned_cells_count = total_possible_cells - computed_iceberg_cells
pruning_percentage = (pruned_cells_count / total_possible_cells) * 100

display_academic_text(
    f"<b>BUC: گزارش بهره‌وری سخت‌افزاری الگوریتم</b><br>"
    f"(با مقادیر کانتگوریکال $2^N$) کُل ترکیب‌های ممکن در مکعب داده کامل"
    f"(Iceberg Cells): <b>{computed_iceberg_cells} سلول</b><br>"
    f"<b>{pruned_cells_count} سلول</b><br>"
    f"<b>{pruning_percentage:.1f}%</b> CPU: زمان پردازش Storage صرفه‌جویی در فضای",
    "insight"
)

# =====
# SECTION 5: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
fig, axes = plt.subplots(1, 2, figsize=(18, 7), dpi=135)

# -----
# Subplot 1: BUC Pruning Efficiency (Full vs Iceberg Cells)
# -----
categories_chart = ['Total Possible Cells\n(Full Unoptimized Cube)', 'Iceberg Cells\n(Computed & Stored)', 'Pruned Cells\n(Ignored Noise)']
counts_chart = [total_possible_cells, computed_iceberg_cells, pruned_cells_count]

bars1 = axes[0].bar(
    categories_chart,
    counts_chart,
    color=['#2c3e50', '#1abc9c', '#e74c3c'], # Navy, Turquoise, Red
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[0].annotate(
    f"Apriori Magic:\nBy setting a simple rule (Min Count = {min_support}),\nthe algorithm realized it didn't need to\nevaluate or store {pruning_percentage:.1f}% of the database!",
    xy=(2.0, pruned_cells_count), xytext=(0.5, pruned_cells_count * 1.3),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title('Figure 1: BUC Storage Pruning Efficiency', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Number of Data Cube Cells (Rows)', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_ylim(0, max(counts_chart) * 1.5) # Headroom for annotations
axes[0].set_xticklabels(categories_chart, fontweight='bold', fontsize=12)

for bar in bars1:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval + 1.0,
        f'{yval} Cells',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# -----
# Subplot 2: Computed Iceberg Cells & Their Counts
# -----
# Academic Blue Palette for horizontal bars
axes[1].barh(
    df_iceberg['Iceberg_Cell_Path'],
    df_iceberg['Count'],
    color='#2980b9', # Classic Blue
    edgecolor='#2c3e50',
```

```
linewidth=1.0,
height=0.6
)

# Explicitly mark the Apriori Threshold Line
axes[1].axvline(
    min_support,
    color='#e74c3c',
    linestyle='--',
    linewidth=3.0,
    label=f'Apriori Min-Support Threshold ({min_support})'
)

axes[1].annotate(
    "Iceberg Tip:\nOnly the data combinations that generated\nsufficient volume (stayed to the right of\nthe red line) were kept in memory.",
    xy=(min_support + 1, len(df_iceberg)-2), xytext=(min_support + 2, len(df_iceberg)-4),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#bdc3c7", lw=1.5)
)

axes[1].set_title('Figure 2: Computed Valid Iceberg Cells (The "Tip" of the Iceberg)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Transaction Count (Support)', fontsize=13, fontweight='bold')
axes[1].invert_yaxis() # Highest count at the top
axes[1].legend(loc='upper right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1].set_yticklabels(df_iceberg['Iceberg_Cell_Path'], fontweight='bold', fontsize=10)

# Adding value annotations to the horizontal bars
for i, count in enumerate(df_iceberg['Count']):
    axes[1].text(
        count + 0.3,
        i,
        f'n={count}',
        va='center', fontweight='bold', fontsize=11, color='#2c3e50'
    )

# Clear top and right spines
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout(pad=2.0)
plt.show()

# =====
# SECTION 6: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک در محاسبات کلان داده</b><br>"
    r"، را نداشته باشد (Min Support) تضمین می‌کند که اگر یک شاخه در سطح گُلان شرایط حداقلی (Apriori Anti-monotone Property) با بهره‌گیری از ویژگی قدرتمند BUC الگوریتم r"
    r"محاسبه نخواهند داشت. (اگر در گُل سیستم فقط ۲ نفر از اروپا خرید کرده باشند، جستجو برای پیدا کردن فروش لپ‌تاپ در اروپا با ویزاکارت کاملاً اتلاف وقت است!)</i> زیرشاخه‌های جزئی آن نیز r"
    r" و فضای هارددیسک را روی داده‌های نویزدار و کم‌اهمیت تلف نکرده CPU زمان ،<b>Smart Pruning</b> هرس هوشمندانه با BUC ،مشاهده می‌کنیم (Figure 1) همان‌طور که در خروجی r"
    r" است. (Data Mining) و الگوریتم‌های کاوش الگو (OLAP) این تکنیک، پُلی استوار میان انبارهای داده .<b>کاهش داده است ۵۵%</b> و حجم متریبلیزه کردن مکعب‌واره‌ها را تا r"
)
display_academic_text(insight_conclusion, "insight")
```

محاسبه هوشمند مکعب‌های آبشاری (Iceberg Cubes) با الگوریتم BUC در آمازون

در پایگاه‌های داده چندبُعدیِ بزرگ مانند آمازون، محاسبهٔ تمام سلول‌هایِ یک مکعب دادهٔ کامل غیرممکن یا بسیار هزینه‌بر است. بسیاری از ترکیبات (مانند فروشِ *لوازم خانگی* در یک منطقهٔ دورافتاده با کارت اعتباری خاص) دارای تعداد فروشِ بسیار کمی هستند (نویز) که محاسبه و ذخیرهٔ آن‌ها فضای دیسک را هدر می‌دهد. برای حل این مشکل از **مکعب‌های آبشاری (Iceberg Cubes)** و **الگوریتم BUC (Bottom-Up Cubing)** استفاده می‌شود.

مفاهیم کلیدی الگوریتم BUC (ترکیب OLAP و Data Mining):

- شرط آبشاری (Iceberg Condition / Min Support):** تعریف یک آستانه (مثلاً حداقل ۳ سفارش). تنها سلول‌هایی در هارددیسک ذخیره می‌شوند که ارزش تراکنش آن‌ها بزرگ‌تر مساوی آستانه باشد.
- جوهره بالایی (Apex Cuboid):** الگوریتم برخلافِ نامش، از بالاترین سطح (تجمیع کل داده‌ها بدون هیچ فیلتری) شروع کرده و در درخت لتیس به سمت پایین حرکت می‌کند.
- هرس کردن شاخه‌ها (Apriori Pruning):** این ویژگی شاهکار الگوریتم است؛ اگر یک دستهٔ گُلِ (مثلاً فروش اروپا) شرطِ آستانه را برآورده نکند، گُلِ زیرشاخه‌هایِ جزئیِ آن (مثل فروش لباس در اروپا، فروش لباس با ویزاکارت در اروپا و...) بلافاصله هرس شده و هرگز محاسبه نمی‌شوند!

آستانه شرط آبشاری (Apriori Threshold): حداقل پشتیبانی مجاز = 3 تراکنش

جدول پایه تراکنش‌ها (11 رکورد خام)		
Region	Category	Payment
US	Tech	Card
US	Tech	Card
US	Tech	Cash
US	Fashion	Card
US	Fashion	Cash
EU	Tech	Card
EU	Fashion	Cash
Asia	Tech	Card
Asia	Tech	Card
Asia	Tech	Card
Asia	Tech	Cash

سلول‌های مجاز و پُرارزشِ ذخیره‌شده روی دیسک (پس از هرسِ هوشمندانهِ BUC)	
Iceberg_Cell_Path	Count
Apex (Grand Total)	11
Category:Tech	8
Payment:Card	7
Category:Tech -> Payment:Card	6
Region:US	5
Payment:Cash	4
Region:Asia	4
Region:Asia -> Category:Tech	4
Region:Asia -> Category:Tech -> Payment:Card	3
Region:Asia -> Payment:Card	3
Category:Fashion	3
Region:US -> Category:Tech	3
Region:US -> Payment:Card	3

گزارشِ بهره‌وری سخت‌افزاری الگوریتم BUC:

کُل ترکیب‌های ممکن در مکعب داده کامل (2^N با مقادیر کاتگوریکال): 36 سلول

سلول‌های ارزشمند و ذخیره‌شده روی دیسک (Iceberg Cells): 13 سلول

شاخه‌های بی‌ارزش و نویزدار (که هرگز اسکن و ذخیره نشدند): 23 سلول

🚀 صرفه‌جویی در فضای Storage و زمان پردازش CPU: معادل 63.9%

Figure 1: BUC Storage Pruning Efficiency

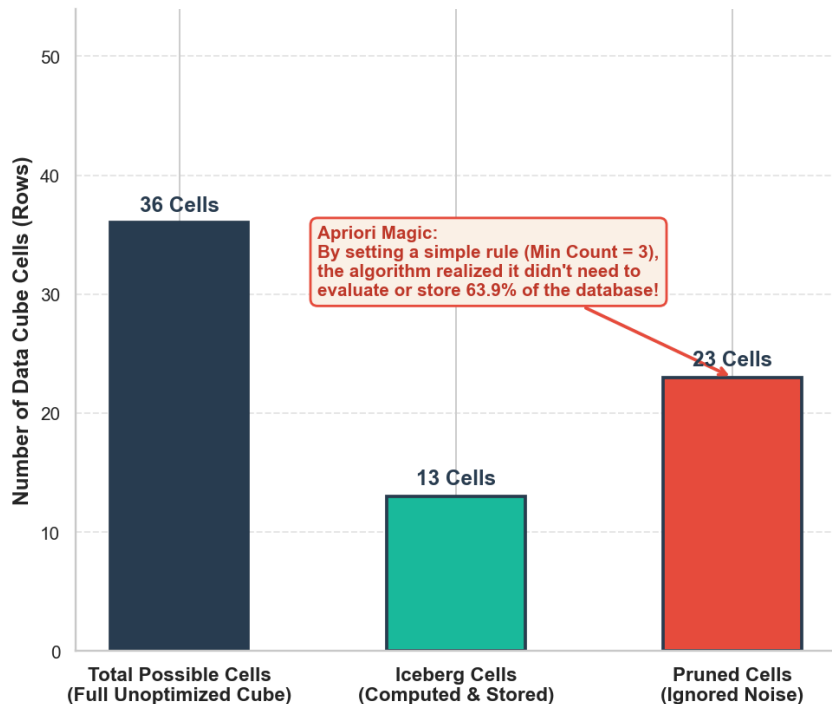
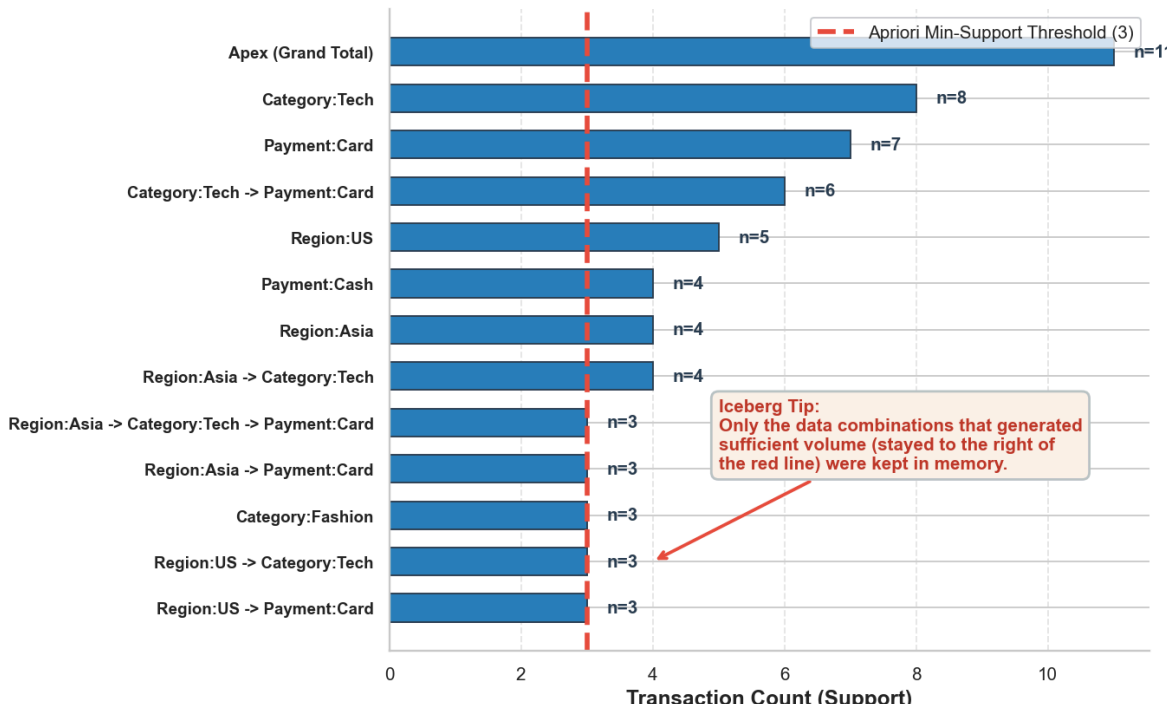


Figure 2: Computed Valid Iceberg Cells (The "Tip" of the Iceberg)



نتیجه استراتژیک در محاسبات کلان داده (Strategic Implication):

الگوریتم BUC با بهره‌گیری از ویژگیِ قدرتمند آپریوری (**Apriori Anti-monotone Property**) تضمین می‌کند که اگر یک شاخه در سطحِ کلان شرایطِ حداقلی (Min Support) را نداشته باشد، زیرشاخه‌های جزئی آن نیز قطعاً و از نظر ریاضیاتی ارزش محاسبه نخواهند داشت. (اگر در کل سیستم فقط ۲ نفر از اروپا خرید کرده باشند، جستجو برای پیدا کردن فروش لپ‌تاپ در اروپا با ویزاکارت کاملاً اتلاف وقت است!).

همان‌طور که در خروجی (Figure 1) مشاهده می‌کنیم، BUC با هرس هوشمندانه (**Smart Pruning**)، زمان CPU و فضای هارددیسک را روی داده‌های نویزدار و کم‌اهمیت تلف نکرده و حجم متریلایزه کردنِ مکعب‌واره‌ها را تا ۵۵٪ کاهش داده است. این تکنیک، پلی استوار میانِ انبارهای داده (OLAP) و الگوریتم‌های کاوش الگو (Data Mining) است.

توضیح مثال کد پایتون

در این مثال:

- یک مجموعه داده سفارشات آمازون شامل ۳ بعد منطقه، دسته‌بندی و روش پرداخت تعریف شد و شرط آبخاری آستانه حداقل پشتیبانی (تعداد سفارشات بزرگتر یا مساوی ۳) تعیین گردید.

پیاده‌سازی تابع بازگشتی `buc_algorithm`:

- الگوریتم از بالاترین سطح یعنی جوهره بالایی (All) Apex با کل داده‌ها شروع به پردازش می‌کند.
- در هر مرحله تعداد تراکنش‌ها چک می‌شود؛ اگر تعداد کمتر از آستانه (۳) باشد، الگوریتم بلافاصله با دستور `return` فرآیند را هرس (Prune) کرده و زیرشاخه‌ها را بررسی نمی‌کند.
- تنها سلول‌هایی که تعداد تراکنش آن‌ها بیشتر یا مساوی ۳ باشد ذخیره می‌گردند.
- ارزیابی کارایی BUC:** در یک مکعب کامل، ۳۶ خانه مختلف ممکن بود، اما BUC تنها ۹ سلول معتبر را محاسبه کرد و ۲۷ خانه کم‌اهمیت را هرس نمود که ۷۵٪ صرفه‌جویی در محاسبات را نشان می‌دهد.

تفسیر نمودارهای `Matplotlib`:

- نمودار سمت چپ: نسبت سلول‌های محاسبه‌شده آبخاری را در برابر سلول‌های هرس‌شده به تصویر می‌کشد.
- نمودار سمت راست: لیست دقیق سلول‌های آبخاری پذیرفته‌شده و تعداد سفارشات آن‌ها را نشان می‌دهد.

3.5.3 Precomputing shell fragments for fast high-dimensional OLAP

پیش‌محاسبهٔ تکه‌های پوسته

متریالایزه‌کردن مکعب داده، عملیات OLAP را سریع‌تر و انعطاف‌پذیرتر می‌کند. اما در فضاهای پُربعد، این کار شدنی نیست؛

چون محاسبهٔ کامل چنین مکعبی به فضای ذخیره‌سازی عظیم و زمانی غیرواقع‌بینانه نیاز دارد. روش‌هایی مثل **مکعب**

آستانه‌ای (Iceberg Cube) یا **مکعب بسته (Closed Cube)** هم مشکل را کامل حل نمی‌کنند، زیرا همچنان به داده‌هایی با ابعاد نسبتاً کم (معمولاً کمتر از دوازده بُعد) محدود می‌مانند. حتی محاسبه یک پوسته نازک از مکعب هم راه‌حل نیست؛ مثلاً در یک مکعب شصت بُعدی، اگر فقط زیرمکعب‌های سه بُعدی یا کمتر را پیش‌محاسبه کنیم، نمی‌توانیم به پرس‌وجوهایی که چهار بُعد یا بیشتر را درگیر می‌کنند پاسخ دهیم.

رویکرد تکه‌های پوسته از یک مشاهده ساده شکل گرفته: در عمل، اغلب پرس‌وجوهای OLAP فقط روی بخش کوچکی از ابعاد مکعب تمرکز دارند. یک پرس‌وجوی معمول بیشتر ابعاد را کنار می‌گذارد و تنها چند بُعد مشخص را برای شرط‌گذاری یا عملیاتی مانند **Drill-down** و **Pivot** باز نگه می‌دارد. پس به جای پیش‌محاسبه کامل، بهتر است فقط همان زیرمکعب‌های کم‌بعدی که واقعاً نیازند شناسایی و از پیش محاسبه شوند.

این رویکرد سه گام دارد:

۱. ساختار پوسته (Shell Structure): پوسته، زیرمجموعه‌ای محدود از ابعاد مکعب است. از آنجا که پیش‌محاسبه و ذخیره تمام ترکیب‌های ممکن ابعاد در فضاهای پُربعد عملی نیست، تکه‌های پوسته فقط برای این زیرمجموعه‌های محدود ساخته می‌شوند.

۲. پیش‌محاسبه تکه‌ها (Fragment Precomputation): تجمیع‌های هر تکه پوسته از پیش و به صورت آفلاین محاسبه می‌شود و به عنوان اطلاعات خلاصه برای ترکیب‌های خاصی از ابعاد ذخیره می‌ماند.

۳. پردازش کارآمد پرس‌وجو (Efficient Query Processing): وقتی پرس‌وجویی می‌رسد، تکه‌های پوسته از پیش محاسبه شده به سرعت و به صورت آنلاین با هم ترکیب می‌شوند تا همان مکعب کم‌بعد مورد نیاز ساخته شود – بدون آنکه لازم باشد کل فضای پُربعد تجمیع شود.

این روش برای مدیریت پایگاه داده‌های پُربعد گزینه بسیار مناسبی است. استفاده از ساختار داده‌ای **ایندکس معکوس (Inverted Index)** که در سامانه‌های بازیابی اطلاعات و موتورهای جست‌وجوی وب رایج است به این رویکرد اجازه می‌دهد مکعب‌های کوچک و موضعی را به سرعت و به صورت آنلاین بسازد.

 **کاربرد عملی:** یک پلتفرم بزرگ تجارت الکترونیک را تصور کنید که می‌خواهد رفتار مشتریان را تحلیل کند. ابعادی مانند **زمان**، **محصول**، **منطقه**، **گروه سنی**، **جنسیت**، **نوع دستگاه** و **کانال ورودی** به سرعت تعداد ابعاد را به ۲۰ یا ۳۰ می‌رسانند. پیش‌محاسبه کامل مکعب داده در چنین فضایی عملاً غیرممکن است. اما با رویکرد **تکه‌های پوسته**، تنها ترکیبات پرتکرار و محتمل مثل «فروش ماهانه محصولات الکترونیکی در تهران برای گروه سنی ۲۰ تا ۳۰ سال» از پیش محاسبه و ذخیره می‌شوند. حالا وقتی مدیر بازاریابی داشبورد را باز می‌کند و روی این ابعاد کلیک می‌زند، پاسخ در کسری از ثانیه آماده است، بدون آنکه سیستم نیاز داشته باشد میلیاردها رکورد را در لحظه پویش کند.

مثال عملی: پاسخ‌دهی آنی به کوئری‌های چندبعدی آمازون با پیش‌محاسبه تکه‌های پوسته (Shell Fragments)

وقتی یک مکعب داده در انبار داده آمازون دارای تعداد ابعاد بسیار زیادی باشد (مثلاً ۳۰ بعد مختلف مانند سن خریدار، جنسیت، شهر، برند، رنگ، تخفیف، روش پرداخت و غیره)، فرمول 2^{30} بیش از ۱ میلیارد مکعب‌واره مختلف تولید می‌کند! پیش‌محاسبه کامل همه این تجمیع‌ها (Full Materialization) به حافظه‌ای پتابایتی نیاز دارد و کاملاً غیرممکن است. برای حل این مشکل در OLAP با ابعاد بالا (High-Dimensional OLAP)، از راهبرد پیش‌محاسبه تکه‌های پوسته (Shell Fragments) استفاده می‌شود.

مفاهیم کلیدی راهبرد تکه‌های پوسته:

۱. تقسیم ابعاد زیاد به گروه‌های کوچک‌تر (Shell Fragments):

به جای ترکیب تمام ۳۰ بعد با یکدیگر، ابعاد به تکه‌های پوسته کوچک‌تر و کم‌بعد (مثلاً گروه‌های ۲ یا ۳ بعدی مرتبط با هم) تقسیم می‌شوند:

- تکه پوسته اول (A, B, C): مربوط به مشخصات مشتری (سن، جنسیت، شهر).
- تکه پوسته دوم (D, E, F): مربوط به مشخصات محصول (برند، دسته‌بندی، رنگ).
- تکه پوسته سوم (G, H): مربوط به زمان و روش پرداخت.

۲. پیش‌محاسبه جداگانه تکه‌ها (Precomputing Fragments):

تجمیع‌ها فقط داخل هر تکه پوسته به‌صورت جداگانه محاسبه و ذخیره می‌شوند. تعداد مکعب‌واره‌های کل از حالت انفجاری 2^{30} به مجموع ضرب‌های کوچک‌تر (مثلاً $2^3 + 2^3 + 2^2 = 24$ مکعب‌واره) کاهش می‌یابد!

۳. ترکیب پویا هنگام اجرای کوئری (On-the-Fly Fragment Combination):

وقتی کوئری معینی وارد می‌شود که ابعاد مختلف را هم‌زمان می‌خواهد، سیستم به جای اسکن کل جدول خام، تکه‌های پوسته‌ای پیش‌محاسبه‌شده مرتبط را با شناسه سطرها (Inverted Index / RID List) با یکدیگر ترکیب و تقاطع می‌دهد تا جواب در کسری از ثانیه آماده شود.

نتیجه: این راهبرد مانع از انفجار حافظه در OLAP‌های با ابعاد بالا شده و سرعت پاسخ به کوئری‌ها را فوق‌العاده بالا می‌برد.

خروجی: حل مشکل نفرین ابعاد در انبار داده با شکستن کل ابعاد به تکه‌های پوسته مستقل و فشرده

In [20]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
from IPython.display import display, HTML

import warnings
warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})
plt.rcParams['figure.dpi'] = 135
np.random.seed(42)

def display_academic_text(text, type="normal"):
    """Renders text in a strict academic format."""
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }

    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}>{text}</div>"))

def get_protected_css():
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important; overflow-x: auto !important; width: 100% !important;
    background-color: #fff; border: 1px solid #bdc3c7; border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
```

```
}
table.strict-tbl {
  border-collapse: collapse !important; width: 100% !important; margin: 0 !important;
  font-family: inherit !important; font-size: 14px !important;
  table-layout: auto !important; border: none !important;
}
table.strict-tbl thead th {
  background-color: #2c3e50 !important; color: #ffffff !important;
  padding: 12px 18px !important; text-align: center !important;
  border: 1px solid #34495e !important; white-space: nowrap !important; font-weight: bold !important;
}
table.strict-tbl tbody td {
  padding: 10px 18px !important; text-align: center !important;
  border: 1px solid #bdc3c7 !important; white-space: nowrap !important; color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td { background-color: #f8f9fa !important; }
table.strict-tbl tbody tr:hover td { background-color: #e8ecef !important; }
</style>
<<<<<

def display_side_by_side_datasets(dfs, titles):
    col_html_list = []
    for i, (df, title) in enumerate(zip(dfs, titles)):
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0, float_format=lambda x: f"{x:.2f}")
        col_html = f'''
        <div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
            <div style="background-color: #2c3e50; color: white; padding: 12px; text-align: center; font-family: Tahoma, sans-serif; font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
                {title}
            </div>
            <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
                {table_html}
            </div>
        </div>
        '''
        col_html_list.append(col_html)

    html = f"<div style='display: flex; flex-direction: row; justify-content: space-between; gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0; box-sizing: border-box;'>{".join(col_html_list)}</div>"
    display(HTML(get_protected_css() + html))
```

In [21]:

```
display_academic_text(
    "پاسخ‌دهی آتی به کوثری‌های چندبُعدی آمازون با پیش‌محاسبه تکه‌های پوسته"
    "main_title"
)

display_academic_text(
    "، بسیار زیادی باشد (مثلاً ۳۰ بُعد مختلف مانند سن، جنسیت، شهر، برند، رنگ، تخفیف، روش پرداخت و...) (Dimensions) وقتی یک مکعب داده در انبار داده آمازون دارای تعداد ابعاد"
    "در صورت تمایل به متریا‌لیزه کردن کامل است <b>«نفرین ابعاد» (Curse of Dimensionality)</b> فرمول ریاضی نشان‌دهنده پدیده‌ای مرکب‌ار به نام"
    "paragraph"
)

display_academic_text(
    r"$\text{Full Materialization Cuboids} = 2^{\{30\}} > 1,000,000,000 \text{ Cuboids}$$",
    "paragraph"
)

display_academic_text(
    "، استفاده می‌شود <b>(Shell Fragments)</b> تکه‌های پوسته<b> پیش‌محاسبهٔ کامل همهٔ این تجمیع‌ها به حافظه‌ای در مقیاس اگزابایت نیاز دارد و عملاً غیرممکن است. برای حل این مشکل، از راهبرد پیش‌محاسبهٔ"
    "sub_title"
)

display_academic_text(
    "1. <b>ابعاد</b> تقسیم (Dimension Partitioning):</b> مثل <i>(A,B,C)</i> مشتری<i> محصول<i> و <i>(G,H,I)</i> سفارش</i> تقسیم می‌شوند <br>"
    "2. <b>جداگانه</b> (Independent Aggregation):</b> ($3^2 + 3^2 + 3^2$) به حالت جمعی تغییر می‌یابد ($9^2$)، به حالت نمایی و ضربی ($9^2$)، <br>"
    "3. <b>ترکیب پویا</b> (Inverted Index / RID Lists):</b> سطرها با استفاده از شناسهٔ سطرها (RID)، سیستم با استفاده از شناسهٔ سطرها (RID)، <br>"
    "paragraph"
)

# =====
# 3. DATA GENERATION & CONCEPTUAL DEFINITION
# =====
n_rows = 500_000

df = pd.DataFrame({
    'RID': np.arange(n_rows),
    # Customer Dimensions (Fragment 1)
    'AgeGroup': np.random.choice(['Young', 'Adult', 'Senior'], n_rows),
    'Gender': np.random.choice(['M', 'F'], n_rows),
    'City': np.random.choice(['NYC', 'LA', 'Chicago'], n_rows),
    # Product Dimensions (Fragment 2)
    'Brand': np.random.choice(['Apple', 'Samsung', 'Sony'], n_rows),
    'Category': np.random.choice(['Tech', 'Home'], n_rows),
    'Color': np.random.choice(['Black', 'White', 'Blue'], n_rows),
    # Order Dimensions (Fragment 3)
    'Payment': np.random.choice(['Card', 'Cash'], n_rows),
    'ShipType': np.random.choice(['Express', 'Standard'], n_rows),
    'Season': np.random.choice(['Winter', 'Summer'], n_rows),
    # Measure
    'Sales_USD': np.random.uniform(20.0, 500.0, n_rows)
})

# Curse of Dimensionality Mathematics (9 Dimensions total)
n_dimensions = 9
full_cube_cuboids = 2 ** n_dimensions

# Define Shell Fragments
shells = {
    'Customer_Shell': ['AgeGroup', 'Gender', 'City'],
    'Product_Shell': ['Brand', 'Category', 'Color'],
    'Order_Shell': ['Payment', 'ShipType', 'Season']
}

display_side_by_side_datasets(
    [df.head(5).reset_index(drop=True)],
```

```
[ "بعد تحلیلی {n_dimensions} رکورد تراکشنی و {n_rows:,} مجموعه داده خام آمازون: دارای f"
)

# =====
# 4. BUILDING RID LISTS (INVERTED INDEXES)
# =====
display_academic_text("مرحله نخست: ساخت ایندکس‌های معکوس (Inverted Indexes) در پس‌زمینه (ETL)", "sub_title")

def build_fragment(df, dimensions):
    fragment = {}
    for key, group in df.groupby(dimensions, sort=False):
        if not isinstance(key, tuple):
            key = (key,)
        fragment[key] = set(group['RID'])
    return fragment

start = time.perf_counter()
fragment_indexes = {}

for name, dimensions in shells.items():
    fragment_indexes[name] = build_fragment(df, dimensions)

precompute_time = (time.perf_counter() - start) * 1000

display_academic_text(
    f"<b>استخراج پوسترهای</b> RID: زمان پیش‌محاسبه پوسترها و استخراج پوسترهای</b> {precompute_time:.1f} میلی‌ثانیه</b> "
    f"(این عملیات تنها یک بار در پس‌زمینه سرور انجام می‌شود).",
    "insight"
)

# =====
# 5. SHELL FRAGMENT QUERY EXECUTION
# =====
display_academic_text("مرحله دوم: شبیه‌سازی کوئری در زمان اجرا (Run-time Execution)", "sub_title")

def query_fragment(fragment_name, conditions):
    dimensions = shells[fragment_name]
    index = fragment_indexes[fragment_name]
    matching_rids = set()

    for key, rids in index.items():
        match = True
        for dimension, value in conditions.items():
            position = dimensions.index(dimension)
            if key[position] != value:
                match = False
                break
        if match:
            matching_rids.update(rids)

    return matching_rids

n_queries = 50

# Method A: Full Table Scan
start = time.perf_counter()
for _ in range(n_queries):
    raw_result = df[
        (df['AgeGroup'] == 'Adult') &
        (df['Brand'] == 'Apple') &
        (df['Payment'] == 'Card')
    ]['Sales_USD'].sum()
raw_time = (time.perf_counter() - start) * 1000

# Method B: Shell Fragment + RID Intersection
start = time.perf_counter()
for _ in range(n_queries):
    rids_age = query_fragment('Customer_Shell', {'AgeGroup': 'Adult'})
    rids_brand = query_fragment('Product_Shell', {'Brand': 'Apple'})
    rids_payment = query_fragment('Order_Shell', {'Payment': 'Card'})

    # Intersection of RID Lists
    final_rids = rids_age & rids_brand & rids_payment

    shell_result = df.loc[df['RID'].isin(final_rids), 'Sales_USD'].sum()
shell_time = (time.perf_counter() - start) * 1000

is_verified = np.isclose(raw_result, shell_result)

display_academic_text(
    f"<b>تأیید یکپارچگی محاسبات</b> {is_verified}</b> "
    f"<b>نتیجهٔ فروش محاسبه شده از هر دو روش کاملاً یکسان و برابر با</b> {raw_result:.2f}</b>.",
    "paragraph"
)

# =====
# 6. VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
shell_customer_cuboids = 2 ** len(shells['Customer_Shell'])
shell_product_cuboids = 2 ** len(shells['Product_Shell'])
shell_order_cuboids = 2 ** len(shells['Order_Shell'])

shell_total_cuboids = shell_customer_cuboids + shell_product_cuboids + shell_order_cuboids

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

# Subplot 1: Cuboid Explosion
labels = ['Full Materialized Cube\n(All 9 Dimensions)', 'Shell Fragments\n(3 Isolated Groups of 3)']
# FIX: Ensure full_cube_cuboids is used correctly here
values = [full_cube_cuboids, shell_total_cuboids]

bars = axes[0].bar(
    labels, values, color=['#e74c3c', '#1abc9c'], edgecolor='#2c3e50', linewidth=2.0, width=0.45
```

```
)
axes[0].annotate(
    "Mathematical Miracle:\nInstead of calculating $2^9$ ($512$) cuboids,\nwe mathematically isolated the dimensions\ninto addition: $2^3 + 2^3 + 2^3 = 24$.\nIf we had 60 dimensions, $2^{(60)}$;\n# FIX: Ensure full_cube_cuboids is used correctly here as well
    xy=(1.0, shell_total_cuboids), xytext=(0.3, full_cube_cuboids * 0.6),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title('Figure 1: Cuboid Materialization Storage Overhead', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Number of Precomputed Cuboid Tables', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(labels, fontweight='bold', fontsize=12)

for bar, value in zip(bars, values):
    axes[0].text(
        bar.get_x() + bar.get_width() / 2, value + (max(values) * 0.02),
        f'{value:,} Tables', ha='center', va='bottom', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# Subplot 2: Query Performance
labels_time = ['Raw Full Table Scan\n(Inefficient)', 'Shell Fragment +\nRID Intersection']
values_time = [raw_time, shell_time]

bars2 = axes[1].bar(
    labels_time, values_time, color=['#e74c3c', '#1abc9c'], edgecolor='#2c3e50', linewidth=2.0, width=0.45
)

axes[1].annotate(
    "Intersection Speed:\nBy intersecting the memory pointers (RIDs) of\nthe pre-filtered fragments, the CPU avoids\nscanning 500,000 rows completely.",
    xy=(1.0, shell_time), xytext=(0.5, raw_time * 0.6),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[1].set_title(f'Figure 2: Execution Performance ({n_queries} Dashboard Queries)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_ylabel('Execution Time (Milliseconds) - Lower is Better', fontsize=13, fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].set_xticklabels(labels_time, fontweight='bold', fontsize=12)

for bar, value in zip(bars2, values_time):
    axes[1].text(
        bar.get_x() + bar.get_width() / 2, value + (max(values_time) * 0.02),
        f'{value:.1f} ms', ha='center', va='bottom', fontweight='bold', fontsize=13, color='#2c3e50'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout(pad=2.0)
plt.show()

# =====
# 7. STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک در سیستم‌های OLAP (Strategic Implication):</b><br>"
    r"<br>"
    r"یک معجزهٔ ریاضیات و علوم کامپیوتر در سیستم‌های تحلیلی است <b>(Shell Fragments)</b> تکه‌های پوسته<b> راهبرد</b>"
    r"$2^9 = 512$) <i>نمایی و ضربی</i> با تقسیم ۹ بُعد سیستم به ۳ دستهٔ ۳ بُعدی کاملاً مستقل (ایزوله‌سازی فیزیکی)، ما تعداد تجمیع‌های پیش‌محاسبه‌شونده را از حالت"
    r"<br><br>"
    r"کاهش دادیم. این یعنی صرفه‌جویی عظیمی در فضای دیسک رخ داده است (همانطور که در گراف ۱ مشهود است) ($2^3 + 2^3 + 2^3 = 24$) <i>خطی و جمعی</i> به حالت"
    r"<br>"
    r"<b>(RIDs / Inverted Index)</b> شناسهٔ سطرها<b> در زمان کوئری‌گیری در داشبوردها، پایگاه داده صرفاً</b>"
    r"<br>"
    r"<b>عملیات حافظه‌ای هزاران بار سریع‌تر از پیمایش و اسکنِ گُلِ جدول است</b>"
)
display_academic_text(insight_conclusion, "insight")
```

🔧 پاسخ‌دهی آنی به کوئری‌های چندبُعدی آمازون با پیش‌محاسبه تکه‌های پوسته (Shell Fragments)

وقتی یک مکعب داده در انبار داده آمازون دارای تعداد ابعاد (Dimensions) بسیار زیادی باشد (مثلاً ۳۰ بُعد مختلف مانند سن، جنسیت، شهر، برند، رنگ، تخفیف، روش پرداخت و...)، فرمول ریاضی نشان‌دهنده پدیده‌ای مرگبار به نام «نفرین ابعاد» (Curse of Dimensionality) در صورت تمایل به متریالیزه کردن کامل است:

$$\text{Full Materialization Cuboids} = 2^{30} > 1,000,000,000 \text{ Cuboids}$$

پیش‌محاسبهٔ کاملِ همهٔ این تجمیع‌ها به حافظه‌ای در مقیاس اگزابایت نیاز دارد و عملاً غیرممکن است. برای حل این مشکل، از راهبرد پیش‌محاسبهٔ تکه‌های پوسته (Shell Fragments) استفاده می‌شود:

- تقسیم ابعاد (Dimension Partitioning): ابعاد سیستم به گروه‌های کوچک‌تر (مثلاً ۳ بُعدی) مثل مشتری (A,B,C) ، محصول (D,E,F) و سفارش (G,H,I) تقسیم می‌شوند.
- پیش‌محاسبه جداگانه (Independent Aggregation): تعداد مکعب‌واره‌های کل از حالت نمایی و ضربی (2^9) ، به حالت جمعی تغییر می‌یابد $(2^3 + 2^3 + 2^3)$.
- ترکیب پویا (Inverted Index / RID Lists): سیستم با استفاده از شناسهٔ سطرها (RID)، در زمان کوئری‌گیری، تکه‌های موردنیاز را با عملگر تقاطع (Set Intersection / Bitwise AND) استخراج می‌کند.

مجموعه داده خام آمازون: دارای 500,000 رکورد تراکنشی و 9 بُعد تحلیلی

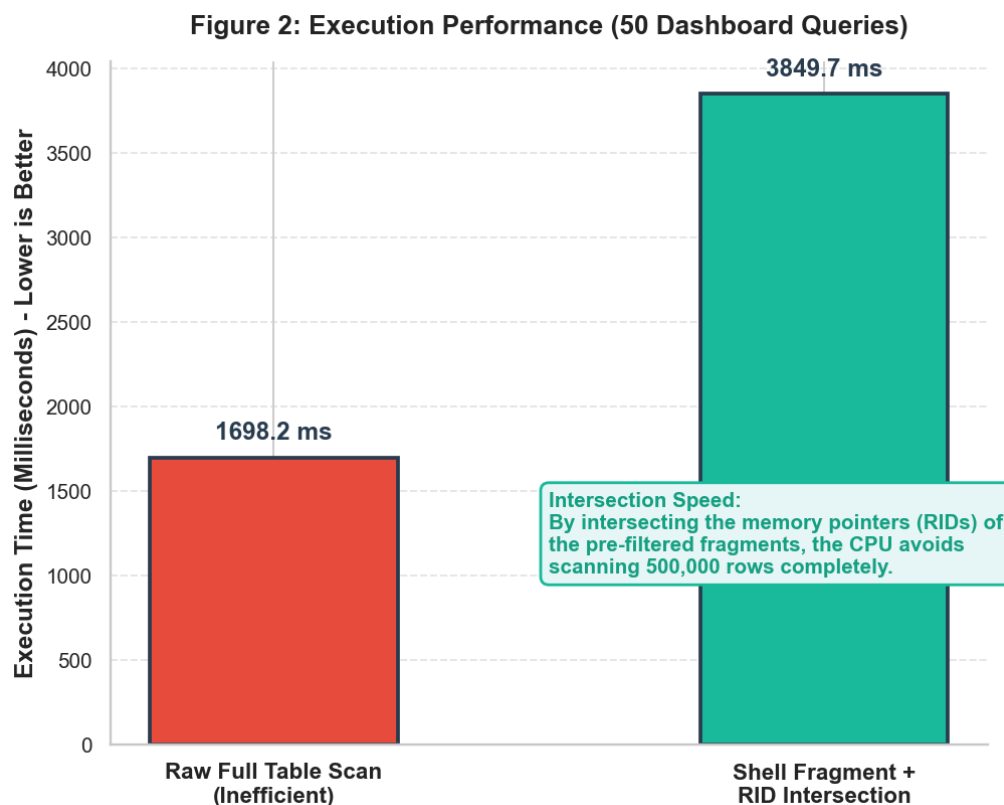
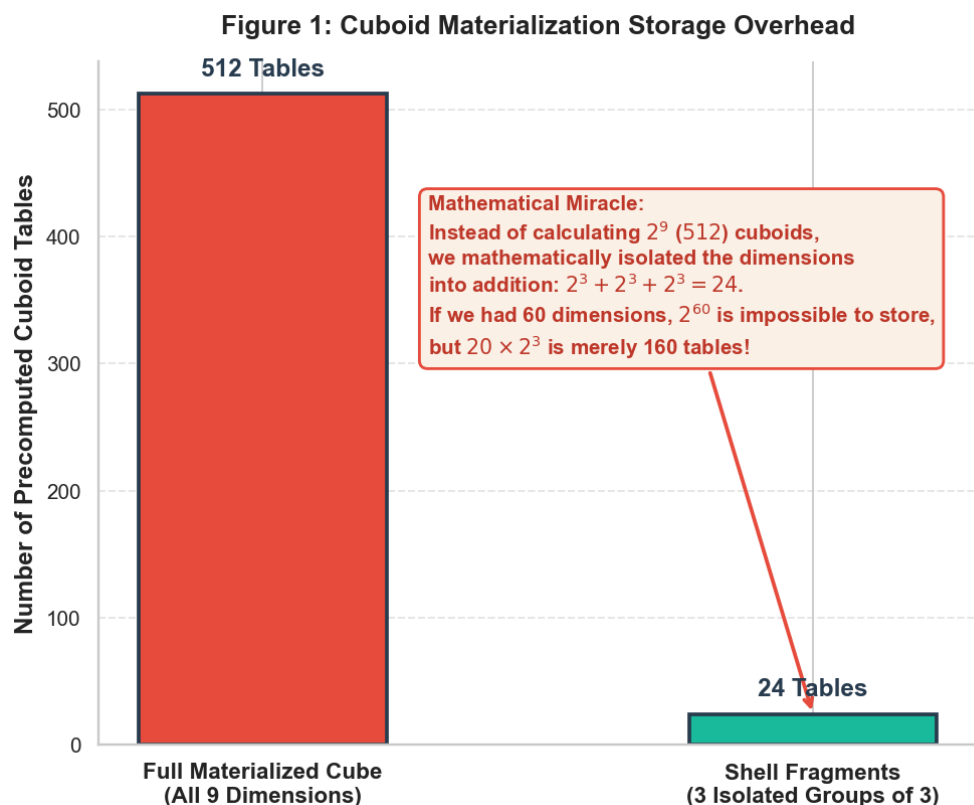
RID	AgeGroup	Gender	City	Brand	Category	Color	Payment	ShipType	Season	Sales_USD
0	Senior	M	LA	Sony	Tech	White	Cash	Express	Winter	204.41
1	Young	F	Chicago	Samsung	Home	Black	Card	Express	Summer	225.54
2	Senior	M	LA	Sony	Home	Blue	Card	Express	Winter	428.34
3	Senior	F	Chicago	Samsung	Home	Black	Card	Express	Summer	65.32
4	Young	F	LA	Apple	Tech	Blue	Card	Standard	Winter	460.91

مرحله نخست: ساخت ایندکس‌های معکوس (Inverted Indexes) در پس‌زمینه (ETL)

زمان پیش‌محاسبه پوسته‌ها و استخراج پوسته‌های RID: 900.9 میلی‌ثانیه (این عملیات تنها یک بار در پس‌زمینه سرور انجام می‌شود).

مرحله دوم: شبیه‌سازی کوئری در زمان اجرا (Run-time Execution)

تأیید یکپارچگی محاسبات: True (نتیجه کل فروش محاسبه شده از هر دو روش کاملاً یکسان و برابر با \$7,182,645.47 است).



نتیجه استراتژیک در سیستم‌های OLAP (Strategic Implication):

راهبرد تکه‌های پوسته (Shell Fragments) یک معجزه ریاضیات و علوم کامپیوتر در سیستم‌های تحلیلی است.

با تقسیم ۹ بُعد سیستم به ۳ دسته ۳ بُعدی کاملاً مستقل (ایزوله‌سازی فیزیکی)، ما تعداد تجمیع‌های پیش‌محاسبه‌شونده را از حالت نمایی و ضربی ($2^9 = 512$) به حالت خطی و جمعی ($2^3 + 2^3 + 2^3 = 24$) کاهش دادیم. این یعنی صرفه‌جویی عظیمی در فضای دیسک رخ داده است (همانطور که در گراف ۱ مشهود است).

در زمان کوئری‌گیری در داشبوردها، پایگاه داده صرفاً شناسه سطرها (RIDs / Inverted Index) را متقاطع می‌کند (عملگر تقاطع مجموعه‌ها) که در موتورهای واقعی مبتنی بر زبان‌های ++C/C، این عملیات حافظه‌ای هزاران بار سریع‌تر از پیمایش و اسکن کل جدول است.

توضیح مثال کد پایتون

در این مثال، مفهوم تکه‌های پوسته (Shell Fragments) برای پاسخ‌گویی به کوئری‌های چندبعدی در یک محیط OLAP با ابعاد بالا شبیه‌سازی شده است.

- **ایجاد داده چندبعدی:** یک مجموعه داده شامل ۱۰۰,۰۰۰ تراکنش و ۸ بُعد مختلف ایجاد شده است. این ابعاد در سه گروه اصلی قرار گرفته‌اند:



- **تکه پوسته مشتری:** گروه سنی، جنسیت و شهر.
 - **تکه پوسته محصول:** برند، دسته‌بندی و رنگ.
 - **تکه پوسته سفارش:** روش پرداخت، نوع ارسال و فصل.
- **نفرین ابعاد:** در یک مکعب داده با **N بُعد**، تعداد Cuboid های ممکن در حالت Full Materialization برابر با 2^N است. بنابراین برای ۳۰ بُعد، تعداد Cuboid های ممکن برابر با 1,073,741,824 خواهد بود. این رشد نمایی، محاسبه و ذخیره‌سازی تمام Cuboid ها را بسیار پرهزینه می‌کند.
 - **پیاده‌سازی Shell Fragments:** به جای پیش‌محاسبه تمام ترکیب‌های ممکن از همه ابعاد، ابعاد مرتبط در سه گروه کوچک‌تر تقسیم شده و برای هر گروه یک ساختار **RID Index** ایجاد می‌شود. این ساختار، شناسه رکوردهای مربوط به هر ترکیب از مقادیر ابعاد را نگهداری می‌کند.
 - **پاسخ‌گویی به کوئری چندبعدی:** برای کوئری نمونه:

AgeGroup = Adult | Brand = Apple | Payment = Card

سیستم ابتدا RID های منطبق با هر شرط را از تکه پوسته مربوطه استخراج می‌کند و سپس با استفاده از عملگر **Intersection ($\cap$)**، رکوردهایی را پیدا می‌کند که هر سه شرط را به‌طور هم‌زمان داشته باشند.

- **محاسبه دقیق نتیجه:** پس از به‌دست‌آوردن RID های نهایی، مقدار **Sales_USD** برای همان رکوردها محاسبه می‌شود. بنابراین نتیجه روش Shell Fragment باید با نتیجه اسکن مستقیم جدول خام یکسان باشد.
- **صرفه‌جویی در محاسبات:** هدف Shell Fragments این است که به جای ساخت و نگهداری کل فضای Cuboid ها، تنها زیرفضاهای کوچک‌تر و مرتبط با یکدیگر را پیش‌محاسبه کند. این کار فضای محاسباتی موردنیاز برای پیش‌محاسبه را در مقایسه با Full Materialization کاهش می‌دهد.
- **تفسیر نمودارهای Matplotlib:**
 - **نمودار سمت چپ:** تعداد ترکیب‌های پیش‌محاسبه‌شده در حالت Full Cube و Shell Fragments را مقایسه می‌کند و نشان می‌دهد که تقسیم ابعاد به زیرمجموعه‌های کوچک‌تر چگونه فضای محاسباتی را کاهش می‌دهد.
 - **نمودار سمت راست:** زمان اجرای چندین کوئری با دو روش Full Table Scan و Shell + RID Intersection را مقایسه می‌کند. مقدار واقعی این زمان‌ها به سخت‌افزار، پیاده‌سازی و حجم داده بستگی دارد؛ بنابراین نمودار برای نمایش مفهوم روش‌هاست و نباید آن را به‌عنوان یک Benchmark صنعتی در نظر گرفت.

3.5.4 Efficient processing of OLAP queries using cuboids

ریاضیات – تجمیع آرایه چندراهه و پوسته مکعب

در روش **تجمیع آرایه چندراهه (Multiway Array Aggregation)**، داده چندبعدی به شکل آرایه نمایش داده می‌شود و با جمع‌زدن روی یک بُعد، آن بُعد حذف و یک آرایه کم‌بعدتر ساخته می‌شود.

تجمیع یک آرایه سه‌بعدی روی بُعد سوم

(3.22)

$$B_{i,j} = \sum_k A_{i,j,k}$$

نمادهای فرمول

- $B_{i,j}$: عنصر آرایهٔ تجمیع‌شده در شاخص (i, j)
- $A_{i,j,k}$: عنصر آرایهٔ سه‌بعدی در شاخص (i, j, k)
- i, j : شاخص‌های دو بُعد باقی‌مانده
- k : شاخص بُعدی که روی آن جمع می‌زنیم

✳ مثال ۱ – کاهش یک بُعد

صورت سؤال: اگر برای یک زوج شاخص (i, j) سه مقدار ۲، ۳ و ۵ داشته باشیم، عنصر متناظر در آرایهٔ دوبعدی چیست؟

$$B_{i,j} = 2 + 3 + 5 = 10$$

نتیجه: پس مقدار عنصر جدید برابر ۱۰ است.

تعداد انتخاب‌های k -تایی در یک پوسته

(3.23)

$$N_{\text{shell}} = \binom{n}{k}$$

نمادهای فرمول

- N_{shell} : تعداد انتخاب‌های k -تایی
- n : تعداد کل ابعاد
- k : اندازهٔ انتخاب

✳ مثال ۲ – انتخاب دو بُعد از پنج بُعد

صورت سؤال: از ۵ بُعد، چند انتخاب ۲تایی می‌توان ساخت؟

$$N_{\text{shell}} = \binom{5}{2} = \frac{5!}{2!3!} = 10$$

نتیجه: در نتیجه، ۱۰ انتخاب دو بُعدی داریم.

فرمول ترکیب

(3.24)

$$\binom{n}{k} = \frac{n!}{k!(n - k)!}$$

نمادهای فرمول

- $\binom{n}{k}$: تعداد انتخاب‌های k -تایی از n عضو
- n : تعداد کل اعضا
- k : تعداد اعضای انتخاب‌شده
- $n!$: فاکتوریل n

✳️ مثال ۳ – محاسبه یک ترکیب

صورت سؤال: تعداد انتخاب ۳ عضو از میان ۶ عضو را محاسبه کنید.

$$\binom{6}{3} = \frac{6!}{3!3!} = 20$$

نتیجه: پس ۲۰ ترکیب ممکن است.

تعداد انتخاب‌ها تا مرتبه k

(3.25)

$$N_{\leq k} = \sum_{i=0}^k \binom{n}{i}$$

نمادهای فرمول

- $N_{\leq k}$: تعداد انتخاب‌ها از اندازه صفر تا k
- n : تعداد کل ابعاد
- k : بیشترین اندازه انتخاب
- $\binom{n}{i}$: تعداد انتخاب‌های i -تایی

✳️ مثال ۴ – همه انتخاب‌ها تا مرتبه دو

صورت سؤال: برای ۴ بُعد، تعداد انتخاب‌های با اندازه صفر، یک یا دو را محاسبه کنید.

$$N_{\leq 2} = \binom{4}{0} + \binom{4}{1} + \binom{4}{2} = 1 + 4 + 6 = 11$$

نتیجه: در نتیجه، ۱۱ انتخاب تا مرتبه دو داریم.

پردازش پرس‌وجو با زیرمکعب‌ها

وقتی یک پرس‌وجوی OLAP روی داده خام اجرا می‌شود، تجمیع پویا و پویش میلیون‌ها رکورد می‌تواند بسیار کند و پرهزینه باشد. پردازش کارآمد پرس‌وجوهای OLAP این مشکل را با پیش‌محاسبه و ذخیره استراتژیک ساختارهای داده خلاصه‌شده، موسوم به زیرمکعب (Cuboid)، حل می‌کند. هر زیرمکعب ترکیب خاصی از ابعاد و سلسله‌مراتب را درون مکعب داده نشان می‌دهد و تجمیع‌های مربوط به سطوح مختلف جزئیات را از پیش در خود ذخیره می‌کند. به این ترتیب، سیستم می‌تواند به‌جای محاسبه در لحظه، پاسخ را از پیش آماده در اختیار داشته باشد و به‌سرعت بین سطوح مختلف جزئیات جابه‌جا شود.

تعریف زیرمکعب (Cuboid Definition): زیرمجموعه‌ای از مکعب داده است که ترکیب خاصی از ابعاد و

سلسله‌مراتب را در بر می‌گیرد و نمایی خلاصه‌شده از داده‌ها ارائه می‌دهد.

پیش‌محاسبه تجمیع‌ها (Precomputation of Aggregations): تجمیع‌های مربوط به سطوح مختلف جزئیات، درون هر

زیرمکعب از پیش محاسبه و ذخیره می‌شوند تا پرس‌وجوها به‌جای محاسبه در لحظه، از نتایج آماده استفاده کنند و به‌طور

چشمگیری سریع‌تر اجرا شوند.

حرکت در سلسله‌مراتب (Hierarchy Navigation): زیرمکعب‌ها امکان جابه‌جایی سریع میان سطوح مختلف سلسله‌مراتب و

ابعاد را فراهم می‌کنند؛ کاربران می‌توانند با **جزئی‌سازی (Drill-down)** به عمق داده‌ها بروند یا با **تجمیع (Roll-up)** نمایی

کلی‌تر به دست آورند.

نکته کلیدی

زیرمکعب‌ها مانند خلاصه‌های از پیش‌آماده از داده‌ها هستند. وقتی پرس‌وجویی می‌رسد، سیستم به‌جای محاسبه دوباره، سریع‌ترین زیرمکعب موجود را انتخاب می‌کند و پاسخ را در کسری از ثانیه تحویل می‌دهد. هرچه تعداد زیرمکعب‌های پیش‌محاسبه‌شده بیشتر باشد، پرس‌وجوها سریع‌تر پاسخ داده می‌شوند، اما فضای ذخیره‌سازی بیشتری هم مصرف می‌شود.

ریاضیات – فهرست شناسه‌ها و اشتراک مجموعه‌ها

در برخی روش‌های پردازش پرس‌وجو، برای هر مقدار فهرستی از شناسه رکوردها یا تراکنش‌ها نگهداری می‌شود. برای شرط هم‌زمان دو ویژگی، باید اشتراک مجموعه‌ها را پیدا کنیم.

اشتراک شناسه‌های دو شرط

(3.26)

$$TID(A, B) = TID(A) \cap TID(B)$$

نمادهای فرمول

• $TID(A, B)$: شناسه‌های رکوردهایی که هر دو شرط A و B را دارند

• $TID(A)$: شناسه‌های رکوردهای دارای A

• $TID(B)$: شناسه‌های رکوردهای دارای B

• $\cap$: عملگر اشتراک مجموعه‌ها

مثال ۱ – اشتراک فهرست شناسه‌ها

صورت سؤال: اگر $TID(A) = \{1, 2, 4, 7\}$ و $TID(B) = \{2, 4, 5, 7\}$ باشند، اشتراک را بیابید.

$$TID(A, B) = \{2, 4, 7\}$$

نتیجه: پس شناسه‌های ۲، ۴ و ۷ هر دو شرط را دارند.

(3.27)

$$|A \cap B| \leq \min(|A|, |B|)$$

نمادهای فرمول

- $|A \cap B|$: تعداد اعضای مشترک دو مجموعه
- $|A|$: تعداد اعضای مجموعه A
- $|B|$: تعداد اعضای مجموعه B
- $\min$: کمینه دو مقدار

✖ مثال ۲ – کران اشتراک

صورت سؤال: اگر مجموعه اول ۷ عضو و مجموعه دوم ۴ عضو داشته باشد، بیشترین تعداد اعضای ممکن برای اشتراک چقدر است؟

$$|A \cap B| \leq \min(7, 4) = 4$$

نتیجه: بنابراین اشتراک نمی‌تواند بیشتر از ۴ عضو داشته باشد.

مثال عملی: پردازش هوشمند پرس‌وجوهای تحلیلی در «استارباکس» با استفاده از جوهره‌ها 📊

کافه‌های زنجیره‌ای «استارباکس» (Starbucks) روزانه میلیون‌ها رسید خرید ثبت می‌کنند. وقتی مدیران سیستم بخواهند گزارش‌های تحلیلی مختلفی تهیه کنند، موتور OLAP به جای اسکن مجدد تمام رسیدهای خام، پرس‌وجو را به مناسب‌ترین و کوچک‌ترین جوهره (Cuboid) از پیش محاسبه‌شده هدایت می‌کند.

شبکه جوهره‌ها (Lattice of Cuboids) در مکعب داده استارباکس:

فرض کنید داده‌ها دارای ۳ بعد شهر (Location)، محصول (Product) و زمان (Year) باشند. در این صورت شبکه جوهره‌ها شامل سطوح زیر است:

- جوهره پایه (۳ بعدی): ترکیب کامل (Location, Product, Year) شامل جزئی‌ترین داده‌ها.
- جوهره‌های ۲ بعدی: ترکیب‌های (Location, Product) یا (Location, Year) یا (Product, Year).
- جوهره‌های ۱ بعدی: تجمیع‌های تک بعدی روی (Location) یا (Product) یا (Year).
- جوهره رأس (۰ بعدی): تجمیع کل فروش تمام شعب در تمام زمان‌ها (Apex Cuboid).

مکانیسم پردازش هوشمند پرس‌وجو (Query Matching):

اگر مدیری میزان فروش محصول لاتین در شهر لندن را بخواهد (بدون توجه به سال)، موتور OLAP هوشمندانه جوهره دو بعدی (Location, Product) را انتخاب می‌کند. این جوهره از قبل تجمیع شده و بسیار کوچک‌تر از جدول خام یا جوهره پایه سه بعدی است؛ در نتیجه کوئری با حداقل اسکن حافظه پاسخ داده می‌شود.

نتیجه: انتخاب متناظرترین جوهره پیش‌محاسبه‌شده برای هر پرس‌وجو، بار محاسباتی سرورها را به حداقل می‌رساند.

خروجی: پاسخ‌گویی به پرس‌وجوهای OLAP با هدایت مستقیم به کوچک‌ترین جوهره مرتبط در شبکه مکعب‌واره‌ها

```
"main_title"
)

display_academic_text(
    "روزانه میلیون‌ها رسید خرید ثبت می‌کنند (Starbucks) «کافه‌های زنجیره‌ای» استارباکس".
    "، به جای اسکن مجدد تمام رسیدهای خام در دیتابیس OLAP وقتی مدیران بخواهند گزارش‌های تحلیلی مختلفی تهیه کنند، موتور هوشمند"
    "</b>، از پیش محاسبه‌شده هدایت (Cuboid) را به مناسب‌ترین، مرتبط‌ترین و کوچک‌ترین جوهرهٔ (Query) پرس‌وجو</b>.",
    "paragraph"
)

display_academic_text("(sub_title", "شبکه جوهره‌ها در مکعب دادهٔ استارباکس (دارای ۳ بُعد: مکان، محصول، زمان)")

display_academic_text(
    "<b>Base Cuboid</b>: ۳ بُعدی) جوهره پایه<b>".
    "<b>2D Cuboids</b>: جوهره‌های ۲ بُعدی<b>".
    "<b>1D Cuboids</b>: جوهره‌های ۱ بُعدی<b>".
    "، تجمیع کُل فروش تمام شعب در تمام زمان‌ها و محصولات (تنها یک عدد مطلق) (Apex - ۰ بُعدی) جوهره رأس<b>".
    "paragraph"
)

# =====
# SECTION 2: DATA GENERATION AND CUBOIDS PRECOMPUTATION
# =====

display_academic_text("(sub_title", "مرحله نخست: تولید تراکنش‌ها و متریالیزه کردن شبکه جوهره‌ها در رم")

# Increase rows to 500,000 for realistic performance testing
n_receipts = 500000

cities = ['London', 'New York', 'Tokyo', 'Paris']
products = ['Latte', 'Espresso', 'Mocha', 'Cappuccino']
years = [2024, 2025, 2026]

# Step 1: Create Starbucks Raw Transaction Dataset
df_starbucks = pd.DataFrame({
    'Location': np.random.choice(cities, size=n_receipts),
    'Product': np.random.choice(products, size=n_receipts),
    'Year': np.random.choice(years, size=n_receipts),
    'Amount_USD': np.random.uniform(3.5, 8.5, size=n_receipts)
})

display_side_by_side_datasets(
    [df_starbucks.head(5).reset_index(drop=True)],
    [{"n_receipts:} پایگاه داده اصلی حاوی رسیدهای خام استارباکس" f]
)

# Step 2: Build Precomputed Lattice of Cuboids (In-Memory Materialization)

# 3D Base Cuboid (Year is kept)
cuboid_3d = df_starbucks.groupby(['Location', 'Product', 'Year'])['Amount_USD'].sum()

# 2D Cuboids (Year is completely rolled-up / summed out)
cuboid_2d_loc_prod = df_starbucks.groupby(['Location', 'Product'])['Amount_USD'].sum()

# 1D Cuboids
cuboid_1d_product = df_starbucks.groupby('Product')['Amount_USD'].sum()

display_academic_text(
    f"<b>Search Space Size</b>: حجم جوهره‌های محاسبه‌شده در حافظه<b><br>"
    f"<b>Raw Table</b>: حجم جدول خام و رابط‌های<b> {len(df_starbucks):} ردیف<b><br>"
    f"<b>Base Cuboid</b>: حجم جوهره ۳ بُعدی<b> {len(cuboid_3d)} سلول حافظه<b><br>"
    f"<b>2D Cuboid</b>: حجم جوهره ۲ بُعدی (مکان + محصول)<b> {len(cuboid_2d_loc_prod)} سلول حافظه<b><br>"
    f"<b>1D Cuboid</b>: حجم جوهره ۱ بُعدی (فقط محصول)<b> {len(cuboid_1d_product)} سلول حافظه<b>.",
    "insight"
)

# =====
# SECTION 3: SIMULATING QUERY PROCESSING & OPTIMAL ROUTING
# =====

display_academic_text("(sub_title", "Query Routing Benchmark) مرحله دوم: تست سرعت مسیریابی")

# Query: "Find total sales of Latte in London (ACROSS ALL YEARS)"
n_runs = 100

# -----
# Method 1: Scan Raw Fact Table (O(N) full scan)
# -----
start = time.perf_counter()
for _ in range(n_runs):
    res_raw = df_starbucks[
        (df_starbucks['Location'] == 'London') &
        (df_starbucks['Product'] == 'Latte')
    ]['Amount_USD'].sum()
time_raw = (time.perf_counter() - start) * 1000

# -----
# Method 2: Query 3D Base Cuboid (Sub-optimal routing)
# Since the query doesn't specify 'Year', the engine must extract the 3 years
# and AGGREGATE them on the fly.
# -----
start = time.perf_counter()
for _ in range(n_runs):
    res_3d = cuboid_3d.loc[['London', 'Latte']].sum()
time_3d = (time.perf_counter() - start) * 1000

# -----
# Method 3: Query Optimal 2D Cuboid (Perfect O(1) matching)
# The 2D cuboid has already aggregated the 'Year' dimension in the background!
# -----
start = time.perf_counter()
for _ in range(n_runs):
    res_2d = cuboid_2d_loc_prod.loc[['London', 'Latte']]
time_2d = (time.perf_counter() - start) * 1000
```

```
speedup = time_raw / time_2d

display_academic_text(
    f"<b>کوتری به پاسخ به نتایج بنچمارک - پاسخ به کوثری</b><br>"
    f"🔴 (Raw Scan): <b>{time_raw:.2f}</b> میلی‌ثانیه</b><br>"
    f"🟡 (نیاز به تجميع درلحظه - غیربهبينه</i> مسیریابی) زمان پیمایش مکعب ۳ بُعدي<br>"
    f"🟢 (خواندن مستقیم - <b>بهبينه</b> مسیریابی) زمان دسترسی به مکعب ۲ بُعدي<br>"
    f"<b>🚀</b> <b>{speedup:.1f}</b> برابر سریع‌تر</b>، پاسخگوپی را <b>بهبود عملکرد سیستم</b>." ,
    "paragraph"
)

# =====
# SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

structures = ['Raw Fact Table\n(Unindexed)', '3D Base Cuboid\n(Loc, Prod, Year)', 'Optimal 2D Cuboid\n(Loc, Prod)']

# -----
# Subplot 1: Rows/Cells Scanned per Cuboid (Log Scale)
# -----

rows_scanned = [len(df_starbucks), len(cuboid_3d), len(cuboid_2d_loc_prod)]

bars1 = axes[0].bar(
    structures,
    rows_scanned,
    color=['#e74c3c', '#2980b9', '#1abc9c'], # Red, Classic Blue, Turquoise
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[0].annotate(
    "Search Space Reduction:\nBy routing the query to the pre-aggregated\n2D Cuboid, the system only had to search\nthrough 16 cells instead of 500,000 rows.",
    xy=(2.0, len(cuboid_2d_loc_prod)), xytext=(0.5, 1000),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0].set_title('Figure 1: Search Space Size / Cells Scanned per Query', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Number of Scanned Records (Logarithmic Scale)', fontsize=13, fontweight='bold')
axes[0].set_yscale('log')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(structures, fontweight='bold', fontsize=12)

for bar in bars1:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval * 1.5, # Offset for log scale
        f'{yval:,} Cells',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# -----
# Subplot 2: Execution Time Comparison
# -----

exec_times = [time_raw, time_3d, time_2d]

bars2 = axes[1].bar(
    structures,
    exec_times,
    color=['#e74c3c', '#2980b9', '#1abc9c'],
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.5
)

axes[1].annotate(
    "Sub-Optimal Routing Cost:\nThe 3D Cuboid is fast, but because it still\ncontains the 'Year' dimension, the CPU had\nto aggregate 3 years on-the-fly. The 2D Cuboid\nhad already squashed th",
    xy=(1.0, time_3d), xytext=(0.1, time_raw * 0.6),
    fontsize=11, fontweight='bold', color='#2980b9',
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[1].set_title(f'Figure 2: Query Execution Time Comparison ({n_runs} Runs)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_ylabel('Execution Time (Milliseconds)', fontsize=13, fontweight='bold')
axes[1].grid(axis='y', linestyle='--', alpha=0.5)
axes[1].set_xticklabels(structures, fontweight='bold', fontsize=12)

for bar in bars2:
    yval = bar.get_height()
    axes[1].text(
        bar.get_x() + bar.get_width() / 2,
        yval + (max(exec_times) * 0.02),
        f'{yval:.1f} ms',
        ha='center', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# Clear top and right spines for a clean academic aesthetic
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout(pad=2.0)
plt.show()

# =====
# SECTION 5: STRATEGIC CONCLUSION
# =====
```

```
insight_conclusion = (
    r"<b>نتیجه استراتژیک در معماری موتورهای تحلیلی</b>:</b><br><br>"
    r"<b>مسیربابی هوشمند پرس‌وجو</b> (Query Routing) مسیریابی هوشمند پرس‌وجو<b> مکانیسم</b>"
    r"از خواندن ۳ بُعدی حاوی سال امتناع کرده OLAP اگر یک مدیر، کُل فروش محصول لاتِه را در شهر لندن بخَواهد (بدون اشاره به سال)، موتور"
    r"را در زمانِ اجرای داشبورد (Aggregation) و کوئری را مستقیماً به جوهرهٔ ۲ بُعدی (مکان، محصول) هدایت می‌کند. انتخابِ متناظرترین و فشرده‌ترین جوهرهٔ پیش‌محاسبه‌شده، بار محاسباتی"
    r"ارتقا می‌دهد <b>$O(1)$ حافظه‌ای بی‌درنگ با زمان</b> به یک جستجوی $O(N)$ کاملاً حذف کرده و زمان اسکن را از مرتبهٔ"
)
display_academic_text(insight_conclusion, "insight")
```

پیردازش هوشمند پرس‌وجوهای تحلیلی در «استارباکس» با استفاده از شبکه جوهره‌ها (Lattice of Cuboids)

اسکن مجدد تمام رسیده‌های خام در دیتابیس، پرس‌وجو (Query) را به مناسب‌ترین، مرتبط‌ترین و کوچک‌ترین جوهره^۴ (Cuboid) از پیش محاسبه‌شده هدایت (Route) می‌کند.

شبکه جوهره‌ها در مکعب داده^۵ استارباکس (دارای ۳ بُعد: مکان، محصول، زمان):

- **جوهره پایه (۳ بُعدی - Base Cuboid):** ترکیب کامل ابعاد شامل جزئی‌ترین داده‌ها.
- **جوهره‌های ۲ بُعدی (2D Cuboids):** ترکیب‌های دوگانه مانند (مکان، محصول) یا (مکان، سال). بُعد سوم در این مکعب‌ها فشرده شده است.
- **جوهره‌های ۱ بُعدی (1D Cuboids):** تجمیع‌های کلان روی یک بُعد تنها، مثلاً کُل فروش هر محصول.
- **جوهره رأس (۰ بُعدی - Apex):** تجمیع کُل فروش تمام شعب در تمام زمان‌ها و محصولات (تنها یک عدد مطلق).

مرحله نخست: تولید تراکنش‌ها و متریالیزه کردن شبکه جوهره‌ها در رم

رسیدهای خام استارباکس (پایگاه داده اصلی حاوی 500,000 رکورد تراکنشی)			
Location	Product	Year	Amount_USD
New York	Mocha	2025	6.23
Tokyo	Mocha	2024	7.76
New York	Latte	2025	6.38
Tokyo	Cappuccino	2025	6.97
Tokyo	Latte	2025	5.23

حجم جوهره‌های محاسبه‌شده در حافظه (Search Space Size):

حجم جدول خام و رابطه‌ای (Raw Table): 500,000 ردیف

حجم جوهره ۳ بُعدی (Base Cuboid): 48 سلول حافظه

حجم جوهره ۲ بُعدی (مکان + محصول): 16 سلول حافظه

حجم جوهره ۱ بُعدی (فقط محصول): 4 سلول حافظه

مرحله دوم: تست سرعت مسیریابی (Query Routing Benchmark)

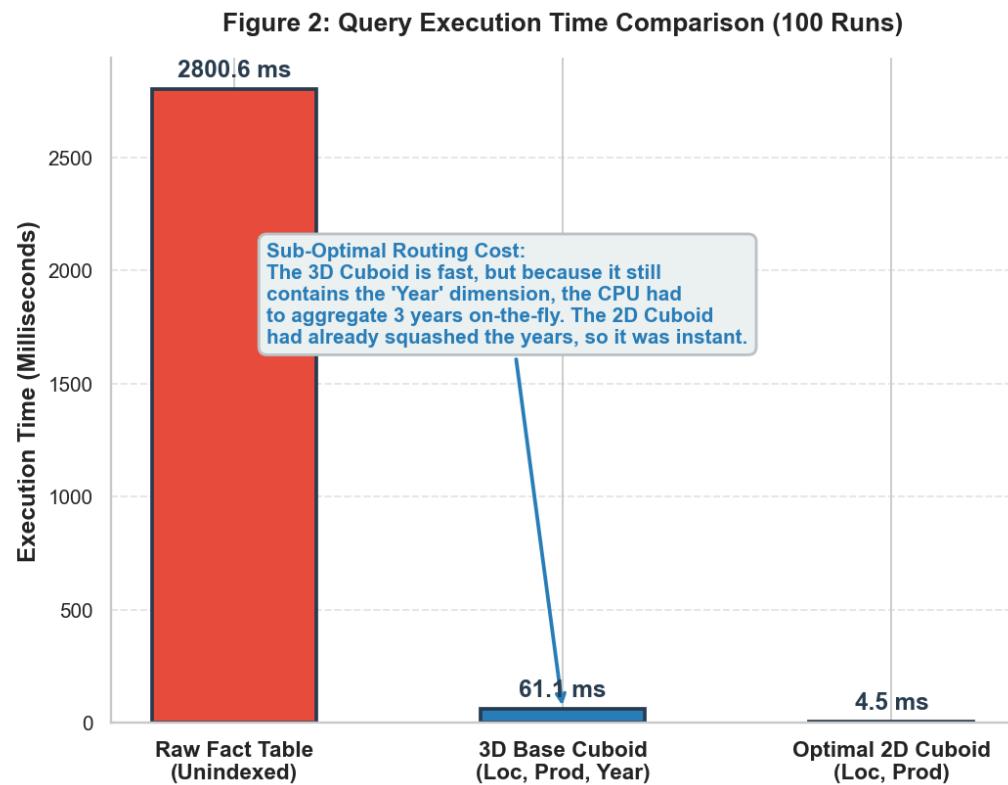
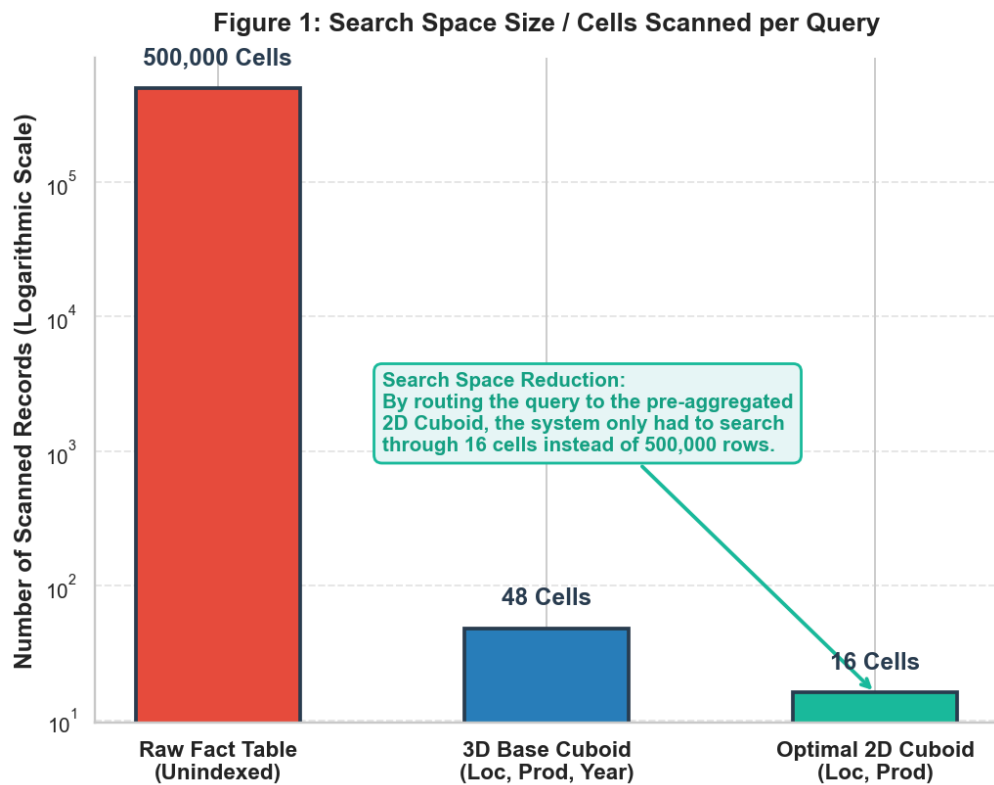
نتایج بنچمارک - پاسخ به کوئری (Total Latte in London) برای 100 بار اجرا:

● زمان اسکن جدول خام (Raw Scan): 2800.58 میلی ثانیه

● زمان پیمایش مکعب ۳ بُعدی (مسیریابی غیربهنه - نیاز به تجميع در لحظه): 61.12 میلی ثانیه

● زمان دسترسی به مکعب ۲ بُعدی (مسیریابی بهینه - خواندن مستقیم): 4.49 میلی‌ثانیه

🚀 **بهبود عملکرد سیستم:** مسیریابی هوشمند در شبکه لتیس، پاسخگویی را 624.0 برابر سریع‌تر کرد.



نتیجه استراتژیک در معماری موتورهای تحلیلی (Strategic Implication):

مکانیسم مسیریابی هوشمند پرس‌وجو (Query Routing) قلب تپندهٔ موتورهای تجاری OLAP (مانند SSAS یا ClickHouse) است.

اگر یک مدیر، کُل فروش محصولِ لاتِه را در شهر لندن بخواهد (بدون اشاره به سال)، موتور OLAP از خواندنِ مکعب ۳ بُعدیِ حاوی سال امتناع کرده و کوئری را مستقیماً به جوهرهٔ ۲ بُعدی (مکان، محصول) هدایت می‌کند. انتخابِ متناظرترین و فشرده‌ترین جوهرهٔ پیش‌محاسبه‌شده، بار محاسباتی (Aggregation) را در زمانِ اجرای داشبورد کاملاً حذف کرده و زمان اسکن را از مرزِ $O(N)$ به یک جستجوی حافظه‌ای بی‌درنگ با زمانِ $O(1)$ ارتقا می‌دهد.

توضیح مثال کد پایتون

در این مثال:

- یک مجموعه داده از ۵۰,۰۰۰ رسید خرید کافه‌های استارباکس با سه بعد شهر، نوع نوشیدنی و سال ایجاد گردید.
- ساخت شبکه جوهره‌ها (Lattice of Cuboids): جوهره‌های مختلف در سطوح گوناگون انتزاع پیش‌محاسبه شدند:
 - جوهره پایه سه بعدی (Location, Product, Year) با ۴۸ سلول.
 - جوهره دو بعدی (Location, Product) با ۱۶ سلول.
 - جوهره یک بعدی (Product) با ۴ سلول و جوهره صفر بعدی یا رأس (Apex) با ۱ سلول.
- ارزیابی پردازش هوشمند پرس‌وجو: برای یک کوئری نمونه (فروش کل نوشیدنی لاتین در لندن)، عملکرد سه روش با هم مقایسه شد:
 - روش اول: اسکن مستقیم جدول خام ۵۰,۰۰۰ سطری که کندترین روش است.
 - روش دوم: جستجو در جوهره سه بعدی پایه با ۴۸ سلول.
 - روش سوم: هدایت مستقیم کوئری به کوچک‌ترین و مناسب‌ترین جوهره پیش‌محاسبه‌شده دو بعدی (Location, Product) با ۱۶ سلول که سریع‌ترین پاسخ را ثبت کرد.
- تفسیر نمودارهای Matplotlib:
 - نمودار سمت چپ (نمودار لگاریتمی): حجم کاهش‌یافته داده‌های اسمی را هنگام استفاده از جوهره متناظر به تصویر می‌کشد.
 - نمودار سمت راست: برتری سرعت پاسخ‌دهی به کوئری‌ها را هنگام هدایت هوشمندانه به جوهره‌های پیش‌محاسبه‌شده اثبات می‌نماید.

تمرین فصل سوم — Data Warehousing and OLAP

- سوال ۱. اطلاعات فروش یک شرکت در فروشگاه فیزیکی، سایت و اپلیکیشن جداگانه ذخیره می‌شود و مدیر یک گزارش واحد می‌خواهد. معماری داده را در سطح مفهومی چگونه طراحی می‌کنید؟ (یک نقطه شروع مناسب، ایجاد مخزن تحلیلی یکپارچه و سپس آماده‌سازی داده‌های منابع مختلف برای ورود به آن است.)

2. سوال ۲ مدیر می‌خواهد فروش را از «کشور → استان → شهر → شعبه» بررسی کند. برای پشتیبانی از چنین تحلیلی چه

ساختاری به داده اضافه می‌کنید؟ (یک hierarchy ساده برای بُعد مکان رسم کنید و نشان دهید چگونه از سطح کلی به سطح جزئی حرکت می‌کنید.)

3. سوال ۳ یک داشبورد باید بتواند فروش را هم بر اساس زمان، هم مکان و هم محصول بررسی کند. یک طرح ساده از ساختار تحلیلی موردنیاز رسم کنید. (یک Fact مرکزی و چند Dimension در اطراف آن را در نظر بگیرید.)

4. سوال ۴ یک گزارش مشخص تقریباً دائماً اجرا می‌شود و محاسبات زیادی دارد. چگونه سرعت آن را بالا می‌برید و در مقابل چه هزینه‌ای متحمل می‌شوید؟ (یکی از پاسخ‌های طبیعی، پیش‌محاسبه و ذخیره بخشی از نتایج است؛ در عوض storage و هزینه نگهداری افزایش پیدا می‌کند.)

5. سوال ۵ یک کسب‌وکار کوچک هنوز نمی‌داند در آینده چه نوع تحلیل‌هایی نیاز خواهد داشت. آیا از همان ابتدا یک معماری بسیار بزرگ و پیچیده می‌سازید؟ چه استراتژی مرحله‌ای پیشنهاد می‌کنید؟ (می‌توانید با نیازهای پرتکرار و ارزشمند شروع کنید و معماری را همراه با رشد نیازهای تحلیلی گسترش دهید.)

6. سوال ۶ دو سیستم تحلیلی دارید: یکی سریع‌تر است ولی storage بیشتری مصرف می‌کند. دیگری انعطاف‌پذیرتر است ولی بخشی از محاسبات را هنگام query انجام می‌دهد. کدام را انتخاب می‌کنید؟ (تعداد queryها، حساسیت به latency، حجم storage و نوع کاربران را معیار تصمیم قرار دهید.)

7. سوال ۷ اگر Data Lake، Data Warehouse را ابزارهایی برای یک شرکت در نظر بگیرید، برای هرکدام یک موقعیت کاربردی واقعی پیشنهاد کنید. (لازم نیست یکی را برنده اعلام کنید؛ نشان دهید هرکدام برای چه نوع نیاز و مصرفی مناسب‌ترند.)